让编码跟上思考的速度

ActionScript 开发者指南

PureMVC



O'REILLY®

Adobe Developer Library

Cliff Hall

ActionScript 开发者 PureMVC 指南

通过实践掌握 PureMVC：这个流行的开源框架采用模型—视图—控制器架构，帮助你开发易维护的应用程序。在这本精简指南中，PureMVC 创始人 Cliff Hall 带你构建一个完整且具有一定复杂度的 Adobe AIR 应用，讲授 PureMVC 开发的基础。

通过清晰的讲解和大量 ActionScript 代码示例，你将学会在日常工作中使用框架类的最佳实践。了解 PureMVC 如何让你专注于应用程序的目的和范围，同时以易维护、可移植的方式承担底层连接工作。

- 详细了解使用 PureMVC 开发应用程序的流程。
- 通过设计数据模式，并创建独立于框架的值对象，完成领域建模。
- 实现独立于框架的视图组件，通过事件和属性公开 API。
- 使用代理模式管理值对象，并隐藏与服务交互的细节。
- 促进视图组件与应用程序其他部分之间的双向通信。
- 通过控制器中的命令对象，将模型层和视图层连接起来。
- 处理视图组件生命周期中的棘手问题，并学习如何复用模型层。

在 oreilly.com 购买本书的 O’Reilly 电子版，即可在该版生命周期内免费获得更新。我们的电子书针对多种格式进行了优化，包括 PDF、EPUB、Mobi、APK 和 DAISY，全部不带 DRM 限制。

Twitter: @oreillymedia
facebook.com/oreilly

US $29.99 CAN $31.99
ISBN: 978-1-449-31456-9

O’REILLY®
oreilly.com

ActionScript 开发者 PureMVC 指南

Cliff Hall

O'REILLY®
北京 · 剑桥 · 法纳姆 · 科隆 · 塞巴斯托波尔 · 东京

ActionScript 开发者 PureMVC 指南
Cliff Hall 著

版权 © 2012 Futurescale, Inc.。保留所有权利。
美国印制。

出版：O’Reilly Media, Inc.，1005 Gravenstein Highway North, Sebastopol, CA 95472。

O’Reilly 图书可供教育、商业或促销用途采购。大多数书目也提供在线版本（http://my.safaribooksonline.com）。详情请联系企业／机构销售部门：(800) 998-9938，或 corporate@oreilly.com。

编辑：Mike Hendrickson、Mary Treseler
制作编辑：Melanie Yarbrough

封面设计：Karen Montgomery
内文设计：David Futato
插图：Robert Romano

第一版修订记录：
2011-12-19　首次发行
版本详情见 http://oreilly.com/catalog/errata.csp?isbn=9781449314569。

Nutshell Handbook、Nutshell Handbook 标志和 O’Reilly 标志均为 O’Reilly Media, Inc. 的注册商标。《ActionScript 开发者 PureMVC 指南》、封面上的厚喙骨顶鸡图像及相关商业外观均为 O’Reilly Media, Inc. 的商标。

制造商和销售商用来区分其产品的许多名称均被声明为商标。本书中出现这些名称时，凡 O’Reilly Media, Inc. 已知其商标声明的，均以全大写或首字母大写形式排印。

尽管本书在编写过程中已尽力谨慎，出版者和作者仍不对任何错误、遗漏，或因使用本书所含信息而导致的损失承担责任。

ISBN：978-1-449-31456-9

[LSI]

1323727631



Adobe Developer Library 是 O’Reilly Media Inc. 与 Adobe Systems, Inc. 合作出版的系列，为使用 Adobe 技术的开发者提供权威资源。这些全面的资源提供学习方案，帮助开发者创建前沿的交互式 Web 应用程序，使其几乎能够触达任何平台上的任何用户。

Adobe Developer Library 通过高品质图书和创新的在线资源，涵盖富互联网应用程序开发的最新工具，直接提供来自源头的专家培训。主题包括 ActionScript、Adobe Flex®、Adobe Flash® 和 Adobe Acrobat®。

有关图书、在线资源等方面的最新消息，请访问 http://adobedeveloperlibrary.com。

谨将本书献给我的妻子 Helen。
她的耐心、爱与支持，
成就了我每一次键盘敲击。

目录

前言 xiii

1. 引言 1

经典 MVC 架构 1
PureMVC AS3 参考实现 3
PureMVC 在应用程序中的作用 4
　不必理会幕后的那个人 5
　认识干活的主力 6
位于边界上的角色 6
　使用 Proxy 作为应用程序的数据源 6
　使用 Mediator 作为视图组件的秘书 9
位于边界之间的角色 12
　让 SimpleCommand 承担大部分思考工作 12
　使用 MacroCommand 执行多个 SimpleCommand 13
其余成员 14
　通知 14
　核心角色 15
　外观 15
为类组织包结构 16
　普通类 16
　框架类 16
　典型的包结构 16
Standard 与 MultiCore 版本 17
编写可移植代码 19

2. 使用 PureMVC 构建应用程序 21

选择平台 21
那是一个漆黑的暴风雨之夜 22
StoryArchitect：小说写作工具 24
　目的陈述 24
　确定范围 25

描述领域 26
构思用户界面 26
持久化策略 27
搭建启动脚手架 28
准备项目与初始包结构 29
创建应用程序常量类 29
创建启动命令 30
创建应用程序外观 31
初始化应用程序外观并调用 startup() 32
由外向内构建 33
步骤 1：值对象 34
步骤 2：视图组件 34
步骤 3：代理 35
步骤 4：中介器 36
步骤 5：命令 37

3. 领域建模 39
设计数据模式 39
回顾实体 40
创建数据模式 41
创建值对象 51
简单示例 51
稍复杂的示例 53
进阶示例 58
创建枚举 66
列出所需枚举 67
基类 67
Enum 子类 68
测试实体 69
Flex 单元测试类 69

4. 实现用户界面 73
两个视图的故事 74
应用程序 76
选择器 79
故事选择器 80
编辑器 82
编辑器控件 84
详情组件 87
条目信息组件 89
笔记组件 91
时间线组件 94
故事磁贴 96

部磁贴 100
章磁贴 102
场景磁贴 105
草稿磁贴 107
选择上下文 109
应用程序事件 112

5. 为模型提供代理 117
Proxy 的生命周期 118
重新审视持久化策略 119
我们需要哪些 Proxy? 120
如何存储数据? 121
建立索引 121
构建 Proxy 122
抽象代理 123
人物阵容代理 128
环境代理 131
故事代理 133
系列代理 138

6. 为视图提供中介 143
Mediator 的生命周期 143
与组件通信 144
与应用程序通信 146
关注的通知 146
处理通知 146
与 Proxy 协作 146
避免获取其他 Mediator 147
Mediator 的多实例性 148
确定中介的粒度 148
构建 Mediator 149
应用程序中介器 149
选择器中介器 151
编辑器中介器 154

7. 应用业务逻辑 159
逻辑的两种类型 159
领域逻辑 159
业务逻辑 159
为什么要区分? 160
Command 的生命周期 160
构建 Command 161
启动命令 161

编辑故事命令 163
应用选择命令 165
移除选择命令 168
应用更改命令 170
放弃更改命令 171
添加条目命令 173
注册 Command 176
应用程序外观 / initializeController() 177

8. 模型进阶主题 179
处理同步行为 179
故事代理 179
应用选择命令 180
处理异步行为 181
分离调用者与响应者 182
同一个调用者／响应者 183
服务请求 184
搜索故事命令 185
故事读取代理 186
复用模型层 188
步骤 1：创建新的 Flex 库项目 189
步骤 2：将库项目加入应用项目的 Flex 构建路径 189
进阶练习：重构启动流程 190
应用程序外观 191
启动命令 192
准备控制器命令 193
准备模型命令 194
准备视图命令 195
重构后的项目 196

9. 视图进阶主题 199
管理弹出窗口 199
以往的做法 199
另一种做法 201
添加、更改和删除故事的用例 201
弹出窗口请求 205
弹出窗口操作事件 206
弹出窗口接口 207
确认弹出窗口 207
故事弹出窗口 209
抽象弹出窗口中介器 211
故事弹出窗口中介器 213
确认弹出窗口中介器 214

管理故事命令 215
延迟实例化 216
应用程序分区 TabNavigator 217
应用程序分区中介器 218
动态添加视图组件 219
游戏棋盘 220
游戏棋盘中介器 221
Flex Mobile 的中介策略 222
典型的 Flex 或 AIR 中介方式 222
Flex Mobile 的中介方式 223
Flex Mobile 示例 224

10. 继续前行 233
在 PureMVC 中使用调试器 233
PureMVC 实用工具 236
其他资源 237

前言

让编码跟上思考的速度

开发大型应用程序时，开发者往往不得不停下来思考：所需的某个类到哪里找，新类应该放在哪里，以及如何把它们连接起来，让数据从存储位置传到界面，供用户交互，或者沿相反方向传送。

无论应用程序的高层有多复杂，在底层，你真正做的事情无非是将数据从 A 点移到 B 点，偶尔再对其进行一些计算。你不应该反复发明完成这些事情的方法；相反，精力应集中在当前用例的需求上。

PureMVC 是一个简单的框架。它提供代码组织方案，并以几种久经考验的设计模式来表达众所周知的模型—视图—控制器概念，从而帮助你减少花在这些底层问题上的思考时间。

它不仅可以帮助独立开发者更快完成工作；如果团队对应用程序中各部分应该放在哪里、彼此如何通信也有一致认识，那么并行实现各个用例时，协作也会更加流畅。这完全可以在不产生开发紧张期常见的意大利面式代码的前提下做到。而且，PureMVC 是一个成熟、功能已经定型的框架，用它编写的遗留代码更易维护，因为其组织方式和实现原则仍然和代码最初写成时一样有效。

本书的目的

作为框架作者，我很幸运，自框架发布以来，几乎一直在持续使用它，将它应用于一个又一个富有挑战的项目。我也有幸与世界各地众多开发者交流，他们与我分享宝贵经验，或同我一起探讨所遇到的问题。

我当然希望把其中最重要的经验传达给读者，但会采用整体性的方式，并以一个并非小打小闹的应用程序为背景。

PureMVC 网站上有大量关于开发细节的讨论，也有文档、演示和实用工具。社区中还有许多介绍如何构建小型演示应用的博客文章。不过，直到现在，还没有资料带领开发者走完构建一个具有一定规模的 PureMVC 应用程序的过程，让大家体会各部分如何组合，以及应按什么顺序构建它们。从应用程序的初步构想到完成成品，以及其间的每一步，你都将逐渐理解哪些职责应该由框架角色承担；同样重要的是，哪些职责不应该由它们承担。



不可能设计出一个能暴露你可能遇到的所有问题的应用程序，但我们仍会涉及框架中所有实际运作的部分。本书着手构建的应用程序 StoryArchitect，其代码放在 com.futurescale.sa 包中。如果有必要用代码演示本书主要应用程序中没有涉及的内容，示例前的文字会先说明一个假设场景，而该示例代码会放在 com.mycompany.myapp 包中。

本书适合谁阅读

对 PureMVC 感兴趣或已经在使用它的 ActionScript 开发者，都能获得可付诸实践的见解。不过，由于示例应用程序使用 AIR 编写，Adobe Flex 和 AIR 开发者将受益最多。

你应当已经具备一些 Adobe ActionScript、Flex、AIR、面向对象编程（类、接口、继承）以及所选 IDE 的使用经验。要充分理解我们将构建的应用程序中的值对象和视图组件，你还应至少略知 ActionScript 的隐式访问器（getter/setter）、XML 处理、Flash 的事件冒泡以及 Flex 的数据绑定。

另外，虽然本书直接面向 ActionScript／Flex／AIR 读者，但正在使用或学习其他编程语言的 PureMVC 移植版本的开发者，同样可以借助本书来理解框架类及其用法。构建视图组件、访问文件系统或服务，以及处理 XML 的部分涉及 Flex 和 AIR 的特定内容。但 PureMVC 框架的角色、职责及协作方式在所有移植版本中都是通用的，阐明这些内容才是本书真正的重点。平台和语言只是具体背景。

致谢

在 O’Reilly，我要感谢 Dan Fauxsmith、Sarah Hake、Mike Hendrickson、Rachel James、Sarah Kim、Sarah Schneider、Karen Shaner、Mary Treseler 和 Melanie Yarbrough，

感谢他们在各自岗位上的出色工作，使本书得以问世。作为一名多年来受益于O’Reilly 图书指导的开发者，能由这一顶尖出版社出版自己的书，是我的荣幸！

PureMVC 贡献者

没有社区的支持，PureMVC 项目绝不可能取得今天的成果。这里无法列出所有人，但我想向以下以各种方式伸出援手的朋友，表达我个人由衷的感谢：

Ahmed Nuaman, Ali Mills, Andy Adamczak, Andy Bulka, Anthony Quinault, Brendan Lee, Brian Knorr, Bruce Phillips, Chandima Cumaranatunge, Chris Pyle, Dan Pedersen, Daniel Garay, Daniel Swid, Daniele Ugoletti, Dave Keen, David Deraedt, David Foley, David Knape, Denis Sheremetov, Denis Volokh, Dmitry Kochetov, Don Stinchfield, Dragos Dascalita, Duncan Hall, Eric La Rocca, Frederic Saunier, Frederic Sullet, Gary Paluk, Greg Jastrab, Hasan Otuome, Jake Dempsey, James Knight, Jari Kemppinen, Jason MacDonald, Javier Julio, Jens Krause, Jhonghee Park, Jim Bachalo, Jim Robson, Jody Hall, Joshua Gottdenker, Justin Wilaby, Luke Bayes, Marcel Overdijk, Marco Secchi, Mark Bathie, Mark Geller, Matt Brailsford, Matthieu Mauny, Michael Oddis, Michael Ramirez, Milos Zikic, Nate Rock, Nathan Levesque, Neil Manuell, Nick Collins, Nicola Bortignon, Omar Gonzalez, Ondina D. F., Paddy Keane, Patrick Lemiuex, Pedr Browne, Philip Sexton, Phuong Tang, Richard Germuska, Roman Pavlenko, Rostislav Siryk, Samuel Asher Rivello, Sasa Tarbuk, Sean Carnell, Simon Bailey, Stefan Richter, Steve Hueners, Thomas Schuessler, Tim Will, Toby de Havilland, Tony DeFusco, Yee Peng Chia, Zhang Ze Yuan, Zhong Xiao Chuan，以及 Zjnue Brzavi。

九型人格系统

本书中简要提及的九型人格系统相关内容，版权归 The Enneagram Institute 所有，© 2011，经许可使用。

本书约定

本书采用以下排版约定：

斜体
表示新术语、URL、电子邮件地址、文件名和文件扩展名。

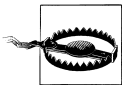
等宽字体
用于程序清单，也用于段落中提及的程序元素，例如变量名或函数名、数据库、数据类型、环境变量、语句和关键字。

等宽粗体
表示用户应按原样输入的命令或其他文本。

等宽斜体
表示应替换为用户提供的值，或根据上下文确定的值的文本。



此图标表示技巧、建议或一般说明。



此图标表示警告或注意事项。

使用代码示例

本书旨在帮助你完成工作。一般来说，你可以在自己的程序和文档中使用本书代码。除非要复制相当大的一部分代码，否则不必联系我们征求许可。例如，编写一个使用本书若干代码片段的程序，无须获得许可；销售或分发收录 O’Reilly 图书示例的 CD-ROM，则需要许可。回答问题时注明本书并引用示例代码，无须获得许可；将本书大量示例代码编入产品文档，则需要许可。

我们欢迎注明出处，但不作强制要求。出处通常包括书名、作者、出版者和 ISBN。例如：“《ActionScript Developer’s Guide to PureMVC》，Cliff Hall 著（O’Reilly）。版权 2012 Futurescale, Inc.，978-1-449-31456-9。”

如果你认为自己对代码示例的使用超出了合理使用或上述许可范围，欢迎通过 permissions@oreilly.com 与我们联系。

Safari® Books Online

Safari Books Online 是一个按需使用的数字图书馆，你可以轻松检索其中超过 7,500 部技术与创意类参考图书及视频，迅速找到所需答案。

订阅后，你可以在线阅读馆藏的任意页面、观看任意视频，也可以在手机和移动设备上读书。你能够在纸质版出版前阅读新书，独家访问编写中的书稿，并向作者提交反馈。你还可以复制粘贴示例代码、整理收藏、下载章节、为关键章节添加书签、创建笔记、打印页面，以及使用大量其他节省时间的功能。

O’Reilly Media 已将本书上传至 Safari Books Online 服务。要完整访问本书，以及 O’Reilly 和其他出版社关于相近主题的电子书，请到 http://my.safaribooksonline.com 免费注册。

联系我们

有关本书的意见和问题，请联系出版者：

O’Reilly Media, Inc.
1005 Gravenstein Highway North
Sebastopol, CA 95472
800-998-9938（美国或加拿大）
707-829-0515（国际或本地）
707-829-0104（传真）

我们为本书建立了网页，列出勘误、示例及其他补充信息。访问地址：

http://www.oreilly.com/catalog/9781449314569

如需发表评论或提出本书相关的技术问题，请发送电子邮件至：

bookquestions@oreilly.com

有关我们的图书、课程、会议和新闻的更多信息，请访问 http://www.oreilly.com。

在 Facebook 上找到我们：http://facebook.com/oreilly

在 Twitter 上关注我们：http://twitter.com/oreillymedia

在 YouTube 上观看我们的视频：http://www.youtube.com/oreillymedia

第 1 章
引言

作为软件开发者，你不仅要创建应用程序，还要与复杂性作斗争。几乎在所有情况下，你编写的程序还必须易于维护，能够轻松应对新增功能、修复缺陷和功能增强的要求。在如今快节奏的商业环境中，尽早、频繁地发布软件已成为普遍要求，因此在开发开始前，很少有时间把系统完整设计好。技术的快速变化又推动着支持或迁移至新硬件平台的需求：应用程序的桌面版、Web 版、平板版和手机版，其设备形态、用例和用户界面各不相同，却正迅速成为标准要求。

面对这些压力，开发者和团队怎样才能持续按时交付健壮、易维护的代码？设计模式长期以来被视为解决应用程序中特定问题的方法。但是，整体应用架构即使由已知模式构成，若规划不当，仍会恶化并变得难以维护。模型—视图—控制器（MVC）这一概念（见图 1-1），尤其是 PureMVC，正是为解决这个问题而生。

经典 MVC 架构

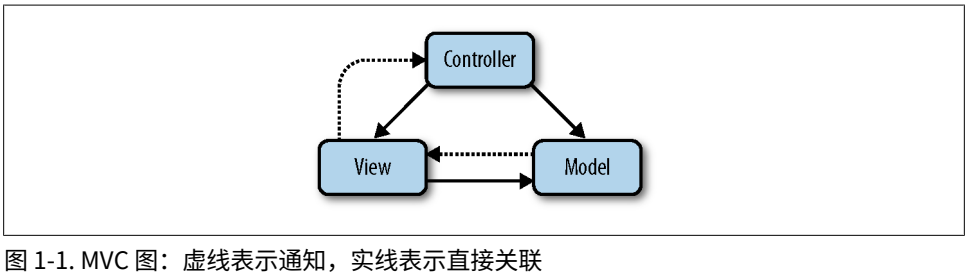


图 1-1. MVC 图：虚线表示通知，实线表示直接关联

本书不是讲历史的地方；只需知道，MVC 在 20 世纪 70 年代首次被提出时，世界要简单得多。计算机能做的事情，以及运行在其上的软件能做的事情，都相对有限。然而，与复杂性的战争当时就已经打响。MVC 成为武器库中的主要武器，因为它瞄准了一个简单却致命的问题：几乎所有提供用户界面、允许用户操作数据的程序，其核心都有这个问题。

要体会 MVC 方案的优雅，必须充分理解问题的本质。这需要进行概括：从应用程序各个运作部件的繁杂细节中退后一步，以更简单的形式观察它。自 MVC 首次被描述以来的二十多年间，尽管发生了种种变化，它所解决的基本问题仍然一如既往地普遍。

不妨想象一下：你的应用程序就像衣橱里积攒了好几周的一大堆脏衣服，必须清洗，否则你就得光着身子去上班。你可以随手抓几件塞进洗衣机，加些洗涤剂，了事。当然，你可能会发现毛巾把柔软的床单磨粗了。为洗软那些已经硬得能自己立起来的袜子而使用的热水，又让你心爱的 T 恤缩水、彩色衣服掉色，把白衣服染花。即使是我们这个行业里最随性的人，大概也会承认，按某种方法把衣物分类值得一做。因此，你会遵循洗衣房中一些不言而喻的最佳实践，把衣物分成几堆。

使用 MVC 时，我们希望分出的这几“堆”是：

- 处理数据的代码（Model，模型）
- 处理用户界面的代码（View，视图）
- 处理业务逻辑的代码（Controller，控制器）

MVC 概念的力量在于，这种高度概括适用于任何具有用户界面和领域模型的应用程序。按这一简单原则分离代码，就能减轻项目面临的一大威胁，斩杀复杂性这头怪兽，并确保在交付那天，你不必光着身子去上班！

先放下洗衣服的比喻，我们把这些堆称为应用程序的“层”：它们是运行时数据和控制流经过的不同层次。MVC 不仅提供这些层，用来分离关键关注点，还给出几条简单规则，规定每一层中的角色（类）应如何与其他层的角色通信：

- 模型层可以通知视图层或控制器层。
- 视图层可以更新模型层，并通知控制器层。
- 控制器层可以更新视图层或模型层。

三个简单的层，三条简单的规则。易于遵循，也易于记忆。



严格的“三层设计”与 MVC 的区别在于：它要求视图层与模型层的所有交互都必须经过控制器层。而在 MVC 中，视图可以更新模型，模型也可以通知视图，完全绕过控制器层。如果愿意，你可以对自己的代码施加严格三层设计的限制，但这并非必需。后文仍会用“层”来指应用程序中的模型、视图和控制器区域，以区别于具体的类；类名始终写作 Model、View 和 Controller。

PureMVC AS3 参考实现

PureMVC 的一个核心目标是独立于语言。因此，尽管大多数语言都有使自身与众不同的强大原生特性，PureMVC 选择的却是几乎所有语言都具备的通用约定。PureMVC 最初的实现，主要受基于 Web 的客户端／服务器环境中客户端需求的影响，具体而言，就是基于 Adobe Flash 平台的富互联网应用程序（RIA）。虽然作者专长于 Flex 开发，但如果框架不依赖 Flex API 特有的类，庞大的 Flash 开发者社区同样可以受益。当时，一个名为 Tamarin 的 ActionScript 引擎还准备为浏览器中的下一代 JavaScript 提供动力。因此，如果框架也不依赖 Flash API 特有的类，就可能让更多人受益。

考虑到这些因素，在选择 MVC 方案之后作出的第一个决定，就是框架只依赖 ActionScript 语言本身的元素，不引用 flash.events.Event，也不依赖 Flex 数据绑定。它不会妨碍你按最佳实践使用这些特性，但内部不会依赖它们。因此，PureMVC 为框架角色之间的通信内置了一套自己的机制，称为 Notification（通知）。ActionScript 与如今常用的其他面向对象语言很相似，因此，只要坚持使用最简单的语言结构，不依赖 AS3 命名空间以及将 XML 作为原生数据类型等强大却不常见的特性，就能大幅增强框架的潜在可移植性。

许多 ActionScript 开发者起初都会对此感到惊讶。甚至有人认为，一个刻意避开 Flex、Flash 和 ActionScript 强大特性的框架，不可能是最优的。当然，这种说法很容易提出，却难以量化或论证。如同艺术一样，你可能喜欢这种实现，也可能不喜欢。但它的有效性已在蛋白质建模、DNA 测序、高可用视频、虚拟化管理程序以及无人机通信系统测试等不同领域得到反复验证。它很可能也足以胜任你正在做的那些精彩项目。

事实上，使用 PureMVC 并不意味着你必须放弃 Flash 平台丰富的特性；那是一种常见误解。框架本身自成一体，

并不意味着你不能通过事件与之通信，或不能使用绑定在 MXML 视图组件内部传递数据。为了某种纯粹主义教条而放弃命名空间和 XML 数据类型这样的语言特性，也实在不明智。关键是要理解，它们与框架结合时，怎样使用、用在哪里最合适。

许多开源软件（OSS）项目起初只有一些想法、少量代码，以及一个没有预设终点的未来，开发者会不断往里添加新功能。重构并废弃旧功能、以新功能取代，会改进项目，却让依赖它的开发者像一直在跑步机上奔跑一样，不得不持续追赶，反复重构自己的代码，才能利用最新、最强的功能。

与之不同，也部分是为了回应这种状况，PureMVC 的 AS3 实现以尽快达到功能定型、缺陷修复完毕、充分成熟的状态为目标。MVC 相当直观，并不难理解。框架的范围在编码开始前就已确定，而在社区使用过程中，其不足也很快被定位并解决。为了避免功能不断膨胀，项目决定通过附加实用工具来扩展，而不改动核心框架。

如今，AS3 实现被称为参考实现，因为它不仅是 Flash 平台上功能完整的框架，还为向其他语言移植提供了逐类、逐方法的模板。本书撰写时，已经有另外十二个主要移植版本：AS2、C#、C++、ColdFusion、Haxe、Java、JavaScript、Objective-C、Perl、PHP、Python 和 Ruby，因此你可以轻松地把 PureMVC 的核心知识应用到这些语言中。履行各角色职责的具体策略可能略有不同，但它们的角色和协作模式保持不变。

PureMVC 在应用程序中的作用

MVC 严格说来并不是一种设计模式，因为它没有给出具有明确角色、职责和协作方式的具体参与者。它更像一种元模式：为应用程序内部关注点的总体分离提供良好建议。遵循它，可以让模型层的类在多个应用程序中复用，让视图层的类在不严重影响应用程序其他部分的情况下复用或重构，也让业务逻辑能够以解耦的方式从应用程序内多个地方触发。

我们完全可以仅凭这些建议编写应用程序。许多开发者确实如此，不使用任何框架也取得了很好的成果。与仅遵照 MVC 建议编码相比，使用 PureMVC 这样的流行框架理应带来额外收益，实际上也确实如此。其中一些好处是：

- 减少高层架构设计、实现及文档编写的工作量。
- 新加入团队的开发者无须先学习一套团队自创的方法。

· 各角色具有明确的职责定位和协作模式，让代码更易理解。

PureMVC 不仅是一个代码库，也是一套核心组织原则。再强调一次，MVC 建议将代码分为三个特定的层。其中一个体现方式就是应用程序的包组织，也就是你为存放类文件而选择的目录安排。组织面向对象的类是一门需要时间和实践来掌握的艺术。技能各异的团队成员往往对其组织方式有不同看法。

你的类不仅要有明确的协作模式，也应该让初次接触项目的人容易找到。创建新类时，放在哪里应当一目了然。再借用一下洗衣服的比喻：如果每天晚上脱衣服时就有不同的筐可用，洗衣日便无须浪费时间重新分类。从一开始，把每件衣物放进正确的筐即可。

你的包结构可能与基本建议略有不同，但只要遵循它，就能获得上述前两项收益：你不必自己重新设计一套组织方式；其他处理你代码的人，如果有 PureMVC 经验，也会知道该到哪里找东西、把东西放在哪里。

不必理会幕后的那个人

如果你确实想了解幕后运作的所有细节，PureMVC 网站的文档深入介绍了每个角色。框架在设计时就考虑到，其任何部分都可以在实际使用中被完整替换，以便你在不继承扩展的情况下重新实现某些角色。这就是每个类都有对应接口的原因。此外，凡是可以安全重写的成员，都采用 public 或 protected，而非 private，使你无须修改框架本身，就能灵活定制。

几乎所有时候，你都会按标准、规定的方式使用那几个相同的角色，因此我们先简要介绍这些类。你可能注意到，这个讲解顺序与在线文档基本相反。在线文档旨在全面说明框架的技术细节，适合希望深入评估、移植或修改 PureMVC 的架构师；但它同时也不得不服务于只想使用框架完成工作的开发者。结果是，后者这个人数更多的群体，起初往往觉得框架过于复杂，而实际上它用起来极其简单。框架设计投入了大量精力，把正确的 MVC 分离变成最容易走通的道路，并尽可能向开发者隐藏复杂性。

因此，本书会尽量以动手实践的方式介绍 PureMVC，跳过或尽量减少抽象层面的讨论。这绝不意味着，与那些熟知每个类和方法的人相比，你就更难练成 PureMVC 高手。恰恰相反，只要专注于真正重要的事情，就可以放心地把内部实现当成“黑箱”。

认识干活的主力

PureMVC 框架由知名设计模式中的角色组成，因此开发者很可能已经理解，或至少熟悉它们。进一步说，框架的编写方式使你在其组成的 11 个类（每个类都有对应接口）中，只需直接接触大约五个。不过，你会使用的这寥寥几个类都是干活的主力，实践中足以轻松覆盖应用程序中除视图组件、值对象和常量类之外的所有部分。

在通常使用的这五个类中，有一个在应用程序中只需编写一次，用于初始化和管理系统；三个分别代表在 MVC 三层中工作的角色；还有一个用来在各层之间传递信息，而且由于需要它的类共享一个便捷方法，你甚至不必亲自实例化它。因此，在日常开发中，真正重要的只有三个类。你会不断继承这三个类，所以必须熟悉它们的角色、职责和协作方式。

它们就是 Proxy、Mediator 和 SimpleCommand。

位于边界上的角色

应用程序所谓的“边界”，就是它与外界交接的地方：面向用户的视图组件，以及文件系统、Web 服务等外部数据源。在这些边界管理输入输出，分别是 Mediator 和 Proxy 的职责。

使用 Proxy 作为应用程序的数据源

围绕应用程序所解决的现实问题领域的一组特有相关术语，通常称为领域模型。应用程序的领域模型很可能由简单的数据载体类来实现，通常称为值对象，用强类型表示数据。框架没有用于表示数据的专门类，因为表示方式实在太多；PureMVC 的目的不是表示数据，而是获取、持久化数据，并将其呈现给用户查看和操作。无论数据采用何种形式，获取和持久化数据都是 Proxy 的首要角色。

Proxy 子类是长生命周期的角色，在应用程序中充当数据源。无论它们负责访问文件系统、远程服务器，还是摄像头、麦克风等硬件，Proxy 通常都会在启动时创建，并在应用程序整个运行期间可用。不过，我们也会探讨其短暂使用的模式。

简单的代理示例

Adobe AIR 有一个很实用的特性，叫加密本地存储（Encrypted Local Store, ELS）。它允许你把键／值对存入本地磁盘上的加密数据库。本例将展示如何用 Proxy 子类在 ELS 中读写值对象。为了演示，我们将保存用户邮件服务器地址和凭据等敏感信息。

我们的方法是，让 EmailConfigProxy 支持获取和设置一个名为 EmailConfigVO 的值对象，其中包含必要的属性。在 EmailConfigProxy 上设置值对象时，我们取出各属性值，以键／值对形式存入 ELS；键是 EmailConfigProxy 中定义的私有常量，值则来自 EmailConfigVO 的相应属性。从 EmailConfigProxy 获取值对象时，getter 只需根据 ELS 当前保存的值，创建并返回一个新的 EmailConfigVO 实例。EmailConfigProxy 中用于获取和设置 EmailConfigVO 的 getter 和 setter，会调用各个 ELS 键／值对对应的私有 getter 和 setter。



本例还展示了本书经常使用的一种 ActionScript 特性：隐式访问器。与大多数语言一样，你当然可以用 getUser() 和 setUser() 方法操作名为 user 的私有变量。但是，使用 ActionScript 的隐式访问器，可以让两个方法组合起来，看上去像一个属性。虽然这是一项比较特别的 ActionScript 特性，但将代码移植到不支持隐式访问器的平台时，只需把属性访问直接改为方法调用，因此在实践中，使用它不会损害可移植性。

EmailConfigProxy

```
package com.mycompany.myapp.model.proxy
{
    import flash.data.EncryptedLocalStore;
    import flash.utils.ByteArray;

    import com.mycompany.myapp.model.vo.EmailConfigVO;

    import org.puremvc.as3.multicore.patterns.proxy.Proxy;

    /**
     * 这是一个示例 Proxy，用于将 MyApp 的
     * 电子邮件配置项持久化到 AIR 的
     * 加密本地存储（ELS）中。
     */
    public class EmailConfigProxy extends Proxy
    {
        public  static const NAME:String        = "EmailConfigProxy";
        private static const EMAIL_HOST:String  = NAME+"/email/config/host";
        private static const EMAIL_PORT:String  = NAME+"/email/config/port";
        private static const EMAIL_USER:String  = NAME+"/email/config/user";
```

```
private static const EMAIL_PASS:String  = NAME+"/email/config/pass";

// 构造函数。把本子类的名称
// 传给 Proxy 构造函数。
public function EmailConfigProxy( )
{
    super ( NAME );
}

// 获取电子邮件配置值对象
public function get emailConfigVO():EmailConfigVO
{
   return new EmailConfigVO ( host, port, user, password );
}

// 设置电子邮件配置值对象
public function set emailConfig( config:EmailConfigVO ):void
{
    host     = config.host;
    port     = config.port;
    user     = String(config.user);
    password = String(config.pass);
}

private function get host( ):String
{
    return retrieve( EMAIL_HOST );
}

private function set host( host:String ):void
{
    store( EMAIL_HOST, host );
}

private function get port( ):Number
{
    return Number( retrieve( EMAIL_PORT ) );
}

private function set port( port:Number ):void
{
    store( EMAIL_PORT, port.toString() );
}

private function get user( ):String
{
    return retrieve( EMAIL_USER );
}

private function set user( user:String ):void
{
    store( EMAIL_USER, user );
}

private function get password( ):String
```

```
        {
            return retrieve( EMAIL_PASS );
        }

        private function set password( pass:String ):void
        {
            store( EMAIL_PASS, pass );
        }

        // 将一个键／值对存入 ELS
        private function store( key:String, value:String ):void
        {
            if (key && value) {
                var bytes:ByteArray = new ByteArray();
                bytes.writeUTFBytes(value);
                EncryptedLocalStore.setItem(key,bytes);
            }
        }

        // 从 ELS 读取一个键／值对
        private function retrieve( key:String ):String
        {
            var bytes:ByteArray = EncryptedLocalStore.getItem( key );
            return (bytes)?bytes.readUTFBytes(bytes.length):null;
        }
    }
}
```

使用 Mediator 作为视图组件的秘书

应用程序的用户界面会以视图组件的层次结构来实现。在 Flash 中，这些组件可能在设计时就放到舞台上；在 Flex 和 AIR 中，它们大多在 MXML 中声明，并在启动时创建。无论哪种情况，也都可以在后续动态实例化组件，并将其插入显示列表。有些组件由你定制，有些则直接使用现成组件。把视图组件接入应用程序的其他部分，就是 Mediator 的角色。

Mediator 子类是长生命周期的角色。其主要作用是，把应用程序对显示列表中某个组件的了解，集中到一个框架子类中。它向该组件提供所需数据，把视图组件的数据传回系统交由业务逻辑处理，或直接更新模型层。它不应成为所谓的“后台代码”（code-behind）类，替视图组件保存状态或管理内部细节。虽然 Mediator 可以通过组件明确公开的属性和方法，间接影响其状态，但视图组件应封装自己的内部实现。Mediator 只应以尽量少的转换，在视图组件和应用程序其他部分之间传递信息；就像主管的私人秘书负责与外界通信，让主管能够专注于自己的工作。

简单的中介器示例

接着前面的例子，某处必然需要一个视图组件，让用户编辑由 EmailConfigProxy 保存的电子邮件配置信息。我们把它称为 EmailConfig，并由 EmailConfigMediator 来照管它。

EmailConfig 视图组件有一个名为 vo 的属性，EmailConfigMediator 可以按需通过它访问 EmailConfigVO 的引用。我们也会演示另一种方式：由视图组件通过自定义 Event 的某个属性，把数据传给 EmailConfigMediator。视图组件使用该值对象的属性填充表单。用户输入或编辑数据后，可以测试配置；测试通过后，便可以保存配置。所有必要的配套行为都交由视图组件封装，例如测试前不启用“保存”按钮、信息未填写完整前不启用“测试”按钮等。构建 EmailConfigMediator 时，我们只需知道组成视图组件 API 的相关属性、方法和 Event。将视图组件视为“黑箱”，有助于企业外包 UI 开发，也便于发挥内部那些擅长 UI、但可能不熟悉 PureMVC 的开发者的能力。

EmailConfigMediator 会监听 EmailConfig 视图组件的某些 Event，它们表示用户想要保存或测试当前配置。它还会关注来自系统其他部分的 Notification（稍后详述），并通过与视图组件交互来处理这些通知。最后，EmailConfigMediator 也会直接与 EmailConfigProxy 协作，以获取或持久化 EmailConfigVO。

EmailConfigMediator

```
package com.mycompany.myapp.view.mediator
{
    import com.mycompany.myapp.controller.constant.MyAppConstants;
    import com.mycompany.myapp.model.proxy.EmailConfigProxy;
    import com.mycompany.myapp.model.proxy.EmailProxy;
    import com.mycompany.myapp.model.vo.EmailConfigVO;
    import com.mycompany.myapp.view.components.EmailConfig;
    import com.mycompany.myapp.view.event.MyAppEvent;

    import org.puremvc.as3.interfaces.INotification;
    import org.puremvc.as3.patterns.mediator.Mediator;

    public class EmailConfigMediator extends Mediator
    {
        public static const NAME:String = "EmailConfigMediator";

        private var emailConfigProxy:EmailConfigProxy;

        // 将此中介器的名称和组件传给 Mediator 构造函数
        public function EmailConfigMediator( viewComponent:EmailConfig )
```

```
{
    super( NAME, viewComponent );
}

// 获取转换为适当类型的视图组件
private function get emailConfig():EmailConfig
{
    return viewComponent as EmailConfig;
}

// 注册时调用。建立直接协作关系。
override public function onRegister():void
{
    // 在视图组件上设置监听器
    emailConfig.addEventListener( MyAppEvent.SAVE_EMAIL_CONFIG,
                                  saveEmailConfig );
    emailConfig.addEventListener( MyAppEvent.TEST_EMAIL_CONFIG,
                                  testEmailConfig );

    // 获取所需的代理
    emailConfigProxy = facade.retrieveProxy(EmailConfigProxy.NAME)
                                            as EmailConfigProxy;
}

// 如果组件通过事件携带配置，就可以从事件中取出它。
// 此处通过 EmailConfigProxy 立即将其保存到 ELS。
private function saveEmailConfig( event:MyAppEvent ):void
{
    emailConfigProxy.emailConfigVO = event.data as EmailConfigVO;
}

// 也可以从视图组件取得数据。
// 此处通过通知发送数据，交给 Command 处理。
private function testEmailConfig( event:MyAppEvent ):void
{
    sendNotification( MyAppConstants.PERFORM_EMAIL_TEST, emailConfig.vo );
}

// 注册时调用。这里列出
// 我们希望接收的通知。
override public function listNotificationInterests():Array
{
    return [ MyAppConstants.SHOW_CONFIG,
             EmailProxy.TEST_RESULT
           ];
}

// 响应此中介器所关注的通知
override public function handleNotification( note:INotification ):void
{
    switch ( note.getName() )
    {
        // 在视图组件上设置电子邮件配置
        case MyAppConstants.SHOW_CONFIG:
            emailConfig.vo = emailConfigProxy.emailConfigVO;
```

```
                    break;

                // 在视图组件上设置电子邮件测试结果
                case MyAppConstants.TEST_RESULT:
                    emailConfig.testResult = note.getBody as Boolean;
                    break;
            }
        }
    }
}
```

位于边界之间的角色

在未严格遵循面向对象设计原则的应用程序中，与服务器通信的类常常还会对接收或发送的数据进行大量计算，甚至创建显示这些数据的视图组件并与之交互。或者，一个用于显示数据的类，还会负责加载、计算和存储数据。将这些边界类的职责分离，已经是避免代码变成一盘乱糟糟的意大利面的重要一步。不过，边界角色仍需要一些帮助，才能保持职责轻量而专注。

远离外部边界上繁忙输入输出的地方，才是应用程序进行思考的所在。无论业务逻辑专属于某个应用，还是由一组应用共享，它都是 SimpleCommand 和 MacroCommand 的职责；本书将二者统称为 Command（命令）。

让 SimpleCommand 承担大部分思考工作

除了处理输入输出，应用程序在运行过程中很可能还需要执行不少纯逻辑操作。例如，启动时准备好应用程序，或对一段文本进行查找替换，这些工作在逻辑上都不属于视图层 Mediator 或模型层 Proxy 的职责范围。

SimpleCommand 子类是短生命周期的角色：按需创建，执行功能，然后退出，等待垃圾回收。与使用带静态方法的类相比，用 SimpleCommand 完成这类工作有明显好处：前者容易助长 MVC 所要避免的意大利面式代码。调用者会与那个类耦合，难以独立复用。由于这些类看起来很适合放各种零碎功能，它们往往逐渐变成所谓的“上帝对象”，体积越来越大，角色也越来越模糊。把逻辑放入独立的类，再以解耦方式触发，就能得到更易理解、重构和复用的代码。

前面的 EmailConfigMediator 示例中有一个 testEmailConfig() 方法，用户在 EmailConfig 视图组件中按下“测试”按钮时会调用它。该组件派发一个携带待测试 EmailConfigVO 的自定义 Event。EmailConfigMediator 再把 EmailConfigVO 放入名为 MyAppConstants.PERFORM_EMAIL_TEST 的 Notification 中发送出去。这里假定，一个名为 PerformEmailTestCommand 的 SimpleCommand 已经注册，用于处理该通知；稍后会说明如何注册。本书后面会看到 Command 中更复杂的逻辑；此处先给出这个 SimpleCommand 可能的实现：

PerformEmailTestCommand

```
package com.mycompany.myapp.controller.command
{
    import com.mycompany.myapp.model.proxy.EmailProxy;
    import com.mycompany.myapp.model.vo.EmailConfigVO;

    import org.puremvc.as3.interfaces.INotification;
    import org.puremvc.as3.patterns.command.SimpleCommand;

    public class PerformEmailTestCommand extends SimpleCommand
    {
        override public function execute( note:INotification ):void
        {
            // 从通知消息体中获取电子邮件配置
            var config:EmailConfigVO = EmailConfigVO( note.getBody() );

            // 获取 EmailProxy
            var emailProxy:EmailProxy;
            emailProxy = EmailProxy( facade.retrieveProxy(EmailProxy.NAME) );

            // 调用电子邮件配置测试。
            // EmailProxy 会将布尔测试结果放入
            // EmailProxy.TEST_RESULT 通知的消息体中发送，
            // 由 EmailConfigMediator 处理。
            emailProxy.testConfiguration( config );
        }
    }
}
```

使用 MacroCommand 执行多个 SimpleCommand

对于较复杂的操作，可以使用 MacroCommand。它只是将任意数量的 SimpleCommand 组合起来，称为其子命令，再按先进先出（FIFO）的顺序执行。实际项目中它并不常见，但本书后面会探讨一种合理用法。

其余成员

现在你已初步认识每天都会编写的主力类：Mediator、Proxy 和 SimpleCommand。学习这些类应该包含什么，同样重要的是不应该包含什么，以及如何建立它们之间的协作、最终让应用程序运转起来，将是本书的重点。

正如先前承诺的，我们会尽量减少对 PureMVC 内部机制的深奥讨论。不过，还有几个你会接触到，或至少应当知道的类。因此，这里先简要介绍，后续章节有需要时再深入说明。

通知

Notification 为这些主力类提供彼此通信的机制。Proxy 虽然可能监听服务组件的 Event，却会用 Notification 向其他 PureMVC 角色广播自身状态。同样，Mediator 可能在视图组件上设置事件监听器，但通常会把这些 Event 转换成向外发送的 Notification，以便向应用程序其他部分传达用户意图。

Notification 有一个名称、一个可选的消息体（可以是任意 Object），以及一个可选的类型（String）。Mediator、Proxy 和 SimpleCommand 都是 Notifier 的子类，因此都会继承名为 sendNotification() 的便捷方法。该方法以上述三个属性为参数，构造新的 Notification 并广播出去。

谁会接收这些通知？请再回头看图 1-1 的 MVC 图。

注意，视图层和控制器层可以接收通知，但模型层不接收。这样做是为了保持模型层的可移植性。确保用户能够与数据交互，是视图层和控制器层共同承担的职责；这两层都可以直接更新模型层。模型层只负责提供数据。如果模型层角色开始关注呈现方面的事情，就会越过自身职责，还可能与某个特定应用程序的实现耦合。

这意味着 Proxy、Mediator 和 SimpleCommand 都能发送 Notification，但只有 Mediator 和 SimpleCommand 可以接收。



Notification 通常简称为“note”。

核心角色

分别代表 MVC 三层的，是名为 Model、View 和 Controller 的三个单例，它们统称为核心角色。它们负责注册并提供你的 Proxy、Mediator、SimpleCommand 和 MacroCommand 子类。

Model 和 View 分别充当 Proxy 实例和 Mediator 实例的注册表；请记住，它们是长生命周期的角色。Notification 发出时，View 还负责通知所有关注它的角色。

Controller 负责维护 SimpleCommand 或 MacroCommand 类名与 Notification 名称之间的映射。当任意角色广播某个已建立映射的 Notification 时，它还会创建并执行相应的类实例。

虽然这些类是 PureMVC 的引擎，但你无须直接操作它们，甚至无须关心它们具体做什么。为什么呢?

外观

外观这一设计模式非常适合为任意复杂的系统提供接口，使系统的使用者始终无须直接与内部角色交互。

在 PureMVC 中，Facade 给开发者的感觉，就像我们把 Model、View 和 Controller 三个类简单地合并成了一个单例。三个核心角色中所有必需的方法，都可以通过它访问。由于这些核心角色实际上仍是独立的类，如有需要，你可以分别替换或修改其中任意一个，尽管几乎不会有这种需要。

可以把 Facade 想成酒店前台。客人到来后，不会直接坐电梯上楼自己找房间，而是由前台办理入住、交给他钥匙，并告诉他房间在哪里。一小时后他回来时，前台还会告诉他制冰机在哪儿，给他一个冰桶；如果他想找地方吃饭，就替他找礼宾人员；如果房间里的灯坏了，就替他叫维修人员。

单例：是好是坏?

PureMVC 中有四个类采用单例，以确保每个类只有一个实例。然而，在面向对象世界中，使用单例通常被视为反模式。它的静态工厂方法意味着，你能在应用程序的任何地方访问该类唯一允许存在的实例。不当使用会让每个类都直接耦合到单例，而不是通过更合适的角色获得所需内容，从而使代码变成意大利面。由于有 Facade，就没有理由直接访问其他单例，因此你可以用自己的实现替换其中任意一个或全部。至于 Facade 自身，按照设计，所有需要它的角色都已持有其引用。只有主应用程序会调用 Facade 的 getInstance() 方法，以传入初始显示列表。因此，那种让单例名声不佳的不必要耦合，

在正常使用 PureMVC 框架时并不会出现。

为类组织包结构

现在你已经认识了将要编写的类，接下来看看它们在典型项目中如何组织成包。

首先，快速回顾最常见的角色：

普通类

视图组件
用户界面的构成部件。
事件
由视图组件派发。
值对象
领域模型中的数据实体。
枚举
选择字段有效取值的枚举。
常量
包含静态常量值的类。

框架类

- `Facade`
- `Mediator`
- `SimpleCommand` / `MacroCommand`

典型的包结构

如前所述，PureMVC 项目有一套推荐的包结构。图 1-2 展示这些常见类在该结构中的组织方式；最顶层的划分确保系统中的所有内容，在概念上都按照 MVC 分类。

这种结构虽然简单到可以很快手动搭建，却具有极强的扩展能力。例如，如果应用程序有很多视图组件，你可以任意深入地细分 view.components 包，将各子系统的用户界面元素分隔开来。如果领域模型很庞大，也可以对 model.vo 包采取同样的做法。把 MVC 分离作为核心组织原则，还有助于你放心地将代码拆成库，或将多个库组合起来，因为你知道这些包结构

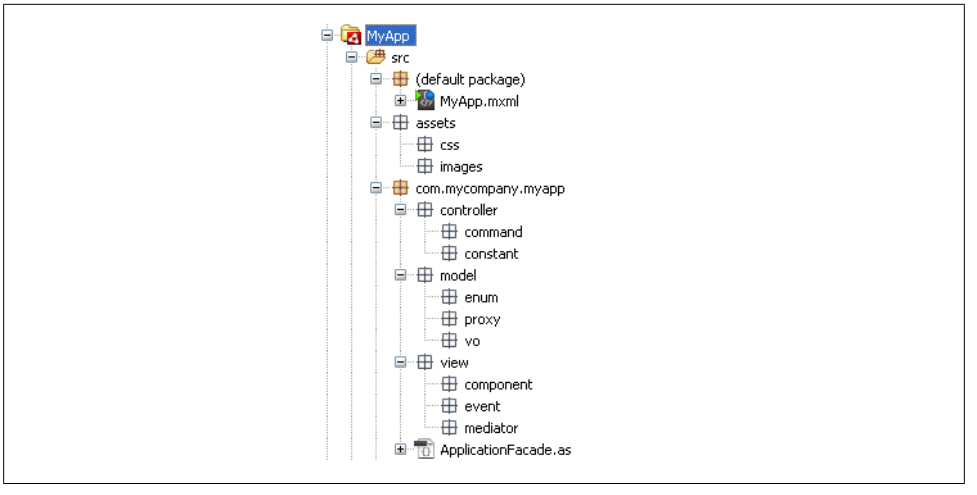


图 1-2. 基于 PureMVC 的应用程序的典型包结构

能够相互衔接。你可能希望把整个模型层提取成一个库，以便在不同应用程序之间复用。即使没有规划多个应用，也可能想把模型层构建为独立的库。如果不允许这个库引用应用程序的视图层和控制器层代码，就能确保 MVC 中最重要的分离得到维持：模型层的隔离。

当然，你可以按自己的喜好组织代码库；框架本身并不关心包结构。但是，MVC 的目的在于分离关注点，让应用程序复杂度增加时，仍能保持可维护性。包组织正是实现这一目的的有力工具。如果你使用 PureMVC，遵循一些通用包组织标准，将使现在和将来的团队成员都受益。想想杜威十进分类法：熟悉它的读者走进任何采用这种方法的图书馆，找到卡片目录，就能找到所需书籍。地方图书馆当然可以建立自己内部自洽的图书定位系统。但对于熟悉大多数图书馆组织方式的人而言，在这种自创体系的图书馆求知，就会平添不必要的认知障碍，直到学会当地的归档规则为止。还记得前言中“让编码跟上思考的速度”的目标吗？尽可能消除这些认知障碍，就能朝这个目标迈进一大步。

Standard 与 MultiCore 版本

PureMVC 发布后不久，就实现了最受期待、却未包含在最初范围内的一项重大变更：MultiCore 版本。

因为 Facade 和各核心角色都只能有一个实例，所以很难使用 Flash 平台的一项强大功能：Flex 模块，或加载本身也使用 PureMVC 的 Flash SWF 文件。

想象一种真正的模块化设计，其解耦程度足够高，能够让第三方基于 PureMVC 的模块，一起运行在基于 PureMVC 的宿主应用程序中。除非模块作者能访问宿主代码，否则就可能发生 Notification 名称冲突，导致行为不稳定。

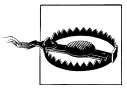
解决方法是，允许宿主和每个模块分别拥有一套独立的 Facade 与 MVC 核心单例。它们由此成为多例（Multiton）：不再只保存一个实例，而是保存一组具名实例的注册表。每个核心便能作为完全独立的程序运行。

核心之间如何通信？无论是宿主到模块、模块到模块，还是模块到宿主，都可以使用接口和直接引用，为各核心公开通信 API。另一种选择是使用随 MultiCore 版本一同发布的实用工具 Pipes。它的灵感来自 Unix 命令行界面：将程序用管道串联，让数据流经各程序进行处理。你可以组合“管道”和分流三通、合流三通、过滤器等“管件”，铺设核心之间的通信路径，然后在核心间发送带类型的消息，并按需附带数据或视图组件引用。

有些应用程序天生适合模块化系统设计。允许把虚拟乐器与音序、混音功能组合使用的音乐制作程序，就是极佳的例子。每种虚拟乐器可以作为一个模块，音序器作为一个模块，混音套件也作为一个模块。这类模块化架构的一大好处，是为第三方插件开发打开了大门。在设想中的音乐制作应用里，第三方可以开发音效和乐器，接入你的应用，使它更强大。你只需发布一个 API 库，其中包含所有模块需要的接口、值对象和消息类，开发者就能开始为你的应用增加价值。如果应用不是模块化的，第三方就无法在不访问整个应用源代码的情况下增强它，而你未必愿意公开这些源码。

MultiCore 是超出 MVC 的又一层分离。它虽然非常强大，却并非人人都需要。它会给开发者带来少量但不可避免的额外复杂度，因此最好将它实现为框架的另一个版本。于是 MultiCore 诞生了，原有版本则被命名为 Standard。

本书使用 Standard 版本。二者差异并不大，因此当你需要模块化方案时，不必担心。第 10 章会提供一些帮助你入门 MultiCore 的资源。



如果团队有条件投入时间进行单元测试，那么即使不编写模块化应用，也应选择 MultiCore，而不是 Standard。原因是，MultiCore 允许你为测试类中的每个测试方法，和／或测试套件中的每个测试类，建立独立核心。而在 Standard 中，一轮测试运行期间所有测试都在同一个核心中进行；除非精心安排初始化和清理，否则前面测试的结果可能影响当前测试。使用 MultiCore 并不要求使用 Flex 模块；你仍然可以按原有方式编写应用。唯一明显的区别是，ApplicationFacade.getInstance() 方法需要一个 String 参数作为键。

编写可移植代码

PureMVC 在实现前作出的几个简单决定，带来了它很高的可移植性。限定范围，只使用最常见的面向对象结构，确保了它易于用其他语言重新实现。由于框架支持许多流行编程语言，当有移植需求时，你的应用程序就有众多潜在目标可选。

如果你只为 Flash 平台编写应用，PureMVC 的可移植性对你可能显得不那么重要。幸运的是，如今 Adobe AIR 可用于许多硬件平台，使 Flash 平台开发者更有舒适而安全的感觉。就像摇滚乐一样，AIR 永不消亡——请在这里想象一段火热的空气吉他独奏（只是略带双关）。我恰好热爱它，也靠它谋生，自然希望情况如此。

但命运可能改变。你用来构建、改进应用程序的平台，也可能像恐龙一样消失，而且原因未必是技术落后。

想想苹果如何通过禁止 Flash 出现在其热门的 iPhone 和 iPad 产品上，打破了 Flash 原本无处不在的局面。有一段时间，它们甚至不允许 AIR 应用进入这些产品。要进入那个庞大市场，唯一选择就是学习 Objective-C，或用 HTML、CSS 和 JavaScript 编写应用。

就在本书书稿交付前不到一周，Adobe 宣布计划把 Flex 捐赠给 Apache 软件基金会，如今 Flex 的未来悬而未决。不再由 Adobe 直接支持后，企业客户会离开它，还是社区会让它继续生存并保持价值？只有时间能回答。但很可能将有大量用 Flex 编写的应用迁移到其他平台。这些应用的整体架构是否可移植，能够让这场不可避免的艰难迁移变得有章可循？还是必须付出巨大代价，从头重新设计？

这里要说明的是，一个主要软件平台的命运，并不完全由“适者生存”法则决定；行业中的硬件和软件巨头同样决定着它。如果代码要长期存活，就需要具备可移植性。

你选用的平台不仅可能消失，而且很多开发机构和独立软件供应商，本来就会面向多个平台开发，因为没有一种语言和工具集能够覆盖他们希望进入的所有市场。想象一下，如果你必须维护用不同语言编写、架构又完全不同的多个应用程序版本，会有多麻烦?

等等，你可能会说，Objective-C 与 ActionScript 完全不同！同一个应用程序在不同平台上的实现，当然会不同，对吧?

没错。问题是：哪些不同是必须的，哪些不同只是因为你选择了另一种做法？考虑代码可移植性时，就需要分清这一点。平台特有的差异肯定存在。但如果有意识地在各平台都选择 PureMVC，你和团队就能围绕同一套架构积累经验，在每个项目中更有效地交流，减少混乱。

PureMVC 方法的一个好处，就是你能移植真正重要的东西：架构。包组织，以及各角色及其职责、协作方式，都可以移植。

除了语言语法的差异，通常必须不同的是视图组件、服务，以及与它们通信的方式；换句话说，就是应用程序的边界。恰好，Mediator 和 Proxy 所隔离的，正是这些边界彼此之间、以及它们与业务逻辑之间的联系。

虽然在目标平台上确实需要重新编写整个应用程序，但无须彻底重新设计架构。

你编写的大多仍是具有相同方法和属性的相同类，只需处理边界处的差异，同时保留逻辑和通知模式。这样，为两个应用新增用例，主要就是在两个地方实现相同的角色。重构现有用例，也不会因架构差异而变得复杂。

这听起来或许有些奇怪，但思考代码可移植性的时机，应在编写之前及编写过程中，而不是直到你被迫移植、却还没有掌握新平台的时候。

第 2 章
使用 PureMVC 构建应用程序

欢迎来到 PureMVC 道场。

这里有一套方法。它不仅是代码库和包组织方案，还包括过去几年里，社区在论坛和工作场所中思考、积累的一套约定与经验。本书谈开发流程的频率，几乎与谈框架用法一样多。不过请记住，规则也可以被打破。就像框架本身一样，这种整体视角的任意部分，你都可以采纳或舍弃，但它们都与精通 PureMVC 的目标有关。这套方法既不是教条，也不是无所不包的终极操作指南；它只提出几个公理，后面的框架使用建议将以它们为前提。只要 PureMVC 开发者至少熟悉这些概念，在加入新项目时，新成员和团队就会因对基本事项有一致预期而轻松许多。这样就能把更多讨论与精力放到实际用例上，少花在普通的基础连接工作上。

注意，本书假定需求规格尚未写好，而你需要访谈客户、确定他需要你做什么、建立领域模型，并为他创建合适的应用程序。除了采用迭代方式，本书不预设任何特定开发方法。采用这里介绍的方法，很容易先构建一个功能有限的原型，再把它逐步演化为功能完整的应用程序。

选择平台

本书将使用 Adobe AIR，构建一个基于 PureMVC 的桌面应用程序。不过，任何 Flash 平台开发者都应该能从这里的思路和示例中获得实用见解。

Flex 与 AIR 的差别，只在于少数库以及编译后程序的运行位置；就我们的目的而言，它们是一样的。主要差异存在于 Flex／AIR 和 Flash／纯 ActionScript 之间，而且主要是视图创建方式不同。Flex 和 AIR 有 MXML，这是一种描述视图层次结构的声明式语言。Flash 可以在 IDE 中创建初始视图；而仅使用 ActionScript 时，

就必须通过代码创建所有视图组件。不论哪种情况，都可以动态创建组件并将其加入视图。Flex 和 AIR 这一侧还提供数据绑定和服务等特性。

这意味着，创建视图并将其连接到 PureMVC 的方式略有不同；在视图组件内部传递数据，以及存储、获取数据的方式也有所不同。除了语法，这些其实就是任意两个面向对象平台之间的主要差异。无论使用哪种语言或平台，创建 PureMVC 应用程序的方法都应大致相同。

选择 Adobe AIR 作为本书应用程序平台，有几个原因。MXML 采用声明式定义组件，比起 Flash IDE 截图，更容易在书中讲解。另外，AIR 已从最初的跨平台桌面环境，扩展到包括 iOS、Android 平板和手机在内的许多移动设备上，因此越来越受欢迎。

如今，构建 Flex 和 AIR 应用有不少很好的工具可选。本书应用使用 Adobe FlashBuilder 4.5 编写，但我们会尽量减少对 IDE 操作细节的讨论。关于你所选 IDE 的使用，已有很多优秀书籍。我们的重点是构建应用程序、利用 ActionScript 和 AIR 的特性，同时让架构在本质上保持可移植。

平台已经选定，现在需要一个应用程序。

那是一个漆黑的暴风雨之夜

我一直在考虑把小说写作当作业余爱好。最后未必写得好，但至少能派上我的打字技能。

研究一番之后，我发现，创作一部优秀小说与构建一个健壮软件应用，有许多相似之处。二者都需要规划和结构。在这两件事中，为参与者定义清晰的角色、职责和协作方式，都是成功的关键。而且，正如编程一样，作者有多少，写出这些页面的方法就有多少。

分析或规划面向对象系统中各角色的一种简便方法，是制作 CRC（类、职责、协作）卡片。拿索引卡和铅笔，在卡片顶部写下拟议中的类名或接口名；下面逐行写该角色的职责；背面列出其协作者，也就是它所知道、或将知道它的角色。这种方法很好，因为你可以快速调整职责归属、改变协作模式，弄清谁需要什么，以及如何获得。

同样的方法也能用于小说规划。每个人物都有一张卡片，描述其在故事中的目标、实现目标的阻碍、性格类型和外貌。故事所谓的“环境”，

就是故事发生的地方。为组成环境的各个场景设定制作描述卡片，可以在起草相关场景之前，先在想象中建立丰富的地点细节。成功的场景也需要由良好规划形成的结构：视点人物是谁、其场景目标是什么、对抗力量是什么（人物、自然力量等）、场景结束时会遭遇何种挫折，都能很快记在一张卡片上。

于是，我开始为一个构思制作卡片，结果它渐渐发展成一个规模很大、可能达到长篇小说篇幅的故事。我希望有一份相当详细的计划，以确定即使只是作为爱好，这也是我愿意投入的故事。我想知道事情如何推进：何时引入或解决支线情节，何时提高利害关系，如何交替安排行动与悬念来增强紧张感。我会在哪里放慢节奏、进行反思？怎样赢得读者对主人公的同情，同时让反派尽可能卑劣又足智多谋？采用这种方法，这些问题都能很快得到初步回答，并随着持续思考不断扩展。索引卡的好处在于，丢弃它和在上面写字一样轻松。因为投入感不重，你的思路能保持灵活。一旦开始打字，无论是代码还是文章，就容易觉得已经进入正式构建阶段，于是本该停留在简单规划层面时，却不知不觉偏离了方向。

我很快发现，即使引入先进的分组工具——回形针，索引卡的数量也会多得惊人。为了按期望的细致程度规划，同时保持灵活性，而且还能离开书桌随处进行，我开始在 BlackBerry PlayBook 平板的文字处理软件中创建文件。有时我会起草已经规划好的场景，以便更好地把握人物阵容和环境；也会记录人物性格与动机方面的笔记，这一点对我颇有挑战。我找到一个很有帮助的网站，介绍各种性格类型及其相关的“情绪中心”和“发展层次”，可以用来塑造更可信的人物。

不久之后，我就有了一堆文件夹，装满这些零散的故事材料。我可以复制文件，再稍作调整，创建某个场景的新草稿。所有写作都在场景层面进行，因此最好各自保存在独立文件中，因为我还不确定所有章节的分界，也可能调整其位置。有时我会用剪切粘贴，把几个场景的当前草稿串成一份临时文档，只为看看衔接是否流畅。但我不会编辑这些临时文档，而是把所有修改做在原始文档或它们的新草稿中。有时，先写后面的场景，反而能让我更清楚前面的情节需要发生什么。

我正在摸索一种将情节规划、资料研究、想象和起草交织起来的流程，希望它能像现代软件开发一样流畅地向前推进。

当然，任何程序员都能看出，这个问题适合用一个程序来解决。巧的是，本书也正需要这样一个程序。

StoryArchitect：小说写作工具

本书将重点构建的 AIR 应用程序，名为 StoryArchitect。

它会做什么？会使用 AIR 的哪些特性？如何存储数据？能在平板上运行吗？不作铺垫、不加解释，直接规定功能集和实现方案当然很容易。但我们在代码层面的选择，会受到对领域了解程度的影响；这里的领域就是小说写作。

上一节向我们提供了开发者在新软件项目中最应了解的两项重要信息：用户想做什么，以及目前是怎样做的。

除非要复制竞争对手的应用，或制作某类产品的变体，否则开发者通常都是从客户口述的需求开始工作。而那些明智地付费请专业人士开发软件的公司，其核心专长大多并不在软件设计领域。

这些出发点虽好却含糊的想法，常常先经过需求编写者的理解，被扩张、扭曲和拉伸，再交给设计师，添上大量花哨功能。最后，结果作为下一项挑战摆到开发者面前，而开发者又恰好喜欢挑战。于是转眼之间，那家只想在线卖饼干的小烘焙店，就多出了火箭飞船游戏，供顾客等待订单处理时玩耍。达到游戏等级可以获得下次购物折扣；当然，发条推文说自己刚买了美味饼干，还能拿到更多优惠券。

听起来都不错，但根据我的经验，这种过程产生的结果几乎从来不是“真正需要的东西”。需求文档里的功能清单，很少能充分说明现有流程的痛点。但如果能从用户视角理解其领域数据和当前流程，就更容易看清，你的技术究竟能在哪些地方真正改善他们的日常工作。

接下来，我们将沿着把潜在客户的想法转化为可运行应用程序的过程前进。你的需求、设计和开发流程大概会有所不同，甚至会随着任职单位或项目变化而不同；但我希望传达一种总体思路，让你有把握地用 PureMVC 开展新项目。

目的陈述

为了让整个团队，包括用户，对软件的核心目的形成一致认识，我们需要一句比电梯演讲还短的话：有意义、能概括全貌的一句话。

StoryArchitect 将让小说作者能够……做什么?

让我们回到探索阶段用户的那些思考。其中有一段对逐渐成形的流程的概括：

> 我正在摸索一种将情节规划、资料研究、想象和起草交织起来的流程……

好，这句话足够清楚，又不像功能清单。它应当指导我们关于功能和实现的所有决定。那么，我们的目的陈述就是：

StoryArchitect 是帮助小说作者将情节规划、资料研究、想象和起草交织开展的工具。

确定范围

作为这款可能很棒的应用的用户，我对它最终应该能做什么，有各种各样的想法。

但我真正需要的，是用它取代目前繁琐的文件管理流程，管理场景草稿、人物与场景设定研究等材料。我还需要一种方式，既能单独编写各场景的草稿，又能用每个场景的当前草稿，动态生成一章、一部或整本书。我应该能写一份场景草稿，然后退回到只读的章节视图，看看加入新场景后整章读起来如何。而且，短篇、中篇、长篇或系列小说的写作过程，本质上都一样，只是规模及场景分组层次逐渐增加，所以我希望用这一个工具就能完成它们。说到底，它是一个理解故事层次结构的简单文字处理器，再加上索引卡的等价功能，让我能够为故事层次中的任意一级、人物或场景设定记录笔记。

未来的宏大构想包括：根据故事各层的描述和笔记生成大纲，以及为每个人物或故事生成情节弧线，从时间和多个维度展示事件如何影响人物或故事线。例如，一场车祸可能严重损害人物的健康，却在后续诉讼之后改善她的财务状况。我相信，可视化工具能帮助我们想象和规划人物多方面命运起伏、或故事情节转折背后的因果关系。但这些留到续篇再说；此刻耳边响起的，正是功能膨胀怪兽诱人的低语。

控制范围的一项重要纪律是：只规划你正在开发的当前版本所必需的功能。务必把高级功能的想法记入清单。如果构建并使用当前版本之后，它们依然有价值，就可以作为下一个版本规划的起点。

现在，我们对准备构建的东西已有一个很粗略的认识。先考虑应用程序将管理哪些事物，再看怎样的用户界面能帮助我们创建和编辑它们。

描述领域

规划项目时，你可能需要对领域多做一些研究。在继续之前，先学习基础知识；本例就是小说写作。幸运的是，上一节的探索阶段已经为 StoryArchitect 的领域模型提供了有价值的线索。回想小说作品的各种组织形式，我们很容易提出一些候选实体。

无论篇幅多长，从六个场景组成的三只小猪与狼的寓言，到横跨多集的银河帝国与反抗军宏大史诗，一个故事（Story）都是由场景（Scene）组成的。

场景看似是故事的原子单位，但请记住，一个场景可以有多份草稿（Draft），其中只有一份被视为当前草稿。最终，故事由所有场景的当前草稿组成。

故事规模变大后，通常会把场景分组为章（Chapter）。

有时，结构更复杂的小说还会把章进一步分组为部（Part）。

非虚构作品和本书这样的参考书，常常把章再划分为节（Section）；但我们专注于小说的应用程序，可以放心省略这个层次。

系列（Series）包含多个故事，通常称作集，但本质上仍是故事；如果是电视剧，有时还会把它们分组为季（Season）。

故事和系列都有关联的人物阵容（Cast），由人物（Character）组成；还有环境（Milieu），由各种场景设定（Setting）组成。通常，同一系列中的所有故事共享人物阵容和环境。

第一轮迭代覆盖这些内容已经绰绰有余。随着开发推进，有些项目可能推迟到后续迭代，这是迭代开发过程的正常部分。最低限度，我们需要能够持久化所有这些条目，并提供可创建和浏览故事的用户界面。设计值对象时，再决定每种领域模型实体要记录哪些细节。

构思用户界面

从前，只需把一张新纸卷进打字机，开始打字。纸页渐渐堆起来，很快就有了一本书。很简单。当然，快速准确地统计词数，或随意在各场景间跳转，这里写一点、那里写一点，就没那么简单了。

至于应用程序的外观，我希望非常干净。它应当像老式 Underwood 打字机一样，有种简单、复古的“纸放上滚筒，开写！”的感觉。起草场景不需要高级文字装饰或格式排版；我们不需要 Microsoft Word。不过，编辑时仍需要控制字号和边距，

而最重要的是，能够轻松创建和扩展故事任意层级的结构，并快速浏览这个结构，以支持非线性的起草方式。我们需要某种时间线，可以逐层进入故事的任意位置，但不用时应能轻松隐藏。创建或打开故事后，应直接来到故事末尾，马上就能继续写作。另外，我们希望方便地访问当前选中条目（故事、部、章等）的名称、描述和笔记。和时间线导航一样，这些详情应当可以隐藏，又容易调出。如果两者还能同时显示就更好了，这样便能一边浏览，一边编辑当前条目的元数据，然后轻松移动到下一个条目继续编辑。我们还应能删除条目，或调整其与同级条目的顺序，例如轻松将第 3 章与第 2 章或第 4 章交换。实际的删除和排序功能留到下一轮迭代，但界面应先具备。

目前用平板写作的一大好处，是能够摆脱办公室的束缚；你正在读的这本书，很大一部分也是这样写的。所以，这款应用最终也必须能在移动设备上使用。但本书不是讲移动开发的，你也未必有平板可供试用。不过我相信，你一定有笔记本或台式电脑，因此第一轮迭代先面向桌面。基于 AIR 构建，可以确保未来存在一条便捷的移动设备迁移路径。一开始就知道这一点，有助于在设计时为将来铺路。例如，避免使用在平板上难以操作的隐藏上下文菜单和树控件，改用大而友好的按钮，以及按需显示的界面组件。

持久化策略

本应用选择 AIR 而非 Flex 的另一个原因，是可以把数据存到本地文件系统，不必为了持久化数据而额外设计服务器端组件。

不过，既然最终会有平板版本，那么希望在桌面和平板上访问同一份数据，也是合理的。这意味着将来某个时候，需要一个可选的服务器端组件，很可能是通过简单 API 暴露云存储功能的服务，如 Amazon S3、Dropbox，或两者兼有。构建时若考虑到这一点，后续迭代就能加入这些在线存储选项，而不必大规模重构应用。

至于数据如何存储，我们有多种方法可选。可以确定，整个应用程序都希望使用强类型对象；根据目前对领域模型的认识，它又具有很强的层次性：系列、季、故事、部、章、场景、草稿。有些层次可选，但总体仍是一棵树。

AIR 提供了名为 SQLite 的关系数据库，允许使用结构化查询语言（SQL）的一种变体，在表中建立索引、存储和检索数据。这种做法，

对于层次化数据可能很麻烦，需要大量表连接，并编写代码确保生成实际查询时，所有内容都得到正确转义。它与简单文件存储、以后再扩展到云端的思路不太契合。不过，由于 SQLite 的整个数据库位于单个文件中，也并非不能设想用它支持协作。最后，把 SQL 查询结果解析为值对象，或将对象转成查询的代码，也可能相当繁琐。

ActionScript 将 XML 作为原生数据类型，而层次化数据正是 XML 的强项。另外，语言内置了遍历 XML 数据、提取所需内容的功能。处理层次化数据，很难想象还有更好的选择。更妙的是，数据始终既可直接处理，又可作为一个简单文件保存。

可是，我们想要的是强类型对象。XML 数据只是由包含字符串的元素和属性组成；我们不希望在操作数据结构时，让应用程序各层都散布各种 XML 操作代码。有没有一种简单的 XML 用法，能够避免这些问题呢?

幸运的是，ActionScript 还提供了隐式访问器：一对 get 和 set 方法组合起来，在类实例上看起来就像一个属性。我们可以通过编写值对象，隐藏其内部数据采用 XML 形式的事实。访问器向应用程序公开带类型的属性，实际数据则存储在 XML 结构中。数据与 XML 之间的序列化和反序列化工作，都封装在值对象自身内部。

将 XML 作为原生数据类型，以及隐式访问器，是 ActionScript 的两项强大特性，在其他面向对象语言中并不常见；但将二者组合使用，仍然可以保持很好的可移植性。把应用移植到不以 XML 为原生类型的语言时，应用程序依然只处理对象。负责持久化的 Proxy 可能要多承担一点职责，或者可以用 SimpleCommand 序列化、反序列化数据。也可以照旧让值对象自己封装 XML 操作；总之，在大多数情况下，应用程序处理值对象的方式仍然不变。

因此，我们将通过存取 XML 文件保存结构信息。先实现本地文件系统存储，同时为后续迭代把这些文件存到线上做好准备。

搭建启动脚手架

和任何项目一样，先有一些只需做一次的准备工作。熟悉结构及标准启动流程（见图 2-1）之后，这个阶段实际上应该不超过 10 分钟。

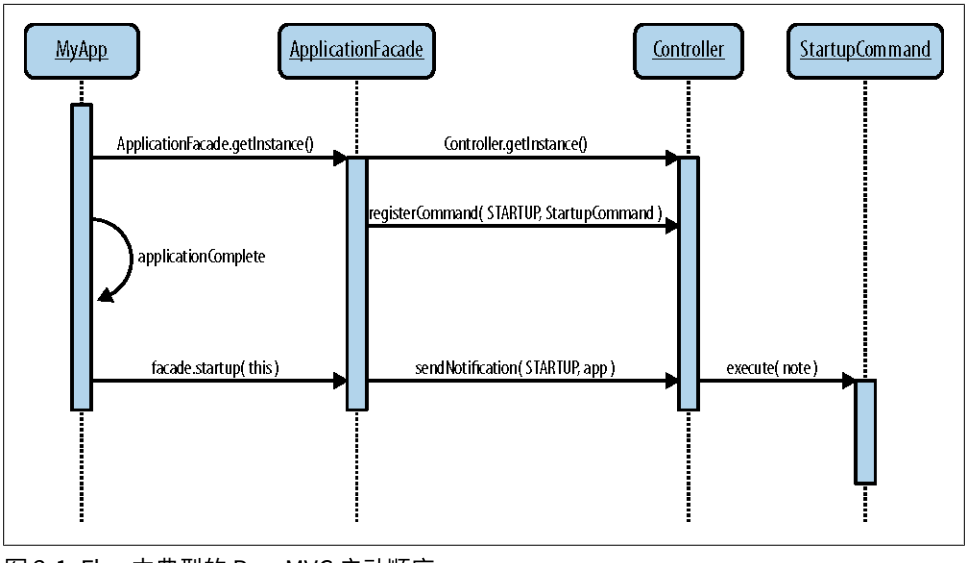


图 2-1. Flex 中典型的 PureMVC 启动顺序

准备项目与初始包结构

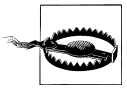
使用你喜欢的 IDE，创建一个新的 Flex Desktop（AIR）项目。默认情况下，Flex／AIR 项目应有 lib 或 libs 文件夹，可放入 SWC 格式的库，供编译到应用程序中。把 PureMVC_AS3_2_0_4.swc 的副本放入该文件夹；这个文件位于 PureMVC AS3 Standard 版本的可下载压缩包中。库放好后，你的类就能使用 PureMVC 框架类。建立基本包结构时，请回顾第 1 章的“典型的包结构”。

创建应用程序常量类

在 controller/constant 包中，创建名为 AppConstants 的 ActionScript 类，用静态常量保存应用程序视图层和控制器层将发送、接收的 Notification 名称。STARTUP 通常是首先定义并使用的一个。后续开发过程中，你会经常回到这个类，添加新的通知名称。



视图层和控制器层通常合称为 App，因为模型层应当能够轻松提取为一个库，并在另一个 App 中复用。



模型层绝不应该依赖 App，但 App 依赖模型层完全没有问题。App 的唯一目的，就是向用户展示领域模型的细节，并允许用户与其交互，因此视图层和控制器层熟悉模型层的角色，是可以理解的。所以，Proxy 发出的通知名称，要么定义在 Proxy 自身上，要么定义在独立的 ModelConstants 文件中。

类

AppConstants.as

代码

```
package com.futurescale.sa.controller.constant
{
  /**
   * App（视图层与控制器层）的通知常量
   */
  public class AppConstants
  {
        public static const STARTUP:String = "Startup"
  }
}
```

创建启动命令

创建一个 StartupCommand，为模型层和视图层的使用做好准备。它会从通知消息体中收到主应用程序的引用，主要工作就是注册 Proxy 和 Mediator。初始视图层次创建完成后，这段最先执行的业务逻辑将建立你定义的 PureMVC 结构，并把应用程序连接到它上面。目前，你还没有确定具体的 Proxy 和 Mediator，因此先创建启动命令，做到从通知消息体中取出应用程序引用这一步即可。



在 StartupCommand 中，一般应先注册 Proxy，再注册 Mediator，因为大多数 Mediator 都会与一个或多个 Proxy 建立长期协作。Mediator 会在注册时获取所需 Proxy 来建立关系。因此，等到与某个 Proxy 协作的 Mediator 注册时，该 Proxy 应当已经注册好了。

类

StartupCommand.as

代码

```
com.futurescale.sa.controller.command
{
  import org.puremvc.as3.interfaces.INotification;
```

```
import org.puremvc.as3.patterns.command.SimpleCommand;

/**
 * 启动 PureMVC 结构，为应用程序的使用做好准备。
 */
public class StartupCommand extends SimpleCommand
{
  override public function execute( note:INotification ):void
  {
    // 从通知中获取应用程序实例
    var app:StoryArchitect = StoryArchitect( note.getBody() );

    // 脚手架阶段到此为止。
    // 以下内容将在后续逐步添加……
    // 注册 Proxy

    // .
    // .
    // .

    // 注册 Mediator
    // .
    // .
    // .
  }
 }
}
```

创建应用程序外观

创建一个名为 ApplicationFacade 的 Facade 子类。它应完成以下工作：

- 实现 getInstance() 方法，使单例实例采用这个子类。
- 重写 initializeController() 方法，先调用父类方法，再注册 StartupCommand，让它响应 STARTUP 通知。
- 添加一个名为 startup() 的 public 便捷方法，供应用程序调用，并以应用实例为参数。该方法只需把应用实例放入 STARTUP 通知的消息体中发送出去。

类

ApplicationFacade.as

代码

```
package com.futurescale.sa
{
  import com.futurescale.sa.controller.command.StartupCommand;
  import com.futurescale.sa.controller.constant.AppConstants;

  import org.puremvc.as3.patterns.facade.Facade;
```

```
public class ApplicationFacade extends Facade
{
 /**
  * 单例实例的工厂方法。
  */
  public static function getInstance( ) : ApplicationFacade
  {
    if ( instance == null ) instance  = new ApplicationFacade( );
    return instance as ApplicationFacade;
  }

  /**
   * 初始化 Controller 并注册 Command。
   */
  override protected function initializeController():void
  {
    super.initializeController();
    registerCommand( AppConstants.STARTUP, StartupCommand );

    // 其余 Command 将在后续逐步注册……
    // .
    // .
    // .

  }

  /**
   * 用于从应用程序中启动
   * PureMVC 结构的便捷方法。
   */
  public function startup( app:MyApp ):void
  {
    sendNotification( AppConstants.STARTUP, app );
  }
 }
}
```

初始化应用程序外观并调用 startup()

在主 AIR Application（或 WindowedApplication、ViewNavigatorApplication）中，执行以下操作：

- 创建一个名为 facade、类型为 ApplicationFacade 的私有变量，通过单例实例工厂方法初始化它。
- 在 applicationComplete 的处理函数中，调用 facade.startup(this)。

类

StoryArchitect.mxml

代码

```
<?xml version="1.0" encoding="utf-8"?>
<s:WindowedApplication
  applicationComplete="facade.startup(this)"
  xmlns:fx="http://ns.adobe.com/mxml/2009"
  xmlns:s="library://ns.adobe.com/flex/spark"
  xmlns:mx="library://ns.adobe.com/flex/mx">

  <fx:Script>
        <![CDATA[
            import com.futurescale.sa.ApplicationFacade;

            // 在构建 MXML 应用程序时初始化 ApplicationFacade
            private var facade:ApplicationFacade = ApplicationFacade.getInstance();
        ]]>
  </fx:Script>

</s:WindowedApplication>
```

由外向内构建

此时已经可以发布应用程序，并看到一个无错误运行的空白界面。在 StartupCommand 中加入 trace 语句，或设置断点，就能确认一切正常。启动脚手架已经完成。那么，接下来做什么？

无论使用何种框架，甚至是否使用框架，每个应用程序都会有领域模型，以及将它呈现给用户的视图组件。通常合理的做法是，先定义值对象，再定义让用户与其交互的视图组件。这些类位于应用程序边界之外。可以想象它们存在于漆黑的虚空中：彼此独立，却像手与手套一样相互契合。应用程序编写出来后，会把它们连接起来。因此，在着手编写连接代码之前，先创建并相互调整值对象和视图组件，是合理的。

这种方法能让你很快做出原型，因为暂时无须担心如何加载数据、再把数据送到视图。值对象完成自身工作，不应需要其他类。视图组件可以轻松在 MXML 中实例化，还可以为自己提供填有模拟数据的值对象，以便先实现数据显示以及与 VO 的交互。这种方法的一个好处是，即使完全不了解框架的人，也能创建值对象和视图组件。它能确保这些类不依赖框架本身，保持可复用性。

值对象和视图组件创建完成后，就可以建立一套使用它们、满足需求的 PureMVC 结构。开始从边界向内构建。首先，需要 Proxy 作为应用程序的数据源；它们就位后，再回到视图，构建把数据交给视图组件的 Mediator。

你会发现，下面的步骤描述的是注意力从模型层转向视图层、又转回来的过程。这种交织式方法从一开始就很有效，也会贯穿迭代开发和遗留系统维护的各个阶段。MVC 各层就位后，新增和修改用例总可以归结为：把一些数据从模型层移到视图层，或反过来。有时还需要视图层和控制器层稍作交流，以响应用户输入、重新组织界面。

步骤 1：值对象

搭好项目的启动脚手架后，第一件事就是创建值对象。考虑应用领域，为识别出的每个实体创建值对象。如果已经知道某些属性的有效取值，还可以为它们定义枚举或常量。

按你喜欢的方式编写这些类，以及使用它们的正式或非正式测试。在构建值对象时，为它们建立一套测试，能很好地帮助你体会操作这些对象的代码将怎样工作；以后也有助于防止更改破坏应用程序。测试提供了实体用法的参考，应及时更新，反映应用中的当前用法。



惯例是在实体名称后加上 VO，使代码中该类的角色清晰可见，例如 InvoiceHeaderVO。这一约定还有助于避免与视图组件重名，因为视图组件常以其显示的领域模型实体命名。参见第 34 页“步骤 2：视图组件”中的示例。

步骤 2：视图组件

接下来，根据领域模型，设计并构建足以满足用例的视图组件。此时可以快速制作组件原型，用强类型值对象形式的模拟数据填充它们。你也会逐渐了解每个视图组件需要什么，以及这些内容应来自显示列表中的父组件，还是来自 Mediator。对需要由 Mediator 或父组件设置的内容，公开 public 属性；为了封装视图组件职责而使用的内部成员，则保留为 private 或 protected。

不要设计过于单薄的视图组件，指望 Mediator 或父组件像操纵木偶一样控制其内部。相反，应公开一组由方法、属性和事件组成的自定义 API，使组件具有清晰的通信协议，只需获得必要内容就能工作。封装组件行为，使它不依赖 Mediator 也能运行。最重要的是，不要让视图组件知道任何框架角色。

以最快的方式测试视图组件：把它们嵌入 MXML 应用，用模拟数据填充，并实际操作。在这个过程中，你可能会发现需要调整值对象。项目初期，可以在第 34 页“步骤 1：值对象”与第 34 页“步骤 2：视图组件”之间来回推进，反复调整值对象和视图组件，直到它们大致达到第一轮迭代的预期形态。

最后，为自己省些麻烦，严格限定视图组件的职责范围。把大型组件组合拆成更小的自定义组件，分别放在独立的类中。视图组件应封装履行自身职责所需的逻辑，因此应尽可能保持简单，让内部工作方式易于理解和修改。



视图组件的命名应描述其功能，而不必描述其实现。例如，一个显示发票抬头表单的组件，最初可能用 Halo Panel 实现，后来又改用 Spark Group。因此，按角色命名更好：使用 InvoiceHeader，而不是 InvoiceHeaderPanel。

步骤 3：代理

决定如何存储和访问值对象，会引导你思考需要哪些 Proxy，以及它们为管理数据应承担哪些职责。起初，可以让它们通过模拟方法生成、接收或发送数据。如果已有服务，或准备使用本地存储，也可以开始编写相关方法。但在第一遍实现时，让主要 Proxy 就位，并大致明确它们如何工作，就已经够了。

Proxy 子类用途很多。它们可以访问 FlashVars 和 URL 参数、本地共享对象（LSO）、加密本地存储（ELS）、文件系统、Web 服务、实时消息传输协议（RTMP，或称“服务器推送”）通道——几乎包括应用能够访问的任何数据源，甚至应用程序状态的临时数据，尽管这类信息并不属于严格意义上的领域模型。

不论数据来自哪里，Proxy 类都有一个 data 属性，你将用隐式访问器把它转换为适当类型。Proxy 子类通常公开一个值对象，或一组同类型值对象的集合。如有必要，它们当然也可以更复杂，但这是常见用法。一个 Proxy 还可以与另一个协作，维护分开管理却彼此相关的数据的完整性，从而降低各个 Proxy 的复杂度。

设想一个 InvoiceProxy，通过远程服务，在服务器数据库中存储、获取和更新 InvoiceVO 对象。这个 Proxy 公开一些方法，

例如 createInvoice、saveInvoice、deleteInvoice 和 findInvoiceByID，供 Command、Mediator 及其他 Proxy 按需调用。因为它们与服务交互，所以本质上都是异步操作。方法被调用后，InvoiceProxy 调用服务，并在稍后处理结果。当结果返回，例如 findInvoiceByID 的结果返回时，InvoiceProxy 可以把 data 属性设为结果中的 InvoiceVO，再发送 InvoiceProxy.INVOICE_LOADED 通知，将 InvoiceVO 放在通知消息体中。



通常按照 Proxy 所提供的数据实体来命名它，例如，用 InvoiceProxy 管理 InvoiceVO。

步骤 4：中介器

由于大多数视图组件已经编写了一定程度，你应该比较了解它们的需求。现在应决定，视图层次中的哪些位置需要中介。并非每个组件都需要 Mediator；有些可以从 Flash 显示列表中的父视图组件接收数据，提供数据也就成了父组件的职责。但数据以及新视图组件，总得在某些位置注入视图层次或从中移除；这些位置正是需要中介的地方。

第一个 Mediator 应服务于顶层应用程序，通常称为 ApplicationMediator；在 Flash 或纯 ActionScript 项目中，则使用 StageMediator 为 Stage 提供中介。用户界面比较简单时，可能只需要这一个 Mediator。不过，多数应用还会为一些应用子组件配置中介。

假设有一个 Invoice 视图组件，其子组件是 InvoiceHeader 和 InvoiceDetails。你可以只为 Invoice 配置中介，让它自己照管子组件，这样只需一个 InvoiceMediator。InvoiceMediator 可以从 InvoiceProxy 获取 InvoiceVO，再将它设置到 Invoice 视图组件上。该组件随后把 InvoiceVO 引用向下传给 InvoiceHeader 和 InvoiceDetails 子组件，可能借助 Flex 绑定，由它们显示 InvoiceVO 中相应的数据。用户与 InvoiceHeader 或 InvoiceDetails 交互时，例如请求保存更改，子组件可以派发一个向上冒泡、经过 Invoice 组件的事件。InvoiceMediator 监听这些事件，并通过把数据交给 Proxy 来响应；也可以把数据放入通知中发送，由 Command 校验并提交。

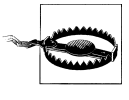
从这个例子可以看出，视图层次中的每个视图组件，并不都需要自己的 Mediator。因此，开始时应尽量少写 Mediator。但请记住，Mediator 不应为了设置或获取数据，深入视图组件实现的许多层级，

从而破坏其封装；父视图组件同样不应这样做。Invoice、InvoiceHeader 和 InvoiceDetails 组件之间的关系很简单。但不论是 Invoice 组件还是 InvoiceMediator，如果访问 invoiceDetails.detailsGrid.selectedItem.id，都会被视为不良的面向对象做法。考虑组件设计和中介粒度时，应以此为警戒。

不过，通信是双向的。Mediator 不仅要关注自己照管的视图组件及其产生的事件，还要关注系统其他部分发来的、可能与该组件相关的 Notification。接着第 35 页“步骤 3：代理”中尚未讲完的例子，假设 InvoiceProxy 发出 InvoiceProxy.INVOICE_LOADED 通知，消息体中包含 InvoiceVO。InvoiceMediator 注册时已声明关注该通知，所以会收到它，并从消息体中取出 InvoiceVO，设置到自己的视图组件上，由组件显示。



按所照管的视图组件为 Mediator 命名，例如，Invoice 视图组件对应 InvoiceMediator。



有时，Flex 会延迟实例化那些在 MXML 中声明、但用户导航到它们之前不显示的子组件，例如 ViewStack 的子组件。为这些组件配置中介时，需要特殊处理。弹出窗口和动态创建的视图组件也是如此。第 9 章会进一步讨论这些问题。

步骤 5：命令

当开始实现 Command 时，除了脚手架阶段创建的 StartupCommand 之外，初始角色大多已经就位。Command 可以承担多种角色，但通常是响应来自其他 Command、Proxy 或 Mediator 的通知，执行某种逻辑；其中经常调用 Proxy，或发送通知，交给另一个 Command 或若干关注该通知的 Mediator 处理。

继续第 36 页“步骤 4：中介器”的例子，假设用户在 InvoiceHeader 组件中修改了 InvoiceVO 的 terms 属性，并点击组件内的按钮保存更改。该组件派发一个冒泡的自定义事件，而 InvoiceMediator 正在 Invoice 组件上监听该事件。InvoiceMediator 的处理方式，是发送 AppConstants.VALIDATE_AND_SUBMIT_INVOICE 通知，并将事件携带的 InvoiceVO 放进通知消息体。此通知已经与 ValidateAndSubmitInvoiceCommand 注册关联；这个命令正如其名，负责校验并提交发票。



按所实现的流程为 Command 命名。例如，ValidateAndSubmitInvoiceCommand 接收视图层生成或修改的发票数据，校验内容，并通过与 Proxy 协作将其提交给服务。

第 3 章
领域建模

必须用某种方式表示应用程序管理的基础数据。在面向对象世界里，这通常表现为带有类型化属性的类。许多因素会影响客户端应用中领域模型实体的最终实现。例如，使用 LCDS、BlazeDS 或 WebORB 时，可以在服务器上创建类，其各属性逐一映射到客户端的对应类。Java 服务在结果中发送这种类的对象时，客户端就会得到已经填充好的 ActionScript 版本对象，无须自己把结果解析到对象上。或者，你可能从 REST 服务收到 JSON 或 XML，再在客户端收到结果时，通过代码把它转换成强类型 ActionScript 对象。使用 Flex 和 AIR 时，还可能希望这些对象具有可绑定属性，使把它们作为数据提供者的视图组件能随其变化而更新。要牢记：这些类不应知道架构框架类（PureMVC），也不应了解视图组件框架（Flex）。它们应该只负责表示数据。可以使用 Flash 或 Flex 定义的集合类型，例如 Vector 或 ArrayCollection，但绝不应使用 Button、DataGrid 或其他显示列表对象。

设计数据模式

前面我们已经考虑了 StoryArchitect 应用程序要管理的实体，也决定用 XML 文件存储数据：最初存到本机，将来可能存到 Amazon S3 这样的在线存储服务。最后，这些决定引导我们把 XML 处理封装在值对象内部。因此，合乎逻辑的起点，就是设计 XML 数据模式。

如果要把数据持久化到数据库，我们会先设计数据库模式，定义所有表和索引。准备把数据存为 XML 时，也没有区别。一边构建应用，一边随意、宽松地定义 XML 实体，是危险的。借助 Altova XMLSpy 这样的 XML Schema 创建和可视化工具（本节图示即用它制作），你可以在

可视化界面中完成设计。最终不仅能清楚理解 XML 结构如何工作，还能得到一套底层规则，用于验证那些声称遵循此模式的 XML 实体。虽然基于模式的验证不属于本书范围，但这显然是先创建数据模式的一项额外收益。



要让 PureMVC 正常工作，就一定要使用包装 XML 的值对象吗？完全不是。不过，这是一种适合与 PureMVC 配合的好模式：它能减轻 Proxy 必须了解 VO 所有细节的负担，也能避免你产生创建职责模糊的工具／辅助类的冲动，因为那些类会增加依赖和复杂度。既然许多数据本来就是 XML 形式，这种模式还能帮助你迅速处理众多现成服务器 API，以及自行从头设计的 API 所返回的结果。

回顾实体

下面按复杂度大致从低到高，列出 StoryArchitect 的领域模型实体：

Note（笔记）
关于任意实体的 URL 和／或自由格式文本。
Setting（场景设定）
故事某一部分发生的地点。
Milieu（环境）
与一个故事或系列关联的一组场景设定。
Character（人物）
故事或系列中的某个个体。
Cast（人物阵容）
与一个故事或系列关联的一组人物。
Draft（草稿）
某场景正文的一个版本。
Scene（场景）
故事的原子单位，可以有多份草稿。
Chapter（章）
一部或一个故事中的一组场景。
Part（部）
一个故事中的一组章。
Story（故事）
一组场景，可以组织为章和部。
Season（季）
系列中故事的一种可选分组。

Series（系列）
共享人物阵容与环境的一组故事。

创建数据模式

下面的图将以图形方式表示各种领域实体的 XML 结构。一般来说，这些元素可以具有属性，并包含其他元素或文本。较复杂的实体可能有多种形式，例如 Story 会根据类型包含不同集合：Part、Chapter 或 Scene。我们将从简单结构逐步讲到复杂结构。每幅图旁都会附上一段具有一定代表性的 XML，供对照参考。

ValueObject（值对象）

VO 必须有唯一 id，其余内容均可选，包括名称、描述、Note 集合、用于排序的序号，以及表示对象是否为存根的标志；存根是需要从存储中加载完整内容的占位对象。其他图中，这些属性和元素会被折叠显示。图中的虚线表示元素或属性可选。这种元素不会单独创建，但其他元素会继承其属性和子元素定义。因此，图 3-1 是唯一不附带 XML 片段的图。

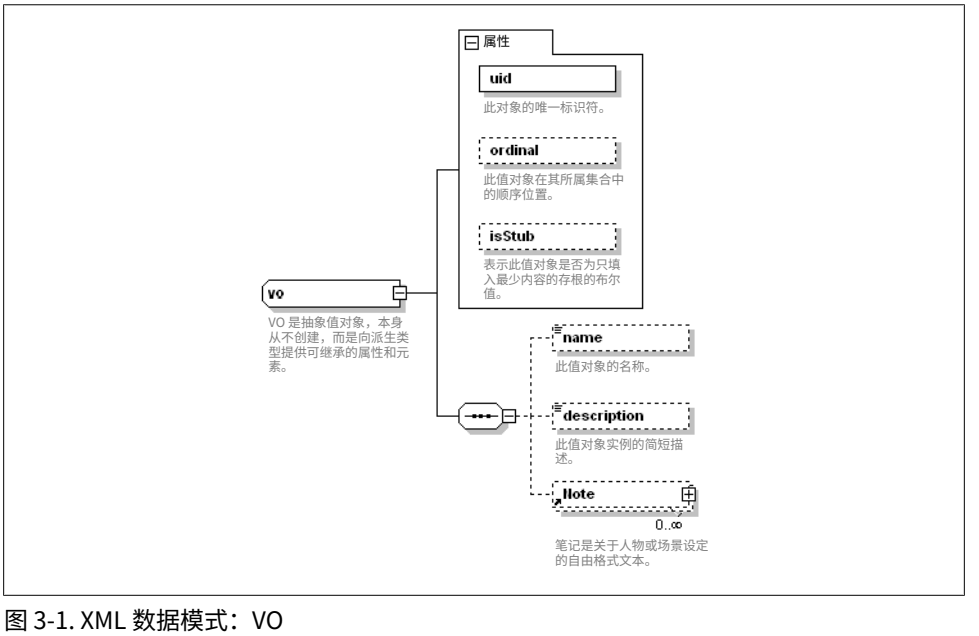


图 3-1. XML 数据模式：VO

Note（笔记）

与下面讨论的所有其他值对象不同，Note 不继承 VO，因为它包含在 VO 内部，不需要 VO 的任何属性。它只是提供一种方式，让我们能为其他任意实体附加自由格式文本和／或 URL，并不比索引卡更复杂。见图 3-2。

```
<Note url="http://bit.ly/Ogz7Q">Interesting article about filaments of Dark Matter
pulling in gas and funneling it across the galaxy.</Note>
```

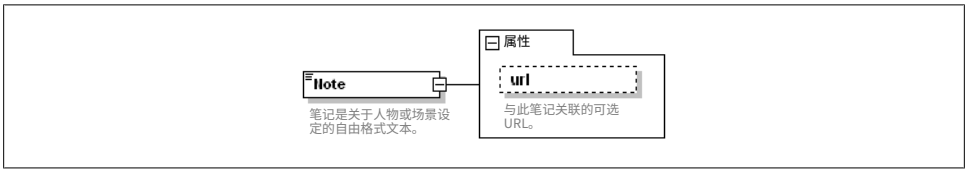


图 3-2. XML 数据模式：Note

Setting（场景设定）

Setting 除了元素名称之外，所需的全部属性都继承自 VO。我们可以为它赋予名称、描述，并添加任意数量的 Note。见图 3-3。

```
<Setting uid="Setting-434514-1">
  <name>Captain Froop's Cabin</name>
  <description>Utilitarian room with a viewscreen filling one wall.</description>
  <Note url="http://bit.ly/qYeGZi">Here is an illustration with the general look.</
Note>
</Setting>
```

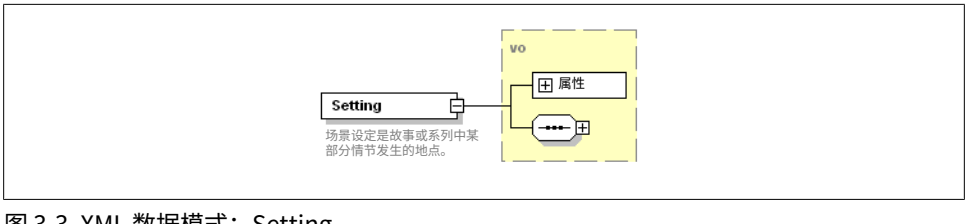


图 3-3. XML 数据模式：Setting

Milieu（环境）

Milieu 是 Setting 集合的简单包装。我们可以给它唯一 ID，将其保存为独立文件，并让同一 Series 中的各个 Story 共享。见图 3-4。

```
<Milieu uid="Milieu-3454553-34">
  <Setting uid="Setting-434514-1">
    <name>Captain Froop's Cabin</name>
    <description>Utilitarian room with a viewscreen filling one wall.</description>
  </Setting>
</Milieu>
```

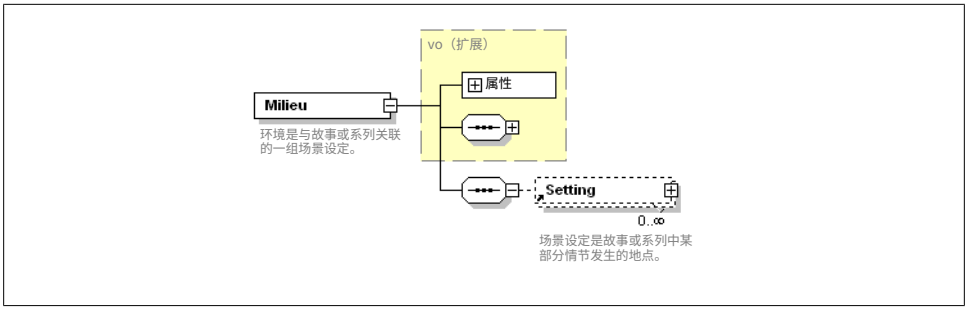


图 3-4. XML 数据模式：Milieu

Character（人物）

Character 在 VO 基础上增加几个属性，按照流行的九型人格系统描述性格。该系统基本上包含九种人格原型、三个“情绪中心”和九个“发展层次”。这是一种非常有力的方法，可用于探索和理解虚构人物的性格与动机，并确定他们在各种情境中会如何行动。见图 3-5。

```
<Character uid="Character-56432464-1" egramType="8" egramCenter="1" egramLevel="4">
  <name>Captain Froop</name>
  <description>Green, slithery, has tentacles. Captain of the Vroomfastian
intergalactic transport ship.</description>
</Character>
```

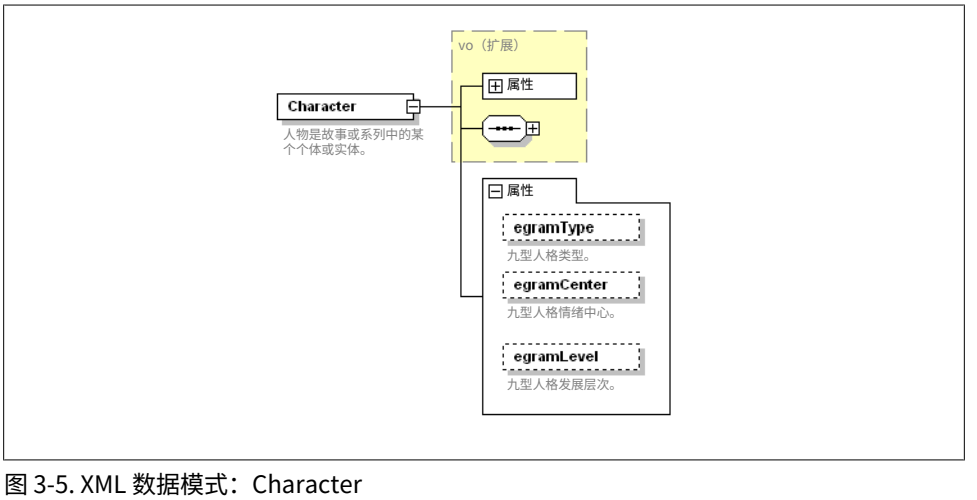


图 3-5. XML 数据模式：Character

Cast（人物阵容）

Cast 继承 VO，是 Character 集合的简单包装。同样，唯一 ID 使我们能够把这个集合，与其关联的一个或多个 Story 或 Series 分开保存。见图 3-6。

```
<Cast uid="Cast-2345213-1">
  <Character uid="Character-56432464-1" egramType="8" egramCenter="1" egramLevel="5">
    <name>Captain Froop</name>
    <description>Green, slithery, has tentacles. Captain of the Vroomfastian
intergalactic transport ship.</description>
  </Character>
  <Character uid="Character-33454645-24" egramType="1" egramCenter="0" egramLevel="4">
    <name>Jerry</name>
    <description>Twelve year old kid. Red hair, freckles.</description>
  </Character>
</Cast>
```

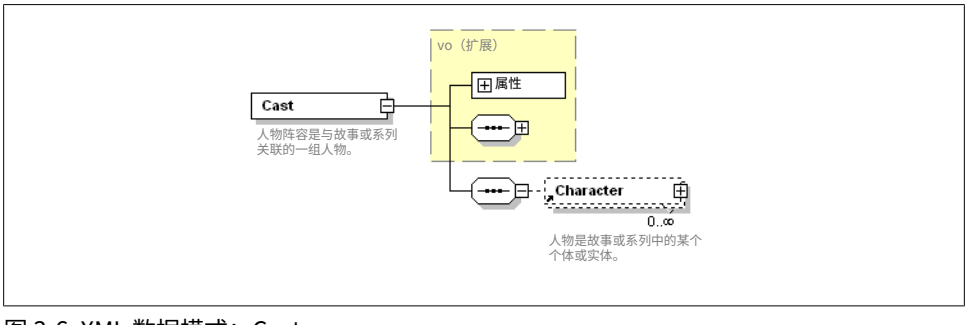


图 3-6. XML 数据模式：Cast

Draft（草稿）

虽然 Draft 继承 VO，但我们不会在其中使用任何子元素；其内容只有文本。它还有一个标志，表示自己是否是所属 Scene 的当前草稿。见图 3-7。

```
<Draft uid="Draft-5443523-22" isCurrent="true">Dawn hit him in the face like a sack of
frozen turnips.</Draft>
```

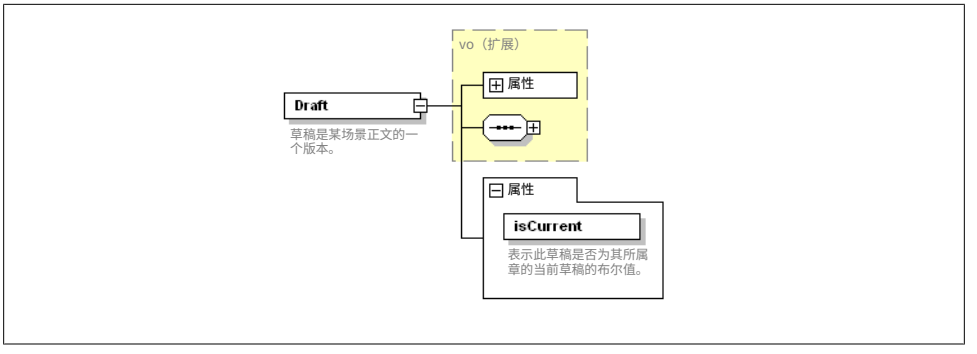


图 3-7. XML 数据模式：Draft

Scene（场景）

Scene 继承 VO，并包含一个 Draft 集合，其中只能有一份被视为当前草稿。

场景通常从某个特定叙述视角写成，包括第一人称（例如目击者叙述）、第二人称（并不常用，因为作品使用“你”来告诉读者正在经历什么）、第三人称（局限于某个特定人物的视角，或采用能随意洞察任何人物的全知视角）。我们将用一个属性记录叙述视角，并在应用程序中定义一个专门的类，记录其有效取值。

另外，如果 Scene 采用故事中某个人物的视角，就把 Character 的唯一 ID 保存为 Scene 的一个属性，以此关联场景与人物。见图 3-8。

```
<Scene uid="Scene-89543346-3" ordinal="1">
  <Draft uid="Draft-9678546-34" ordinal="1" isCurrent="true">A harsh klaxon echoed
through the ship as the autopilot disengaged from hyperspace.</Draft>
</Scene>
```

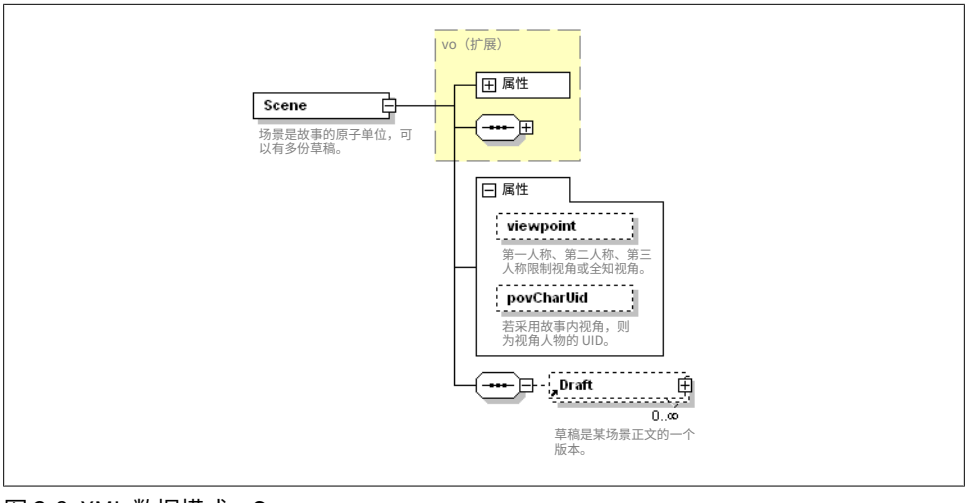


图 3-8. XML 数据模式：Scene

Chapter（章）

Chapter 是相当简单的实体。它继承 VO，并包含一组 Scene。编号方面，Chapter 可以按整个 Story 编号，也可以按其所在 Part 编号。见图 3-9。

```
<Chapter uid="Chapter-54435453-6" ordinal="12">
  <Scene uid="Scene-89543346-3" ordinal="1">
   <Draft uid="Draft-5465344-43" ordinal="1" isCurrent="true">It was a dark and stormy
night.</Draft>
  </Scene>
  <Scene uid="Scene-56562546-2"  ordinal="1" />
  <Scene uid="Scene-78678346-12" ordinal="2" />
</Chapter>
```

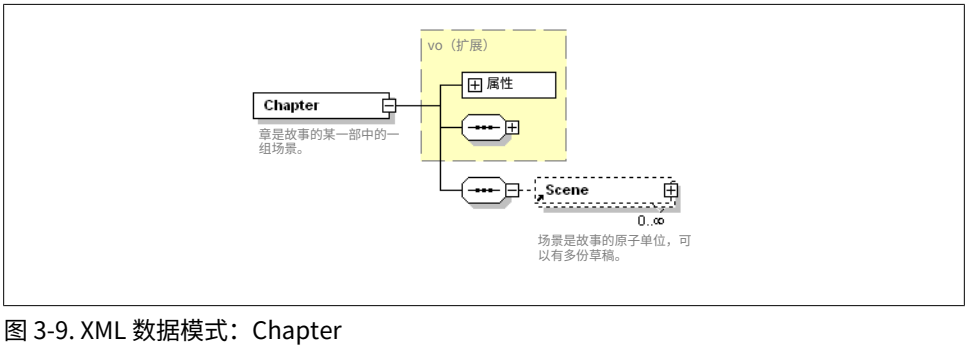


图 3-9. XML 数据模式：Chapter

Part（部）

Part 只是另一种分组机制，因此与 Chapter 很相似。它继承 VO 的属性，并包含一组 Chapter。划分为部不会让书的篇幅增加，只会使结构更复杂。通常，Part 用来把发生在不同时期的章分组，或把故事的大块内容分配给某一个人物的视角。

Part 在故事中是可选的，但如果 Story 使用 Part，那么其中的 Chapter 就位于各个 Part 元素内部。序号根据该 Story 中下一个可用的 Part 序号分配。见图 3-10。

```
<Part uid="Part-12345678-5" ordinal="1">
  <name>The Early Years</name>
  <Chapter uid="Chapter-3454656-32" ordinal="1">
    <Scene uid="Scene-56562546-2" ordinal="1" />
    <Scene uid="Scene-78678346-12" ordinal="2" />
  </Chapter>
  <Chapter uid="Chapter-7867834-45" ordinal="2" />
</Part>
```

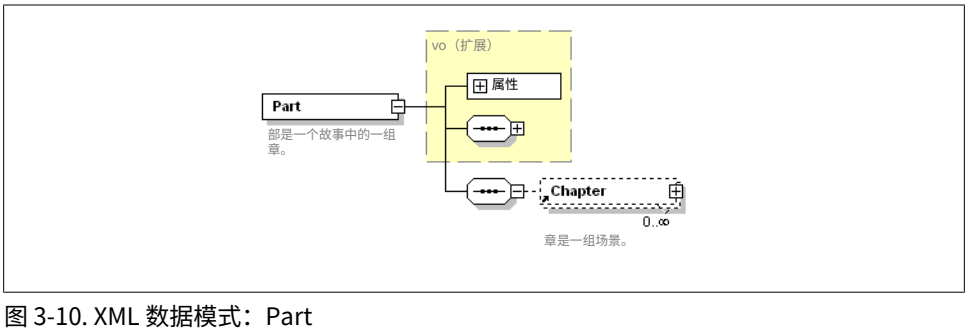


图 3-10. XML 数据模式：Part

Story（故事）

Story 继承 VO 的属性和元素，并可包含一个 Note 集合。它还能表示几种不同类型，用来说明场景如何组织。

简单的短篇故事只需一组 Scene。普通的中篇或长篇故事则包含一组 Chapter，Chapter 当然又包含 Scene。更复杂的作品可能使用 Part，由 Part 包含 Chapter，再由 Chapter 包含 Scene。图中的“开关”表示这种选择：包含 Part、Chapter 或 Scene 集合中的一种，而不是混合包含。

另外，Story 元素还可包含 Cast 和 Milieu 元素。这就是 VO 的 isStub 属性发挥作用的地方。Cast 和 Milieu 会保存到独立文件中，因为同一 Series 中的多个 Story 可能共享它们。这意味着，保存 Story 时，需要把 Cast 和 Milieu 替换成

存根；加载 Story 时，再按唯一 id 加载 Cast 和 Milieu，替换掉这些存根，然后交给视图层。见图 3-11。

```
<Story uid="Story-12345678-1" ordinal="1" isStub="false" type="O">
  <name>Untitled Scifi Story</name>
  <description>The Vroomfastians have been traveling the Dark Matter Highway for quite
awhile. They've just pulled off at our exit, and Man are they hungry!</description>
  <Note url="http://bit.ly/Ogz7Q">Interesting article about filaments of Dark Matter
pulling in gas and funneling it across the galaxy.</Note>
  <Note>How about "Once, Upon the Dark Matter Highway" for a title?</Note>
  <Cast/>
  <Milieu/>
  <Scene uid="Scene-89543346-3" ordinal="1">
    <Draft uid="Draft-5465344-43" ordinal="1" isCurrent="false">It was a dark and
stormy night.</Draft>
    <Draft uid="Draft-5443523-22" ordinal="2" isCurrent="false">Dawn hit him in the
face like a sack of frozen turnips.</Draft>
    <Draft uid="Draft-9678546-34" ordinal="3" isCurrent="true">A harsh klaxon echoed
through the ship as the autopilot disengaged from hyperspace.</Draft>
  </Scene>
  <Scene uid="Scene-56562546-2" ordinal="2" />
  <Scene uid="Scene-78678346-12" ordinal="3" />
</Story>
```

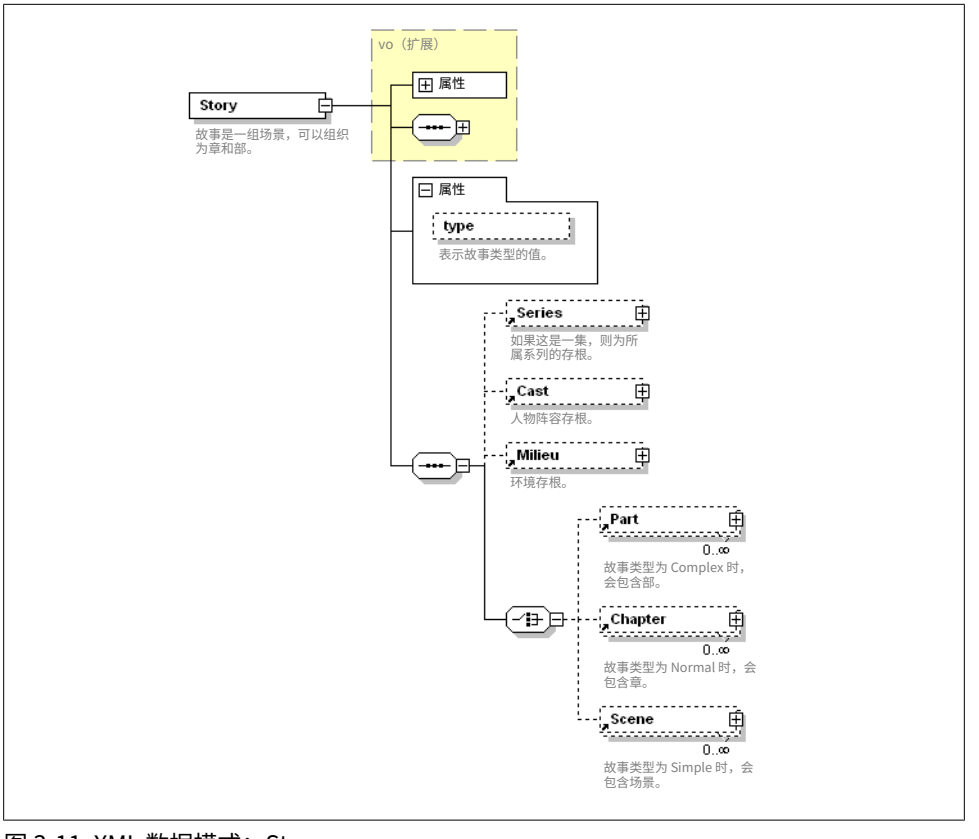


图 3-11. XML 数据模式：Story

Season（季）

Season 是一组 Story 元素的简单包装。Season 在 Series 中可选；但如果 Series 使用 Season，那么 Story 元素就保存在 Season 元素内部。见图 3-12。

```
<Season uid="Season-43534573-1" ordinal="1">
  <Story uid="Story-56562546-41" ordinal="1" />
  <Story uid="Story-78678346-24" ordinal="2" />
</Story>
```

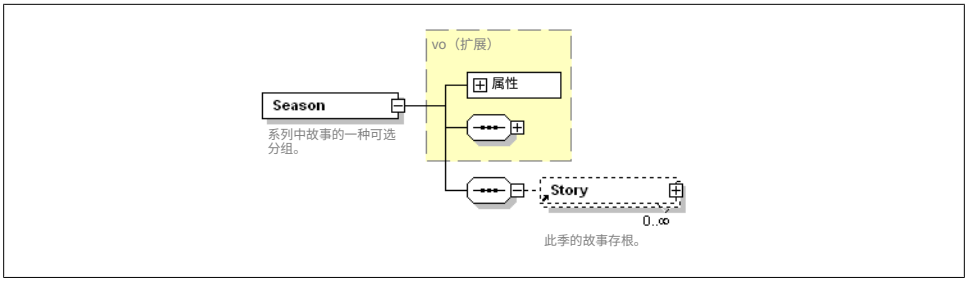


图 3-12. XML 数据模式：Season

Series（系列）

Series 继承 VO 的属性和元素，并可包含一个 Note 集合。另外，Series 元素还可包含 Cast 和 Milieu 元素，供该系列中的所有 Story 共享。见图 3-13。

```
<Series uid="Story-12345678-1" ordinal="1" isStub="false" useSeasons="false">
  <name>Untitled Scifi Series</name>
  <description>A young boy becomes a central figure in a saga of galactic intrigue when
he and his father are abducted by aliens and given a powerful secret to keep.</
description>
  <Note url="http://bit.ly/Ogz7Q">Interesting article about filaments of Dark Matter
pulling in gas and funneling it across the galaxy.</Note>
  <Cast/>
  <Milieu/>
  <Story uid="Story-56562546-2" ordinal="1">
    <name>Once, Upon the Dark Matter Highway</name>
    <description>The Vroomfastians have been traveling the Dark Matter Highway for
quite awhile. They've just pulled off at our exit, and Man are they hungry!</
description>
  </Story>
  <Story uid="Scene-78678346-12" ordinal="2" />
</Series>
```

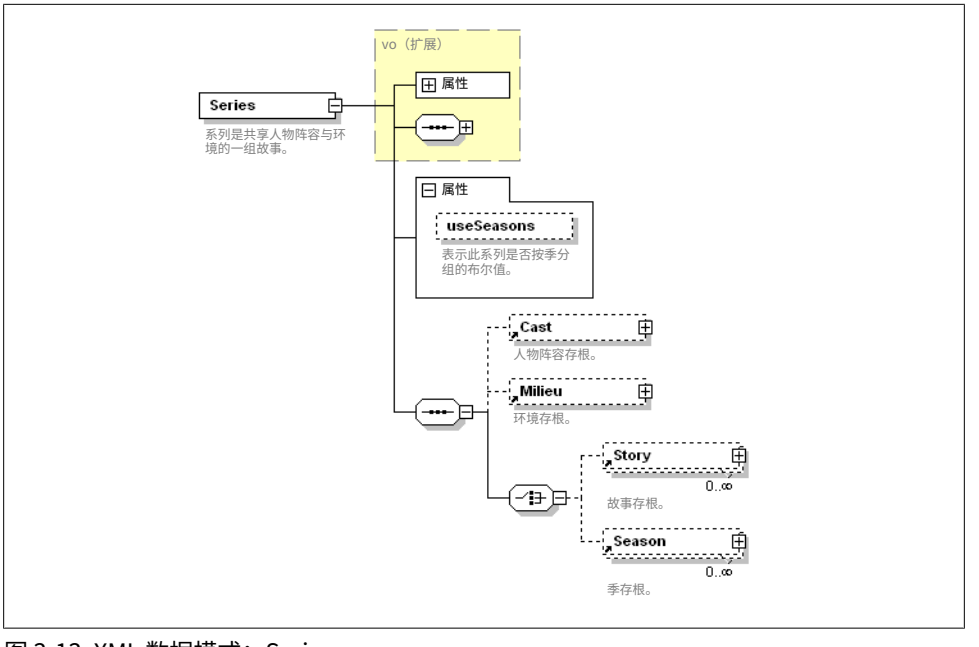


图 3-13. XML 数据模式：Series

创建值对象

经过 XML 数据模式的设计，我们已经清楚值对象类需要公开哪些属性。这些值对象会自行完成序列化，因此将使用隐式访问器，也就是成对的函数，负责获取和设置带类型的值，而在调用者看来只有一个属性。这样，就能把带类型的属性存入 XML 结构，或从 XML 结构中读出。当然，这里不会列出所有 VO 的源代码，但理解这些类如何工作很重要。我们将按复杂度，检查其中三个重要的类。



构建 VO 时，重要的是不要让它引用任何 PureMVC 类。它应该能够独立测试，也能够在 PureMVC 应用程序之外使用。

简单示例

先从一个简单却重要的数据载体 NoteVO 开始。系统中的其他 VO 通过继承 ValueObject，都能拥有一个 NoteVO 集合。

注意，NoteVO 的类声明之前有一个 [Bindable] 标签。我们会为所有 VO 都这样做。可以确定，视图组件将与

值对象协作，以表示和展示数据。[Bindable] 标签使持有 NoteVO 实例的视图组件，能够在 NoteVO 属性变化时更新。和使用 XML 原生数据类型一样，这也是在保持架构中 MVC 分离的同时，积极利用平台功能的例子。

另外，这个类的构造函数接受一个可选的 XML 参数。从文件系统读取原始 XML 并创建相应值对象时，我们把 XML 传给对应值对象类的构造函数。如果没有传入 XML，就必须创建一个有效的默认元素，因为后续与对象的所有交互，都通过访问器进行，而访问器负责管理 XML 结构的各个部分。

注意访问器如何完成各值的序列化和反序列化，使数据始终保存在 XML 结构中，只需写出 xml 属性就能保存。url 访问器在 XML 元素的某个属性中存取 String，text 访问器则管理元素本身的文本内容。

NoteVO

```
package com.futurescale.sa.model.vo
{
    /**
     * Note 是关于其他任意实体的 URL 和／或自由格式文本。
     */
    [Bindable] public class NoteVO
    {
        /**
         * XML 元素名称。
         */
        public static const ELEMENT:String = "Note";

        /**
         * 构造 NoteVO。
         */
        public function NoteVO( xml:XML=null )
        {
            this.xml = (xml)?xml:<{ELEMENT}/>;
        }

        /**
         * 此 Note 的文本。
         */
        public function set text( t:String ):void
        {
            x.setChildren(t);
        }
        public function get text():String
        {
            return xml.toString();
        }

        /**
```

```
             * 可选的 URL。
             */
            public function set url( u:String ):void
            {
                xml.@url = u;
            }
            public function get url():String
            {
                return xml.@url;
            }

            /**
             * 此 Note 的 XML 表示。
             */
            public function set xml( x:XML ):void
            {
                this.x = x;
            }
            public function get xml():XML
            {
                return x;
            }
            protected var x:XML;
        }
    }
```

稍复杂的示例

作为其他所有 VO（NoteVO 除外）的基类，ValueObject 提供了一些很有价值的可继承功能。

注意 name getter 的工作方式。如果没有 <name/> 元素，就根据 XML 元素名和序号生成名称。例如，Chapter 可以使用我们设置的独有名称；如果没有设置，就采用类似 Chapter 52 这样按规则生成的名称。

ValueObject 的另一个重要功能是 getNextOrdinal 方法，用于确定子对象的下一个可用序号。往 StoryVO 中添加新的 ChapterVO 时，需要知道应给它哪个章号，这由 getNextOrdinal 决定。还有一个按序号排序 ValueObject 的方法，确保无论 SceneVO 按什么顺序加入 ChapterVO，都能按正确顺序排序显示。另外，也有按名称排序 ValueObject 的方法，使 CastVO 和 MilieuVO 能够以合理顺序呈现 CharacterVO 和 SettingVO。

getStub() 方法返回 ValueObject 的 XML 元素表示，其中只包含 ValueObject 中定义的属性。起初看起来，这似乎只是一次直接的克隆操作，但请记住，系统中的其他 VO 都会继承 ValueObject，getStub() 实际是在这些子类上调用的。应用程序中有些地方不需要完整 VO。例如，保存 StoryVO 时，会把它们单独存放在一个文件夹中；但为了得到可用

StoryVO 的列表，而不必逐一打开并完整读取，就需要某种索引。索引由 StoryVO 的存根组成，其中 isStub 为 true，并且只包含基本的 ValueObject 属性。SeriesVO 中的 StoryVO 列表，也是需要存根的地方。加载 SeriesVO 时，我们希望看到各集的一些基本信息，却不希望每个 Story 的全部内容都作为 Series 的一部分保存。相反，应保存带有 StoryVO 存根的 SeriesVO，之后再按需加载各个故事。

可继承功能中，或许最重要的是为任意子类提供带唯一标识符的默认元素。你可能已经注意到，第 41 页“ValueObject”中的 XML 片段，其 uid 属性都遵循相似的格式。这个格式由构造函数调用的 validateOrProvideDefault() 方法生成。在远程数据库场景中，视图端新建的对象，通常要添加到数据库后才有 ID。但这里，对象创建时就能轻松即时生成自己的 ID。生成唯一 ID 的做法是：把元素名，例如 Story 或 Character，放在一个数值前面；该数值由自纪元起经过的秒数，加上本次运行期间已创建的 ValueObject 子类实例数量得到。

ValueObject

```
package com.futurescale.sa.model.vo
{
    import mx.formatters.DateFormatter;

    /**
     * 值对象的基类。
     */
    [Bindable] public class ValueObject
    {
        /**
         * 用于分隔拼接文本的常量。
         */
        public static const DOUBLE_SPACE:String = "\n\n";
        public static const DASH:String = "-";
        public static const SPACE:String = " ";

        /**
         * 构造 ValueObject。必须传入元素名称，
         * XML 则可选。会验证 XML 是否属于
         * 指定的元素类型；否则使用该类型的
         * 默认 XML 元素替换。
         */
        public function ValueObject( element:String, xml:XML=null )
        {
            this.xml = validateOrProvideDefault( element, xml );
        }

        /**
         * 获取此值对象的存根。
```

```
 *
 * 保存父对象之前，需要把父 XML 结构中的
 * 某些对象替换为存根。例如，Series 中的
 * 所有 Story 元素，以及 Story 和 Series
 * 中的 Cast 与 Milieu。
 */
public function getStub( ):XML
{
    var stub:XML = validateOrProvideDefault( xml.localName() );
    var vo:ValueObject = new ValueObject( xml.localName(), stub );
    vo.isStub = true;
    vo.uid = this.uid;
    vo.name = this.name;
    vo.ordinal = this.ordinal;
    vo.description = this.description;
    return stub;
}

/**
 * 值对象的名称。如果未设置名称，
 * 则将元素名与 ordinal 属性拼接。
 */
public function set name( n:String ):void
{
    xml.name = n;
}
public function get name():String
{
    return (String(xml.name)!="") ?
            String(xml.name) : String(xml.localName()+SPACE+ordinal);
}

/**
 * 此值对象在集合中的序号位置。
 */
public function set ordinal( o:Number ):void
{
    xml.@ordinal = o;
}
public function get ordinal():Number
{
    return Number(xml.@ordinal);
}

/**
 * 此 ValueObject 的唯一标识符。
 */
public function set uid( u:String ):void
{
    xml.@uid = u;
}
public function get uid():String
{
    return String(xml.@uid);
}
```

```
/**
 * 此 ValueObject 是否为尚未填充的存根?
 */
public function set isStub( s:Boolean ):void
{
    xml.@isStub = s;
}
public function get isStub():Boolean
{
    return (x.@isStub=="true");
}

/**
 * 此 ValueObject 的简短描述。
 */
public function set description( d:String ):void
{
    xml.description = d;
}
public function get description():String
{
    return String(xml.description);
}

/**
 * 包含此 ValueObject 的 NoteVO 的 Vector。
 */
public function get notes():Vector.<NoteVO>
{
    var v:Vector.<NoteVO> = new Vector.<NoteVO>();
    var xl:XMLList = xml.child(NoteVO.ELEMENT);
    for each ( var nx:XML in xl ) {
        var note:NoteVO = new NoteVO(nx);
        v.push(note);
    }
    return v;
}
public function set notes( v:Vector.<NoteVO> ):void
{} // 只读，但可绑定

/**
 * 为 ValueObject 添加 Note。
 */
public function addNote( note:NoteVO ):void
{
    xml.appendChild( note.xml );
}

/**
 * 值对象的 XML 表示。重写
 * 访问器以提供默认值。
 */
public function set xml( x:XML ):void
{
```

```
    this.x = x;
}
public function get xml():XML
{
    return x;
}
protected var x:XML;

/**
 * 按序号排序 ValueObject 的函数。
 */
public static function ordinalSort( v1:ValueObject, v2:ValueObject ):Number
{
    var retval:Number = 0;                              // 相等
    if (v1.ordinal < v2.ordinal) { retval = -1; }       // 小于
    else if (v1.ordinal > v2.ordinal) { retval = 1; }   // 大于
    return retval;
}

/**
 * 按名称排序 ValueObject 的函数。
 */
public static function nameSort( v1:ValueObject, v2:ValueObject ):Number
{
    var retval:Number = 0;                             // 相等
    if (v1.name < v2.name) { retval = -1; }            // 小于
    else if (v1.name > v2.name) { retval = 1; }        // 大于
    return retval;
}

/**
 * 获取子元素的下一个序号。
 */
protected function getNextOrdinal( xl:XMLList ):Number
{
    var n:Number = 0;
    var x:XML;
    for each ( x in xl ) {
        var o:Number = Number(x.@ordinal);
        if (o > n) n = o;
    }
    return ++n;
}

/**
 * 验证传入的 XML 元素非 null，且
 * 元素名称正确。否则，提供带有唯一
 * 标识符的默认元素。
 */
protected function validateOrProvideDefault( elementName:String, xml:XML ):XML
{
    if (xml == null || xml.localName() != elementName) {
        var u:String = elementName + DASH + new Date().time +
                       DASH + uidRuntimeUniquifier++;
        xml = <{elementName} uid={u}/>
```

```
            }
            return xml;
        }

        /**
         * 本次运行期间创建的唯一值对象数量。
         * 用于确保同一毫秒内创建的两个同类型
         * 对象具有不同的 uid。
         */
        protected static var uidRuntimeUniquifier:Number = 0;
    }
}
```

进阶示例

本应用中最复杂的示例是 StoryVO。回顾其 XML 数据模式，它支持三种不同的故事类型，决定其中包含 PartVO、ChapterVO 还是 SceneVO 的集合。

StoryVO 既是持有这些集合的数据载体，又从 ValueObject 继承了 getNextOrdinal() 方法，因此可以轻松传入子元素的 XMLList，取得下一个可用序号。它的集合访问器也稍复杂一些：不再只是管理属性或元素中的简单值，而要根据 XML 元素列表，构建值对象集合。本实现将使用 ActionScript 的 Vector 类，使我们能拥有一个集合，并指定它所能容纳的对象类型。另外，在这个具体实现中，集合访问器是不对称的。例如，可以获取 PartVO 列表，却不能设置它，因为它是只读的。新的 PartVO 通过 getNewPart() 或 addPart() 添加。getNewPart() 会创建一个新的 PartVO，自动分配下一个可用的 Part uid，然后调用 addPart()，把 XML 元素实际加入 StoryVO 的 XML 结构。需要从其他地方，例如另一个 StoryVO，加入已有 PartVO 时，则直接调用 addPart()。注意，addPart() 还会在传入 PartVO 已有序号、且序号落在现有范围内时，重新排列集合中的 PartVO；否则就分配下一个更大的序号。

如前所述，故事有简单、普通和复杂三种类型。在 XML 数据模式中，这由 <Story/> 元素的 type 属性表示。我们只保存 0、1 或 2 来代表这些类型，但在代码里，这些数值不容易记忆。注意，StoryVO 的 type 访问器使用了 StoryTypeEnum。该类枚举此字段的有效值，并赋予它们常量名称。因此，如果调用代码要直接设置 type 字段，也必须了解 StoryTypeEnum 类。为了封装 type 属性的处理，你会看到三个布尔属性的访问器：useScenes、useChapters 和 useParts。现在，拿到一个 StoryVO 实例后，只需写 storyVO.useParts=true，就会以 StoryTypeEnum.COMPLEX 调用 type setter，进而将 storyVO.xml 的 type 属性设为 2。

StoryVO 的另一个重要功能，是通过 getText() 方法取得整个故事的只读文本。无论场景如何分组，它都会由所有场景当前草稿的正文组成。PartVO、ChapterVO 和 SceneVO 也实现了同一方法。各层依次遍历直接子对象并调用其 getText()；SceneVO 则直接取得 text 属性，然后把文本拼接起来，在场景间留出双倍间距。如果有多个章，还会为各章添加标题；部也是如此。

如果你认为值对象应该只是被动的数据载体，那么把这么多逻辑放进 VO，可能显得过多。但这些代码总得放在某处，而且应用程序中凡是处理 VO 的地方，都可能需要它们。有些人会创建所谓的辅助类或工具类，将这些逻辑全部放进去，理由是把逻辑与数据分离，想必是好事。接着，他们又把处理这些对象的方法设为静态，从应用程序各处访问。结果，代码不仅依赖值对象，如今还依赖那个辅助类。

StoryVO

```
package com.futurescale.sa.model.vo
{
    import com.futurescale.sa.model.enum.StoryTypeEnum;

    import mx.utils.StringUtil;

    /**
     * Story 可以是独立作品，也可以是 Series 的一集。
     * 其中的 Chapter 可以按需分组为 Part，
     * 并具有关联的 Cast、Milieu，以及一个或
     * 多个 Plot。此值对象表示
     * 一个 Story。
     */
    [Bindable] public class StoryVO extends ValueObject
    {
        /**
         * XML 元素名称。
         */
        public static const ELEMENT:String = "Story";

        /**
         * 构造 StoryVO。
         */
        public function StoryVO( xml:XML=null )
        {
            super( ELEMENT, xml );
        }

        /**
         * Cast 存根。
         */
        public function get cast():CastVO
        {
            return ( x.Cast[0] )?new CastVO( x.Cast[0] ):null;
```

```
}
public function set cast( c:CastVO ):void
{
    x.Cast = c.xml;
}

/**
 * Milieu 存根。
 */
public function get milieu():MilieuVO
{
    return ( x.Milieu[0] )?new MilieuVO( x.Milieu[0] ):null;
}
public function set milieu( m:MilieuVO ):void
{
    x.Milieu = m.xml;
}

/**
 * 若为某系列的一集，则为 Series 存根。
 */
public function get series():SeriesVO
{
    return ( x.Series[0] )?new SeriesVO( x.Series[0] ):null;
}
public function set series( s:SeriesVO ):void
{
    x.Series = s.xml;
}

/**
 * 这是某个 Series 的一集吗?
 */
public function get isEpisode():Boolean
{
    return ( x.Series[0] );
}
public function set isEpisode( e:Boolean ):void
{} // 只读，但可绑定

/**
 * 这是哪种类型的 Story?
 * StoryTypeEnum.SIMPLE = 仅含场景
 * StoryTypeEnum.NORMAL = 包含章和场景
 * StoryTypeEnum.COMPLEX = 包含部、章和场景
 */
public function get type():StoryTypeEnum
{
    return (x.@type!=undefined)?StoryTypeEnum.list[ Number( x.@type ) ]:null;
}
public function set type( t:StoryTypeEnum ):void
{
    x.@type = t.ordinal;
}
```

```
/**
 * 此 Story 是否只使用 Scene?
 */
public function get useScenes():Boolean
{
    return ( this.type == StoryTypeEnum.SIMPLE );
}
public function set useScenes(u:Boolean):void
{
    this.type = StoryTypeEnum.SIMPLE;
}

/**
 * 此 Story 是否使用 Chapter?
 */
public function get useChapters():Boolean
{
    return ( this.type == StoryTypeEnum.NORMAL );
}
public function set useChapters(u:Boolean):void
{
    this.type = StoryTypeEnum.NORMAL;
}

/**
 * 此 Story 是否使用 Part?
 */
public function get useParts():Boolean
{
    return ( this.type == StoryTypeEnum.COMPLEX );
}
public function set useParts(u:Boolean):void
{
    this.type = StoryTypeEnum.COMPLEX;
}

/**
 * SIMPLE Story 的 Scene，按序号排序的 Vector。
 */
public function get scenes():Vector.<SceneVO>
{
    var v:Vector.<SceneVO> = new Vector.<SceneVO>();
    var xl:XMLList = xml..Scene;
    for each ( var sx:XML in xl ) {
        var scene:SceneVO = new SceneVO(sx);
        v.push(scene);
    }
    v.sort(ValueObject.ordinalSort);
    return v;
}
public function set scenes( v:Vector.<SceneVO> ):void
{} // 只读，但可绑定

/**
 * NORMAL Story 的 Chapter，按序号排序的 Vector。
```

```
 */
public function get chapters():Vector.<ChapterVO>
{
    var v:Vector.<ChapterVO> = new Vector.<ChapterVO>();
    var xl:XMLList = xml..Chapter;
    for each ( var cx:XML in xl ) {
        var chapter:ChapterVO = new ChapterVO(cx);
        v.push(chapter);
    }
    v.sort(ValueObject.ordinalSort);
    return v;
}
public function set chapters( v:Vector.<ChapterVO> ):void
{} // 只读，但可绑定

/**
 * COMPLEX Story 的 Part，按序号排序的 Vector。
 */
public function get parts():Vector.<PartVO>
{
    var v:Vector.<PartVO> = new Vector.<PartVO>();
    if (useParts) {
        var xl:XMLList = xml.child(PartVO.ELEMENT);
        for each ( var px:XML in xl ) {
            var part:PartVO = new PartVO(px);
            v.push(part);
        }
        v.sort(ValueObject.ordinalSort);
    }
    return v;
}
public function set parts( v:Vector.<PartVO> ):void
{} // 只读，但可绑定

/**
 * 为 Story 获取一个新 Scene。
 */
public function getNewScene():SceneVO
{
    var scene:SceneVO = new SceneVO();
    addScene( scene );
    return scene;
}

/**
 * 为 Story 获取一个新 Chapter。
 */
public function getNewChapter(  ):ChapterVO
{
    var chapter:ChapterVO = new ChapterVO();
    addChapter( chapter );
    return chapter;
}

/**
```

```
 * 为 Story 获取一个新 Part。
 */
public function getNewPart():PartVO
{
    var part:PartVO = new PartVO();
    addPart( part );
    return part;
}


/**
 * 向 Story 添加 Scene。如果加入的 Scene
 * 已有序号，且小于下一个场景序号，
 * 则重新编号现有 Scene；否则，
 * 分配下一个场景序号。
 *
 * 如果没有 part 参数，强制故事类型为 NORMAL。
 */
public function addScene( scene:SceneVO ):void
{
    if ( scene.ordinal > 0 && scene.ordinal < nextPartOrdinal ) {
        var scenes:Vector.<SceneVO> = this.scenes;
        for each (var svo:SceneVO in scenes ){
            if (svo.ordinal >= scene.ordinal) svo.ordinal++;
        }
    } else {
        scene.ordinal = nextSceneOrdinal;
    }
    xml.appendChild( scene.xml );
    useScenes=true;
}

/**
 * 向 Story 添加 Chapter。
 * 如果加入的 Chapter 已有序号，且小于
 * 下一个章序号，则重新编号现有 Chapter；
 * 否则，分配下一个章序号。
 *
 * 如果没有 part 参数，强制故事类型为 NORMAL。
 */
public function addChapter( chapter:ChapterVO ):void
{
    // 插入时重新编号各章
    if ( chapter.ordinal > 0 && chapter.ordinal < nextChapterOrdinal ) {
        var chapters:Vector.<ChapterVO> = this.chapters;
        for each ( var cvo:ChapterVO in chapters ){
            if ( cvo.ordinal >= chapter.ordinal ) cvo.ordinal++;
        }
    } else {
        chapter.ordinal = nextChapterOrdinal;
    }
    xml.appendChild( chapter.xml );
    useChapters=true;
}
```

```
/**
 * 向 Story 添加 Part。如果加入的 Part
 * 已有序号，且小于下一个部序号，
 * 则重新编号现有 Part；否则，
 * 为其分配下一个部序号。
 *
 * 强制故事类型为 COMPLEX。
 */
public function addPart( part:PartVO ):void
{
    if ( part.ordinal > 0 && part.ordinal < nextPartOrdinal ) {
        var parts:Vector.<PartVO> = this.parts;
        for each (var pvo:PartVO in parts ){
            if (pvo.ordinal >= part.ordinal) pvo.ordinal++;
        }
    } else {
        part.ordinal = nextPartOrdinal
    }
    xml.appendChild( part.xml );
    useParts=true;
}

/**
 * 获取 Story 中每个 Scene 的当前 Draft 文本。
 * 可以选择在文本开头
 * 包含故事标题。
 *
 * 如果使用 Part，且数量多于一个，
 * 则包含各部标题。
 *
 * 如果使用 Chapter，且数量多于一个，
 * 则包含各章标题。
 */
public function getText( includeHeading:Boolean=false ):String
{
    var text:String = includeHeading ? name + DOUBLE_SPACE : "";
    if (useParts) {
        var partList:Vector.<PartVO> = parts;
        if (partList.length > 1) {
            // 多个部，包含标题
            for each ( var pvo:PartVO in partList) {
                text += pvo.getText(true) + DOUBLE_SPACE;
            }
        } else if (partList.length == 1){
            // 单个部，不含标题
            text += PartVO(partList[0]).getText(false);
        }
    } else if (useChapters) {
        var chapterList:Vector.<ChapterVO> = chapters;
        if (chapterList.length > 1) {
            // 多个章，包含标题
            for each ( var cvo:ChapterVO in chapterList) {
                text += cvo.getText(true) + DOUBLE_SPACE;
            }
        } else if (chapterList.length == 1){
```

```
            // 单个章，不含标题
            text += ChapterVO(chapterList[0]).getText(false);
        }
    } else if (useScenes) {
        var sceneList:Vector.<SceneVO> = scenes;
        if (sceneList.length > 1) {
            // 多个场景，包含标题
            for each ( var svo:SceneVO in sceneList) {
                text += svo.getText(true) + DOUBLE_SPACE;
            }
        } else if (sceneList.length == 1){
            // 单个场景，不含标题
            text += SceneVO(sceneList[0]).getText(false);
        }
    }

    // 故事的只读文本
    return StringUtil.trim( text );
}

/**
 * Story 全文的词数。
 */
public function get wordCount():Number
{
    var wc:Number = 0;
    var storyText:String = getText(false);
    if ( storyText.length > 0 ) {
        wc = storyText.split(/\s+/g).length;
    }
    return wc;
}

public function set wordCount( c:Number ):void
{} // 只读，但可绑定

/**
 * 获取此 Story 的下一个 Part 序号。
 * 用于包含部、章和场景的 COMPLEX 故事。
 */
private function get nextPartOrdinal():Number
{
    return getNextOrdinal( xml..Part );
}

/**
 * 获取此 Story 的下一个 Chapter 序号。
 * 用于包含场景和章的 NORMAL 故事。
 */
private function get nextChapterOrdinal():Number
{
    return getNextOrdinal( xml..Chapter );
}

/**
```

```
         * 获取此 Story 的下一个 Scene 序号。
         * 用于仅包含场景的 SIMPLE 故事。
         */
        private function get nextSceneOrdinal():Number
        {
            return getNextOrdinal( xml..Scene );
        }
    }
}
```

创建枚举

有些语言，例如 Java，内置了枚举功能。遗憾的是，ActionScript 没有。那么，枚举究竟是什么，为什么我们需要关心它?

枚举是一种把数据字段的所有可能值，列举为某个类的静态成员的方法。例如，先看看怎样用最简单的方式，枚举 StoryVO 的 type 属性的有效值。

像下面这样写有什么问题吗?

```
package com.futurescale.sa.model.constant
{
    public class StoryConstants
    {
        public static const SIMPLE:String  = "0";
        public static const NORMAL:String  = "1";
        public static const COMPLEX:String = "2";
    }
}
```

完全没有问题，不过还有几个功能，若能具备就更好了。

首先，我们希望有一份可能值的列表，用于填充选择故事类型的可视化控件。还需要什么？最好除了数值本身，还有这些有效值的名称。如果组合框只以 0、1、2 作为故事类型选项，用户体验并不好；显示“简单”“普通”“复杂”就更合适。若组合框旁还能显示名称的解释，例如“仅场景”“章和场景”“部、章和场景”，就更好了。组合框尚未选择时，最好还有另一版列表，包含一个“未选择”选项，并为它设负值，使视图组件能判断是否已作出选择。这样，遍历有效值的业务逻辑就无须考虑“未选择”条目。另外，有时我们希望单独控制列表项的顺序，而不受数值影响。最后，还需要一种方法，根据两个 Enum 的复合内容判断它们是否相等。

这些锦上添花的功能，当然可以硬编码到负责收集或显示故事类型的视图组件里。但应用程序中往往有多个地方，

都需要解释故事类型，因此 Enum 提供了一种方式，把这些相关内容集中到我们关心的有效值上。如上所示，它们使用起来与简单常量一样容易。实现虽然稍复杂，但这些职责总要放在某处，而 Enum 这个角色很合适。



一定要使用枚举吗？PureMVC 是否在某种程度上必须依赖它？完全不是。不过，它是常与 PureMVC 配合使用的一种模式，能减轻其他角色必须了解 VO 及其字段所有有效值的负担。Enum 是另一类领域模型实体：它的所有属性值都预先确定，不打算根据用户输入动态创建。和 VO 一样，它不是框架角色；它帮助框架角色避免承担超出其角色范围的职责。

列出所需枚举

我们已经开始理解枚举的角色。那么，在我们的应用程序中，具体哪些地方需要它们呢？

- StoryTypeEnum：StoryVO 的 type 属性的有效值。
- ViewPointEnum：SceneVO 的 viewpoint 属性的有效值。
- EgramTypeEnum：CharacterVO 的 egramType 属性的有效值。
- EgramCenterEnum：CharacterVO 的 egramCenter 属性的有效值。
- EgramLevelEnum：CharacterVO 的 egramLevel 属性的有效值。

基类

现在看看 Enum 类。它会像 ValueObject 为 VO 所做的那样，为子类提供基本功能，包括序号、名称和描述。它还有一个 equals 方法，用来比较当前实例与另一个 Enum 实例的属性。

Enum

```
package com.futurescale.sa.model.enum
{
    /**
     * 枚举的基类。
     */
    [Bindable] public class Enum
    {
        public var name:String;
        public var ordinal:Number;
        public var description:String;
```

```
        public function Enum( ordinal:Number, name:String, description:String )
        {
            this.name = name;
            this.ordinal = ordinal;
            this.description = description;
        }

        /**
         * 与另一个枚举实例比较。
         */
        public function equals( enum:Enum ):Boolean
        {
            return ( ( this.name        == enum.name )    &&
                     ( this.ordinal     == enum.ordinal ) &&
                     ( this.description == enum.description )
                   );
        }
    }
}
```

Enum 子类

Enum 子类的职责包括：为某个数据字段的有效值定义静态常量，而这些常量实际上就是该类的静态实例。它们通过 list getter 提供这些常量，返回一个数组。另外还有 comboList getter，返回一个 ArrayCollection，其中包含该列表，并在最前面增加“未选择”选项。ArrayCollection 实现了 IList，而基于列表的 Spark 控件需要该接口。基类提供的 equals 方法通常可以直接使用，因为它比较的是任何 Enum 子类都继承的三个基本属性。不过，你的子类可能还有更多属性需要比较，或者你不仅希望基本属性相符，还希望被比较实例所属的类也相符。你可以重写 equals，检查实例类型和额外属性，再执行父类方法以确认基本属性匹配，从而严格确定这些条件。

接下来，我们看看一个简单的 Enum 子类 StoryTypeEnum 如何实现。

StoryTypeEnum

```
package com.futurescale.sa.model.enum
{
    import mx.collections.ArrayCollection;

    /**
     * Story 类型的枚举。
     */
    public class StoryTypeEnum extends Enum
    {
        public static const NONE:StoryTypeEnum     =
                new StoryTypeEnum( -1, "--None Selected--", "Choose a Story Type" );
```

```
        public static const SIMPLE:StoryTypeEnum   =
                new StoryTypeEnum(  0, "Simple",   "Scenes only" );

        public static const NORMAL:StoryTypeEnum   =
                new StoryTypeEnum(  1, "Normal",  "Chapters and Scenes" );

        public static const COMPLEX:StoryTypeEnum  =
                new StoryTypeEnum(  2, "Complex", "Chapters, Parts, and Scenes" );

       public function StoryTypeEnum( ordinal:Number, name:String, description:String )
        {
            super( ordinal, name, description );
        }

        public static function get list():Array
        {
            return [ SIMPLE, NORMAL, COMPLEX ];
        }

        public static function get comboList():ArrayCollection
        {
            return new ArrayCollection( [ NONE ].concat( list ) );
        }

        override public function equals( enum:Enum ):Boolean
        {
            return ( enum is StoryTypeEnum && super.equals( enum ) );
        }
    }
}
```

测试实体

刚刚构建的这些类，是整个应用程序的基础。其他所有代码的目的，只是把这些类的实例交给用户操作，并将其持久化。尤其是这些实体还封装了即时序列化和反序列化逻辑，因此在继续之前进行测试非常重要。如果不能确定领域模型足够可靠，那么排查使用这些实体的代码时，就始终带着不可接受的不确定性。跳过测试省下的时间，日后肯定会在调试中损失掉。你不一定非要用测试框架编写正式测试，但使用常见框架，是最快享受到单元测试好处的途径。

Flex 单元测试类

FlashBuilder 集成了 FlexUnit 测试框架，使创建和运行测试非常容易，因此本应用选择使用它。大多数流行测试框架的原理相近，主要区别是标注各方法的元数据。不同 IDE 创建新测试类的方式有所不同，但基本上，

对于每个待测试的类，都可以让 IDE 生成一个测试类骨架，其中的方法用于执行待测类的各部分代码。多数框架允许在整个测试类执行前后，以及每个测试方法前后，进行初始化和清理。我们将看一个简单示例：NoteVO 的 FlexUnit 测试类。在测试类中，逐个测试 NoteVO 的方法，先从构造函数开始。构造函数用两个独立测试方法测试，因为未传入 XML 时，类会自行提供默认值，因此有两种不同的行为需要测试。对象创建并填充后，只需尽可能多地对它作出断言。如果任何断言不成立，就报告相应的信息。

当然，单元测试不是灵丹妙药。你没有想到的边界情况，仍可能在以后暴露缺陷；也可能有人添加代码破坏某些功能，却没有运行测试来确认代码仍能通过。但如果测试足够全面，运行后出现图 3-14 所示、表示全部通过的大绿条，就能让你对作为开发基础的这些代码产生很强的信心。

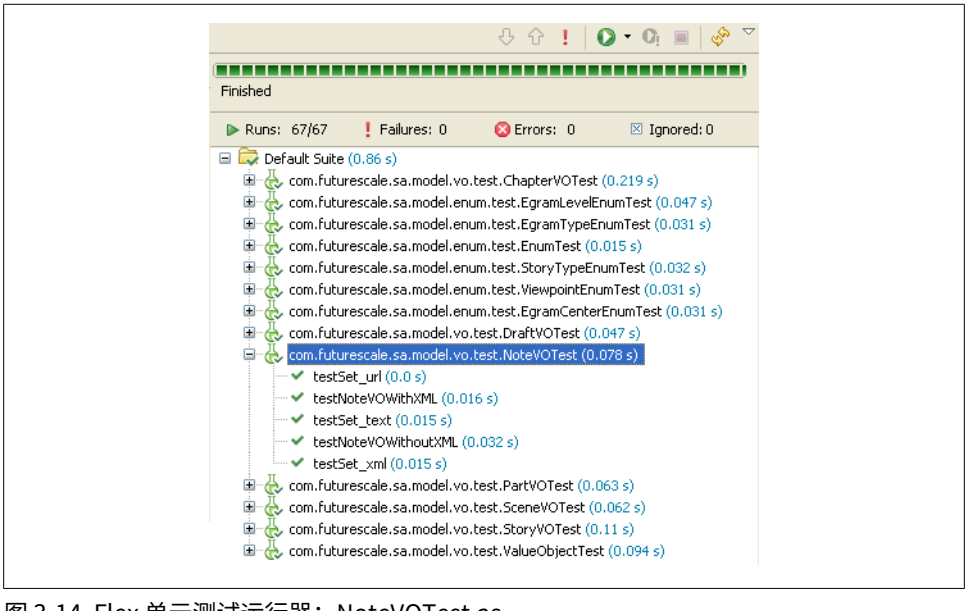


图 3-14. Flex 单元测试运行器：NoteVOTest.as

NoteVOTest

```
package com.futurescale.sa.model.vo.test
{
    import com.futurescale.sa.model.vo.NoteVO;
    import flexunit.framework.Assert;
    public class NoteVOTest
    {
        [Test] public function testNoteVOWithXML():void
```

```
        {
            var url:String = "http://google.com";
            var text:String = "Some stuff I'm keeping in the cloud.";
            var xml:XML = <Note url={url}>{text}</Note>;
            var vo:NoteVO = new NoteVO( xml );
            if (! vo is NoteVO ) Assert.fail("Construction with xml failed");
            if ( vo.text != text ) Assert.fail("text match failed");
            if ( vo.url != url ) Assert.fail("url match failed");
        }

        [Test] public function testNoteVOWithoutXML():void
        {
            var vo:NoteVO = new NoteVO();
            if (! vo is NoteVO ) Assert.fail("Construction without xml failed");
            if ( vo.xml.localName() != NoteVO.ELEMENT )
                   Assert.fail("default xml not provided");
        }

        [Test] public function testSet_text():void
        {
            var text:String = "Bunnies are nice!";
            var vo:NoteVO = new NoteVO();
            vo.text = text;
            if ( vo.text != text ) Assert.fail("text match failed");
        }

        [Test] public function testSet_url():void
        {
            var url:String = "http://puremvc.org";
            var vo:NoteVO = new NoteVO();
            vo.url = url;
            if ( vo.url != url ) Assert.fail("url match failed");
        }

        [Test] public function testSet_xml():void
        {
            var url:String = "http://puremvc.org";
            var text:String = "Code at the Speed of Thought!";
            var xml:XML = <Note url={url}>{text}</Note>;
            var vo:NoteVO = new NoteVO();
            vo.xml = xml;
            if ( vo.text != text ) Assert.fail("text match failed");
            if ( vo.url != url ) Assert.fail("url match failed");
        }
    }
}
```

第 4 章
实现用户界面

构成用户界面的各个类，应当封装各自的行为和外观。视图组件可能需要外部提供数据，或者偶尔调用它的方法，才能获得要显示的内容，或得知自己需要切换到另一种视觉状态。但是，一旦获得输入和触发信号，组件就应能自行完成这些视觉状态的转换。当组件需要与应用程序的其他部分通信时，应仅通过派发事件，或设置子组件的属性、调用子组件的方法来完成。在 PureMVC 应用程序中，视图组件绝不应了解 PureMVC 框架类及其子类。

先构建值对象，再构建视图组件，随后才深入 PureMVC 机制，这样安排有一个好处：Mediator、Proxy 和 Command 类尚未创建，因此你暂时根本无法访问它们。由于值对象和枚举已经建立，我们在构建视图组件时，可以用模拟的 VO 填充它们，无须先完成系统其余所有部分，再由那些部分提供数据。事实上，你应当能够将视图组件的开发交给完全不了解 PureMVC 的另一家公司、团队或团队成员，仍然顺利完成工作。如果开发团队来自另一家公司，可以考虑把 Model 类重新打包为独立的库，供他们构建视图组件时使用。

实现视图组件时，最重要的是记住：组件应公开一组由事件、属性和方法构成的 API，供外界与之交互。这套 API 应隐藏组件的内部实现。例如，不应要求调用者深入组件内部，去设置其中声明的 DataGrid 实例的 dataProvider 属性。相反，组件应公开一个可绑定的公共属性，例如 displayItems。然后，将 DataGrid 实例的 dataProvider 属性绑定到 displayItems。这意味着，负责照管该组件的 Mediator 子类既不知道，也不关心组件如何实现；它只知道，要显示数据，就应设置 displayItems 属性。以后，这个视图组件可以重构为

使用一个 ID 不同的下拉列表，而 Mediator 完全不受这一变化影响。

此外，与面向对象应用程序中的任何类一样，应尽量避免给某个视图组件堆积过多职责。当你发现大量嵌套结构、相关逻辑和变量都声明在同一处时，就应考虑创建自定义组件，替换大段 MXML。有人可能认为，这会增加复杂度，因为类的数量增多了。但实际上，它能简化视图组件、明确其职责，并提高复用性；本章将对此作出展示。

两个视图的故事

我们在白板上进行了大量推敲，考虑如何将 StoryArchitect 应用程序的目标转化为用户界面，最终确定应用程序包含两个主要视图：Chooser（选择器）和 Editor（编辑器）。打开应用程序后，你可以选择多种操作来创建和管理 Story 与 Series。你可以选中一个 Story，将它导出为文件、删除它、修改名称等。也可以选择开始写作，随后进入 Editor，在那里完成写作。Editor 将提供为 Story 任意部分添加描述和笔记的功能，以及用于浏览和扩展 Story 的时间线。

虽然处理 Series 的功能是应用程序的重要组成部分，但相对于写作 Story 这一主要目标，它和 Cast、Milieu 的管理一样，重要性要低得多。本书将重点讨论与 Story 直接相关的主要用例和用户界面元素。请记住，第一次迭代的主要目标是：创建并持久化 Story，编辑和扩展它的结构，为结构中的任意部分命名，并为其添加描述和笔记。

下面，我们来查看用户界面中一些较为重要的部分。除了主应用程序和组件之外，我们还将定义一个名为 AppEvent、能够携带数据的冒泡事件，以及一个类似值对象的 SelectionContext 类，用于在视图组件之间共享当前选择的信息。

在协作关系方面，请注意，这些视图组件基本上只了解自己的直接子组件、AppEvent 类、SelectionContext 类、Enum 及其子类，以及 ValueObject 及其子类。视图组件内部没有任何地方引用 PureMVC 的 Facade、Mediator、Command、Proxy 或 Notification 类。唯一的例外是主应用程序本身：它必须了解 Facade，才能引导启动过程。视图组件公开属性以接收数据，发送事件以表达用户意图，观察并遵从共享的选择上下文，而且仅与自己的直接子组件，以及外界提供的数据对象交互。这些数据对象可以来自父组件，也可以来自 Mediator。

打开应用程序后，在第一个视图中，你可以添加或选择一个 Story 或 Series，并使用与所选项目相关的各种功能。这个视图称为 Chooser 视图，如图 4-1 所示。创建和管理 Story 等功能会打开弹出窗口，收集并应用用户输入。

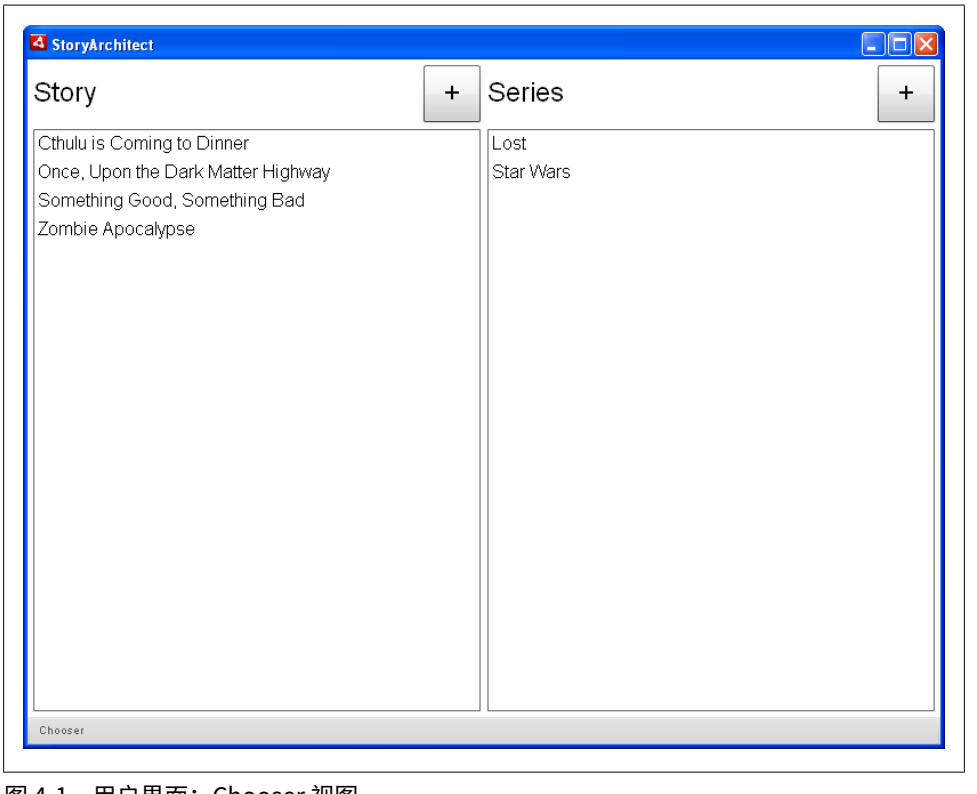


图 4-1．用户界面：Chooser 视图

创建 Story 或从列表中选择一个 Story 后，你会进入 Editor 视图（见图 4-2），立即开始编写该 Story 最后一个 Scene 的当前 Draft，无论各个 Scene 是如何分组的。你还可以展开时间线和详情组件，以浏览 Story、扩展其内容并添加批注。

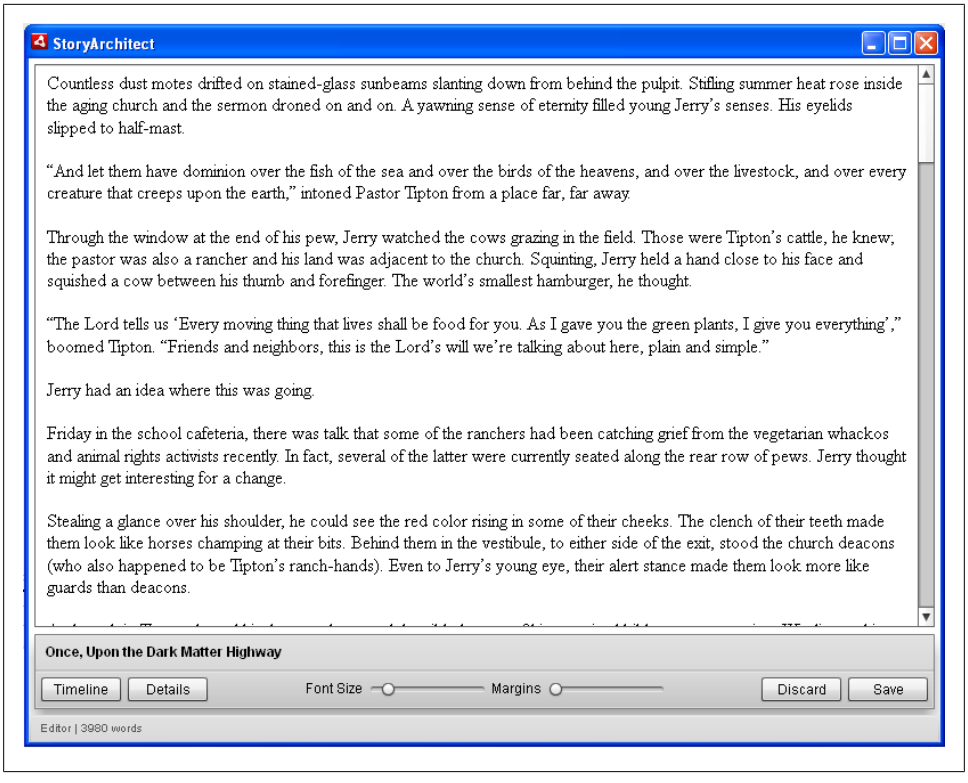


图 4-2．用户界面：Editor 视图

应用程序

类

`StoryArchitect.mxml`

职责

- 声明并布局 Chooser 和 Editor 组件。
- 初始化 PureMVC Facade。
- 触发 PureMVC 启动过程，传入应用程序引用以便建立中介关系。
- 定义 Starting、Chooser 和 Editor 三种显示模式。
- 公开可绑定的公共属性，用于设置词数。
- 公开可绑定的公共属性，用于设置显示模式。
- 公开可绑定的公共属性，用于设置 SelectionContext。
- 向需要 SelectionContext 的子组件提供该对象。

- 根据显示模式，控制子组件是否可见、是否参与布局。
- 在状态栏中显示当前模式；处于 Editor 模式时，还显示词数。
- 监听 Editor 组件的 ReportWordCount 类型 AppEvent，事件发生时更新 wordCount 属性。

协作关系

运行时，Flash 首先构建主应用程序 StoryArchitect，初始化它的显示列表和变量。因此，这里非常适合发起 PureMVC 机制的初始化。为此，StoryArchitect 了解 ApplicationFacade，以便初始化它并触发启动过程。主应用程序是“视图组件不应了解 PureMVC Facade”这条规则的例外。应用程序与 Facade 的交互应仅限于履行这两项职责。其他任何视图组件都没有理由了解 Facade。

StoryArchitect 还了解 SelectionContext 类。这是专为本应用程序设计的结构，让组件可以获知当前选择。当为 StoryArchitect 设置 SelectionContext 时，它会通过 Flex 绑定传给子组件。你会注意到，界面中的许多视图组件都具备这个可绑定属性，并会将其继续传给自己的子组件。例如，借助它，我们可以让 Timeline 组件自动展开到 Story 最后一个 Part 中、最后一个 Chapter 的最后一个 Scene 的当前 Draft。稍后将更详细地介绍这个类。

StoryArchitect 了解并控制其直接子组件 Chooser 和 Editor。

ApplicationFacade 了解 StoryArchitect，并公开一个便捷方法，将应用程序传给 StartupCommand。StartupCommand 会短暂引用它，以注册 ApplicationMediator；后者负责协调 StoryArchitect 与系统其余部分之间的通信。

代码

```
<?xml version="1.0" encoding="utf-8"?>
<!-- StoryArchitect 应用程序 -->
<s:WindowedApplication xmlns:editor="com.futurescale.sa.view.component.editor.*"
                       xmlns:chooser="com.futurescale.sa.view.component.chooser.*"
                       xmlns:s="library://ns.adobe.com/flex/spark"
                       xmlns:fx="http://ns.adobe.com/mxml/2009"
                       applicationComplete="facade.startup(this); // 启动应用    "
                       minWidth="800" minHeight="600">
    <fx:Script>
        <![CDATA[
            import com.futurescale.sa.ApplicationFacade;
            import com.futurescale.sa.view.context.SelectionContext;

            public static const MODE_STARTING:String = "Starting...";
            public static const MODE_CHOOSER:String  = "Chooser";
```

```
            public static const MODE_EDITOR:String   = "Editor";

            // 在各视图组件之间共享的选择上下文。
            [Bindable] public var context:SelectionContext;

            // 词数（在 Editor 模式下显示于状态栏）
            [Bindable] public var wordCount:String = "";

            // 显示模式（Chooser/Editor）
            [Bindable] private var mode:String = MODE_STARTING;

            // 初始化 PureMVC Facade
            private var facade:ApplicationFacade = ApplicationFacade.getInstance();

            /**
             * 设置应用程序的显示模式；如果显示的是 Editor，
             * 则根据所选 Story 初始化词数。

             */
            public function setMode( mode:String ):void
            {
                this.mode=mode;
                if (mode == MODE_EDITOR && context.story) {
                    wordCount = context.story.wordCount.toString();
                } else {
                    wordCount = "";
                }
            }

        </fx:Script>

        <!-- 布局 -->
        <s:layout>
            <s:VerticalLayout horizontalAlign="center"/>
        </s:layout>

        <!-- 状态栏 -->
        <s:status>{mode}{(wordCount != "")?" | "+wordCount+" words":""}</s:status>

        <!-- 选择器 -->
        <chooser:Chooser id="chooser" width="100%" height="100%"
                         includeInLayout="{mode == MODE_CHOOSER}"
                         visible="{mode == MODE_CHOOSER}"/>

        <!-- 编辑器 -->
        <editor:Editor id="editor" width="100%" height="100%"
                       includeInLayout="{mode == MODE_EDITOR}"
                       visible="{mode == MODE_EDITOR}"
                       reportWordCount="wordCount=String(event.data)"
                       context="{context}"/>

    </s:WindowedApplication>
```

选择器

类

Chooser.mxml

职责

- 声明并布局 StoryChooser 和 SeriesChooser 组件。
- 公开可绑定的公共属性，用于设置 StoryVO 列表。
- 公开可绑定的公共属性，用于设置 SeriesVO 列表。
- 向子组件提供 Story 和 Series 列表。

协作关系

Chooser 只了解并控制自己的直接子组件 StoryChooser 和 SeriesChooser。请注意，Chooser 不需要访问 SelectionContext 对象，因为它的子组件仅显示简单列表，并在用户作出选择、按下按钮时派发待处理的事件。

Chooser 的父组件 StoryArchitect 应用程序了解 Chooser；StartupCommand 也了解它，并会短暂引用它以注册 ChooserMediator。ChooserMediator 了解 Chooser，并协调它与系统其余部分之间的通信，也间接协调其 StoryChooser 和 SeriesChooser 子组件的通信。用于填充子组件的 Story 和 Series 列表，由 ChooserMediator 设置到 Chooser 上。子组件派发的冒泡事件，也由在 Chooser 上设置监听器的 ChooserMediator 处理。

代码

```
<?xml version="1.0" encoding="utf-8"?>
<s:HGroup xmlns:s="library://ns.adobe.com/flex/spark"
          xmlns:fx="http://ns.adobe.com/mxml/2009"
          xmlns:chooser="com.futurescale.sa.view.component.chooser.*"
          verticalAlign="middle" horizontalAlign="center"
          paddingLeft="5" paddingRight="5" width="100%"
          paddingBottom="5" paddingTop="5" height="100%"
         >
    <fx:Script>
        <![CDATA[
            import mx.collections.ArrayCollection;

            // 故事列表
            [Bindable] public var storyList:ArrayCollection;

            // 系列列表
            [Bindable] public var seriesList:ArrayCollection;

    </fx:Script>
```

```
        <!-- 故事选择器 -->
        <chooser:StoryChooser id="storyChooser" storyList="{storyList}"
                              width="50%" height="100%"/>

        <!-- 系列选择器 -->
        <chooser:SeriesChooser id="seriesChooser" seriesList="{seriesList}"
                               width="50%" height="100%"/>

    </s:HGroup>
```

故事选择器



图 4-3．用户界面：故事选择器

类

`StoryChooser.mxml`

职责

· 声明一个列表，显示可供选择的 StoryVO。

- 声明并布局用于操作所选项目的各个按钮。
- 根据选择，控制按钮是否可见、是否参与布局。
- 声明一个醒目的“添加”（+）按钮，用于添加新 Story。
- 按下按钮时派发相应的事件。

协作关系

StoryChooser 了解并控制自己的直接子组件：Flex 的 Button、Label 和 List 组件。它还了解 AppEvent 类，在某些按钮被按下时，会创建并派发这种事件。

只有父组件 Chooser 了解 StoryChooser。

代码

```
<?xml version="1.0" encoding="utf-8"?>
<s:VGroup xmlns:fx="http://ns.adobe.com/mxml/2009"
          xmlns:s="library://ns.adobe.com/flex/spark"
          minWidth="390" minHeight="290">

    <fx:Script>
        <![CDATA[
            import com.futurescale.sa.view.event.AppEvent;

            import mx.collections.ArrayCollection;

            // StoryVO 列表
            [Bindable] public var storyList:ArrayCollection;

            // 创建并派发 AppEvent
            private function sendEvent( type:String, data:Object=null ):void
            {
                dispatchEvent( new AppEvent(type,data ) );
            }

    </fx:Script>

    <!-- 选择器标题栏 -->
    <s:HGroup fontSize="24" verticalAlign="middle"
              width="100%" height="50">

        <!-- 标题 -->
        <s:Label text="Story" width="100%"/>

        <!-- 添加按钮 -->
        <s:Button label="+" width="50" height="100%"
                  click="sendEvent( AppEvent.ADD_STORY )" />

    </s:HGroup>

    <!-- 故事列表 -->
    <s:List labelField="name" id="lstStories"
```

```
            dataProvider="{storyList}" fontSize="16"
            width="100%" height="100%"/>

        <!-- 控件 -->
        <s:HGroup fontSize="16" verticalAlign="middle"
                  visible="{lstStories.selectedItem != null}"
                  includeInLayout="{lstStories.selectedItem != null}"
                  width="100%" height="100">

            <!-- 人物阵容 / 环境 -->
            <s:VGroup height="100%" width="25%">
                <s:Button label="Cast"   height="50%" width="100%"/>
                <s:Button label="Milieu" height="50%" width="100%"/>
            </s:VGroup>

            <!-- 笔记 / 大纲 -->
            <s:VGroup height="100%" width="25%">
                <s:Button label="Notes"   height="50%" width="100%"/>
                <s:Button label="Outline" height="50%" width="100%"/>
            </s:VGroup>

            <!-- 管理 / 导出 -->
            <s:VGroup height="100%" width="25%">

                <!-- 管理 -->
                <s:Button label="Manage" height="50%" width="100%"
                          click="sendEvent( AppEvent.MANAGE_STORY,
                                            lstStories.selectedItem )" />
                <!-- 导出 -->
                <s:Button label="Export" height="50%" width="100%"/>
            </s:VGroup>

            <!-- 写作 -->
            <s:Button label="Write" fontSize="24"
                      click="sendEvent( AppEvent.EDIT_STORY,
                                        lstStories.selectedItem )"
                      height="100%" width="25%"/>

        </s:HGroup>

    </s:VGroup>
```

编辑器

类

`Editor.mxml`

职责

- 声明并布局一个 TextArea，用于写作或显示只读的汇总文本。
- 声明并布局 Controls 子组件。

- 公开可绑定的公共属性，用于设置 SelectionContext。
- 向需要 SelectionContext 的子组件提供该对象。
- 提供公共函数，将焦点设置到文本编辑器。
- 编辑文本时，更新所选 DraftVO 的文本。
- 编辑文本时，派发事件，报告当前词数。
- 使用 Controls 组件提供的值，控制 TextArea 的字号和宽度百分比。
- 声明 Flex 元数据，说明该组件会派发 ReportWordCount 类型的 AppEvent。
- 监听 Controls 实例的 SelectScene 和 SelectDraft 类型 AppEvent，并在事件发生时将焦点设置到 TextArea；这些 AppEvent 仍会继续冒泡，并由 Mediator 处理。

协作关系

Editor 了解并控制自己的直接子组件：Flex TextArea 和自定义 Controls 组件。它还了解 AppEvent 类，在文本被编辑时会创建并派发这种事件。它也了解 SelectionContext 类，并在方法和绑定表达式中引用该类。

Editor 的父组件 StoryArchitect 应用程序了解 Editor；StartupCommand 也了解它，并会短暂引用它以注册 EditorMediator。EditorMediator 将协调 Editor 与系统其余部分之间的通信。

代码

```
<?xml version="1.0" encoding="utf-8"?>
<!-- 编辑器 -->
<s:VGroup xmlns:fx="http://ns.adobe.com/mxml/2009"
          xmlns:s="library://ns.adobe.com/flex/spark"
          xmlns:mx="library://ns.adobe.com/flex/mx"
          xmlns:editor="com.futurescale.sa.view.component.editor.*"
          paddingBottom="5" paddingTop="5" horizontalAlign="center"
          paddingLeft="5" paddingRight="5"
          minWidth="400" minHeight="300" >

    <fx:Metadata>
       [Event(name="reportWordCount", type="com.futurescale.sa.view.event.AppEvent")]
    </fx:Metadata>

   <fx:Script>
        <![CDATA[
            import com.futurescale.sa.view.context.SelectionContext;
            import com.futurescale.sa.view.event.AppEvent;

            // 选择上下文
            [Bindable] public var context:SelectionContext;
```

```
            // 用户在文本编辑器中输入时调用
            private function textEdit():void
            {
                if (context.draft) {
                    // 使用编辑器中的最新文本更新草稿
                    context.draft.text = textEditor.text;

                    // 派发事件，报告故事的当前词数。
                    var event:AppEvent = new AppEvent( AppEvent.REPORT_WORDCOUNT,
                                                       context.story.wordCount );
                    dispatchEvent( event );
                }
            }

        </fx:Script>

        <!-- 文本编辑器 -->
        <s:TextArea id="textEditor" borderVisible="{context.draft != null}"
                    percentWidth="{100 - controls.editMarginPct}" height="100%"
                    visible="true" editable="{context.draft != null}"
                    text="{context.selectedText}" change="textEdit()"
                    fontSize="{controls.editFontSize}" fontFamily="serif"
                    paddingLeft="10" paddingRight="10"
                    paddingTop="10" paddingBottom="10"/>

        <!-- 控件 -->
        <editor:Controls id="controls" width="100%"
                         selectDraft="textEditor.setFocus()"
                         selectScene="textEditor.setFocus()"
                         context="{context}"/>
    </s:VGroup>
```

编辑器控件

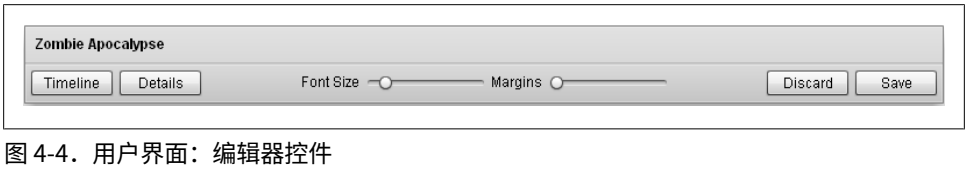


图 4-4．用户界面：编辑器控件

类

Controls.mxml

职责

- 以 Panel 为基础，采用垂直布局，并用所选 StoryVO 的名称作为标题。
- 在控制栏中声明用于放弃或保存编辑的按钮。

- 在控制栏中声明用于展开 Timeline 和 Details 组件的按钮。
- 在控制栏中声明滑块，以控制文本编辑器的字号和边距。
- 声明 Timeline 和 Details 组件，并根据按钮状态控制它们是否可见、是否参与布局。
- 未选中“Timeline”和“Details”按钮时，仅显示 Panel 标题栏和控制栏。
- 公开可绑定的公共属性，分别表示字号和边距百分比。
- 公开可绑定的公共属性，用于设置 SelectionContext。
- 向需要 SelectionContext 的子组件提供该对象。
- 按下“Discard”（放弃）或“Save”（保存）按钮时，派发相应的 AppEvent。
- 声明 Flex 元数据，说明该组件会派发 SelectScene 和 SelectDraft 类型的 AppEvent。

协作关系

Controls 组件了解并控制自己的直接子组件：自定义 Timeline 和 Details 组件，以及在控制栏区域声明的 Flex Panel、Button、Label 和 HSlider 组件。它还了解 AppEvent 类，在文本被编辑时会创建并派发这种事件。它也了解 SelectionContext 类，并在方法和绑定表达式中引用该类。

只有父组件 Editor 了解 Controls 组件。

代码

```
<?xml version="1.0" encoding="utf-8"?>
<!-- 控件 -->
<s:Panel xmlns:fx="http://ns.adobe.com/mxml/2009"
         xmlns:s="library://ns.adobe.com/flex/spark"
         xmlns:timeline="com.futurescale.sa.view.component.timeline.*"
         xmlns:details="com.futurescale.sa.view.component.details.*"
         title="{(context.series) ? context.series.name : context.story.name }"
         width="100%" minHeight="0">

    <fx:Metadata>
        [Event(name="selectScene", type="com.futurescale.sa.view.event.AppEvent")]
        [Event(name="selectDraft", type="com.futurescale.sa.view.event.AppEvent")]
    </fx:Metadata>

    <fx:Script>
        <![CDATA[
            import com.futurescale.sa.view.context.SelectionContext;
            import com.futurescale.sa.view.event.AppEvent;
```

```
        [Bindable] public var editFontSize:Number=16;
        [Bindable] public var editMarginPct:Number=0;

        // 选择上下文
        [Bindable] public var context:SelectionContext;

        private function discardChanges():void
        {
            var event:AppEvent;
            if ( context.story ) {
                event = new AppEvent( AppEvent.DISCARD_STORY, context.story );
            } else if ( context.series ) {
                event = new AppEvent( AppEvent.DISCARD_SERIES, context.series );
            }
            dispatchEvent( event );
        }

        private function saveChanges():void
        {
            var event:AppEvent;
            if ( context.story ) {
                event = new AppEvent( AppEvent.SAVE_STORY, context.story );
            } else if ( context.series ) {
                event = new AppEvent( AppEvent.SAVE_SERIES, context.series );
            }
            dispatchEvent( event );
        }

    </fx:Script>

    <!-- 详情 -->
    <details:Details id="details"
                     context="{context}"
                     width="100%" height="100%"
                     visible="{detailsButton.selected}"
                     includeInLayout="{detailsButton.selected}"/>

    <!-- 时间线 -->
    <timeline:Timeline id="timeline"
                       context="{context}"
                       story="{context.story}"
                       width="100%" height="100%"
                       visible="{timelineButton.selected}"
                       includeInLayout="{timelineButton.selected}"/>

    <!-- 控制栏 -->
    <s:controlBarContent>

        <!-- 切换时间线 -->
        <s:ToggleButton id="timelineButton" label="Timeline"/>

        <!-- 切换详情 -->
        <s:ToggleButton id="detailsButton" label="Details"/>

        <!-- 间隔 -->
```

```
            <s:Spacer width="100%"/>

            <!-- 字号 -->
            <s:Label text="Font Size"/>
            <s:HSlider id="fontSlider" change="editFontSize=fontSlider.value"
                       minimum="12" maximum="48" value="{editFontSize}"/>

            <!-- 边距大小 -->
            <s:Label text="Margins"/>
            <s:HSlider id="marginSlider" change="editMarginPct=marginSlider.value"
                       minimum="0" maximum="75" value="{editMarginPct}"/>

            <!-- 间隔 -->
            <s:Spacer width="100%"/>

            <!-- 放弃或保存修改 -->
            <s:Button label="Discard" click="discardChanges()"/>
            <s:Button label="Save" click="saveChanges()"/>

        </s:controlBarContent>

        <!-- 面板布局 -->
        <s:layout>
            <s:VerticalLayout gap="0"/>
        </s:layout>

        <!-- 控制栏布局 -->
        <s:controlBarLayout>
            <s:HorizontalLayout verticalAlign="middle"
                                paddingLeft="5" paddingRight="5"
                                paddingBottom="5" paddingTop="5"/>
        </s:controlBarLayout>

    </s:Panel>
```

详情组件



图 4-5．用户界面：展开 Details 组件的编辑器控件

类

Details.mxml

职责

- 声明 ItemInfo 和 Notes 组件。
- 公开可绑定的公共属性，用于设置 SelectionContext。
- 向需要 SelectionContext 的子组件提供该对象。

协作关系

Details 组件了解并控制自己的直接子组件，即自定义 ItemInfo 和 Notes 组件。它还了解 SelectionContext 类，并在方法和绑定表达式中引用该类。

目前只有父组件 Controls 了解 Details 组件。在将来的迭代中，Chooser 组件也会了解它；这样，当用户选中顶层 VO 时，就能直接修改项目信息、添加笔记，而无须进入 Editor。

代码

```
<?xml version="1.0" encoding="utf-8"?>
<!-- 详情 -->
<s:HGroup xmlns:fx="http://ns.adobe.com/mxml/2009"
          xmlns:s="library://ns.adobe.com/flex/spark"
          xmlns:details="com.futurescale.sa.view.component.details.*"
          width="100%" height="100%">

    <fx:Script>
        <![CDATA[
            import com.futurescale.sa.view.context.SelectionContext;

            // 选择上下文
            [Bindable] public var context:SelectionContext;


    </fx:Script>

    <!-- 项目信息 -->
    <details:ItemInfo context="{context}" width="50%" height="100%" />

    <!-- 笔记 -->
    <details:Notes context="{context}" width="50%" height="100%" />

</s:HGroup>
```

项目信息组件

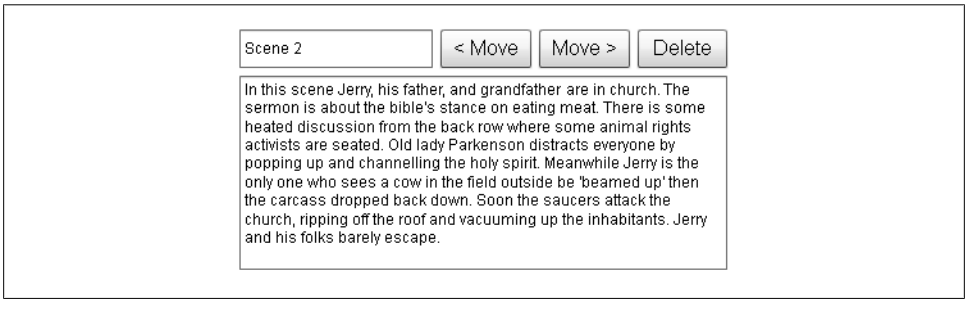


图 4-6．用户界面：项目信息组件

类

`ItemInfo.mxml`

职责

- 声明 TextInput 和 TextArea，分别显示所选项目（一个 ValueObject）的名称和描述。
- 响应用户编辑，更新所选项目的名称和描述。
- 声明按钮，用于删除所选项目，或调整它在同级项目列表中的顺序；这些功能留待后续迭代实现。
- 仅在所选项目不是顶层 VO（CastVO、MilieuVO、StoryVO 或 SeriesVO）时，显示“Move”（移动）和“Delete”（删除）按钮。
- 公开可绑定的公共属性，用于设置 SelectionContext。

协作关系

ItemInfo 组件了解并控制自己的直接子组件，即 Flex Button、TextInput 和 TextArea 组件。它还了解 SelectionContext 类，并在方法和绑定表达式中引用该类。它也了解顶层 VO（CastVO、MilieuVO、StoryVO 或 SeriesVO），因为选中这些对象时，必须隐藏“Move”和“Delete”按钮。

只有父组件 Details 了解 ItemInfo 组件。

代码

```
<?xml version="1.0" encoding="utf-8"?>
<!-- 项目信息 -->
<s:VGroup xmlns:fx="http://ns.adobe.com/mxml/2009"
          xmlns:s="library://ns.adobe.com/flex/spark"
          xmlns:details="com.futurescale.sa.view.component.details.*"
          width="100%" height="100%" paddingTop="5" paddingBottom="5"
```

```
          paddingLeft="5" paddingRight="5">
    <fx:Script>
        <![CDATA[
            import com.futurescale.sa.model.vo.CastVO;
            import com.futurescale.sa.model.vo.MilieuVO;
            import com.futurescale.sa.model.vo.SeriesVO;
            import com.futurescale.sa.model.vo.StoryVO;
            import com.futurescale.sa.model.vo.ValueObject;
            import com.futurescale.sa.view.context.SelectionContext;

            // 选择上下文
            [Bindable] public var context:SelectionContext;

            // 编辑名称时，更新所选项目的名称
            private function nameEdit():void
            {
                if (context.selectedItem) context.selectedItem.name = itemName.text;
            }

            // 编辑描述时，更新所选项目的描述
            private function descEdit():void
            {
                if (context.selectedItem) context.selectedItem.description =
itemDesc.text;
            }

            // 根据所选项目，显示或隐藏移动和删除按钮
            private function showButtons( vo:ValueObject ):Boolean
            {
                return (!(vo is StoryVO) &&
                        !(vo is SeriesVO) &&
                        !(vo is CastVO) &&
                        !(vo is MilieuVO) );
            }

    </fx:Script>

    <s:HGroup width="100%">

        <!-- 项目名称 -->
        <s:TextInput id="itemName" width="100%" height="30" prompt="Name"
                     change="nameEdit()" text="{context.selectedItem.name}"/>

        <!-- 将项目左移 -->
        <s:Button label="&lt; Move" fontSize="16" height="100%"
                  visible="{showButtons( context.selectedItem )}"
                  includeInLayout="{showButtons( context.selectedItem )}"/>

        <!-- 将项目右移 -->
        <s:Button label="Move &gt;" fontSize="16" height="100%"
                  visible="{showButtons( context.selectedItem )}"
                  includeInLayout="{showButtons( context.selectedItem )}"/>

        <!-- 删除项目 -->
        <s:Button label="Delete" fontSize="16" height="100%"
```

```
                visible="{showButtons( context.selectedItem )}"
                includeInLayout="{showButtons( context.selectedItem )}"/>


    </s:HGroup>

    <!-- 项目描述 -->
    <s:TextArea id="itemDesc" width="100%" height="100%"
                change="descEdit()" prompt="Description"
                text="{context.selectedItem.description}"/>

</s:VGroup>
```

笔记组件



图 4-7．用户界面：笔记组件

类

Notes.mxml

职责

- 声明 Flex Label 和 List，用于显示所选项目的 Note 列表。
- 声明 Flex Button，用于添加和删除 Note。
- 声明 Flex TextArea 和 TextInput，用于编辑所选 Note。
- 声明 Flex Button，用于启动浏览器，查看所选 Note 的 URL。
- 提供标签函数，为列表中的 Note 生成名称，因为 Note 没有 name 字段。
- 在列表中选中 Note 时，填充表单字段。
- 响应用户编辑，更新所选 Note 的 text 和 url。
- 按下“Add Note”（+，添加笔记）按钮时，派发相应事件；删除功能留待后续迭代实现。
- 公开可绑定的公共属性，用于设置 SelectionContext。

· 将所选项目中保存 NoteVO 的 Vector 包装为 ArrayCollection，供 List 的 dataProvider 使用。

协作关系

Notes 组件了解并控制自己的直接子组件，即各个 Flex Button、TextInput、List 和 TextArea 组件。它还了解 SelectionContext 类，并在方法和绑定表达式中引用该类。它也了解 NoteVO、ValueObject 和 AppEvent 类，并会派发用于添加和选择 Note 的 AppEvent。

只有父组件 Details 了解 Notes 组件。

代码

```
<?xml version="1.0" encoding="utf-8"?>
<!-- 笔记 -->
<s:HGroup xmlns:fx="http://ns.adobe.com/mxml/2009"
          xmlns:s="library://ns.adobe.com/flex/spark"
          xmlns:details="com.futurescale.sa.view.component.details.*"
          width="100%" height="100%">
    <fx:Script>
        <![CDATA[
            import com.futurescale.sa.model.vo.NoteVO;
            import com.futurescale.sa.model.vo.ValueObject;
            import com.futurescale.sa.view.context.SelectionContext;
            import com.futurescale.sa.view.event.AppEvent;

            import mx.collections.ArrayCollection;

             // 选择上下文
            [Bindable] public var context:SelectionContext;

            // 将 Vector 包装为集合，供列表使用。
            public function wrapNotes( vo:ValueObject ):ArrayCollection
            {
                var notes:Array = new Array();
                for each (var note:NoteVO in vo.notes) {
                    notes.push(note);
                }
                return new ArrayCollection( notes );
            }

            // 为所选项目添加笔记
            private function addNote():void
            {
                var event:AppEvent = new AppEvent( AppEvent.ADD_NOTE );
                event.data = context.selectedItem;
                dispatchEvent( event );
            }

            // 从所选项目的笔记列表中选择一条笔记
            private function selectNote():void
```

```
        {
            var event:AppEvent = new AppEvent( AppEvent.SELECT_NOTE );
            event.data = noteList.selectedItem;
            dispatchEvent( event );
        }

        // 为列表中的笔记提供标签
        private function noteLabelFunction(item:Object):String
        {
            var label:String = "Note ";
            var notes:Vector.<NoteVO> = context.selectedItem.notes;
            for ( var i:int = 0; i<notes.length; i++ ) {
                if ( notes[i].xml === NoteVO(item).xml ) {
                    label += String(i+1);
                    break;
                }
            }
            return label;
        }

        // 编辑文本时，更新所选笔记的文本
        private function textEdit():void
        {
            if (context.note) context.note.text = noteText.text;
        }

        // 编辑 URL 时，更新所选笔记的 URL
        private function urlEdit():void
        {
            if (context.note) context.note.url = noteURL.text;
        }

        // 打开浏览器，查看所选笔记的 URL
        private function openURL():void
        {
            var urlRequest:URLRequest = new URLRequest(context.note.url);
            navigateToURL(urlRequest);
        }

</fx:Script>

<!-- 笔记列表管理 -->
<s:VGroup height="100%"
          paddingRight="0" paddingLeft="0"
          paddingBottom="5" paddingTop="5">

    <!-- 标签 -->
    <s:HGroup height="30" width="100%" verticalAlign="bottom" >
        <s:Label text="Notes" fontSize="16"/>
    </s:HGroup>

    <!-- 笔记列表 -->
    <s:List id="noteList" width="100%" height="100%"
            dataProvider="{ wrapNotes( context.selectedItem ) }"
            change="selectNote()"
```

```
            labelFunction="noteLabelFunction"/>

        <!-- 笔记按钮 -->
        <s:HGroup width="100%">
            <s:Button label="-"  width="50%"
                      enabled="{noteList.selectedItem != null}"/>
            <s:Button label="+" width="50%" click="addNote()"/>
        </s:HGroup>

    </s:VGroup>

    <!-- 笔记表单 -->
    <s:VGroup height="100%" width="100%" enabled="{context.note != null}"
              paddingRight="5" paddingLeft="5"
              paddingBottom="5" paddingTop="5">

        <!-- 笔记文本 -->
        <s:TextArea id="noteText" text="{context.note.text}"
                    prompt="Note Text..." change="textEdit()"
                    width="100%" height="100%"/>

        <!-- 笔记 URL -->
        <s:HGroup width="100%"
                  paddingRight="0" paddingLeft="0"
                  paddingBottom="0" paddingTop="0">

            <!-- URL 输入 -->
            <s:TextInput id="noteURL" width="100%" prompt="Note URL..."
                         text="{context.note.url}" change="urlEdit()"  />

            <!-- 前往按钮 -->
            <s:Button label="Go" click="openURL()"
                      enabled="{context.note.url.length != 0}" />

        </s:HGroup>

    </s:VGroup>
```

时间线组件



图 4-8．用户界面：展开 Timeline 组件的编辑器控件

类

Timeline.mxml

职责

- 以 Flex Scroller 类为基础。
- 声明 Flex HGroup，用于容纳可滚动的内容。
- 公开公共属性，用于设置要显示的 StoryVO。
- 设置 StoryVO 时，创建 StoryTile，并用它替换 HGroup 现有的全部子组件。
- 提供标签函数，为 List 中的笔记生成名称，因为 Note 没有 name 字段。
- 公开可绑定的公共属性，用于设置 SelectionContext。
- 向显示的 StoryTile 提供 SelectionContext 对象。

协作关系

Timeline 组件了解并控制自己的直接子组件，即 Flex HGroup 和自定义 StoryTile 组件。它还了解 SelectionContext 类，并将其对象传给 StoryTile；也了解 StoryVO，用它创建 StoryTile。

只有父组件 Controls 了解 Timeline 组件。

代码

```
<?xml version="1.0" encoding="utf-8"?>
<!-- 时间线 -->
<s:Scroller xmlns:fx="http://ns.adobe.com/mxml/2009"
         xmlns:s="library://ns.adobe.com/flex/spark">

    <fx:Script>
        <![CDATA[
            import com.futurescale.sa.model.vo.StoryVO;
            import com.futurescale.sa.view.context.SelectionContext;

            // 选择上下文
            [Bindable] public var context:SelectionContext;

            // 故事
            public function set story( storyVO:StoryVO ):void {
                _story = storyVO;
                storyGroup.removeAllElements();
                if (storyVO) {
                    var storyTile:StoryTile = new StoryTile();
                    storyTile.context       = context;
                    storyTile.story         = story;
                    storyGroup.addElement( storyTile );
                }
            }
```

```
            public function get story( ):StoryVO  {
                return _story;
            }
            private var _story:StoryVO;

        </fx:Script>

        <!-- 故事组 -->
        <s:HGroup id="storyGroup" width="100%" height="100%"
                  paddingRight="5" paddingBottom="5"
                  paddingLeft="5"/>

    </s:Scroller>
```

故事磁贴

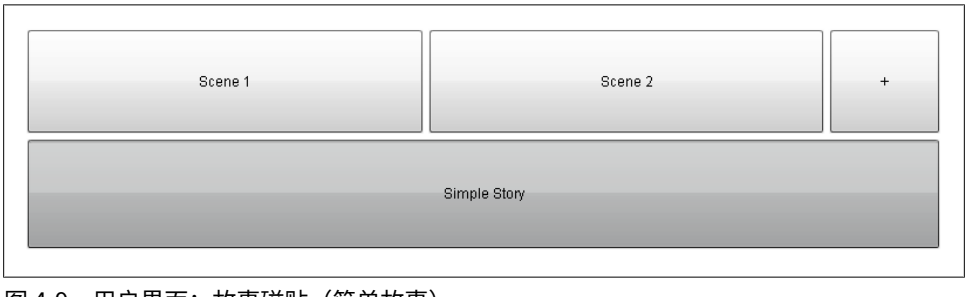


图 4-9．用户界面：故事磁贴（简单故事）



图 4-10．用户界面：故事磁贴（普通故事）

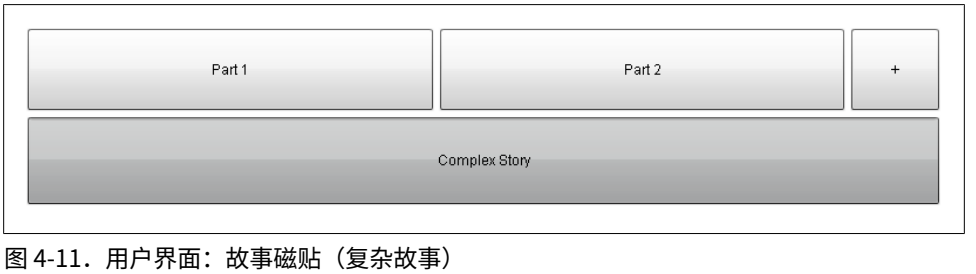


图 4-11．用户界面：故事磁贴（复杂故事）

类

StoryTile.mxml

职责

- 以 Flex VGroup 类为基础。
- 声明 Flex HGroup，用于容纳 PartTile、ChapterTile 或 SceneTile 实例。
- 声明 Flex ToggleButton，用于选择 StoryVO。
- 公开公共属性，用于设置要显示的 StoryVO。
- 选中 StoryVO 时，创建相应的 PartTile、ChapterTile 或 SceneTile 实例，替换 HGroup 现有的全部子组件。
- 将适当的磁贴组件加入 HGroup 后，在末尾添加一个 AddTile，并设置它，使其派发添加另一个子项所需的事件。
- 在 StoryVO 被选中或取消选择时，移除并重新创建全部磁贴，从而产生展开或折叠这些磁贴的效果。
- 公开可绑定的公共属性，用于设置 SelectionContext。
- 向显示的 PartTile、ChapterTile 或 SceneTile 实例提供 SelectionContext 对象。

协作关系

StoryTile 组件了解并控制自己的直接子组件，即 Flex HGroup，以及自定义 PartTile、ChapterTile、SceneTile 和 AddTile 组件。它了解 SelectionContext 类，并将其对象传给各个磁贴组件。它也了解 StoryVO、PartVO、ChapterVO 和 SceneVO，以便根据故事类型，用这些对象创建适当的磁贴组件。

目前只有父组件 Timeline 了解 StoryTile 组件。在后续迭代中，当为 Timeline 组件添加 Series 功能时，SeasonTile 组件也将了解它。

代码

```
<?xml version="1.0" encoding="utf-8"?>
<!-- 故事磁贴 -->
<s:VGroup xmlns:fx="http://ns.adobe.com/mxml/2009"
          xmlns:s="library://ns.adobe.com/flex/spark"
          height="100%" width="100%">

    <fx:Script>
        <![CDATA[
            import com.futurescale.sa.model.vo.ChapterVO;
            import com.futurescale.sa.model.vo.PartVO;
            import com.futurescale.sa.model.vo.SceneVO;
            import com.futurescale.sa.model.vo.StoryVO;
            import com.futurescale.sa.view.context.SelectionContext;
```

```
import com.futurescale.sa.view.event.AppEvent;

// 选择上下文
[Bindable] public var context:SelectionContext;

// 故事。
[Bindable] public function set story( storyVO:StoryVO ):void
{
    _story = storyVO;
}
public function get story():StoryVO
{
    return _story;
}
private var _story:StoryVO;

// 根据选中的故事移除或添加磁贴
private function changeStorySelection( selection:Boolean ):Boolean
{
    if (!selection) {
        removeTiles();
    } else {
        createTiles();
    }
    return selection;
}

// 创建磁贴
private function createTiles():void
{
    removeTiles();
    var addTile:AddTile = new AddTile();

    if ( story.useScenes )
    {
        // 创建 SceneTile
        var scenes:Vector.<SceneVO> = story.scenes;
        for (var s:int=0; s< scenes.length; s++ ) {
            var sceneTile:SceneTile = new SceneTile();
            sceneTile.context = context;
            sceneTile.scene   = scenes[s];
            tileGroup.addElement( sceneTile );
        }

        // 创建“添加场景”磁贴
        addTile.addType     = AppEvent.ADD_SCENE;
        addTile.addTarget   = story;
    }
    else if ( story.useChapters )
    {
        // 创建 ChapterTile
        var chapters:Vector.<ChapterVO> = story.chapters;
        for (var c:int=0; c<chapters.length; c++ ) {
            var chapterTile:ChapterTile = new ChapterTile();
            chapterTile.context = context;
```

```
                chapterTile.chapter = chapters[c];
                tileGroup.addElement( chapterTile );
            }

            // 创建“添加章”磁贴
            addTile.addType     = AppEvent.ADD_CHAPTER;
            addTile.addTarget   = story;

        }
        else if ( story.useParts )
        {
            // 创建 PartTile
            var parts:Vector.<PartVO> = story.parts;
            for (var p:int=0; p< parts.length; p++ ) {
                var partTile:PartTile = new PartTile();
                partTile.context = context;
                partTile.part    = parts[p];
                tileGroup.addElement( partTile );
            }

            // 创建“添加部”磁贴
            addTile.addType     = AppEvent.ADD_PART;
            addTile.addTarget   = story;
        }
        tileGroup.addElement( addTile );
        tileGroup.percentHeight=100;
    }

    // 移除所有现有磁贴
    private function removeTiles():void
    {
        tileGroup.removeAllElements();
        tileGroup.percentHeight=0;
    }

    // 选择故事
    private function selectStory():void
    {
        var appEvent:AppEvent = new AppEvent(AppEvent.SELECT_STORY);
        appEvent.data         = story;
        dispatchEvent(appEvent);
    }


</fx:Script>

<!-- 磁贴组 -->
<s:HGroup id="tileGroup" width="100%"/>

<!-- 故事按钮 -->
<s:ToggleButton id="storyButton" click="selectStory()"
                selected="{changeStorySelection(context.story.uid == story.uid)}"
                height="100%" width="100%" minWidth="150" minHeight="25"
                label="{story.name}"/>
```

```
</s:VGroup>
```

部磁贴

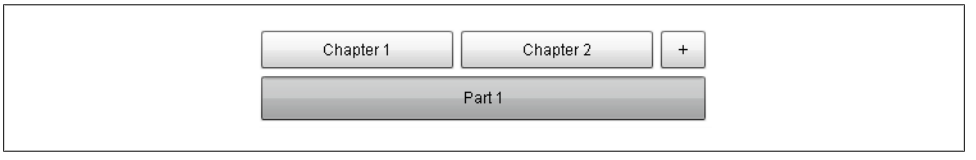


图 4-12．用户界面：部磁贴

类

PartTile.mxml

职责

- 以 Flex VGroup 类为基础。
- 声明 Flex HGroup，用于容纳 ChapterTile 实例。
- 声明 Flex ToggleButton，用于选择 PartVO。
- 公开公共属性，用于设置要显示的 PartVO。
- 选中 PartVO 时，创建 ChapterTile 实例，替换 HGroup 现有的全部子组件。
- 将适当的磁贴组件加入 HGroup 后，在末尾添加一个 AddTile，并设置它，使其派发添加另一个子项所需的事件。
- 在 PartVO 被选中或取消选择时，移除或重新创建全部磁贴，从而产生展开或折叠这些磁贴的效果。
- 公开可绑定的公共属性，用于设置 SelectionContext。
- 向显示的 ChapterTile 实例提供 SelectionContext 对象。

协作关系

PartTile 组件了解并控制自己的直接子组件，即 Flex HGroup，以及自定义 ChapterTile 和 AddTile 组件。它还了解 SelectionContext 类，并将其对象传给各个磁贴组件。它也了解 PartVO 和 ChapterVO，以便用这些对象创建适当的磁贴组件。最后，它了解 AppEvent 类，用于告知 AddTile 应当派发哪一种事件。

只有父组件 StoryTile 了解 PartTile 组件。

代码

```
<?xml version="1.0" encoding="utf-8"?>
<!-- 部磁贴 -->
```

```
<s:VGroup xmlns:fx="http://ns.adobe.com/mxml/2009"
          xmlns:s="library://ns.adobe.com/flex/spark"
          xmlns:mx="library://ns.adobe.com/flex/mx"
          height="100%" width="100%">

    <fx:Script>
        <![CDATA[
            import com.futurescale.sa.model.vo.ChapterVO;
            import com.futurescale.sa.model.vo.PartVO;
            import com.futurescale.sa.view.context.SelectionContext;
            import com.futurescale.sa.view.event.AppEvent;

            // 选择上下文
            [Bindable] public var context:SelectionContext;

            // 部
            [Bindable] public var part:PartVO;

            // 根据选中的部移除或添加磁贴
            private function changePartSelection( selection:Boolean ):Boolean
            {
                if (!selection) {
                    removeTiles();
                } else {
                    createTiles();
                }
                return selection;
            }

            // 创建磁贴
            private function createTiles():void
            {
                // 创建章磁贴
                removeTiles();
                var chapters:Vector.<ChapterVO> = part.chapters;
                for (var i:int=0; i<chapters.length; i++ ) {
                    var chapterTile:ChapterTile = new ChapterTile();
                    chapterTile.context = context;
                    chapterTile.chapter = chapters[i];
                    chapterGroup.addElement( chapterTile );
                }

                // 创建“添加章”磁贴
                var addTile:AddTile = new AddTile();
                addTile.addType     = AppEvent.ADD_CHAPTER;
                addTile.addTarget   = part;
                chapterGroup.addElement( addTile );
                chapterGroup.percentHeight=100;
            }

            // 移除磁贴
            private function removeTiles():void
            {
                chapterGroup.removeAllElements();
                chapterGroup.percentHeight=0;
```

```
            }

            // 切换部的选中状态
            private function selectPart():void
            {
                var event:AppEvent;
                if (partButton.selected) {
                    event = new AppEvent(AppEvent.SELECT_PART);
                } else {
                    event = new AppEvent(AppEvent.DESELECT_PART);
                    removeTiles();
                }
                event.data    = part;
                event.related = context.story;
                dispatchEvent(event);
            }


        </fx:Script>

        <!-- 章组 -->
        <s:HGroup id="chapterGroup" width="100%"/>

        <!-- 部按钮 -->
        <s:ToggleButton id="partButton" click="selectPart()"
                        selected="{changePartSelection(context.part.uid == part.uid)}"
                        height="100%" width="100%" minWidth="150" minHeight="25"
                        label="{part.name}"/>

    </s:VGroup>
```

章磁贴

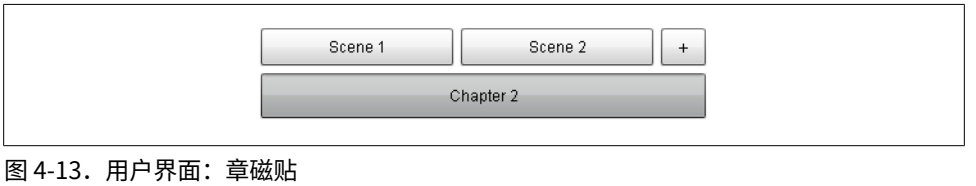


图 4-13．用户界面：章磁贴

类

ChapterTile.mxml

职责

- 以 Flex VGroup 类为基础。
- 声明 Flex HGroup，用于容纳 SceneTile 实例。
- 声明 Flex ToggleButton，用于选择 ChapterVO。
- 公开公共属性，用于设置要显示的 ChapterVO。

- 选中 ChapterVO 时，创建 SceneTile 实例，替换 HGroup 现有的全部子组件。
- 将适当的磁贴组件加入 HGroup 后，在末尾添加一个 AddTile，并设置它，使其派发添加另一个子项所需的事件。
- 在 ChapterVO 被选中或取消选择时，移除或重新创建全部磁贴，从而产生展开或折叠这些磁贴的效果。
- 公开可绑定的公共属性，用于设置 SelectionContext。
- 向显示的 SceneTile 实例提供 SelectionContext 对象。

协作关系

ChapterTile 组件了解并控制自己的直接子组件，即 Flex HGroup，以及自定义 SceneTile 和 AddTile 组件。它还了解 SelectionContext 类，并将其对象传给各个磁贴组件。它也了解 SceneVO 和 ChapterVO，以便用这些对象创建适当的磁贴组件。最后，它了解 AppEvent 类，用于告知 AddTile 应当派发哪一种事件。

只有两种可能的父组件 StoryTile 和 PartTile 了解 ChapterTile 组件。

代码

```
<?xml version="1.0" encoding="utf-8"?>
<!-- 章磁贴 -->
<s:VGroup xmlns:fx="http://ns.adobe.com/mxml/2009"
          xmlns:s="library://ns.adobe.com/flex/spark"
          xmlns:mx="library://ns.adobe.com/flex/mx"
          height="100%" width="100%">

    <fx:Script>
        <![CDATA[
            import com.futurescale.sa.model.vo.ChapterVO;
            import com.futurescale.sa.model.vo.SceneVO;
            import com.futurescale.sa.view.context.SelectionContext;
            import com.futurescale.sa.view.event.AppEvent;

            // 选择上下文
            [Bindable] public var context:SelectionContext;

            // 章
            [Bindable] public var chapter:ChapterVO;

            // 根据选中的章移除或添加磁贴
            private function changeChapterSelection( selection:Boolean ):Boolean
            {
                if (!selection) {
                    removeTiles();
                } else {
                    createTiles();
                }
```

```
            return selection;
        }

        // 创建磁贴
        private function createTiles():void
        {
            // 创建 SceneTile
            removeTiles();
            var scenes:Vector.<SceneVO> = chapter.scenes;
            for (var i:int=0; i<scenes.length; i++ ) {
                var sceneTile:SceneTile = new SceneTile();
                sceneTile.context = context;
                sceneTile.scene   = scenes[i];
                sceneGroup.addElement( sceneTile );
            }

            // 创建“添加场景”磁贴
            var addTile:AddTile = new AddTile();
            addTile.addType     = AppEvent.ADD_SCENE;
            addTile.addTarget   = chapter;
            addTile.addArgument = (context.part)?context.part:context.story;
            sceneGroup.addElement( addTile );
            sceneGroup.percentHeight=100;
        }

        // 移除磁贴
        private function removeTiles():void
        {
            sceneGroup.removeAllElements();
            sceneGroup.percentHeight=0;
        }

        // 切换章的选中状态
        private function selectChapter():void
        {
            var event:AppEvent;
            if (chapterButton.selected) {
                event = new AppEvent(AppEvent.SELECT_CHAPTER);
            } else {
                event = new AppEvent(AppEvent.DESELECT_CHAPTER);
                removeTiles();
            }
            event.data    = chapter;
            event.related = (context.part)?context.part:context.story;
            dispatchEvent(event);
        }


    </fx:Script>

    <!-- 场景组 -->
    <s:HGroup id="sceneGroup" width="100%"/>

    <!-- 章按钮 -->
    <s:ToggleButton id="chapterButton" click="selectChapter()"
```

```
                    selected="{changeChapterSelection(context.chapter.uid ==
chapter.uid)}"
                    height="100%" width="100%" minWidth="150" minHeight="25"
                    label="{chapter.name}"/>

</s:VGroup>
```

场景磁贴

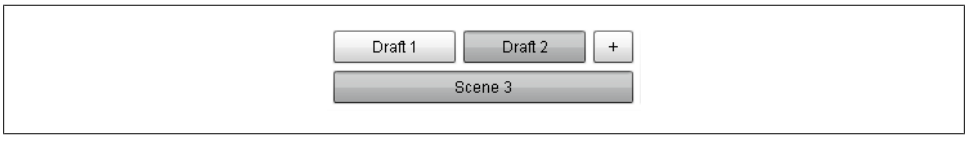


图 4-14．用户界面：场景磁贴

类

SceneTile.mxml

职责

- 以 Flex VGroup 类为基础。
- 声明 Flex HGroup，用于容纳 DraftTile 实例。
- 声明 Flex ToggleButton，用于选择 SceneVO。
- 公开公共属性，用于设置要显示的 SceneVO。
- 选中 SceneVO 时，创建 DraftTile 实例，替换 HGroup 现有的全部子组件。
- 将适当的磁贴组件加入 HGroup 后，在末尾添加一个 AddTile，并设置它，使其派发添加另一个子项所需的事件。
- 在 SceneVO 被选中或取消选择时，移除或重新创建全部磁贴，从而产生展开或折叠这些磁贴的效果。
- 公开可绑定的公共属性，用于设置 SelectionContext。
- 向显示的 DraftTile 实例提供 SelectionContext 对象。

协作关系

SceneTile 组件了解并控制自己的直接子组件，即 Flex HGroup，以及自定义 DraftTile 和 AddTile 组件。它还了解 SelectionContext 类，并将其对象传给各个磁贴组件。它也了解 SceneVO 和 DraftVO，以便用这些对象创建适当的磁贴组件。最后，它了解 AppEvent 类，用于告知 AddTile 应当派发哪一种事件。

只有三种可能的父组件 StoryTile、PartTile 和 ChapterTile 了解 SceneTile 组件。

代码

```
<?xml version="1.0" encoding="utf-8"?>
<!-- 场景磁贴 -->
<s:VGroup xmlns:fx="http://ns.adobe.com/mxml/2009"
          xmlns:s="library://ns.adobe.com/flex/spark"
          height="100%" width="100%">

    <fx:Script>
        <![CDATA[
            import com.futurescale.sa.model.vo.DraftVO;
            import com.futurescale.sa.model.vo.SceneVO;
            import com.futurescale.sa.view.context.SelectionContext;
            import com.futurescale.sa.view.event.AppEvent;

            // 选择上下文
            [Bindable] public var context:SelectionContext;

            // 场景
            [Bindable] public var scene:SceneVO;

            // 根据选中的场景移除或添加磁贴
            private function changeSceneSelection( selection:Boolean ):Boolean
            {
                if (!selection) {
                    removeTiles();
                } else {
                    createTiles();
                }
                return selection;
            }

            // 创建磁贴
            private function createTiles():void
            {
                // 创建草稿磁贴
                removeTiles();
                var drafts:Vector.<DraftVO> = scene.drafts;
                for (var i:int=0; i<drafts.length; i++ ) {
                    var draftTile:DraftTile = new DraftTile();
                    draftTile.context = context;
                    draftTile.draft   = drafts[i];
                    draftGroup.addElement( draftTile );
                }

                // 创建“添加草稿”磁贴
                var addTile:AddTile = new AddTile();
                addTile.addType     = AppEvent.ADD_DRAFT;
                addTile.addTarget   = scene;
                draftGroup.addElement( addTile );
                draftGroup.percentHeight=100;
            }

            // 移除磁贴
            private function removeTiles():void
            {
```

```
                draftGroup.removeAllElements();
                draftGroup.percentHeight=0;
            }

            // 切换场景的选中状态
            private function selectScene():void
            {
                var event:AppEvent;
                if (sceneButton.selected) {
                    event = new AppEvent(AppEvent.SELECT_SCENE);
                } else {
                    event = new AppEvent(AppEvent.DESELECT_SCENE);
                    removeTiles();
                }
                event.data    = scene;
                event.related = (context.chapter)?context.chapter:context.story;
                dispatchEvent(event);
            }

        </fx:Script>

        <!-- 草稿组 -->
        <s:HGroup id="draftGroup" width="100%"/>

        <!-- 场景按钮 -->
        <s:ToggleButton id="sceneButton" click="selectScene()"
                        selected="{changeSceneSelection(context.scene.uid == scene.uid)}"
                        height="100%" width="100%" minWidth="150" minHeight="25"
                        label="{scene.name}"/>

    </s:VGroup>
```

草稿磁贴



图 4-15．用户界面：草稿磁贴

类

DraftTile.mxml

职责

- 以 Flex VGroup 类为基础。
- 声明 Flex HGroup，用于填充高度，使本磁贴与其他磁贴恰好等高，因为本磁贴没有子项。
- 声明 Flex ToggleButton，用于选择 DraftVO。

- 公开公共属性，用于设置要显示的 DraftVO。
- 单击 ToggleButton 时，应设置所选 SceneVO 的 currentDraft，并派发相应事件以选择 DraftVO。
- 公开可绑定的公共属性，用于设置 SelectionContext。
- 向显示的 DraftTile 实例提供 SelectionContext 对象。

协作关系

DraftTile 组件了解并控制自己的直接子组件，即 Flex HGroup 和 Button 组件。它还了解 SelectionContext、DraftVO 和 AppEvent 类。

只有父组件 SceneTile 了解 DraftTile 组件。

代码

```
<?xml version="1.0" encoding="utf-8"?>
<!-- 草稿磁贴 -->
<s:VGroup xmlns:fx="http://ns.adobe.com/mxml/2009"
          xmlns:s="library://ns.adobe.com/flex/spark"
          xmlns:component="com.futurescale.sa.view.component.*"
          height="100%" width="100%">

    <fx:Script>
        <![CDATA[
            import com.futurescale.sa.model.vo.DraftVO;
            import com.futurescale.sa.model.vo.SceneVO;
            import com.futurescale.sa.view.context.SelectionContext;
            import com.futurescale.sa.view.event.AppEvent;

            // 选择上下文
            [Bindable] public var context:SelectionContext;

            // 草稿
            [Bindable] public var draft:DraftVO;

            // 切换草稿的选中状态
            public function selectDraft():void
            {
                context.scene.currentDraft = draft;
                var appEvent:AppEvent = new AppEvent( AppEvent.SELECT_DRAFT );
                appEvent.data = draft;
                appEvent.related = context.scene;
                dispatchEvent(appEvent);
            }


    </fx:Script>

    <!-- 填充组 -->
    <s:HGroup width="100%"/>
```

```
        <!-- 草稿按钮 -->
        <s:ToggleButton id="draftButton" click="selectDraft()"
                        selected="{context.draft.uid == draft.uid}"
                        height="100%" width="100%" minWidth="75" minHeight="25"
                        label="{draft.name}"/>

    </s:VGroup>
```

选择上下文

类

`SelectionContext.as`

职责

- 公开公共方法，用于选择各种 ValueObject 子类的对象。这些方法应确保取消选择相关项目，或 Story、Series 层次结构中位于所选项目之下的项目。例如，选择 Part 时，应清除当前 Chapter、Scene、Draft 和 Note 的选择，但不清除 Story、Season、Series 的选择，也不清除 Cast 和 Milieu 的选择。选择 Story 时，则同时选择它的 Cast 和 Milieu，并清除 Part、Chapter、Scene、Draft 和 Note 的选择。
- 公开一个可绑定、类型为 ValueObject 的公共属性 selectedItem；该属性应始终指向选择方法当前选中的项目。因此，selectStory(story:StoryVO) 不仅将 story 属性设置为传入的 StoryVO，也会同时设置 selectedItem:ValueObject 属性。selectedItem 属性供 ItemInfo 和 Notes 视图组件使用，因为它们可以处理任意 ValueObject。
- 公开可绑定的公共属性，用于读取当前选中的 CastVO、ChapterVO、CharacterVO、DraftVO、CastVO、MilieuVO、NoteVO、PartVO、SceneVO、SeasonVO、SeriesVO、SettingVO、StoryVO 和 ValueObject。

协作关系

SelectionContext 类了解 CastVO、ChapterVO、CharacterVO、DraftVO、CastVO、MilieuVO、NoteVO、PartVO、SceneVO、SeasonVO、SeriesVO、SettingVO、StoryVO 和 ValueObject 类，并为这些类维护整个系统对操作中所选项目的统一认识。

几乎所有视图组件都了解 SelectionContext 类，并根据其属性确定与选择有关的行为。多种 Command 也了解 SelectionContext 类，包括 AddItemCommand、DeleteItemCommand、ApplySelectionCommand、RemoveSelectionCommand、ApplyChangesCommand、DiscardChangesCommand、StartupCommand 等。

代码

```
package com.futurescale.sa.view.context
{
    import com.futurescale.sa.model.vo.CastVO;
    import com.futurescale.sa.model.vo.ChapterVO;
    import com.futurescale.sa.model.vo.CharacterVO;
    import com.futurescale.sa.model.vo.DraftVO;
    import com.futurescale.sa.model.vo.MilieuVO;
    import com.futurescale.sa.model.vo.NoteVO;
    import com.futurescale.sa.model.vo.PartVO;
    import com.futurescale.sa.model.vo.SceneVO;
    import com.futurescale.sa.model.vo.SeasonVO;
    import com.futurescale.sa.model.vo.SeriesVO;
    import com.futurescale.sa.model.vo.SettingVO;
    import com.futurescale.sa.model.vo.StoryVO;
    import com.futurescale.sa.model.vo.ValueObject;

    /**
     * 用户界面中当前选中的项目。
     *
     * 调用选择方法时，Series/Story 层次结构中
     * 位于所选项目之下的项目，以及所选 Note、
     * Cast 和 Milieu，都会按需要自动设为 null。


     */
    [Bindable] public class SelectionContext
    {
        public static const NAME:String = "SelectionContext";

        public function selectSeries( series:SeriesVO ):void
        {
            this.selectedItem = series;
            this.series  = series;
            this.season  = null;
            this.story   = null;
            this.part    = null;
            this.chapter = null;
            this.scene   = null;
            this.draft   = null;
            this.note    = null;
            if ( series ) {
                this.cast   = series.cast;
                this.milieu = series.milieu;
                this.setting   = null;
                this.character = null;
            }
        }

        public function selectSeason( season:SeasonVO ):void
        {
            this.selectedItem = season;
            this.season  = season;
            this.story   = null;
            this.part    = null;
```

```
        this.chapter = null;
        this.scene   = null;
        this.draft   = null;
        this.note    = null;
    }

    public function selectStory( story:StoryVO ):void
    {
        this.selectedItem = story;
        this.story   = story;
        this.part    = null;
        this.chapter = null;
        this.scene   = null;
        this.draft   = null;
        this.note    = null;
        if ( story ) {
            this.cast   = story.cast;
            this.milieu = story.milieu;
            this.setting   = null;
            this.character = null;
        }
        selectedText = (story)? story.getText(true) : "";
    }

    public function selectPart( part:PartVO ):void
    {
        this.selectedItem = part;
        this.part    = part;
        this.chapter = null;
        this.scene   = null;
        this.draft   = null;
        this.note    = null;
        selectedText = (part)? part.getText(true) : "";
    }

    public function selectChapter( chapter:ChapterVO ):void
    {
        this.selectedItem = chapter;
        this.chapter = chapter;
        this.scene   = null;
        this.draft   = null;
        this.note    = null;
        selectedText = (chapter)? chapter.getText(true) : "";
    }

    public function selectScene( scene:SceneVO ):void
    {
        this.selectedItem = scene;
        this.scene   = scene;
        this.draft   = null;
        this.note    = null;
        selectedText = (scene)? scene.getText(true) : "";
        if ( scene && scene.currentDraft ) selectDraft( scene.currentDraft );
    }
```

```
        public function selectDraft( draft:DraftVO ):void
        {
            this.scene.currentDraft = draft;
            this.draft   = draft;
            this.note    = null;
            selectedText = (draft)? draft.text : "";
        }

        public function selectNote( note:NoteVO ):void
        {
            this.note    = note;
        }

        public function selectCharacter( character:CharacterVO ):void
        {
            this.selectedItem = character;
            this.character = character;
            this.note   = null;
        }

        public function selectSetting( setting:SettingVO ):void
        {
            this.selectedItem = setting;
            this.setting = setting;
            this.note   = null;
        }

        public var selectedText:String;
        public var selectedItem:ValueObject;

        public var series:SeriesVO;
        public var season:SeasonVO;
        public var story:StoryVO;
        public var part:PartVO;
        public var chapter:ChapterVO;
        public var scene:SceneVO;
        public var draft:DraftVO;
        public var note:NoteVO;
        public var cast:CastVO;
        public var milieu:MilieuVO;
        public var character:CharacterVO;
        public var setting:SettingVO;
    }
```

应用事件

类

AppEvent.as

职责

有些人喜欢为系统中使用的每一种 Event 都建立单独的类，这完全可行。在本应用程序中，我们将使用一个自定义 Event 类，让它支持多种不同的类型。

通过 AppEvent，我们正在建立一套由两部分组成的通信约定中的第一部分。AppEvent 类定义的常量，将用户界面产生的所有意图集中在一处表示；这些意图将在某个位置处理，可能是显示列表中更高的层级，也可能是系统更深处的 Mediator 和 Command。这套约定的另一部分是 AppConstants 文件，我们将在其中以类似方式定义 View 层和 Controller 层共享的所有通知名称。两个文件经常会出现重复内容，因此人们可能会考虑，是否应将它们合并为一个文件。答案是否定的。事件名称与通知名称并不总是一一对应，你很快就会看到这一点。最好将两者分开，让 Mediator 负责转换。采用这种由两部分组成的通信约定，也符合前面所作的承诺：可以将整个用户界面的构建交给完全不了解 PureMVC 的开发者或团队。它让界面开发者决定应派发哪些事件，而让 PureMVC 开发者决定如何最妥善地协调处理这些事件。

还请注意，我们先把动词和名词定义为私有常量，再将它们组合成用于 AppEvent 类型的公共常量。这不是必须的，但可以让事件命名更加清晰。

- 扩展 Flash Event 类。
- 定义公共静态常量，表示有效的 AppEvent 类型。
- 定义 Object 类型的公共属性，分别表示 data 和 related 数据。
- 构造函数接受 type、data 和 related 参数，并将它们设置到相应的公共属性。
- 构造函数应调用父类构造函数，传入收到的 type，并将 bubbles 和 cancelable 参数均设为 true。

协作关系

AppEvent 类只了解它所扩展的 Flash Event 类，以及用作其公共属性类型的 ActionScript Object 类。

几乎所有视图组件都了解 AppEvent 类，并根据它的公共静态类型常量派发事件。各种 Mediator 也了解 AppEvent，并监听视图组件发出的这些事件。许多 Command 同样了解它，包括 AddItemCommand、DeleteItemCommand、ApplySelectionCommand、RemoveSelectionCommand、ApplyChangesCommand 和 DiscardChangesCommand。Mediator 将 AppEvent 放入 Notification 中转发给这些 Command，Command 再解析事件，以完成自己的业务逻辑。

代码

```
package com.futurescale.sa.view.event
{
    import flash.events.Event;

    public class AppEvent extends Event
    {
        // 构成 AppEvent 类型的各个部分
        private static const ADD:String             = "add";
        private static const DELETE:String          = "delete";
        private static const MANAGE:String          = "manage";
        private static const SELECT:String          = "delect";
        private static const DESELECT:String        = "deselect";
        private static const EXPORT:String          = "export";
        private static const OUTLINE:String         = "outline";
        private static const EDIT:String            = "edit";
        private static const DISCARD:String         = "discard";
        private static const SAVE:String            = "save";
        private static const REPORT:String          = "report";
        private static const MOVE:String            = "move";

        private static const SERIES:String          = "Series";
        private static const SEASON:String          = "Season";
        private static const EPISODE:String         = "Episode";
        private static const STORY:String           = "Story";
        private static const PART:String            = "Part";
        private static const CHAPTER:String         = "Chapter";
        private static const SCENE:String           = "Scene";
        private static const DRAFT:String           = "Draft";
        private static const CAST:String            = "Cast";
        private static const CHARACTER:String       = "Character";
        private static const MILIEU:String          = "Milieu";
        private static const SETTING:String         = "Setting";
        private static const NOTE:String            = "Note";
        private static const WORDCOUNT:String       = "WordCount";
        private static const AHEAD:String           = "Ahead";
        private static const BACK:String            = "Back";

        // AppEvent 类型
        public static const ADD_SERIES:String       = ADD+SERIES;
        public static const ADD_SEASON:String       = ADD+SEASON;
        public static const ADD_EPISODE:String      = ADD+EPISODE;
        public static const ADD_STORY:String        = ADD+STORY;
        public static const ADD_PART:String         = ADD+PART;
        public static const ADD_CHAPTER:String      = ADD+CHAPTER;
        public static const ADD_SCENE:String        = ADD+SCENE;
        public static const ADD_DRAFT:String        = ADD+DRAFT;
        public static const ADD_CHARACTER:String    = ADD+CHARACTER;
        public static const ADD_SETTING:String      = ADD+SETTING;
        public static const ADD_NOTE:String         = ADD+NOTE;

        public static const DELETE_SERIES:String    = DELETE+SERIES;
        public static const DELETE_SEASON:String    = DELETE+SEASON;
        public static const DELETE_EPISODE:String   = DELETE+EPISODE;
        public static const DELETE_STORY:String     = DELETE+STORY;
```

```
public static const DELETE_PART:String      = DELETE+PART;
public static const DELETE_CHAPTER:String   = DELETE+CHAPTER;
public static const DELETE_SCENE:String     = DELETE+SCENE;
public static const DELETE_DRAFT:String     = DELETE+DRAFT;
public static const DELETE_CHARACTER:String = DELETE+CHARACTER;
public static const DELETE_SETTING:String   = DELETE+SETTING;
public static const DELETE_NOTE:String      = DELETE+NOTE;

public static const DISCARD_STORY:String    = DISCARD+STORY;
public static const DISCARD_SERIES:String   = DISCARD+SERIES;

public static const EXPORT_SERIES:String    = EXPORT+SERIES;
public static const EXPORT_STORY:String     = EXPORT+STORY;

public static const EDIT_SEREIES:String     = EDIT+SERIES;
public static const EDIT_STORY:String       = EDIT+STORY;

public static const SAVE_STORY:String       = SAVE+STORY;
public static const SAVE_SERIES:String      = SAVE+SERIES;

public static const MANAGE_SERIES:String    = MANAGE+SERIES;
public static const MANAGE_STORY:String     = MANAGE+STORY;
public static const MANAGE_CAST:String      = MANAGE+CAST;
public static const MANAGE_MILIEU:String    = MANAGE+MILIEU;

public static const MOVE_AHEAD:String       = MOVE+AHEAD;
public static const MOVE_BACK:String        = MOVE+BACK;

public static const OUTLINE_SERIES:String   = OUTLINE+SERIES;
public static const OUTLINE_STORY:String    = OUTLINE+STORY;

public static const REPORT_WORDCOUNT:String = REPORT+WORDCOUNT;

public static const SELECT_SERIES:String    = SELECT+SERIES;
public static const SELECT_SEASON:String    = SELECT+SEASON;
public static const SELECT_STORY:String     = SELECT+STORY;
public static const SELECT_PART:String      = SELECT+PART;
public static const SELECT_CHAPTER:String   = SELECT+CHAPTER;
public static const SELECT_SCENE:String     = SELECT+SCENE;
public static const SELECT_DRAFT:String     = SELECT+DRAFT;
public static const SELECT_CHARACTER:String = SELECT+CHARACTER;
public static const SELECT_SETTING:String   = SELECT+SETTING;
public static const SELECT_NOTE:String      = SELECT+NOTE;

public static const DESELECT_PART:String    = DESELECT+PART;
public static const DESELECT_CHAPTER:String = DESELECT+CHAPTER;
public static const DESELECT_SCENE:String   = DESELECT+SCENE;

public var data:Object;     // 可选的数据对象
public var related:Object;  // 可选的关联数据对象

public function AppEvent( type:String, data:Object=null, related:Object=null )
{
    super(type, true, true);
    this.data = data;
```

```
                this.related = related;
            }
        }
    }
```

第 5 章
为模型提供代理

PureMVC 框架的 Proxy 类相当简单。它只有几个在注册和移除时调用的生命周期方法；一个 Object 类型的 data 属性，以及配套的 getData 和 setData 方法；此外，它还可以发送通知，与 Mediator 进行单向通信，或触发 Command 的执行。

这个类简单得似乎毫无价值，就像一张白纸，在《麦田里的守望者》的第一个字落在纸上之前，看上去也可能如此。但正因为它足够简单，再加上 PureMVC Model 为系统中的 Proxy 提供了注册表，它实际上非常有用。它没有用多少规则约束你；作为最佳实践，它不应了解任何 View 层或 Controller 层的类，要求基本上就这些。



Proxy 通常是系统中长期存在的角色：在启动时注册，并在整个运行期间一直存在。因此，那些可能动态创建和移除的 Proxy 实例称为临时 Proxy（transient Proxy）。

通常，你会继承框架的 Proxy 类，再加入自己的功能。不过，正如你将看到的，我们的 Proxy 子类还会利用临时 Proxy 实例，跟踪所创建或加载到内存中的每个 VO。这是 Proxy 最简单的用途：将任意一份数据暂存起来，以便稍后按名称快速取回。例如，我们也完全可以在 StoryProxy 上用一个 Array 属性保存所有 StoryVO，但这样，每次查找特定对象时，都必须遍历 Array，直到找到目标，或者把 Array 当作 Java 风格的 Map，即哈希表或关联数组来使用。PureMVC Model 已经替我们完成了这些工作，而且经过了单元测试。Proxy 是一个极其轻量的包装，因此在几乎所有情况下都是更好的选择。不妨把 PureMVC Model 想象成银行金库，把临时 Proxy 想象成里面的保管箱。每个简单的箱子都有唯一钥匙，里面可以放入任何东西。



Proxy 类有一个 facade 属性，引用应用程序的 Facade。它可以注册、移除和获取其他 Proxy，甚至可以操作 Mediator。这个属性来自其父类 Notifier；sendNotification() 便捷方法也继承自那里。请将这股力量用于善事，而非恶事：克制自己，不要利用 Proxy 的 facade 引用去做任何与 View 层或 Controller 层有关的事。

Proxy 生命周期

在多数情况下，你编写的 Proxy 子类会在框架基础 Proxy 之上添加功能，用于与远程服务或文件系统交换数据、执行身份验证、订阅 RTMP（服务器推送）通道、进行套接字通信等。对于系统其余部分，Proxy 是数据的管理者，也是通往外部世界的入口。通常，它们在启动过程的“模型准备”阶段只注册一次；最简单的情况下，这一阶段可能只是 StartupCommand 中、注册各个 Mediator 之前的那段代码。

整个运行期间，Command 和 Mediator 会按名称获取需要通信的 Proxy；该名称通常是 Proxy 子类自身定义的常量。随后，它们调用 Proxy 的方法，或设置、获取其中保存的数据。但是，Proxy 不能接收 Notification。这样设计，是为了避免 Proxy 引用系统中可能定义通知名称的其他类，进而使 Proxy 子类与特定应用程序绑定，降低复用性。这也符合第 1 章所示的经典 MVC 图：Model 层不是通知的潜在接收者。

在 Proxy 子类的构造函数中，我们调用父类构造函数，传入唯一名称和数据对象引用。名称用于把 Proxy 注册到 Model 中；数据对象可能为 null，也经常初始化为集合类型，随后由父类设置到 data 属性。由于 data 的类型是 Object，我们通常创建一个 getter，将 data 转换为实际类型，并赋予 cast 或 story 这样有意义的名称。

有时，为了维护数据完整性，各个 Proxy 之间也需要通信。注册之后，它们可以像 Mediator 或 Command 一样，获取另一个 Proxy 并与之交互。不过，这就要求 Controller 层的逻辑按正确顺序注册 Proxy，以确保在请求某个 Proxy 时，它已经存在。以后修改 StartupCommand 的人可能并不容易看出这种依赖。因此，当一个 Proxy 需要与另一个 Proxy 通信时，最好从一开始就通过构造函数参数要求传入所依赖的 Proxy，明确建立这种关系。本章将对此进行展示。

最后，请记住，Proxy 具有 onRegister 和 onRemove 方法；框架会在 Proxy 生命周期中的相应时刻调用它们。

那些通常会放在类构造函数中的操作，实际上应推迟到 onRegister() 被调用时再做。可能引发问题的操作并不多，但你应当先完成 Proxy 的

注册，再做其他事情。把操作放在构造函数中而不是 onRegister() 中，并不会节省任何 CPU 周期。试图尽早发起服务调用，就是一种在某些情况下可能给你带来麻烦的做法。如果服务调用返回后，Proxy 会发送通知，告知数据已经就绪，并由此开始与应用程序其余部分通信，那么在 Proxy 注册之前就发起获取请求，便存在一个风险：数据返回时，Proxy 可能还没有注册。在 PureMVC MultiCore 中，只有注册之后才能访问 Facade，因此此前还无法发送 Notification。也就是说，如果调用在 Proxy 注册之前返回，它无论如何也无法把结果通知其他对象。养成将这些操作放在 onRegister() 中，而不是构造函数中的习惯，就不会遇到这个问题，也无须因使用 PureMVC Standard 还是 MultiCore 而采用不同做法。

如果创建了 Flex RemoteObject 这样的服务组件，你通常会在它上面设置事件监听器。如果 Proxy 子类有可能在程序运行结束之前被移除，框架就会调用 onRemove()。除非在 onRegister() 中设置事件监听器时指定了“弱引用”，否则应在 onRemove() 中移除这些监听器。如果 Proxy 的生命周期允许它在程序结束前被移除，还可以考虑在 onRemove() 中发送最后一条 Notification，让可能持有该 Proxy 引用的 Mediator 将引用设为 null。这样能够确保该 Proxy 的所有引用都被移除，使它可以被垃圾回收。

重新审视持久化策略

最初考虑持久化问题时，我们选择把简单的 XML 文件写入文件系统，作为持久化机制。不过，我们也考虑过将来可能希望把这些文件存储在某个在线位置。这个想法听起来很好，却带来了一个需要在继续之前考虑的问题。我们不想本末倒置，先写下一大堆面向未来的代码，最后却发现它们并非真正所需。但我们也不想因为连一套能够按预期逐步迭代的策略都没有想过，就构建出阻碍已知未来目标的方案，把自己逼入死角。

问题的核心在于同步行为与异步行为的区别。

与远程服务通信时，我们总是采用异步方式。客户端向服务器发出请求，执行线程则继续处理自己的事务。稍后，服务器响应到来，打断当前的执行流程。这时，我们必须记起发起调用时正在做什么，通常会使用异步令牌模式的某种变体，才能确定如何处理服务器返回的数据。

使用 AIR 写入文件系统时，我们可以选择同步或异步方式。

因此，乍一看，似乎直接使用 AIR 的异步文件系统访问方法就行了。等到确定使用哪一种在线服务时，再增强 Proxy，使其能够与文件系统或该服务通信。我们也可以采用更巧妙的办法：用 Delegate（委托）模式增强 Proxy，将 I/O 职责委托给一个可以互换的类。Proxy 使用适当的 Delegate 类，与文件系统或服务通信，自身完全不受影响。

但是，同步代码更容易编写和理解。一个方法可以取回文件，然后立即对它执行操作。异步代码则必须把这类操作拆成两部分：一部分在取回文件之前执行，另一部分在文件返回之后执行。如果每项操作都必须这样拆分，代码的规模、复杂度以及出错风险都会明显增加。现阶段就实现异步代码，显然属于本末倒置：为了未来迭代中的目标，我们现在就编写更复杂的代码。可等到那时，我们也不希望发现，必须把全部简单的同步代码彻底重写，才能改成异步。怎样才能现在编写最简单的代码，又确保复杂需求到来时，它们仍然能派上用场呢?

与其假定未来系统只能写入本地或在线位置中的一个，为什么不设计成两阶段过程：先同步写入本地文件系统，再异步镜像到在线位置？这样，应用程序天然就具备离线工作模式。我们需要建立同步机制：启动时，如果处于在线状态，就检查在线服务，下载所有比本地副本更新的文件。这样，无论你是在与他人协作，还是先在台式机上编辑 Story，随后想带着一杯薄荷朱利普鸡尾酒和一台 iPad 到阳台继续写作，都可以无缝衔接。

因此，我们将同步写入本地文件系统，享受代码简单带来的好处，同时确信以后可以轻松添加在线服务。

需要哪些 Proxy?

我们知道，Proxy 必须把 XML 存储到本地文件系统。前面，我们定义了 ValueObject 及其多个子类，它们封装各种 XML 片段，再将这些片段组装成由 StoryVO 和 SeriesVO 表示的大型层次结构。因此，很显然，我们会希望用不同的 Proxy，分别管理 StoryVO 和 SeriesVO 的持久化。但还必须处理 CastVO 和 MilieuVO：一个 SeriesVO 及其全部 StoryVO 会共享这些对象。虽然本书不会从用户界面角度讨论它们的用例，但管理它们的持久化会带来一个有趣的问题，本章将完整解决。因此，我们还需要用于管理 CastVO 和 MilieuVO 的 Proxy。

如何存储数据？

我们用四个 Proxy 存储四种不同类型的 XML 数据，因此，顺理成章的做法是在本地文件系统的四个不同文件夹中分别存储它们。这些文件夹位于 AIR 为本应用程序提供的 Application Storage Directory（应用程序存储目录）下。由于每个 ValueObject 都有自己的唯一 ID，只需在 StoryVO、SeriesVO、CastVO 或 MilieuVO 的 ID 后面加上 .xml，就得到了文件名。

这样就结束了吗？还没有……

索引

回看第 4 章的图 4-1。图中展示了应用程序的 Chooser 视图，其中 Story 和 Series 列表按名称排序。你认为这些列表应当如何获得？难道要打开 Story 和 Series 文件夹中的每一个文件，仅仅为了读取名称吗？如果其中有许多非常大的文件，又该怎么办？这是一种蛮力做法，显然可以改进。决定不用 SQLite、改为直接写 XML 文件时，我们也放弃了一项能力：通过快速查询，获取仅包含所需字段的项目列表。

但请记住，在设计 VO 时，我们添加了 isStub 属性，用于判断 ValueObject 子类是仅包含基类信息的存根，还是包含全部数据的完整对象。之所以这样设计，是因为我们知道，不希望把一个 SeriesVO 的全部 StoryVO 都存进同一个文件。想象一下，如果把《迷失》全部七季、每季约 26 集的文本都放进一个文件，会是什么情形？存根也让我们能够分别存储 CastVO 和 MilieuVO，同时在保存的 StoryVO 和 SeriesVO 文件中保留对应存根。现在看来，这些存根还可以充当各类数据的 XML 索引文件中的条目。保存新 StoryVO 时，可以把完整 XML 存入数据文件，再把存根形式添加到索引文件中；这个索引文件就是一个简单的 XMLList。图 5-1 展示了 Story 文件。

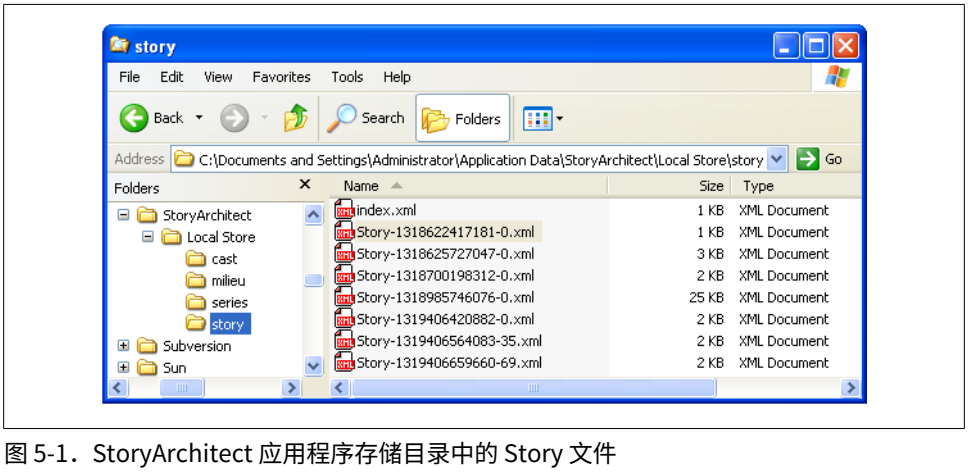


图 5-1．StoryArchitect 应用程序存储目录中的 Story 文件

index.xml 文件示例

```
<?xml version="1.0" encoding="utf-8"?>
<index>
  <Story uid="Story-1319406659660-69" isStub="true">
    <name>Complex Story</name>
  </Story>
  <Story uid="Story-1318700198312-0" isStub="true">
    <name>Cthulu is Coming to Dinner</name>
    <description>arrrgh!</description>
  </Story>
  <Story uid="Story-1319406564083-35" isStub="true">
    <name>Normal Story</name>
  </Story>
  <Story uid="Story-1318985746076-0" isStub="true">
    <name>Once, Upon the Dark Matter Highway</name>
  </Story>
  <Story uid="Story-1319406420882-0" isStub="true">
    <name>Simple Story</name>
  </Story>
  <Story uid="Story-1318625727047-0" isStub="true">
    <name>Something Good, Something Bad</name>
    <description>That's life.</description>
  </Story>
  <Story uid="Story-1318622417181-0" isStub="true">
    <name>Zombie Apocalypse</name>
    <description>Braaaains!</description>
  </Story>
</index>
```

构建 Proxy

到目前为止，我们已经确定需要以下四个 Proxy 子类：

- `CastProxy`
- `MilieuProxy`
- `SeriesProxy`
- `StoryProxy`

我们知道，它们所存储的结构之间存在依赖关系，并将通过允许 Proxy 彼此协作来管理这些依赖。我们也知道，这些 Proxy 在文件系统 I/O 和索引方面，将执行许多相同的操作。

这是一个经典的面向对象设计决策：应通过继承，还是通过组合，来添加这些相似功能？有人认为应始终优先采用其中一种，但真正遇到问题时，你需要根据眼前的约束作出决定。

前面顺带提及的 Delegate 模式，就是组合方式的一个例子。如果希望能够替换这部分公共功能，那么

组合是一个不错的选择。如果为类提供这种功能的对象实现了某个接口，就可以轻松用该接口的另一个实现替换它。

但在当前场景中，也就是文件系统与索引功能，我们并不打算替换这些功能。即使将来增加向云端异步镜像的功能，也始终会先使用 AIR 与文件系统进行同步交互。因此，在这里，继承是最合理的方式。

我们将创建扩展 Proxy 的 AbstractProxy 类，并在其中加入全部公共功能。前面列出的 Proxy 将扩展 AbstractProxy，再添加应用程序所需的方法，以持久化各个 VO，并自动维护它们之间的依赖。设计 AbstractProxy 时，我们有机会隐藏实际文件系统交互和索引处理的繁杂细节，让子类只需处理 ValueObject。

请注意，这里也会体现组合方式。让 SeriesProxy 和 StoryProxy 持有 CastProxy 和 MilieuProxy 的引用，并允许前两者把部分职责委托给后两者，就是组合的一个例子。



如前所述，如果要对 Proxy 类进行单元测试，应使用 PureMVC MultiCore，这样每项测试都能得到一个全新的 PureMVC 核心。但如果采用 Delegate 模式，用一个不依赖框架的类处理 I/O，再通过组合让 Proxy 引用这个非框架 Delegate，就可以轻松地对委托功能进行单元测试，而完全不涉及框架。

抽象代理

类

`AbstractProxy.as`

职责

AbstractProxy 从不直接实例化，只供其他类继承。它将为子类提供持久化和获取 VO 所需的一切，同时把文件系统访问隐藏在私有方法中，使子类所了解的方法和属性都只处理 VO。它还会在内存和磁盘上，为子类负责管理的 VO 类型自动生成排序后的索引。

- 提供一个函数，连接到子类指定名称的文件夹；如果文件夹不存在则创建它，并确保即使其中没有任何项目，也始终包含索引文件。
- 提供添加、保存和删除 ValueObject 的函数，并自动更新文件系统和内存中的索引。

- 提供一个函数，将所连接文件夹的索引中的 ValueObject 存根作为 ArrayCollection 返回。
- 确保索引按 ValueObject 的 name 字段排序。

协作关系

AbstractProxy 类了解 Flash 的 File、FileMode 和 FileStream 类，用它们执行文件系统 I/O。它了解 Flex 的 ArrayCollection、Sort 和 SortField 类，用它们创建按名称排序的索引。当然，它也了解自己所扩展的 PureMVC Proxy 类，以及需要持久化的自定义 ValueObject 类。

只有它的子类 SeriesProxy、StoryProxy、CastProxy 和 MilieuProxy 了解 AbstractProxy 类。

代码

```
package com.futurescale.sa.model.proxy
{
    import com.futurescale.sa.model.vo.ValueObject;

    import flash.filesystem.File;
    import flash.filesystem.FileMode;
    import flash.filesystem.FileStream;

    import mx.collections.ArrayCollection;

    import org.puremvc.as3.patterns.proxy.Proxy;

    import spark.collections.Sort;
    import spark.collections.SortField;

    /**
     * 为子类提供简单的 VO 到磁盘的 API，
     * 并自动维护按名称排序的索引。
     */
    public class AbstractProxy extends Proxy
    {

        // 文件扩展名
        private const FILE_EXT:String = ".xml";

        // 索引文件名
        private const INDEX:String    = "index"+FILE_EXT;

        // 要访问的文件夹，由 connectToFolder() 设置
        private var folder:File;

        /**
         * 构造函数。
         *
         * 名称由子类提供；data 属性用于保存
         * VO 存根索引，初始化为 ArrayCollection。
```

```
 */
public function AbstractProxy( name:String )
{
    super( name, new ArrayCollection() );
}

/**
 * 按名称排序的 VO 存根索引。
 */
public function get index():ArrayCollection
{
    return data as ArrayCollection;
}

/**
 * 为索引创建 VO 存根的抽象工厂方法。
 * 子类必须重写此方法，并返回适当类型的
 * VO 存根，以便加入索引。

 */
protected function getVoInstance( xml:XML ):ValueObject
{
    return null;
}

/**
 * 设置受保护的 folder 属性。
 *
 * 如果文件夹不存在，还会创建它并写入
 * 空索引；否则，从现有文件夹读取索引。

 */
protected function connectToFolder( name:String ):void
{
    folder = File.applicationStorageDirectory.resolvePath( name );
    if ( !folder.exists ) {
        folder.createDirectory();
        writeIndex();
    } else {
        readIndex();
    }
}

/**
 * 将值对象的 XML 属性写入文件，
 * 并更新磁盘和内存中的索引。
 */
protected function writeVO( vo:ValueObject ):void
{
    var file:File = getFile( vo );
    writeFile( file, vo.xml );
    updateIndex( vo );
}

/**
```

```
 * 根据传入的存根，从磁盘读取相应 VO，
 * 并返回 XML。
 */
protected function readVO( voStub:ValueObject ):XML
{
    var file:File = getFile( voStub );
    return XML ( readFile( file ) );
}

/**
 * 根据传入的 VO，从磁盘删除相应 VO，
 * 并更新索引。
 */
protected function deleteVO( vo:ValueObject ):void
{
    var file:File = getFile( vo );
    if ( file.exists ) file.deleteFile();
    updateIndex( vo, true );
}

/**
 * 从磁盘读取索引 XML，并在内存中
 * 创建 VO 存根索引。
 */
private function readIndex():void
{
    // 读取索引 XML
    var file:File = folder.resolvePath( INDEX );
    var xml:XML = readFile( file );

    // 根据 XML 构建索引
    var ac:ArrayCollection =  new ArrayCollection();
    for each ( var ix:XML in xml.children() )
    {
        var vo:ValueObject = getVoInstance( ix );
        ac.addItem( vo );
    }

    // 按名称对索引排序
    var sort:Sort = new Sort();
    var field:SortField = new SortField( "name" );
    sort.fields = [ field ];
    ac.sort = sort;
    ac.refresh();

    // 将索引设为代理的数据
    setData( ac );
}

/**
 * 将内存中的 VO 存根索引写入
 * 磁盘上的索引 XML 文件。
 */
private function writeIndex():void
{
```

```
    var file:File = folder.resolvePath( INDEX );
    var xml:XML = <index></index>;
    var ivo:ValueObject;
    for ( var i:int=0; i < index.length; i++ ){
        ivo = ValueObject( index.getItemAt(i) );
        xml.appendChild( ivo.getStub() );
    }
    writeFile( file, xml );
}

/**
 * 更新内存中的 VO 存根索引，替换或移除
 * 传入的 VO 存根，对索引排序，再写入
 * 磁盘上的索引 XML 文件。
 */
private function updateIndex( vo:ValueObject, removing:Boolean=false ):void
{
    // 查找并移除已存在的项目
    for ( var i:int=0; i < index.length; i++ ){
        var ivo:ValueObject = ValueObject( index.getItemAt(i) );
        if ( ivo.uid == vo.uid ) {
            index.removeItemAt( i );
            break;
        }
    }

    // 若并非删除操作，将新建或更新的项目加入索引
    if ( ! removing ) index.addItem( getVoInstance( vo.getStub() ) );

    // 重新对索引排序
    index.refresh();

    // 持久化索引
    writeIndex();
}

/**
 * 将 XML 写入文件。
 */
private function writeFile( file:File, xml:XML ):void
{
    var stream:FileStream = new FileStream()
    stream.open( file, FileMode.WRITE );
    stream.writeUTFBytes( xml );
    stream.close();
}

/**
 * 从文件读取 XML 并返回。
 */
private function readFile( file:File ):XML
{
    var stream:FileStream = new FileStream()
    stream.open( file, FileMode.READ );
    var xml:XML = XML( stream.readUTFBytes( stream.bytesAvailable ) );
```

```
                stream.close();
                return xml;
            }

            /**
             * 获取指定 VO 对应的 File 对象。
             */
            private function getFile( vo:ValueObject ):File
            {
                var filename:String = vo.uid + FILE_EXT;
                return folder.resolvePath( filename );
            }
        }
    }
```

人物阵容代理

类

`CastProxy.as`

职责

CastProxy 只需要负责创建、保存和删除 CastVO，并确保已加载的 VO 缓存在内存中，删除时也从内存移除。在启用人物阵容编辑功能的未来迭代中，CastProxy 还会负责加载 CastVO。

- 扩展 AbstractProxy，继承持久化和索引功能。
- 在 CastProxy 注册时连接 cast 文件夹，而非在构造函数中连接。
- 重写父类 getVoInstance 方法，使用传入该方法的 XML 创建并返回 CastVO 实例。这是必要的，因为父类只了解 ValueObject；因此，当父类读取 XML 时，需要用这个方法创建正确的 ValueObject 子类。
- 提供一个函数，创建新 CastVO，按其 ID 将它缓存在临时 Proxy 实例中，再写入文件系统，并返回 CastVO 存根。
- 提供保存 CastVO 的函数。
- 提供删除 CastVO 的函数，并确保其关联的临时 Proxy 从内存中移除。

协作关系

CastProxy 类了解 CastVO、ValueObject、AbstractProxy 和 Proxy 类。

StoryProxy 和 SeriesProxy 了解 CastProxy；在下一次迭代中，用于编辑 Story 或 Series 的人物阵容的 Command 也会了解它。

代码

```
package com.futurescale.sa.model.proxy
{
    import com.futurescale.sa.model.vo.CastVO;
    import com.futurescale.sa.model.vo.ValueObject;

    import org.puremvc.as3.patterns.proxy.Proxy;

    /**
     * CastProxy 为 CastVO 提供持久化、
     * 索引和缓存功能。
     */
    public class CastProxy extends AbstractProxy
    {
        // 注册名称
        public  static const NAME:String        = "CastProxy";

        // 用于存储的文件系统文件夹名称
        private static const FOLDER:String      = "cast";

        // 此 Proxy 发送的通知
        public static const CAST_CREATED:String = NAME+"/"+FOLDER+"/created";
        public static const CAST_SAVED:String   = NAME+"/"+FOLDER+"/saved";
        public static const CAST_DELETED:String = NAME+"/"+FOLDER+"/deleted";

        /**
         * 构造函数。
         */
        public function CastProxy( )
        {
            super( NAME );
        }

        /**
         * Proxy 注册时，由框架调用。
         */
        override public function onRegister():void
        {
            connectToFolder( FOLDER );
        }

        /**
         * 创建、保存、缓存并返回一个新的 CastVO。
         *
         * 只有保存新的 StoryVO 或 SeriesVO 时，才创建新的 CastVO。
         * 创建并保存后，使用临时 Proxy 实例将它缓存起来，
         * 因而以后使用时不必重新加载。
         * 返回存根，是因为它只会用于
         * 要保存的 StoryVO 或 SeriesVO 中。

         */
        public function createCast( name:String ):CastVO
        {
            // 创建人物阵容并命名
            var cast:CastVO = new CastVO();
```

```
        cast.name = name;

        // 缓存人物阵容
        facade.registerProxy( new Proxy( cast.uid, cast ) );

        // 保存人物阵容并更新索引
        writeVO( cast );

        // 通知所有观察者
        sendNotification( CAST_CREATED, cast );

        // 将存根返回给调用者
        return new CastVO( cast.getStub() );
    }

    /**
     * 保存 CastVO。
     */
    public function saveCast( cast:CastVO ):void
    {
        // 保存人物阵容并更新索引
        writeVO( cast );

        // 通知所有观察者
        sendNotification( CAST_SAVED, cast );
    }

    /**
     * 删除人物阵容。
     *
     * 如果存在保存已缓存 CastVO 的临时 Proxy，
     * 则先移除它，再从文件系统删除 CastVO。
     * 索引会自动更新。
     */
    public function deleteCast( cast:CastVO ):void
    {
        // 移除缓存的人物阵容
        if ( facade.hasProxy( cast.uid ) ) {
            facade.removeProxy( cast.uid );
        }
        // 从文件系统删除人物阵容并更新索引
        deleteVO( cast );

        // 使用存根通知所有观察者，让 CastVO 可以被垃圾回收
        sendNotification( CAST_DELETED, new CastVO( cast.getStub() ) );
    }

    /**
     * VO 工厂方法；父类建立索引时需要。
     */
    override protected function getVoInstance( xml:XML ):ValueObject
    {
        return new CastVO( xml );
    }
```

```
        }
    }
```

环境代理

类

`MilieuProxy.as`

职责

MilieuProxy 同样简单，只需创建、保存和删除 MilieuVO，并确保已加载的 VO 缓存在内存中，删除时也从内存移除。在启用环境编辑功能的未来迭代中，MilieuProxy 还会负责加载 MilieuVO。

- 扩展 AbstractProxy，继承持久化和索引功能。
- 在 MilieuProxy 注册时连接 milieu 文件夹，而非在构造函数中连接。
- 重写父类 getVoInstance 方法，使用传入该方法的 XML 创建并返回 MilieuVO 实例。
- 提供一个函数，创建新 MilieuVO，按其 ID 将它缓存在临时 Proxy 实例中，再写入文件系统，并返回 MilieuVO 存根。
- 提供保存 MilieuVO 的函数。
- 提供删除 MilieuVO 的函数，并确保其关联的临时 Proxy 从内存中移除。

协作关系

MilieuProxy 类了解 CastVO、ValueObject、AbstractProxy 和 Proxy 类。

StoryProxy 和 SeriesProxy 了解 MilieuProxy；在下一次迭代中，用于编辑 Story 或 Series 的人物阵容的 Command 也会了解它。

代码

```
package com.futurescale.sa.model.proxy
{
    import com.futurescale.sa.model.vo.MilieuVO;
    import com.futurescale.sa.model.vo.ValueObject;

    import org.puremvc.as3.patterns.proxy.Proxy;

    /**
     * MilieuProxy 为 MilieuVO 提供持久化、
     * 索引和缓存功能。
     */
    public class MilieuProxy extends AbstractProxy
```

```
{
    // 注册名称
    public  static const NAME:String          = "MilieuProxy";

    // 用于存储的文件系统文件夹名称
    private static const FOLDER:String        = "milieu";

    // 此 Proxy 发送的通知
    public static const MILIEU_CREATED:String = NAME+"/"+FOLDER+"/created";
    public static const MILIEU_SAVED:String   = NAME+"/"+FOLDER+"/saved";
    public static const MILIEU_DELETED:String = NAME+"/"+FOLDER+"/deleted";

    /**
     * 构造函数。
     */
    public function MilieuProxy( )
    {
        super( NAME );
    }

    /**
     * Proxy 注册时，由框架调用。
     */
    override public function onRegister():void
    {
        connectToFolder( FOLDER );
    }

    /**
     * 创建、保存、缓存并返回一个新的 MilieuVO。
     *
     * 只有保存新的 StoryVO 或 SeriesVO 时，才创建新的 MilieuVO。
     * 创建并保存后，使用临时 Proxy 实例将它缓存起来，
     * 因而以后使用时不必重新加载。
     * 返回存根，是因为它只会用于
     * 要保存的 StoryVO 或 SeriesVO 中。
     */
    public function createMilieu( name:String ):MilieuVO
    {
        // 创建环境并命名
        var milieu:MilieuVO = new MilieuVO();
        milieu.name = name;

        // 缓存环境
        facade.registerProxy( new Proxy( milieu.uid, milieu ) );

        // 保存环境并更新索引
        writeVO( milieu );

        // 通知所有观察者
        sendNotification( MILIEU_CREATED, milieu );

        // 将存根返回给调用者
        return new MilieuVO( milieu.getStub() );
    }
```

```
        /**
         * 保存 MilieuVO。
         */
        public function saveMilieu( milieu:MilieuVO ):void
        {
            // 保存环境并更新索引
            writeVO( milieu );

            // 通知所有观察者
            sendNotification( MILIEU_SAVED, milieu );
        }

        /**
         * 删除 MilieuVO。
         *
         * 如果存在保存已缓存环境的临时 Proxy，
         * 则先移除它，再从文件系统删除环境。
         * 索引会自动更新。
         */
        public function deleteMilieu( milieu:MilieuVO ):void
        {
            // 移除缓存的环境
            if ( facade.hasProxy( milieu.uid ) ) {
                facade.removeProxy( milieu.uid );
            }

            // 从文件系统删除环境并更新索引
            deleteVO( milieu );

            // 使用存根通知所有观察者，让环境对象可以被垃圾回收
            sendNotification( MILIEU_DELETED, new MilieuVO( milieu.getStub() ) );
        }

        /**
         * VO 工厂方法；父类建立索引时需要。
         */
        override protected function getVoInstance( xml:XML ):ValueObject
        {
            return new MilieuVO( xml );
        }
    }
}
```

故事代理

类

StoryProxy.as

职责

StoryProxy 负责创建、加载、保存和删除 StoryVO，并确保已加载的 VO 缓存在内存中，删除时也从内存移除。它有一些职责涉及 CastProxy 和 MilieuProxy：添加不属于任何 Series 的 Story 时，必须创建并持久化新的 Cast 和 Milieu；删除不属于任何 Series 的 Story 时，必须从内存和磁盘移除其 Cast 和 Milieu。此外，添加 Story 时，StoryProxy 还必须根据故事类型创建新的 Chapter、Part 或 Scene，使 Story 已经可以开始编辑，用户不必手动添加。由于 VO 内部的逻辑，这也会创建它们下面的全部层级。例如，复杂 Story 在保存之前会创建一个 Part，而该 Part 自动包含一个 Chapter，Chapter 中有一个 Scene，Scene 中又有一个 Draft。

- 扩展 AbstractProxy，继承持久化和索引功能。
- 要求构造函数传入 CastProxy 和 MilieuProxy 引用，并保存到私有属性中。
- 在 StoryProxy 注册时连接 story 文件夹，而非在构造函数中连接。
- 重写父类 getVoInstance 方法，使用传入该方法的 XML 创建并返回 StoryVO 实例。
- 提供添加新 StoryVO 的函数，按 ID 将其缓存在临时 Proxy 实例中。
- 添加 StoryVO 时，如果 cast 和 milieu 属性为 null，就让 CastProxy 和 MilieuProxy 分别创建新的 CastVO 和 MilieuVO，并将存根随 StoryVO 一起保存；Story 不属于任何 Series 时，这两个属性会为 null。
- 添加 StoryVO 时，确保保存之前已按故事类型创建默认的 Scene、Chapter 或 Part。
- 提供加载 StoryVO 的函数：存在缓存时使用缓存；需要加载时，加载后予以缓存；同时允许显式重新缓存，以支持放弃修改的情形。
- 提供保存 StoryVO 的函数。
- 提供删除 StoryVO 的函数，确保其关联的临时 Proxy 从内存移除；如果不属于任何 Series，还应删除其 CastVO 和 MilieuVO。

协作关系

StoryProxy 类了解 StoryTypeEnum、StoryVO、ValueObject、AbstractProxy 和 Proxy 类。

SeriesProxy 了解 StoryProxy；各种 Command 也了解它，包括 StartupCommand、ManageStoryCommand、EditStoryCommand、ApplyChangesCommand 和 DiscardChangesCommand。

代码

```
package com.futurescale.sa.model.proxy
{
    import com.futurescale.sa.model.enum.StoryTypeEnum;
    import com.futurescale.sa.model.vo.StoryVO;
    import com.futurescale.sa.model.vo.ValueObject;

    import org.puremvc.as3.interfaces.IProxy;
    import org.puremvc.as3.patterns.proxy.Proxy;

    /**
     * StoryProxy 为 StoryVO 提供持久化、
     * 索引和缓存功能。
     */
    public class StoryProxy extends AbstractProxy
    {
        // 注册名称
        public static  const NAME:String         = "StoryProxy";

        // 用于存储的文件系统文件夹名称
        private static const FOLDER:String       = "story";

        // 此 Proxy 发送的通知
        public static const STORY_ADDED:String   = NAME+"/"+FOLDER+"/added";
        public static const STORY_SAVED:String   = NAME+"/"+FOLDER+"/saved";
        public static const STORY_DELETED:String = NAME+"/"+FOLDER+"/deleted";

        // 已知的协作对象
        private var milieuProxy:MilieuProxy;
        private var castProxy:CastProxy;

        /**
         * 构造函数。
         *
         * StoryProxy 与 MilieuProxy、CastProxy
         * 协作，确保领域数据保持一致。
         * 要求通过构造函数传入它们，
         * 既保证它们存在，
         * 又封装了彼此的关系。

         */
        public function StoryProxy( milieuProxy:MilieuProxy,
                                    castProxy:CastProxy )
        {
            super( NAME );

            this.milieuProxy = milieuProxy;
            this.castProxy   = castProxy;
        }
```

```
/**
 * Proxy 注册时，由框架调用。
 */
override public function onRegister():void
{
    connectToFolder( FOLDER );
}

/**
 * 加载故事。
 *
 * 如果故事因先前的加载或添加而已被缓存，
 * 则返回缓存的 VO；否则，先加载并缓存，
 * 然后再返回。
 *
 * 可以选择强制从磁盘重新缓存，
 * 例如用户放弃修改时。
 */
public function loadStory( storyStub:StoryVO,
                           recache:Boolean = false ):StoryVO
{
    // 可选择强制从磁盘加载并重新缓存
    if ( recache ) facade.removeProxy( storyStub.uid );

    var cacheProxy:IProxy;
    if ( facade.hasProxy( storyStub.uid ) ) {
        cacheProxy = facade.retrieveProxy( storyStub.uid );
    } else {
        var story:StoryVO = new StoryVO( readVO( storyStub ) );
        cacheProxy = new Proxy( story.uid, story );
        facade.registerProxy( cacheProxy );
    }
    return cacheProxy.getData() as StoryVO;
}

/**
 * 添加故事。
 *
 * 将故事写入文件系统，并使用临时 Proxy 实例
 * 在内存中缓存，因而以后使用时不必重新加载。
 * 此方法还会为故事填充各层级，
 * 直至第一个草稿。
 */
public function addStory( story:StoryVO ):void
{
    // 缓存故事
    facade.registerProxy( new Proxy( story.uid, story ) );

    // 若该故事属于某个系列，它已有 Cast 和 Milieu；
    // 否则，创建它们。
    if ( story.cast == null ) {
        story.cast = castProxy.createCast( story.name );
    }
    if ( story.milieu == null ) {
```

```
            story.milieu = milieuProxy.createMilieu( story.name );
        }
        // 填充故事结构
        switch ( story.type )
        {
            case StoryTypeEnum.SIMPLE:
                story.getNewScene();
                break;

            case StoryTypeEnum.NORMAL:
                story.getNewChapter();
                break;

            case StoryTypeEnum.COMPLEX:
                 story.getNewPart();
                break;
        }

        // 保存到文件系统，并更新索引
        writeVO( story );

        // 通知所有观察者
        sendNotification( STORY_ADDED, story );
    }

    /**
     * 保存故事。
     */
    public function saveStory( story:StoryVO ):void
    {
        // 保存到文件系统，并更新索引
        writeVO( story );

        // 通知所有观察者
        sendNotification( STORY_SAVED, story );
    }

    /**
     * 删除故事。
     *
     * 如果存在保存已缓存故事的临时 Proxy，
     * 则先移除它，再从文件系统删除故事。
     * 索引会自动更新。
     */
    public function deleteStory( story:StoryVO ):void
    {
        // 移除缓存的故事
        if ( facade.hasProxy( story.uid ) ) {
            facade.removeProxy( story.uid );
        }

        // 若不属于任何系列，则移除人物阵容和环境
        if ( ! story.isEpisode ) {
            castProxy.deleteCast( story.cast );
            milieuProxy.deleteMilieu( story.milieu );
```

```
            }

            // 从文件系统删除，并更新索引
            deleteVO( story );

            // 使用存根通知所有观察者，让故事对象可以被垃圾回收
            sendNotification( STORY_DELETED, new StoryVO( story.getStub() ) );
        }

        /**
         * VO 工厂方法；父类建立索引时需要。
         */
        override protected function getVoInstance( xml:XML ):ValueObject
        {
            return new StoryVO( xml );
        }
    }
}
```

系列代理

类

`SeriesProxy.as`

职责

SeriesProxy 负责创建、加载、保存和删除 SeriesVO，并确保已加载的 VO 缓存在内存中，删除时也从内存移除。添加 Series 时，必须创建并持久化新的 Cast 和 Milieu。此外，如果该 Series 按 Season 组织，还要为它填充一个新的 Season。删除 Series 时，还必须删除 Cast、Milieu 以及全部 Story。

- 扩展 AbstractProxy，继承持久化和索引功能。
- 要求构造函数传入 StoryProxy、CastProxy 和 MilieuProxy 引用，并保存到私有属性中。
- 在 SeriesProxy 注册时连接 series 文件夹，而非在构造函数中连接。
- 重写父类 getVoInstance 方法，使用传入该方法的 XML 创建并返回 SeriesVO 实例。
- 提供添加新 SeriesVO 的函数，按 ID 将其缓存在临时 Proxy 实例中。
- 添加 SeriesVO 时，如果 cast 和 milieu 属性为 null，就让 CastProxy 和 MilieuProxy 分别创建新的 CastVO 和 MilieuVO，并将存根随 SeriesVO 一起保存。

- 提供加载 SeriesVO 的函数：存在缓存时使用缓存；需要加载时，加载后予以缓存；同时允许显式重新缓存，以支持放弃修改的情形。
- 提供保存 SeriesVO 的函数。
- 提供删除 SeriesVO 的函数，并确保其关联的临时 Proxy 从内存中移除。
- 删除 SeriesVO 时，将它的每个 StoryVO，以及 CastVO 和 MilieuVO，都从文件系统和内存中移除。

协作关系

SeriesProxy 类了解 SeriesVO、StoryVO、ValueObject、AbstractProxy 和 Proxy 类。

多种 Command 了解 SeriesProxy，包括 StartupCommand、ManageSeriesCommand、EditStoryCommand、ApplyChangesCommand 和 DiscardChangesCommand。

代码

```
package com.futurescale.sa.model.proxy
{
    import com.futurescale.sa.model.vo.SeriesVO;
    import com.futurescale.sa.model.vo.StoryVO;
    import com.futurescale.sa.model.vo.ValueObject;

    import org.puremvc.as3.interfaces.IProxy;
    import org.puremvc.as3.patterns.proxy.Proxy;

    /**
     * SeriesProxy 为 SeriesVO 提供持久化、
     * 索引和缓存功能。
     */
    public class SeriesProxy extends AbstractProxy
    {
        // 注册名称
        public  static const NAME:String          = "SeriesProxy";

        // 用于存储的文件系统文件夹名称
        private static const FOLDER:String        = "series";

        // 此 Proxy 发送的通知
        public static const SERIES_ADDED:String   = NAME+"/"+FOLDER+"/added";
        public static const SERIES_SAVED:String   = NAME+"/"+FOLDER+"/saved";
        public static const SERIES_DELETED:String = NAME+"/"+FOLDER+"/deleted";

        // 已知的协作对象
        private var storyProxy:StoryProxy;
        private var milieuProxy:MilieuProxy;
        private var castProxy:CastProxy;

        /**
```

```
 * 构造函数。
 *
 * SeriesProxy 与 StoryProxy、MilieuProxy
 * 和 CastProxy 协作，确保领域数据保持一致。
 * 要求通过构造函数传入它们，
 * 既保证它们存在，
 * 又封装了彼此的关系。

 */
public function SeriesProxy( storyProxy:StoryProxy,
                             milieuProxy:MilieuProxy,
                             castProxy:CastProxy )
{
    super( NAME );

    this.storyProxy  = storyProxy;
    this.milieuProxy = milieuProxy;
    this.castProxy   = castProxy;
}

/**
 * Proxy 注册时，由框架调用。
 */
override public function onRegister():void
{
    connectToFolder( FOLDER );
}

/**
 * 加载系列。
 *
 * 如果系列因先前的加载或添加而已被缓存，
 * 则返回缓存的 VO；否则，先加载并缓存后返回。
 * 索引会自动更新。

 *
 * 可以选择强制从磁盘重新缓存，
 * 例如用户放弃修改时。
 */
public function loadSeries( seriesStub:SeriesVO,
                            recache:Boolean = false ):SeriesVO
{
    if ( recache ) facade.removeProxy( seriesStub.uid );

    var cacheProxy:IProxy;
    if ( facade.hasProxy( seriesStub.uid ) ) {
        cacheProxy = facade.retrieveProxy( seriesStub.uid );
    } else {
        var series:SeriesVO = new SeriesVO( readVO( seriesStub ) );
        cacheProxy = new Proxy( series.uid, series );
        facade.registerProxy( cacheProxy );
    }
    return cacheProxy.getData() as SeriesVO;
}
```

```
/**
 * 添加系列。
 *
 * 将系列写入文件系统，并使用临时 Proxy 实例
 * 在内存中缓存，因而以后使用时不必重新加载。

 *
 * 创建、缓存并保存 Cast 和 Milieu，并将存根
 * 添加到……（原文此句未完）
 */
public function addSeries( series:SeriesVO ):void
{
    // 缓存系列
    facade.registerProxy( new Proxy( series.uid, series ) );

    // 创建人物阵容和环境
    series.cast   = castProxy.createCast( series.name );
    series.milieu = milieuProxy.createMilieu( series.name );

   // 填充系列结构
   if (series.useSeasons) series.getNewSeason();

   // 保存系列并更新索引
    writeVO( series );

    // 通知所有观察者
    sendNotification( SERIES_ADDED, series );
}

/**
 * 保存系列。
 */
public function saveSeries( series:SeriesVO ):void
{
    // 保存系列并更新索引
    writeVO( series );

    // 通知所有观察者
    sendNotification( SERIES_SAVED, series );
}

/**
 * 删除系列。
 *
 * 如果存在保存已缓存系列的临时 Proxy，
 * 则先移除它，再从文件系统删除系列。
 * 索引会自动更新。
 *
 * 同时移除人物阵容、环境和全部剧集。
 */
public function deleteSeries( series:SeriesVO ):void
{
    // 移除缓存的系列
    if ( facade.hasProxy( series.uid ) ) {
        facade.removeProxy( series.uid );
```

```
            }

            // 删除人物阵容和环境
            castProxy.deleteCast( series.cast );
            milieuProxy.deleteMilieu( series.milieu );

            // 删除剧集
            var episode:StoryVO;
            for each ( episode in series.stories ) {
                storyProxy.deleteStory( episode );
            }

            // 从文件系统删除系列，并更新索引
            deleteVO( series );

            // 使用存根通知所有观察者，让系列对象可以被垃圾回收
            sendNotification( SERIES_DELETED, new SeriesVO( series.getStub() ) );
        }

        /**
         * VO 工厂方法；父类建立索引时需要。
         */
        override protected function getVoInstance( xml:XML ):ValueObject
        {
            return new SeriesVO( xml );
        }
    }
}
```

第 6 章
为视图提供中介

俗话说，沟通是双向的。这正是 Mediator 存在的核心道理。

Mediator 的存在，是为了促进视图组件与应用程序其余部分之间的双向通信。因此，它的大部分职责都是处理来自视图组件的 Event，以及来自应用程序其余部分的 Notification。这使应用程序中的其他角色与组件彼此解耦。

如果 Mediator 是应用程序中唯一真正了解某个视图组件的类，那么，当组件的实现必须改变时，对系统其余部分的影响就能降到最低。而且，如果视图组件封装得当，内部实现的变化，例如把 List 换成 ComboBox，既不会改变它的协作方式，也不会对 Mediator 产生任何影响。

Mediator 类有一个 facade 属性，引用 ApplicationFacade。它可以注册、移除，并在适用时获取其他 Mediator、Proxy 和 Command。这个属性来自其父类 Notifier；sendNotification() 便捷方法也继承自那里。在这些操作中，只有获取其他 Mediator 被视为不良实践。下面会讨论这一点。

Mediator 生命周期

通常，你会把视图组件传入 Mediator 子类的构造函数；这最常发生在 StartupCommand 的“视图准备”阶段。主应用程序创建时，会把自身引用传给 ApplicationFacade 的 startup() 方法，从而触发 StartupCommand。在该命令中，通常会为主应用程序以及它的一些顶层子组件建立中介关系。



Mediator 还可以通过其他方式取得视图组件，例如自行创建一个组件，或在注册之后的某个时刻通过 Notification 接收组件。“视图进阶主题”一章会讨论其中一些情形，不过，前面描述的方式在大多数情况下最为常见。

Mediator 子类的构造函数通常只接受一个强类型的视图组件参数。例如，ChooserMediator 只接受 Chooser 组件。在构造函数内部，我们调用父类构造函数，传入唯一名称和视图组件引用。名称通常由 Mediator 子类中的 NAME 常量定义；父类会把组件引用设置到 viewComponent 属性。由于 viewComponent 的类型是 Object，我们通常创建一个私有或受保护的 getter，将其转换为实际类型，并赋予 app 或 chooser 这样有意义的名称。类中的其他方法处理视图组件时，就通过这个 getter 引用它。

Mediator 通常长期存在，在整个运行期间保持注册，但并非始终如此。例如，在以 ViewNavigator 为主应用程序的 Flex Mobile 项目中，视图组件（称为 View）的生命周期就截然不同。ViewNavigator 像一个栈，你可以将 View 压入和弹出。在这种环境中，中介管理的对象不断变化，因为离开某个组件时，ViewNavigator 就会销毁它。你可以监听应用程序发出的 added 事件，并通过一个 Command 移除旧 Mediator，再为新的 View 添加新的 Mediator。Flex Mobile 应用程序的中介管理示例见第 9 章。

无论 Mediator 存活多久、何时创建，只要视图组件会派发 Mediator 需要接收的 Event，就应在 Mediator 子类的 onRegister() 方法中设置监听器。也应在这个方法中获取 Mediator 需要协作的各个 Proxy。除非在 onRegister() 中设置监听器时指定了“弱引用”，否则一定要在 onRemove() 中移除这些监听器。这样可以确保，Mediator 从 PureMVC View 中移除时，对它的引用都已消失，从而可以被垃圾回收。

与组件通信

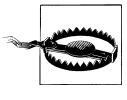
编写视图组件时，我们不会让它深入应用程序、获取 Proxy 来取得数据。相反，我们公开属性和方法，让外部能够通过它们把数据传入组件。同样，我们也不会在系统其他位置获取组件并操作它，而是向它的 Mediator 发送 Notification，由 Mediator 负责告知组件。

可以让视图组件像一位薪酬过高的 CEO：坐在办公室里，所有东西都由别人送到面前。他不会自己去茶水间拿咖啡，而是

由秘书理所当然地送来。如果他需要把一份重要的签署文件通过快递急送到另一家公司，他只需叫一声秘书；秘书便会从他的办公桌上取走文件，并安排快递上门取件。
同样，送来重要文件的快递员也不会直接闯进 CEO 的办公室，而会先把包裹交给秘书，再由秘书按照 CEO 喜欢的方式通知他。

在这个场景中，Mediator 扮演秘书的角色。这是商业活动中一种简单而普遍的模式，它非常适合用来确保：视图组件由一个充分了解其需求的类悉心照管。

在 Mediator 中，我们为视图组件设置的监听器所对应的事件处理器，最终通常会把 Event 转换成 Notification，再发送出去，交给其他 Mediator 或某个 Command 接收。
我们可能只是在收到 Event Y 时，发送 Notification X；也可能把 Event 本身或它的某个属性放进 Notification 的 body；还可能从视图组件中提取一些数据，再通过 Notification 的 body 属性发送出去。



有一件事不应做：利用 Mediator 维护状态、执行大量计算，或进行其他数据操作。Mediator 应负责协调通信，而非思考决策；把思考交给 Command，只需发送一条 Notification 就能找到它们。

依赖注入

过去几年，Flex 社区对“依赖注入”（dependency injection，简称 DI）的强大能力讨论颇多。一些其他框架的拥护者把 DI 宣传成 PureMVC 缺少的某种神奇新功能。然而，当 Mediator 把 VO 传入视图组件时，无论通过调用方法还是设置属性，做的正是依赖注入。视图组件依赖值对象才能工作，而 Mediator 将这个 VO 注入视图组件。

PureMVC 确实没有的是依赖编译器的 DI。例如，Flex 数据绑定就是通过编译器指令完成的 DI。每次给视图组件的某个属性加上 [Bindable] 标签，都是在告诉 Flex 编译器：当该属性变化时，希望把数据重新注入那些在绑定表达式中引用它的依赖组件。这就形成了对 Flex 编译器和框架的依赖，需要由它们解释并落实这一要求，从而限制了应用程序的可移植性。

不过，我们的视图组件本来也无法直接移植；例如移植到 Python 时，必须完全重写。如果没有直接对应的数据绑定机制，就需要让视图组件多承担一点职责，将绑定行为封装在内部。但是，如果 PureMVC 本身必须依靠自定义编译器指令才能

工作，那么它以及你的应用程序架构，就无法如此轻松地移植到其他语言了。

与应用程序通信

通知关注列表

Mediator 处理来自视图组件的 Event 时，最常见的响应是使用 sendNotification() 方法发送对应的 Notification。这可以触发单个 Command，也可以通知任意数量对此感兴趣的 Mediator，包括发送该 Notification 的 Mediator 自身。那么，所谓“感兴趣的 Mediator”是什么意思呢?

PureMVC 的 Notification 系统采用发布/订阅方式，是对 Flash Event 系统的补充，而不是试图取代它。Mediator 通过 listNotificationInterests() 方法返回一个 Array，在其中列出希望接收的 Notification 名称，以表达自己对这些通知的关注。框架注册 Mediator 时，会调用这个方法。这意味着，Mediator 不必持有通知发送者的引用；通知可能来自任何地方。框架已将 Mediator 注册为该 Notification 的 Observer，因此，每当其他角色发送这条通知时，所有感兴趣的 Mediator 都会被通知，也就是其 handleNotification() 方法会被调用。

处理通知

在 handleNotification() 方法中，与事件处理器一样，我们通常只执行简单操作，例如使用 Notification 的 body 中的数据设置视图组件的属性，或调用它的方法。由于我们的目标是避免在响应 Notification 时执行逻辑和数据操作，最佳实践是使用简单的 switch 语句，为 Mediator 关注的每个 Notification 名称设置一个 case。这种方式的简单性恰恰是优点：Mediator 无须为每种关注的 Notification 实现单独的处理方法，也无须提供这些方法的引用。使用 switch 处理 Notification，会温和地促使你保持响应简单，例如设置视图组件属性、调用方法，或再发送一条 Notification。

与 Proxy 协作

响应 Notification 时，或者在事件处理器中，我们还可能直接与 Proxy 通信。你应该还记得，Proxy 无法监听 Notification，而是由 Command、Mediator 或其他 Proxy 操作。回想第 1 章的经典 MVC 图：View 层和 Controller 层一样，都可以直接更新 Model

层。那么，什么时候应发送 Notification，让 Command 处理，而不是直接与 Proxy 通信呢？如果只是从视图组件取出一份数据，再传给 Proxy 的某个方法，就应走最短的路径，直接调用 Proxy。如果需要执行逻辑、转换或验证数据，或者与多个 Proxy 通信，最好把这些操作放进 Command，由 Mediator 发送一条简单的 Notification 来触发。还有一种很适合由 Notification 触发 Command 的情况：用户界面中的多个位置都需要启动同一功能，例如按钮、键盘快捷键和选项菜单，而这些位置分别由不同的 Mediator 监听。只要 Mediator 发送同名 Notification，就可以触发同一个 Command。另一种办法是，把同一个 Command 注册到多个不同的 Notification 名称上，让它们都能触发该命令。



如果 Mediator 需要与某个 Proxy 协作，应在 onRegister() 方法中获取该 Proxy 的引用。这正是启动过程中先准备 Model、再准备 View 的主要原因：当 Mediator 注册并开始建立协作关系时，Proxy 已经注册好了。

避免获取其他 Mediator

在 Mediator 中，应避免获取另一个 Mediator 来调用其方法，或读写其属性。为了维护数据模型的完整性，Proxy 可以通过直接引用彼此而自由协作；但对于 Mediator，我们没有理由采用如此紧密的耦合。Mediator 能够发送和接收 Notification，因此可以以更松散的方式建立联系。

谈到 View 层的松耦合，不妨设想一个 Story Viewer Flex 应用程序：读者可以在 Web 浏览器中阅读 StoryArchitect 所生成的 XML 文件中的故事。它使用 Timeline 视图组件导航，却不需要 Editor 的各种编辑功能。我们可能创建一个 Viewer 组件，仅复用 Timeline 和 TextArea。这意味着，我们要注册 ViewerMediator，而不再注册 EditorMediator。假设 Chooser 增加了一种模式，完全不显示编辑按钮，只派发选择事件。如果 ChooserMediator 原本的响应方式，是获取 EditorMediator，再把选中的 StoryVO 交给它，那么就会产生我们不希望看到的、对 EditorMediator 的紧耦合。相反，ChooserMediator 应发送一条 EditorMediator 感兴趣的 Notification；即使它需要 EditorMediator 返回某些信息，也可以请求后者再发送一条响应 Notification。这样，ChooserMediator 完成了自己的职责，却没有承担对另一个角色的依赖。ViewerMediator 同样可以轻松关注这条选择通知，这正是发布/订阅方式的妙处。

Mediator 多实例

通常，对于某个 Mediator 子类，在它所照管的视图组件的生命周期中，只会注册一个实例；这常常意味着整个运行期间都是同一个实例。但如果应用程序需要同一视图组件的多个实例，而且每个实例都需要中介管理，该怎么办？Notification 还有一个 type 属性。它可以以 String 形式携带额外信息，由 Mediator 按需要解释。借助它，我们可以注册同一 Mediator 的多个实例。

假设在 StoryArchitect 将来的一次迭代中，我们决定同时打开多个 Story：将它们放在选项卡容器中，每个选项卡都有独立的 Editor 组件实例。这时，就需要多次注册 EditorMediator，让每个实例照管一个 Editor。可以把关联 StoryVO 的唯一 uid 属性作为名称，传给 Mediator 的父类构造函数。但处理 Notification 时，每个 EditorMediator 实例如何知道通知是否发给自己，从而决定是否响应呢？Notification 的 type 属性可以帮助我们。若要把通知发给负责 StoryVO “Story-123456789-0”对应 Editor 的 Mediator，只需将该 Story 的 uid 作为 type 参数传给 sendNotification()。随后，在 EditorMediator 的 handleNotification() 方法中，只有当 Notification 的 type 等于该 Mediator 的名称时，才执行操作；名称可通过 getMediatorName() 方法取得。

确定中介管理的粒度

我们的某些视图组件层次相当深。仅在 Timeline 组件内部，就可能包含 StoryTile、PartTile、ChapterTile、SceneTile 和 DraftTile 这样的层级。每个组件都需要一个 Mediator 吗？还是只需要给 Timeline 配一个 Mediator？甚至 Timeline 本身只是 Controls 的子组件，而 Controls 又是 Editor 的子组件，那么 Timeline 有必要拥有 Mediator 吗？是不是有些令人困惑？中介管理做到什么程度才算过头？每个按钮都需要 Mediator 吗？答案取决于视图层次的构建方式，以及每个组件究竟需要多少专门照管。最佳方法是自上而下考虑需求，而不是自下而上。

通常，我们会认为应用程序本身需要 Mediator。在本例中，StoryArchitect 的需求很简单：需要注入 SelectionContext，并且我们希望能够通知 ApplicationMediator，切换 StoryArchitect 的显示模式。例如，在 Editor 中放弃或保存修改之后，返回 Chooser。

Chooser 和 Editor 是用户界面的两个主要视图，而且它们的需求差异足够大，因此，分别建立中介关系，比把所有事情堆进一个庞大的 ApplicationMediator 更好。子组件所需的信息，可以由父组件通过绑定或组合提供。例如，SelectionContext 从 StoryArchitect 开始，经过 Editor、Controls，一直向下绑定到 Timeline。再往下，各个磁贴组件会自行构造

自己的子组件，并把 SelectionContext 像传家宝一样，一代代向下传递。用户单击按钮时，这些磁贴产生的 AppEvent 会沿显示列表向上冒泡，供 EditorMediator 监听；随后，事件会被放进 Notification 发送出去，交由 Command 处理。

构建 Mediator

在 StoryArchitect 中，我们利用视图层次本身向下传递共享的 SelectionContext，并让最深层子组件的事件向上冒泡。因此，实际上并不需要太多 Mediator。我们还会使用一些弹出窗口完成操作，但它们的处理方式将在第 9 章介绍。

到目前为止，我们已经确定需要以下三个 Mediator 子类：

- `ApplicationMediator`
- `ChooserMediator`
- `EditorMediator`

在本例中，这些 Mediator 子类之间没有需要共享的公共功能，因此无须像 AbstractProxy 那样再创建一个 AbstractMediator。这些类都会直接扩展 Mediator，并重写或添加必要的方法，以便与自己的视图组件以及系统其余部分通信。请观察下面的实现：并非每个视图组件都会派发 Mediator 需要处理的事件，或让这类事件从其内部冒泡出来。同样，有些 Mediator 不关注任何 Notification。对于每一对 Mediator 与视图组件，事件处理和通知处理职责的分配都不同，通常两者都会涉及一些。这里的几个 Mediator 恰好各自只承担其中一种。

应用程序中介器

类

`ApplicationMediator.as`

职责

ApplicationMediator 不需要监听应用程序发出的任何 Event。注册时，它需要通知 StoryArchitect 显示 Chooser。此后，它只需响应 Notification，显示 Editor，例如在 Chooser 中选定 Story 并单击“Write”按钮后；或显示 Chooser，例如在 Editor 中保存或放弃修改后。

· 注册时，获取保存 SelectionContext 的临时 Proxy，并将 SelectionContext 设置到 StoryArchitect 的 context 属性。

- 注册时，还应将应用程序设为 StoryArchitect.MODE_CHOOSER 模式。
- 请注意，StoryArchitect 当前不会派发或冒泡出任何我们需要处理的 Event。
- 声明关注 AppConstants.SHOW_EDITOR 和 AppConstants.SHOW_CHOOSER 两种 Notification。
- 处理这些 Notification 时，将应用程序设置为相应的显示模式。
- 定义名为 app 的私有 getter，将 viewComponent 属性转换为 StoryArchitect 类型。

协作关系

ApplicationMediator 类了解 StoryArchitect、AppConstants、SelectionContext、Proxy 和 Mediator 类。

只有注册它的 StartupCommand 了解 ApplicationMediator。

代码

```
package com.futurescale.sa.view.mediator
{
    import com.futurescale.sa.controller.constant.AppConstants;
    import com.futurescale.sa.view.context.SelectionContext;

    import org.puremvc.as3.interfaces.INotification;
    import org.puremvc.as3.interfaces.IProxy;
    import org.puremvc.as3.patterns.mediator.Mediator;

    /**
     * 协调与应用程序组件之间的通信。
     */
    public class ApplicationMediator extends Mediator
    {
        /**
         * Mediator 的注册名称。
         */
        public static const NAME:String = "ApplicationMediator";

        /**
         * 使用应用程序实例构造。
         */
        public function ApplicationMediator( app:StoryArchitect )
        {
            super( NAME, app );
        }

        /**
         * Mediator 注册时调用。
         */
        override public function onRegister():void
```

```
            {
                // 设置 SelectionContext
                var scProxy:IProxy = facade.retrieveProxy( SelectionContext.NAME );
                app.context = SelectionContext( scProxy.getData() );

                // 设置初始显示模式
                app.setMode( StoryArchitect.MODE_CHOOSER );
            }

            /**
             * 此 Mediator 关注的通知。
             */
            override public function listNotificationInterests():Array
            {
                return [ AppConstants.SHOW_EDITOR,
                         AppConstants.SHOW_CHOOSER,
                        ];
            }

            /**
             * 处理此 Mediator 关注的通知。
             */
            override public function handleNotification( note:INotification ):void
            {
                switch ( note.getName() )
                {
                    case AppConstants.SHOW_EDITOR:
                        app.setMode( StoryArchitect.MODE_EDITOR );
                        break;

                    case AppConstants.SHOW_CHOOSER:
                        app.setMode( StoryArchitect.MODE_CHOOSER );
                        break;
                }
            }

            /**
             * 将视图组件转换为正确的类型。
             */
            private function get app():StoryArchitect
            {
                return viewComponent as StoryArchitect;
            }
        }
    }
```

选择器中介器

类

ChooserMediator.as

职责

ChooserMediator 不关注任何 Notification，但会监听 Chooser 的多种 AppEvent。例如，从列表中选择 Story 并单击“Write”按钮时，会产生 AppEvent.EDIT_STORY。它使用一个方法处理全部这些事件，通过 switch 语句发送相应 Notification，而不是为每种事件单独编写处理方法。由于 Mediator 对 Event 的处理通常不应超出这种程度，使用带 switch 语句的单一方法，既高效，也易读。这种写法还与 Notification 的处理方式保持一致，并让 Mediator 类保持简短。

- 注册时，获取 StoryProxy 和 SeriesProxy，并填充 Chooser 的 storyList 和 seriesList。
- 在视图组件上设置监听器，监听 Chooser 派发的重要 AppEvent。
- 通过派发相应的 Notification 来处理 AppEvent。请注意，这里的 AppEvent 与相应 Notification 一一对应，因此可以用包含 switch 语句的单一方法处理，与 handleNotification() 的通常实现方式相同。
- 定义名为 chooser 的私有 getter，将 viewComponent 属性转换为 Chooser 类型。

协作关系

ChooserMediator 类了解 Chooser、AppConstants、AppEvent 和 Mediator 类。它还与 StoryProxy 和 SeriesProxy 协作，获取 Chooser 所需的 Story 和 Series 列表。

只有注册它的 StartupCommand 了解 ChooserMediator。

代码

```
package com.futurescale.sa.view.mediator
{
    import com.futurescale.sa.controller.constant.AppConstants;
    import com.futurescale.sa.model.proxy.SeriesProxy;
    import com.futurescale.sa.model.proxy.StoryProxy;
    import com.futurescale.sa.view.component.chooser.Chooser;
    import com.futurescale.sa.view.event.AppEvent;

    import org.puremvc.as3.patterns.mediator.Mediator;

    public class ChooserMediator extends Mediator
    {
        /**
         * Mediator 的注册名称。
         */
        public static const NAME:String = "ChooserMediator";
```

```
/**
 * 使用 StoryChooser 组件实例构造。
 */
public function ChooserMediator( chooser:Chooser )
{
    super( NAME, chooser );
}

/**
 * Mediator 注册时调用。
 */
override public function onRegister():void
{
    // 与所需代理建立协作关系
    storyProxy = StoryProxy( facade.retrieveProxy( StoryProxy.NAME ) );
    seriesProxy = SeriesProxy( facade.retrieveProxy( SeriesProxy.NAME ) );

    // 设置所需数据
    chooser.storyList  = storyProxy.index;
    chooser.seriesList = seriesProxy.index;

    // 设置组件的事件监听器
    chooser.addEventListener( AppEvent.ADD_EPISODE,   handleAppEvent );
    chooser.addEventListener( AppEvent.ADD_STORY,     handleAppEvent );
    chooser.addEventListener( AppEvent.ADD_SERIES,    handleAppEvent );
    chooser.addEventListener( AppEvent.EDIT_STORY,    handleAppEvent );
    chooser.addEventListener( AppEvent.MANAGE_STORY,  handleAppEvent );
    chooser.addEventListener( AppEvent.MANAGE_SERIES, handleAppEvent );
}

/**
 * 处理来自视图组件的 AppEvent。
 */
private function handleAppEvent( event:AppEvent ):void
{
    switch ( event.type )
    {
        case AppEvent.ADD_EPISODE:
            sendNotification( AppConstants.ADD_EPISODE );
            break;

        case AppEvent.ADD_STORY:
            sendNotification( AppConstants.MANAGE_STORY );
            break;

        case AppEvent.ADD_SERIES:
            sendNotification( AppConstants.MANAGE_SERIES );
            break;

        case AppEvent.EDIT_STORY:
            sendNotification( AppConstants.EDIT_STORY, event.data );
            break;

        case AppEvent.MANAGE_STORY:
            sendNotification( AppConstants.MANAGE_STORY, event.data );
```

```
                    break;

                case AppEvent.MANAGE_SERIES:
                    sendNotification( AppConstants.MANAGE_SERIES, event.data );
                    break;
            }
        }

        /**
         * 将视图组件转换为正确的类型。
         */
        private function get chooser():Chooser
        {
            return viewComponent as Chooser;
        }

        private var storyProxy:StoryProxy;
        private var seriesProxy:SeriesProxy;
    }
}
```

编辑器中介器

类

EditorMediator.as

职责

与 ChooserMediator 一样，EditorMediator 不关注任何 Notification。不过，它会监听 Editor 发出的多种 AppEvent，并发送相应的 Notification。主要区别在于：EditorMediator 发出的每一种 Notification，都可能由两种或更多不同的 AppEvent 类型触发，因此，它会将 AppEvent 一并放入通知的 body，交由接收者处理。读到第 7 章时，你会看到，由这些通知触发的 Command 都会检查 AppEvent，并根据类型采取相似却具体不同的操作。例如，ApplySelectionCommand 由 AppConstants.APPLY_SELECTION 触发，它会针对所选项目，如 Story 或 Scene，调用 SelectionContext 上对应的方法。这种方式比为每一种可选项目都创建不同的 Command 更精简，例如 SelectStoryCommand、SelectPartCommand 等，因为这些命令本质上都在做同一件事。

- 注册时，在视图组件上设置监听器，监听 Editor 派发或从内部向上冒泡的重要 AppEvent。
- 通过派发相应 Notification 来处理 AppEvent。请注意，这个 Mediator 如何按功能类别，将大量具体的 AppEvent 转换成较少种类的 Notification；由这些通知触发的 Command 将检查 AppEvent，以确定应执行的正确操作。

· 定义名为 editor 的私有 getter，将 viewComponent 属性转换为 Editor 类型。

协作关系

EditorMediator 类了解 Editor、AppConstants、AppEvent 和 Mediator 类。

只有注册它的 StartupCommand 了解 EditorMediator。

代码

```
package com.futurescale.sa.view.mediator
{
    import com.futurescale.sa.controller.constant.AppConstants;
    import com.futurescale.sa.view.component.editor.Editor;
    import com.futurescale.sa.view.event.AppEvent;

    import org.puremvc.as3.patterns.mediator.Mediator;

    /**
     * 协调与 Editor 组件之间的通信。
     */
    public class EditorMediator extends Mediator
    {
        /**
         * Mediator 的注册名称。
         */
        public static const NAME:String = "EditorMediator";

        /**
         * 使用 Editor 组件实例构造。
         */
        public function EditorMediator( editor:Editor )
        {
            super( NAME, editor );
        }

        /**
         * Mediator 注册时调用。
         */
        override public function onRegister():void
        {
            // 设置组件的事件监听器
            editor.addEventListener( AppEvent.ADD_STORY,        handleAppEvent );
            editor.addEventListener( AppEvent.ADD_PART,         handleAppEvent );
            editor.addEventListener( AppEvent.ADD_CHAPTER,      handleAppEvent );
            editor.addEventListener( AppEvent.ADD_SCENE,        handleAppEvent );
            editor.addEventListener( AppEvent.ADD_DRAFT,        handleAppEvent );
            editor.addEventListener( AppEvent.ADD_NOTE,         handleAppEvent );
            editor.addEventListener( AppEvent.DELETE_STORY,     handleAppEvent );
            editor.addEventListener( AppEvent.DELETE_PART,      handleAppEvent );
            editor.addEventListener( AppEvent.DELETE_CHAPTER,   handleAppEvent );
            editor.addEventListener( AppEvent.DELETE_SCENE,     handleAppEvent );
```

```
        editor.addEventListener( AppEvent.DELETE_DRAFT,     handleAppEvent );
        editor.addEventListener( AppEvent.DELETE_NOTE,      handleAppEvent );
        editor.addEventListener( AppEvent.SELECT_STORY,     handleAppEvent );
        editor.addEventListener( AppEvent.SELECT_PART,      handleAppEvent );
        editor.addEventListener( AppEvent.SELECT_CHAPTER,   handleAppEvent );
        editor.addEventListener( AppEvent.SELECT_SCENE,     handleAppEvent );
        editor.addEventListener( AppEvent.SELECT_DRAFT,     handleAppEvent );
        editor.addEventListener( AppEvent.SELECT_NOTE,      handleAppEvent );
        editor.addEventListener( AppEvent.DESELECT_PART,    handleAppEvent );
        editor.addEventListener( AppEvent.DESELECT_CHAPTER, handleAppEvent );
        editor.addEventListener( AppEvent.DESELECT_SCENE,   handleAppEvent );
        editor.addEventListener( AppEvent.DISCARD_STORY,    handleAppEvent );
        editor.addEventListener( AppEvent.DISCARD_SERIES,   handleAppEvent );
        editor.addEventListener( AppEvent.SAVE_STORY,       handleAppEvent );
        editor.addEventListener( AppEvent.SAVE_SERIES,      handleAppEvent );
    }

    /**
     * 处理来自视图组件的 AppEvent。
     */
    private function handleAppEvent( event:AppEvent ):void
    {
        switch ( event.type )
        {
            // 用于添加项目的 AppEvent。
            case AppEvent.ADD_STORY:
            case AppEvent.ADD_PART:
            case AppEvent.ADD_CHAPTER:
            case AppEvent.ADD_SCENE:
            case AppEvent.ADD_DRAFT:
            case AppEvent.ADD_NOTE:
                sendNotification(  AppConstants.ADD_ITEM, event );
                break;

            // 用于删除项目的 AppEvent。
            case AppEvent.DELETE_STORY:
            case AppEvent.DELETE_PART:
            case AppEvent.DELETE_CHAPTER:
            case AppEvent.DELETE_SCENE:
            case AppEvent.DELETE_DRAFT:
            case AppEvent.DELETE_NOTE:
                sendNotification( AppConstants.DELETE_ITEM, event );
                break;

            // 用于选择项目的 AppEvent。
            case AppEvent.SELECT_STORY:
            case AppEvent.SELECT_PART:
            case AppEvent.SELECT_CHAPTER:
            case AppEvent.SELECT_SCENE:
            case AppEvent.SELECT_DRAFT:
            case AppEvent.SELECT_NOTE:
                sendNotification( AppConstants.APPLY_SELECTION, event );
                break;

            // 用于取消选择项目的 AppEvent。
```

```
                case AppEvent.DESELECT_PART:
                case AppEvent.DESELECT_CHAPTER:
                case AppEvent.DESELECT_SCENE:
                    sendNotification( AppConstants.REMOVE_SELECTION, event );
                    break;

                // 用于放弃修改的 AppEvent。
                case AppEvent.DISCARD_STORY:
                case AppEvent.DISCARD_SERIES:
                    sendNotification( AppConstants.DISCARD_CHANGES, event );
                    break;

                // 用于保存修改的 AppEvent。
                case AppEvent.SAVE_STORY:
                case AppEvent.SAVE_SERIES:
                    sendNotification( AppConstants.APPLY_CHANGES, event );
                    break;
            }
        }

        /**
         * 将视图组件转换为正确的类型;
         */
        private function get editor():Editor
        {
            return viewComponent as Editor;
        }
    }
}
```

第 7 章
应用业务逻辑

Model 层和 View 层的各个角色都已定义，现在可以把它们串联起来，完成实际工作。这正是 Controller 层的专长。但我们希望在这里完成哪些工作呢?

两种逻辑

领域逻辑

ValueObject、AbstractProxy 及其子类中的全部代码，都属于领域逻辑。它们只负责表示和持久化领域模型的数据结构。

VO 封装了将数据表示为强类型对象和属性所需的逻辑，以供程序其余部分操作；本例中，就是在 XML 结构与对象之间转换数据。VO 也经常没有任何逻辑，只有属性。如果 XML 解析逻辑比较简单，也可以放在 Proxy 中；如果很多，则可能放在 Proxy 使用的某种委托或辅助类中。

Proxy 和 VO 负责可靠地持久化和获取数据。我们不希望依赖应用程序中的 Command 来确保数据完整性，也不希望让 Command 直接负责数据持久化。

业务逻辑

相比之下，业务逻辑关系到应用程序本身的目标或业务，负责实现应用程序的用例。例如，为编辑打开一个 Story：取得 StoryVO，随后对 SelectionContext 作出相应调用，选中该 Story 最后一个 Scene 的当前 Draft，无论各个 Scene 是怎样分组的。这些代码虽然操作领域模型数据，目的却是让 View 层准备好显示它。这种功能服务于应用程序的一个具体用例：Editor 打开时，应停在 Story 的末尾，准备继续增添悬念

和情节。它与数据的表示或持久化毫无关系。如果决定制作一个完全不提供编辑功能的 Viewer 应用程序，就不会有这个特定用例。因此很明显，它不属于 Model 层；Model 层应独立于应用程序用例的各种变化。

为什么要区分?

始终应考虑一种可能：你希望重新打包 Model 层，让它在另一个应用程序中复用，例如 StoryArchitect 的 Viewer 版本。我们希望，表示和持久化领域模型所需的几乎全部内容，都位于 model 包中，方便将其移入库中，具体见第 8 章。因此，如果把领域逻辑与业务逻辑混在 Command 里，重新打包 Model 层以供其他应用程序复用，就不会像原本应该的那样干净利落。我们必须记住，哪些具体 Command 也要移入库，并在两个应用程序中分别注册。更不用说，如果根本不区分领域逻辑和业务逻辑，两者很可能已经混杂在同一个 Command 中。如果这个职责边界模糊的 Command 还执行着只适用于原应用程序的业务逻辑，那么要把它移入库中共享领域逻辑，就可能并不容易。

Command 生命周期

通常在 ApplicationFacade 的 initializeController() 方法中注册 Command，将一个或多个 Notification 名称与指定 Command 类关联。

与 Proxy 和 Mediator 不同，Command 没有 onRegister() 和 onRemove() 方法；它是一个短暂存在的角色。当任何其他角色发送它所注册的某种 Notification 时，Command 才会被实例化。工作完成后，框架不再保留对它的任何引用，因此，只要其他角色也没有持有引用，它就应能被垃圾回收。所以，应避免在 Command 中给任何对象设置监听器。此外，Command 应当无状态；不过，如果需要跟踪某个信息，例如命令已经运行多少次，可以把它保存在类变量中，而不是实例变量中。

Command 有两种形式：SimpleCommand 和 MacroCommand。

使用 SimpleCommand 时，只需重写 execute() 方法，并把逻辑放在其中。构造完成后，该方法会立即执行，触发命令的 Notification 也会作为参数传入。

使用 MacroCommand 时，应重写 initializeMacroCommand() 方法，并调用一次或多次 addSubCommand()，传入 Command 类名。MacroCommand 的 execute() 方法被调用时，会按先进先出（FIFO）的顺序，依次执行各个“子命令”。子命令可以混合使用 MacroCommand 和 SimpleCommand。每个子命令都会收到最初的 Notification，而该通知可以

在执行过程中被修改。另请注意，除非还希望能够单独执行这些子命令，否则只需把它们注册到 MacroCommand，无须通过 ApplicationFacade 的 initializeController() 方法将它们注册到 Controller。

本章介绍的 Command 全部是 SimpleCommand，我们将重点关注应用程序中的一些业务逻辑。稍后的第 8 章，在把领域模型重新打包为独立库时，会把 StartupCommand 拆成由 MacroCommand 触发的两个 SimpleCommand，届时就能看到一个很好的 MacroCommand 示例。

SimpleCommand 类有一个 facade 属性，引用 ApplicationFacade。这意味着，它可以注册、移除和获取 Mediator、Proxy，也可以注册和移除 Command。facade 属性来自其父类 Notifier；sendNotification() 便捷方法也继承自那里。

构建 Command

对应用程序业务逻辑的考察，将从 StartupCommand 开始；在其中，我们会准备好 Model 层和 View 层，使它们能够工作。随后，查看由 Chooser 和 Editor 触发的一些 Command。本章重点讨论与下列操作相关的命令：编辑 Story、应用和取消选择、保存和放弃对 Story 的修改，以及向 Story 添加项目，包括 Part、Chapter、Scene、Draft 和 Note。

关于创建、删除 Story 或 Series，或更改其名称的 Command，将留到第 9 章讨论，因为这些命令使用了专为本应用程序建立的弹出窗口管理子系统；不过，该子系统也很容易复用。

启动命令

类

`StartupCommand.as`

职责

StartupCommand 的职责有些特殊。它运行时，所有 Proxy 和 Mediator 都尚未注册，因此，我们需要注册它们：先用 Proxy 准备 Model，再用 Mediator 准备 View。这个顺序很重要，因为 Mediator 通常会在 onRegister() 中获取 Proxy，以建立协作关系；所以，那时 Proxy 应当已经可用。

- 扩展 SimpleCommand。
- 实现启动过程的常见阶段。
- 按正确顺序注册全部 AbstractProxy 子类，以满足它们通过构造函数强制建立的相互依赖。

- 创建供许多视图组件和 Command 共享的 SelectionContext 实例，并用临时 Proxy 缓存；Proxy 的名称使用 SelectionContext.NAME。
- 从 Notification 的 body 中取得 StoryArchitect 应用程序引用。
- 为 StoryArchitect 应用程序及其两个顶层组件 chooser 和 editor 建立中介关系。
- 注册弹出窗口的 Mediator，第 9 章将介绍。

协作关系

StartupCommand 类了解全部 Proxy 和 Mediator 子类。就已知协作对象的数量而言，它是一个相当特殊的角色。在第 8 章，我们会把 Model 准备阶段和 View 准备阶段拆成两个不同的 Command，让这种情况更合理一些。

只有 ApplicationFacade 了解 StartupCommand，并在其 initializeController() 方法中注册它。

代码

```
package com.futurescale.sa.controller.command.startup
{
    import com.futurescale.sa.model.proxy.CastProxy;
    import com.futurescale.sa.model.proxy.MilieuProxy;
    import com.futurescale.sa.model.proxy.SeriesProxy;
    import com.futurescale.sa.model.proxy.StoryProxy;
    import com.futurescale.sa.view.context.SelectionContext;
    import com.futurescale.sa.view.mediator.ApplicationMediator;
    import com.futurescale.sa.view.mediator.ChooserMediator;
    import com.futurescale.sa.view.mediator.EditorMediator;
    import com.futurescale.sa.view.popup.mediator.AlertPopupMediator;
    import com.futurescale.sa.view.popup.mediator.ConfirmationPopupMediator;
    import com.futurescale.sa.view.popup.mediator.SeriesPopupMediator;
    import com.futurescale.sa.view.popup.mediator.StoryPopupMediator;

    import org.puremvc.as3.interfaces.INotification;
    import org.puremvc.as3.patterns.command.SimpleCommand;
    import org.puremvc.as3.patterns.proxy.Proxy;

    public class StartupCommand extends SimpleCommand
    {
        override public function execute( note:INotification ):void
        {
            // 模型准备阶段
            //
            // 为领域模型创建并注册各个 Proxy。
            var castProxy:CastProxy     = new CastProxy();
            var milieuProxy:MilieuProxy = new MilieuProxy();
            var storyProxy:StoryProxy   = new StoryProxy( milieuProxy, castProxy );
            var seriesProxy:SeriesProxy = new SeriesProxy( storyProxy, milieuProxy,
                                                           castProxy);
```

```
            facade.registerProxy( castProxy );
            facade.registerProxy( milieuProxy );
            facade.registerProxy( storyProxy );
            facade.registerProxy( seriesProxy );

            // 视图准备阶段
            //
            // 从通知 body 中取得应用程序
            var app:StoryArchitect = StoryArchitect( note.getBody() );

            // 注册一个便于保存 SelectionContext 的 Proxy。
            //
            // SelectionContext 跟踪 View 中选中的项目，
            // 不属于领域模型；但可以利用框架的 Proxy 类，
            // 快速缓存这个仅由 View 和 Controller 层
            // 使用的数据实体。
            var selectionContext:SelectionContext = new SelectionContext();
            var scProxy:Proxy = new Proxy( SelectionContext.NAME, selectionContext );
            facade.registerProxy( scProxy );

            // 为初始视图组件建立中介关系
            facade.registerMediator( new ApplicationMediator( app ) );
            facade.registerMediator( new EditorMediator( app.editor ) );
            facade.registerMediator( new ChooserMediator( app.chooser ) );

            // 注册弹出窗口的中介器
            facade.registerMediator( new AlertPopupMediator() );
            facade.registerMediator( new ConfirmationPopupMediator() );
            facade.registerMediator( new StoryPopupMediator() );
            facade.registerMediator( new SeriesPopupMediator() );
        }
    }
}
```

编辑故事命令

类

EditStoryCommand.as

职责

用户在 Chooser 中选中一个故事，并单击醒目的“Write”按钮时，EditStoryCommand 会取得完整的 StoryVO 和 SelectionContext。它找到并选中 Story 的最后一个 Scene，不论故事属于哪种类型，并在 SelectionContext 中选中所有对应项目。例如，对于复杂 Story，要选中最后一个 Part、最后一个 Chapter、最后一个 Scene 及其 Draft。随后，它触发 Editor 的显示。由于 Editor 持有 SelectionContext 引用，打开后就会定位到所选位置。

- 扩展 SimpleCommand。

- 从 Notification 的 body 中取得 StoryVO。
- 从临时 Proxy 中获取 SelectionContext。
- 如果 StoryVO 是存根，则从磁盘加载完整 StoryVO。
- 找到最后一个 Scene 的当前 Draft，选中 Story 的最末端以开始编辑，无论该 Story 中的 Scene 是怎样分组的。
- 选中 Draft 时，还必须在 SelectionContext 中选中 Story 结构中位于它上方的全部元素。
- 发送 AppConstants.SHOW_EDITOR 通知；无须在通知 body 中传递 StoryVO，因为 Editor 会使用 SelectionContext 中选中的 Story。

协作关系

EditStoryCommand 类了解 StoryVO、PartVO、ChapterVO、SceneVO、SelectionContext 和 AppConstants。当用户在 Chooser 中选中 Story 并单击“Write”按钮时，该命令被触发。这里的目标是，将 SelectionContext 设置为 Story 最后一个场景的当前 Draft，并让用户在 Editor 中直接到达该位置，准备继续添加文本。

只有 ApplicationFacade 了解 EditStoryCommand，并在其 initializeController() 方法中注册它。

代码

```
package com.futurescale.sa.controller.command.story
{
    import com.futurescale.sa.controller.constant.AppConstants;
    import com.futurescale.sa.model.proxy.StoryProxy;
    import com.futurescale.sa.model.vo.ChapterVO;
    import com.futurescale.sa.model.vo.PartVO;
    import com.futurescale.sa.model.vo.SceneVO;
    import com.futurescale.sa.model.vo.StoryVO;
    import com.futurescale.sa.view.context.SelectionContext;

    import org.puremvc.as3.interfaces.INotification;
    import org.puremvc.as3.interfaces.IProxy;
    import org.puremvc.as3.patterns.command.SimpleCommand;

    /**
     * 为编辑故事做好准备。
     */
    public class EditStoryCommand extends SimpleCommand
    {
        override public function execute( note:INotification ):void
        {
            // 从通知 body 中取得故事。
            var story:StoryVO = StoryVO( note.getBody() );

            // 获取 SelectionContext
```

```
                var scProxy:IProxy = facade.retrieveProxy( SelectionContext.NAME );
                var context:SelectionContext = SelectionContext( scProxy.getData() );

                // 获取 StoryProxy
                var storyProxy:StoryProxy =
                               StoryProxy( facade.retrieveProxy( StoryProxy.NAME ) );

                // 确保取得完整的 Story
                if ( story.isStub ) {
                    story = storyProxy.loadStory( story );
                }

                // 选中故事最末端，以开始编辑
                context.selectStory( story );
                var part:PartVO;
                var chapter:ChapterVO;
                var scene:SceneVO;
                if ( story.useScenes )
                {
                    scene = story.scenes[ story.scenes.length-1 ];
                    context.selectScene( scene );
                    context.selectDraft( scene.currentDraft );
                }
                else if ( story.useChapters )
                {
                    chapter = story.chapters[ story.chapters.length-1 ];
                    scene   = chapter.scenes[ chapter.scenes.length-1 ];
                    context.selectChapter( chapter );
                    context.selectScene( scene );
                    context.selectDraft( scene.currentDraft );
                }
                else if ( story.useParts )
                {
                    part    = story.parts[ story.parts.length-1 ];
                    chapter = part.chapters[ part.chapters.length-1 ];
                    scene   = chapter.scenes[ chapter.scenes.length-1 ];
                    context.selectPart( part );
                    context.selectChapter( chapter );
                    context.selectScene( scene );
                    context.selectDraft( scene.currentDraft );
                }

                // 通知 View 编辑故事。
                sendNotification( AppConstants.SHOW_EDITOR );
            }
        }
    }
```

应用选择命令

类

ApplySelectionCommand.as

职责

用户在 Timeline 中打开 Story 的某个部分，或在 Details 组件中打开某条 Note 时，ApplySelectionCommand 会执行，并收到一个 AppEvent。它检查这个 AppEvent，确定应在 SelectionContext 中选中什么。SelectionContext 与视图组件共享，视图组件通过它决定显示内容。

- 扩展 SimpleCommand。
- 从 Notification 的 body 中取得 AppEvent。
- 从临时 Proxy 中获取 SelectionContext。
- 根据 AppEvent 的类型，调用 SelectionContext 上相应的选择方法。
- 调用 SelectionContext 的选择方法时，先传入 null 调用一次，再传入 AppEvent 的 data 属性调用一次。如果所选对象与已选对象相同，这样做能强制重新计算视图层次中的绑定，让视图重新绘制自身。
- 此外，选择 Story 时，调用 StoryProxy 的 loadStory() 方法，用完整填充的 StoryVO 替换 AppEvent.data 中可能只是存根的 StoryVO。如果已经加载，则从临时 Proxy 缓存中返回；否则，先加载、缓存，再返回。

协作关系

ApplySelectionCommand 类了解 AppEvent、大多数 ValueObject 子类、SelectionContext 和 Proxy。在未来的迭代中，我们还会添加对 SeasonVO、CharacterVO 和 SettingVO 的选择支持。

只有 ApplicationFacade 了解 ApplySelectionCommand，并在其 initializeController() 方法中注册它。

代码

```
package com.futurescale.sa.controller.command.edit
{
    import com.futurescale.sa.model.proxy.StoryProxy;
    import com.futurescale.sa.model.vo.ChapterVO;
    import com.futurescale.sa.model.vo.DraftVO;
    import com.futurescale.sa.model.vo.NoteVO;
    import com.futurescale.sa.model.vo.PartVO;
    import com.futurescale.sa.model.vo.SceneVO;
    import com.futurescale.sa.model.vo.StoryVO;
    import com.futurescale.sa.view.context.SelectionContext;
    import com.futurescale.sa.view.event.AppEvent;

    import org.puremvc.as3.interfaces.INotification;
    import org.puremvc.as3.interfaces.IProxy;
```

```
import org.puremvc.as3.patterns.command.SimpleCommand;

/**
 * 处理以 AppEvent 形式传入的选择新项目的请求。
 * 在这里，我们解析事件，并在视图组件共享的
 * SelectionContext 中选中该项目。

 */
public class ApplySelectionCommand extends SimpleCommand
{
    override public function execute( note:INotification ):void
    {
        // 从通知 body 中取得事件
        var event:AppEvent = AppEvent( note.getBody() );

        // 获取 SelectionContext
        var scProxy:IProxy = facade.retrieveProxy( SelectionContext.NAME );
        var context:SelectionContext = SelectionContext( scProxy.getData() );

        switch ( event.type )
        {
            case AppEvent.SELECT_STORY:
                context.selectStory( null );
                context.selectStory( getStory( StoryVO( event.data ) ) );
                break;

            case AppEvent.SELECT_PART:
                context.selectPart( null );
                context.selectPart( PartVO( event.data ) );
                break;

            case AppEvent.SELECT_CHAPTER:
                context.selectChapter( null );
                context.selectChapter( ChapterVO( event.data ) );
                break;

            case AppEvent.SELECT_SCENE:
                context.selectScene( null );
                context.selectScene( SceneVO( event.data ) );
                break;

            case AppEvent.SELECT_DRAFT:
                context.selectDraft( null );
                context.selectDraft( DraftVO( event.data ) );
                break;

            case AppEvent.SELECT_NOTE:
                context.selectNote( null );
                context.selectNote( NoteVO( event.data ) );
                break;
        }
    }

    /**
     * 根据存根获取完整的 StoryVO。
```

```
         */
        private function getStory( stub:StoryVO ):StoryVO
        {
          var proxy:StoryProxy = StoryProxy( facade.retrieveProxy( StoryProxy.NAME ) );
            return proxy.loadStory( stub );
        }
    }
}
```

取消选择命令

类

RemoveSelectionCommand.as

职责

用户关闭 Timeline 中的某一部分时，例如 Timeline 已显示某个 Chapter 的 Scene，此时再单击该“Chapter”按钮，就必须从 SelectionContext 中移除该项目及其下方各层级的选择。例如，单击已选中的 Chapter 以取消选择时，应改为选中包含它的 Story 或 Part；这样，SelectionContext 中相应选择方法的内部逻辑，就会取消对该 Chapter 及其 Scene 的选择。

- 扩展 SimpleCommand。
- 从 Notification 的 body 中取得 AppEvent。
- 从临时 Proxy 中获取 SelectionContext。
- 根据 AppEvent 的类型，调用 SelectionContext 上适当的选择方法；实际选中的是 AppEvent 的 data 属性所指项目的父项，从而清除 Story 层次结构中位于该父项之下的所有选择。
- 取消选择 PartVO 时，调用 SelectionContext 的 selectStory() 方法；StoryVO 位于 AppEvent 的 related 属性中。
- 取消选择 ChapterVO 时，AppEvent 的 related 属性中可能是 StoryVO，也可能是 PartVO，应选择相应对象。
- 取消选择 SceneVO 时，AppEvent 的 related 属性中可能是 StoryVO，也可能是 ChapterVO，应选择相应对象。
- 调用选择方法时，先传入 null 调用一次。如果该对象与同类型当前选中的对象相同，这样能够使视图层次中的绑定重新计算，强制视图重新绘制。

协作关系

RemoveSelectionCommand 类了解 AppEvent、若干 ValueObject 子类、SelectionContext 和 Proxy。

只有 ApplicationFacade 了解 RemoveSelectionCommand，并在其 initializeController() 方法中注册它。

代码

```
package com.futurescale.sa.controller.command.edit
{
    import com.futurescale.sa.model.proxy.StoryProxy;
    import com.futurescale.sa.model.vo.ChapterVO;
    import com.futurescale.sa.model.vo.PartVO;
    import com.futurescale.sa.model.vo.StoryVO;
    import com.futurescale.sa.view.context.SelectionContext;
    import com.futurescale.sa.view.event.AppEvent;

    import org.puremvc.as3.interfaces.INotification;
    import org.puremvc.as3.interfaces.IProxy;
    import org.puremvc.as3.patterns.command.SimpleCommand;

    /**
     * 处理以 AppEvent 形式传入的取消选择项目的请求。
     * 在这里，我们解析事件，并在视图组件共享的
     * SelectionContext 中选择相应项目。

     */
    public class RemoveSelectionCommand extends SimpleCommand
    {
        override public function execute( note:INotification ):void
        {
            // 从……取得事件（原文此句未完）
            var event:AppEvent = AppEvent( note.getBody() );

            // 获取 SelectionContext
            var scProxy:IProxy = facade.retrieveProxy( SelectionContext.NAME );
            var context:SelectionContext = SelectionContext( scProxy.getData() );

            switch ( event.type )
            {
                case AppEvent.DESELECT_PART:
                    context.selectStory(null);
                    context.selectStory( StoryVO( event.related ) );
                    break;

                case AppEvent.DESELECT_CHAPTER:
                    if  ( event.related is PartVO ) {
                        context.selectPart(null);
                        context.selectPart( PartVO( event.related ) );
                    } else {
                        context.selectStory(null);
                        context.selectStory( StoryVO( event.related ) );
                    }
                    break;

                case AppEvent.DESELECT_SCENE:
                    if  ( event.related is ChapterVO ) {
                        context.selectChapter(null);
```

```
                    context.selectChapter( ChapterVO( event.related ) );
                } else {
                    context.selectStory(null);
                    context.selectStory( StoryVO( event.related ) );
                }
                break;
            }
        }
    }
}
```

应用修改命令

类

`ApplyChangesCommand.as`

职责

用户在 Editor 中单击“Save”按钮时，ApplyChangesCommand 会执行。它只需保存正在编辑的 Story 或 Series，再让用户返回 Chooser。

- 扩展 SimpleCommand。
- 从 Notification 的 body 中取得 AppEvent。
- 如果 AppEvent 的类型是 SAVE_STORY，则获取 StoryProxy，并以 AppEvent 的 data 属性中的 StoryVO 为参数，调用 saveStory() 方法。
- 如果 AppEvent 的类型是 SAVE_SERIES，则获取 SeriesProxy，并以 AppEvent 的 data 属性中的 SeriesVO 为参数，调用 saveSeries() 方法。

协作关系

ApplyChangesCommand 类了解 AppConstants、AppEvent、StoryProxy、SeriesProxy、StoryVO 和 SeriesVO。在 Editor 中编辑 Story，或在未来迭代中编辑 Series 时，单击“Save”按钮就会发出该 AppEvent。

只有 ApplicationFacade 了解 ApplyChangesCommand，并在其 initializeController() 方法中注册它。

代码

```
package com.futurescale.sa.controller.command.edit
{
    import com.futurescale.sa.controller.constant.AppConstants;
    import com.futurescale.sa.model.proxy.SeriesProxy;
    import com.futurescale.sa.model.proxy.StoryProxy;
    import com.futurescale.sa.model.vo.SeriesVO;
    import com.futurescale.sa.model.vo.StoryVO;
    import com.futurescale.sa.view.event.AppEvent;
```

```
        import org.puremvc.as3.interfaces.INotification;
        import org.puremvc.as3.patterns.command.SimpleCommand;

        public class ApplyChangesCommand extends SimpleCommand
        {
            override public function execute( note:INotification ):void
            {
                // 从……取得事件（原文此句未完）
                var event:AppEvent = AppEvent( note.getBody() );

                // 可能需要的 Proxy
                var storyProxy:StoryProxy;
                var seriesProxy:SeriesProxy;

                // 将实体保存到磁盘
                switch ( event.type )
                {
                    case AppEvent.SAVE_STORY:
                       storyProxy = StoryProxy( facade.retrieveProxy( StoryProxy.NAME ) );
                        var story:StoryVO = StoryVO( event.data );
                        storyProxy.saveStory( story );
                        break;

                    case AppEvent.SAVE_SERIES:
                     seriesProxy = SeriesProxy( facade.retrieveProxy( StoryProxy.NAME ) );
                        var series:SeriesVO = SeriesVO( event.data );
                        seriesProxy.saveSeries( series );
                        break;

                }

                // 让用户返回选择器
                sendNotification( AppConstants.SHOW_CHOOSER );
            }
        }
    }
```

放弃修改命令

类

`DiscardChangesCommand.as`

职责

用户在 Editor 中单击“Discard”按钮时，DiscardChangesCommand 会执行。它先清除 SelectionContext 中对已修改 Story 或 Series 的引用，再重新加载该 Story 或 Series，覆盖我们一直在编辑的缓存版本。随后，让用户返回 Chooser。

- 扩展 SimpleCommand。

- 从 Notification 的 body 中取得 AppEvent。
- 如果 AppEvent 的类型是 DISCARD_STORY，则以 null 调用 SelectionContext 的 selectStory() 方法，取得 StoryProxy，再调用它的 loadStory() 方法。传入 AppEvent 的 data 属性中的 StoryVO，并将 recache 参数设为 true，以确保移除缓存中的 StoryVO，用从磁盘加载的对象替换。
- 如果 AppEvent 的类型是 DISCARD_SERIES，则以 null 调用 SelectionContext 的 selectSeries() 方法，取得 SeriesProxy，再调用它的 loadSeries() 方法。传入 AppEvent 的 data 属性中的 SeriesVO，并将 recache 参数设为 true，以确保移除缓存中的 SeriesVO，用从磁盘加载的对象替换。
- 发送 AppConstants.SHOW_CHOOSER 通知，让用户返回 Chooser。

协作关系

DiscardChangesCommand 类了解 AppConstants、AppEvent、StoryProxy、SeriesProxy、StoryVO、SeriesVO 和 SelectionContext。在 Editor 中编辑 Story，或在未来迭代中编辑 Series 时，单击“Discard”按钮就会发出该 AppEvent。

只有 ApplicationFacade 了解 DiscardChangesCommand，并在其 initializeController() 方法中注册它。

代码

```
package com.futurescale.sa.controller.command.edit
{
    import com.futurescale.sa.controller.constant.AppConstants;
    import com.futurescale.sa.model.proxy.SeriesProxy;
    import com.futurescale.sa.model.proxy.StoryProxy;
    import com.futurescale.sa.model.vo.SeriesVO;
    import com.futurescale.sa.model.vo.StoryVO;
    import com.futurescale.sa.view.context.SelectionContext;
    import com.futurescale.sa.view.event.AppEvent;

    import org.puremvc.as3.interfaces.INotification;
    import org.puremvc.as3.interfaces.IProxy;
    import org.puremvc.as3.patterns.command.SimpleCommand;

    public class DiscardChangesCommand extends SimpleCommand
    {
        override public function execute( note:INotification ):void
        {
            // 从……取得事件（原文此句未完）
            var event:AppEvent = AppEvent( note.getBody() );

            // 获取 SelectionContext
            var scProxy:IProxy = facade.retrieveProxy( SelectionContext.NAME );
            var context:SelectionContext = SelectionContext( scProxy.getData() );
```

```
                // 可能需要的 Proxy
                var storyProxy:StoryProxy;
                var seriesProxy:SeriesProxy;

                // 从选择上下文移除实体，并从磁盘重新缓存
                switch ( event.type )
                {
                    case AppEvent.DISCARD_STORY:
                       storyProxy = StoryProxy( facade.retrieveProxy( StoryProxy.NAME ) );
                        var story:StoryVO = context.story;
                        context.selectStory( null );
                        storyProxy.loadStory( story, true );
                        break;

                    case AppEvent.DISCARD_SERIES:
                     seriesProxy = SeriesProxy( facade.retrieveProxy( StoryProxy.NAME ) );
                        var series:SeriesVO = context.series;
                        context.selectSeries( null );
                        seriesProxy.loadSeries( series, true );
                        break;
                }

                // 让用户返回选择器
                sendNotification( AppConstants.SHOW_CHOOSER );
            }
        }
    }
```

添加项目命令

类

`AddItemCommand.as`

职责

用户在 Timeline 或 Details 组件中添加项目时，例如向 Part 添加 Chapter，或向 Scene 添加 Note，AddItemCommand 会执行。它根据 AppEvent 的类型，例如 AppEvent.ADD_CHAPTER 或 AppEvent.ADD_NOTE，添加适当的项目，并在 SelectionContext 中选中该项目。

- 扩展 SimpleCommand。
- 从 Notification 的 body 中取得 AppEvent。
- 从临时 Proxy 中获取 SelectionContext。
- 如果 AppEvent 的类型是 ADD_PART，则取得选中的 Story，调用其 getNewPart() 方法；随后，先以 null、再以选中的 Story 调用 SelectionContext 的 selectStory() 方法，使视图层次重新绘制，并显示新 Part。

- 如果 AppEvent 的类型是 ADD_CHAPTER，则取得选中的 Part；如果没有，则取得 Story。调用其 getChapter() 方法，再先以 null、后以选中的 Part 或 Story，调用 SelectionContext 上对应的选择方法，使视图层次重新绘制，并显示新 Chapter。
- 如果 AppEvent 的类型是 ADD_SCENE，则取得选中的 Chapter；如果没有，则取得 Story。调用其 getScene() 方法，再先以 null、后以选中的 Chapter 或 Story，调用 SelectionContext 上对应的选择方法，使视图层次重新绘制，并显示新 Scene。
- 如果 AppEvent 的类型是 ADD_DRAFT，则取得选中的 Scene，调用其 getNewDraft() 方法，再先以 null、后以选中的 Scene，调用 SelectionContext 的 selectScene() 方法，使视图层次重新绘制，并显示新 Draft。
- 如果 AppEvent 的类型是 ADD_NOTE，则取得选中的项目，即一个 ValueObject 类型的引用，无论实际选中的是哪种对象。调用它的 addNote() 方法，再先把 SelectionContext 的 selectedItem 属性设为 null，随后设为选中的项目，使视图层次重新绘制，并显示新 Note。

协作关系

AddItemCommand 类了解 ValueObject 及其大多数子类、AppEvent、SeriesVO 和 SelectionContext。在 Editor 中编辑 Story，或在未来迭代中编辑 Series 时，单击某个“添加”（+）按钮以添加 Note、Draft、Scene、Chapter 或 Part，就会发出该 AppEvent。未来的迭代中，这个 Command 还会负责向 Series 添加 Episode 和 Season。

只有 ApplicationFacade 了解 AddItemCommand，并在其 initializeController() 方法中注册它。

代码

```
package com.futurescale.sa.controller.command.edit
{
    import com.futurescale.sa.model.vo.ChapterVO;
    import com.futurescale.sa.model.vo.DraftVO;
    import com.futurescale.sa.model.vo.NoteVO;
    import com.futurescale.sa.model.vo.PartVO;
    import com.futurescale.sa.model.vo.SceneVO;
    import com.futurescale.sa.model.vo.SeasonVO;
    import com.futurescale.sa.model.vo.StoryVO;
    import com.futurescale.sa.model.vo.ValueObject;
    import com.futurescale.sa.view.context.SelectionContext;
    import com.futurescale.sa.view.event.AppEvent;

    import org.puremvc.as3.interfaces.INotification;
    import org.puremvc.as3.interfaces.IProxy;
    import org.puremvc.as3.patterns.command.SimpleCommand;

    /**
```

```
 * View 以 AppEvent 的形式发出了添加新项目的请求。
 * 在这里，我们解析事件，添加项目，
 * 然后触发对该项目的选择。

 */
public class AddItemCommand extends SimpleCommand
{
    override public function execute( notification:INotification ):void
    {
        var event:AppEvent = AppEvent( notification.getBody() );

        // 获取 SelectionContext
        var scProxy:IProxy = facade.retrieveProxy( SelectionContext.NAME );
        var context:SelectionContext = SelectionContext( scProxy.getData() );

        var season:SeasonVO;
        var story:StoryVO;
        var part:PartVO;
        var chapter:ChapterVO;
        var scene:SceneVO;
        var draft:DraftVO;
        var vo:ValueObject;

        switch ( event.type )
        {
            case AppEvent.ADD_PART:
                // 向选中的故事添加部
                story = context.story;
                part = story.getNewPart();
                context.selectStory( null );
                context.selectStory( story );
                break;

            case AppEvent.ADD_CHAPTER:
                if ( context.part )
                {
                    // 向选中的部添加章
                    part = context.part
                    chapter =  part.getNewChapter( );
                    context.selectPart( null );
                    context.selectPart( part );
                } else {
                    // 向选中的故事添加章
                    story = context.story;
                    chapter =  story.getNewChapter( );
                    context.selectStory( null );
                    context.selectStory( story );
                }
                break;

            case AppEvent.ADD_SCENE:
                if ( context.chapter )
                {
                    // 向选中的章添加场景
                    chapter = context.chapter;
```

```
                    scene =  chapter.getNewScene( );
                    context.selectChapter( null );
                    context.selectChapter( chapter );
                } else {
                    // 向选中的故事添加场景
                    story = context.story;
                    scene =  story.getNewScene( );
                    context.selectStory( null );
                    context.selectStory( story );
                }
                break;

            case AppEvent.ADD_DRAFT:
                // 向选中的场景添加草稿
                scene = context.scene;
                draft = scene.getNewDraft();
                context.selectScene( null );
                context.selectScene( scene );
                break;

            case AppEvent.ADD_NOTE:
                // 向选中的项目添加笔记
                context.note = new NoteVO();
                vo = context.selectedItem;
                vo.addNote( context.note );
                context.selectedItem = null
                context.selectedItem = vo;
                break;
        }
    }
}
}
```

注册 Command

我们没有在 StartupCommand 中一并注册 Command，是因为通常已经在 ApplicationFacade 中重写 initializeController()，至少注册一条 STARTUP 通知来引导应用程序启动。既然如此，把全部 Command 的注册集中在一处是合理的。不过，对于 ApplicationFacade 这样长期存在的角色，仅仅为了注册一次，就要了解这么多类，也确实有些多。

如果愿意，可以把其他 Command 的注册，也就是除 STARTUP 之外的注册，移入 StartupCommand。若决定这样做，可以考虑将它们放进独立的 SimpleCommand，再让一个 MacroCommand 形式的 StartupCommand 把它作为子命令执行。StartupCommand 按顺序调用 addSubCommand()，传入 PrepareControllerCommand、PrepareModelCommand 和 PrepareViewCommand。这样，Proxy 和 Mediator 的 onRegister() 方法发出的通知，都已经有对应 Command 可以响应。



无论选择在哪里注册 Command，都要记住：如果忘了注册，Command 就不会执行！

在前面几章构建视图组件及其 Mediator 时，你已经看到：用户界面派发的 AppEvent 会被转换成各种 Notification，再由这些通知触发 Command。现在，让我们回到 ApplicationFacade 的 initializeController()，把第一次迭代所需的 Command 注册全部放进去。

应用程序 Facade / initializeController()

类

`ApplicationFacade.as`

方法

`initializeController()`

职责

通常，我们会在重写的 ApplicationFacade.initializeController() 方法中注册全部 Command，因为至少 StartupCommand 必须由 ApplicationFacade 注册。不过，关于为什么以及如何将这项工作重构到一个 Command 中，请参阅第 8 章的示例和讨论。

- 必须调用 super.initializeController()。
- 为每一组 Notification 与 Command 调用 registerCommand()。
- 一个 Notification 名称只能注册一个 Command，但同一个 Command 可以注册到多个 Notification 名称。

代码

```
/**
 * 注册各个 Command。
 */
override protected function initializeController():void
{
    super.initializeController();

    // 由应用程序发起
    registerCommand( AppConstants.STARTUP,          StartupCommand );

    // 由 Chooser 发起
    registerCommand( AppConstants.MANAGE_STORY,     ManageStoryCommand );
    registerCommand( AppConstants.MANAGE_SERIES,    ManageSeriesCommand );
    registerCommand( AppConstants.EDIT_STORY,       EditStoryCommand );
```

```
        registerCommand( StoryProxy.STORY_ADDED,        EditStoryCommand );

        // 由 Editor 发起
        registerCommand( AppConstants.ADD_ITEM,         AddItemCommand );
        registerCommand( AppConstants.DELETE_ITEM,      DeleteItemCommand );
        registerCommand( AppConstants.APPLY_SELECTION,  ApplySelectionCommand );
        registerCommand( AppConstants.REMOVE_SELECTION, RemoveSelectionCommand);
        registerCommand( AppConstants.DISCARD_CHANGES,  DiscardChangesCommand );
        registerCommand( AppConstants.APPLY_CHANGES,    ApplyChangesCommand);
    }
```

第 8 章
模型进阶主题

本章将讨论几个与模型有关的常见问题：同步行为与异步行为，以及模型层的复用。

处理同步行为

在 StoryArchitect 应用程序中，我们的 Proxy 子类以同步方式与文件系统交互。也就是说，读取文件的结果在下一行代码中就已经可用，你可以立即处理获取到的数据。这很方便，因为调用方通常是在执行某项工作的过程中发起调用，并希望继续完成这项工作。

下面是 StoryArchitect 中的一个示例，展示了一个 Proxy 方法，以及调用它的 Command 如何在执行用例时立即使用获取到的数据。ApplySelectionCommand 必须选中一个故事（这是一项与视图有关的工作），但如果故事只是一个存根（例如列表中的故事），就必须确保它在放入 SelectionContext 时已经完整加载。请注意，Command 使用返回结果是多么直接。

故事代理

类

`StoryProxy.as`

方法

`loadStory()`

代码

```
/**
 * 加载一个故事。
 *
 * 如果故事已因之前的加载或添加而被缓存，
```

```
     * 则返回缓存的 VO;
     * 否则先加载并缓存它。
     *
     * 也可以选择强制从磁盘重新缓存
     * （例如丢弃用户更改时）。
     */
    public function loadStory( storyStub:StoryVO, recache:Boolean = false ):StoryVO
    {
        // 可选择强制从磁盘加载并重新缓存
        if ( recache ) facade.removeProxy( storyStub.uid );

        var cacheProxy:IProxy;
        if ( facade.hasProxy( storyStub.uid ) ) {
            cacheProxy = facade.retrieveProxy( storyStub.uid );
        } else {
            var story:StoryVO = new StoryVO( readVO( storyStub ) );
            cacheProxy = new Proxy( story.uid, story );
            facade.registerProxy( cacheProxy );
        }
        return cacheProxy.getData() as StoryVO;
    }
```

应用选中项命令

类

ApplySelectionCommand.as

方法

execute(), getStory()

代码

```
    override public function execute( note:INotification ):void
    {
        // 从中获取事件
        var event:AppEvent = AppEvent( note.getBody() );

        // 获取 SelectionContext
        var scProxy:IProxy = facade.retrieveProxy( SelectionContext.NAME );
        var context:SelectionContext = SelectionContext( scProxy.getData() );

        switch ( event.type )
        {
                 ...

            case AppEvent.SELECT_STORY:
                context.selectStory( null );
                context.selectStory( getStory( StoryVO( event.data ) ) );
                break;

                 ...
```

```
    }
}

/**
 * 根据存根获取完整的 StoryVO。
 */
private function getStory( stub:StoryVO ):StoryVO
{
  var proxy:StoryProxy = StoryProxy( facade.retrieveProxy( StoryProxy.NAME ) );
    return proxy.loadStory( stub );
}
```

处理异步行为

正如本书前面所讨论的，StoryArchitect 的未来版本可能会把数据镜像到服务器，以便你在多台设备上工作。这就需要与远程服务器通信。

与远程服务器通信时，我们总是采用异步方式。我们发出请求，过了一段时间，服务器传回的响应便会打断当前流程。如果你要获取一个对象，就不能在发起调用后立即使用它，而必须等待结果。此外，你还必须记住发出请求时正在做什么，因为这可能影响你对结果的处理方式。

在 Proxy 内部，这仅意味着要将发起调用的代码与随后处理结果的代码拆分为至少两个方法：一个用于发起调用，一个用于处理结果，也可能另有一个方法用于处理故障响应。

如前面的同步示例所示，Command 在执行应用程序的某个用例时，常常会在一大段代码的中间调用 Proxy 的持久化或读取操作。这意味着应用程序中 Mediator 和 Command 的构建方式，在一定程度上取决于 Proxy 表现为同步、异步，还是二者混合。这也是我们先构建 Proxy，再构建 Mediator 和 Command 的原因之一。在 StoryArchitect 中，我们知道通信将采用同步方式，因此这种安排很顺利。不过，让我们考虑使用异步服务的情况；这种情况要常见得多。

设想一个 Command 只是调用 Proxy 的某个异步方法。稍后，响应返回，Proxy 通过一个 Notification 将结果发出。

接下来会发生什么?

- 能否执行另一个 Command，并且／或者通知若干个 Mediator?
- 发起调用的 Command 能否以某种方式取得结果并继续处理?

这两个问题的答案都是可以。

以上是处理用例中异步中断的两种方式，我们将在这里讨论。无论采用哪一种，由于异步通信本身的特点，都需要更复杂的响应

处理机制。请记住，目标并不是寻找获取数据的最短路径，而是对执行业务逻辑的 Command 完全隐藏远程通信的具体方式。将读取操作封装在 Proxy 中，这样即使以后必须改变服务器通信的方式（例如，从读取并解析普通文件中的 XML，改为与 WebORB 服务器通信并获取已经构造好的 ActionScript 对象），业务逻辑也不会受到影响。无论数据从服务传入时是什么形式，View 层和 Controller 层请求和接收的都是具有明确类型的对象。

分离调用方与响应方

这是最常见的做法。一个角色在 Proxy 上发起请求，另外一个或多个角色（Mediator 和／或 Command）处理结果。

我们前面讨论过一个假想的阅读器应用，它可能会使用某种云服务 API。不过，现在假设我们已经创建了自己的 WebORB、BlazeDS 或 LCDS 后端，并通过一个远程 Java 服务开放对 StoryVO 的访问。同时，我们还有一个在浏览器中运行的 Flex 阅读器应用（也可能有桌面版和移动版），其基本功能只是让用户浏览故事列表并打开某个故事阅读。

我们来讨论这样一个用例：用户从列表中选择了一个故事，于是触发 LoadStoryCommand。阅读器并不特别需要选择故事内部的各个部分，因此我们甚至没有注册 SelectionContext；Mediator 只是把选中的 StoryVO 存根放在 Notification 的消息体中发送出去，供 Command 使用。LoadStoryCommand 将与一个专用的 StoryReadingProxy 交互，后者负责与远程阅读服务通信，获取完整的 StoryVO。

- LoadStoryCommand 调用 StoryReadingProxy，传入要加载的 StoryVO 存根，发送 AppConstants.READ_STORY 通知，然后退出。
- ViewerAppMediator 关注 AppConstants.READ_STORY，收到通知后要求其视图组件 ViewerApp 从 Chooser 切换到 Reader。
- 稍后，服务器响应返回，StoryReadingProxy 发送 StoryReadingProxy.STORY_RETRIEVED 通知，并将 StoryVO 放入消息体。
- ReaderMediator 关注 StoryReadingProxy.STORY_RETRIEVED，收到通知后取出其中的 StoryVO，并将其设置到视图组件 Reader 上。
- 视图组件 Reader 立即显示 StoryVO，供用户阅读。

这里其实不需要再用代码说明。回想第 1 章，我们已经展示过一个非常类似的示例：PerformEmailTestCommand 调用 EmailProxy 的 testConfiguration() 方法，随后异步结果通过 EmailProxy.TEST_RESULT 通知发出，由 EmailConfigMediator 处理。

由同一对象调用并响应

有时，我们希望在 Proxy 上调用远程读取方法的那个 SimpleCommand 实例，能够接收异步响应，以便利用获取到的信息继续执行业务逻辑。

这种方式最大的难点，在于让 Command 与领域逻辑解耦。

在 Flex 开发中，一种常见做法是让类似 Command 的类实现 Flex 的 IResponder 接口，然后由它自己发起调用，或者由 Proxy 发起调用并把这个 Command 设置为响应方。这样，结果就会直接传回 Command 的 result 或 fault 方法。问题在于，这种做法混淆了 Command 的职责：通过实现 IResponder 并处理 ResultEvent 和 FaultEvent，Command 实际上也参与了持久化机制。

如第 7 章所述，我们希望持久化机制被封装在 Model 层中，让 Command 专注于业务逻辑。Command 只是需要一份数据，不需要知道数据来自某个服务，也不需要解析 ResultEvent 和 FaultEvent。如果它承担了这些工作，那么以后改变持久化机制时，不仅领域逻辑会受影响，业务逻辑也会受影响，因为这个 Command 同样需要重构。

在本例中，假设我们已经在 Chooser 中选中了一个故事，希望对它进行文本搜索，但必须先加载完整的故事。我们将使用 SearchStoryCommand 加载故事；故事加载完成后，它会收到通知，然后执行搜索操作。

我们将用一个名为 ServiceRequest 的类规范请求，并封装回调机制。它使用了一个不太为人所知的 PureMVC 类 Observer；框架正是用它来通知感兴趣的 Mediator 并触发 Command 的（提示：对于 Command 注册中涉及的所有 Notification，Controller 就是感兴趣的 Observer）。ServiceRequest 将继承 Observer，使调用方能够指定自己的身份，并公开请求数据和响应数据的属性。StoryReadingProxy 将接收 ServiceRequest，而不再接收 StoryVO。它会一直持有 ServiceRequest，直到结果返回，然后调用其 notifyObserver() 方法，向发起调用的 Command 回传一个 Notification。Command 的 execute() 方法结束后仍然存在、没有被垃圾回收，是因为 ServiceRequest 还持有对它的引用。只要 Proxy 继续保留 ServiceRequest，就仍然能够向这个 Command 作出响应。

· SearchStoryCommand 在其 execute() 方法中：
- 分别从 Notification 的 body 和 type 中取出 StoryVO 存根与搜索词。
- 创建一个 ServiceRequest，将 StoryVO 存根作为 requestData，将自身的 searchStory() 方法作为回调，并将自身作为调用方。

- 调用 StoryReadingProxy.loadStory()，并传入 ServiceRequest。
· StoryReadingProxy 在其 loadStory() 方法中：
- 调用 Flex RemoteObject 实例上的一个方法，并立即获得一个 Flex AsyncToken 作为返回值（这个令牌会保留在内存中，并随服务响应一起返回）。
- 将 ServiceRequest 存放在允许动态属性的 AsyncToken 对象上。
- 调用 AsyncToken 的 addResponder() 方法，将自身设置为此次调用的响应方（因为 StoryReadingProxy 实现了 Flex 的 IResponder 接口，所以它的 result() 和 fault() 方法随后会被调用）。
· 稍后，服务器响应返回，StoryReadingProxy 的 result() 方法被调用，并执行以下操作：
- 从 ResultEvent 的 result 属性中取出 StoryVO。
- 从 ResultEvent 的 token 属性中取出 AsyncToken。
- 从 AsyncToken 动态设置的 request 属性中取出 ServiceRequest。
- 将加载好的 StoryVO 设置到 ServiceRequest 的 resultData 属性。
- 创建一个 Notification，将 ServiceRequest 放入消息体。
- 调用 ServiceRequest 的 notifyObserver()，通知 Observer（即 SearchStoryCommand）。
· SearchStoryCommand 的回调方法 searchStory() 收到通知后，检查通知名称，并采取以下两种操作之一：
- 对于 ServiceRequest.RESULT_OK，在故事全文中搜索指定搜索词，并通过 AppConstants.SEARCH_RESULT 通知发出首次出现的位置。
- 对于 ServiceRequest.RESULT_FAIL，通过 AppConstants.REPORT_FAILURE 通知发出失败消息，交由 View 层的标准错误报告机制处理。

服务请求

类

ServiceRequest.as

代码

```
package com.futurescale.sa.model.request
{
    import org.puremvc.as3.patterns.observer.Observer;

    public class ServiceRequest extends Observer
```

```
    {
        public static const RESULT_OK:String = "result/ok";
        public static const RESULT_FAIL:String = "result/fail";

        public var hasCallback:Boolean = false;
        public var requestData:Object;
        public var resultData:Object;

        public function ServiceRequest( requestData:Object = null,
                                        callback:Function  = null,
                                        caller:Object      = null ) {
            // 保存 Observer 信息
            super( callback, caller );

            // 保存请求数据
            this.requestData = requestData;

            // 记录是否指定了完整的 Observer 信息
            hasCallback = ( callback != null && caller != null );
        }
    }
}
```

搜索故事命令

类

SearchStoryCommand.as

代码

```
package com.futurescale.sa.controller.command.story
{
    import com.futurescale.sa.controller.constant.AppConstants;
    import com.futurescale.sa.model.proxy.StoryReadingProxy;
    import com.futurescale.sa.model.request.ServiceRequest;
    import com.futurescale.sa.model.vo.StoryVO;

    import org.puremvc.as3.interfaces.INotification;
    import org.puremvc.as3.interfaces.IProxy;
    import org.puremvc.as3.patterns.command.SimpleCommand;

    public class SearchStoryCommand extends SimpleCommand
    {
        private var term:String; // 搜索词

        override public function execute( note:INotification ):void
        {
            // 从通知的 type 中获取搜索词
            term = note.getType();

            // 从通知的 body 中获取故事存根
            var storyVO:StoryVO = StoryVO( note.getBody );
```

```
            // 获取 StoryReadingProxy
            var proxy:IProxy = facade.retrieveProxy( StoryReadingProxy.NAME );
            var readerProxy:StoryReadingProxy = StoryReadingProxy( proxy );

            // 创建 ServiceRequest
            var request:ServiceRequest =
                new ServiceRequest( storyVO, searchStory, this );

            // 加载故事
            readerProxy.loadStory( request );
        }

        private function searchStory( note:INotification ):void
        {
            // 从通知的 body 中获取已完成的请求
            var request:ServiceRequest = ServiceRequest( note.getBody() );

            // 处理结果
            switch ( note.getName() ) {

                // RESULT_OK：查找首次出现的位置，发送包含结果的通知
                case ServiceRequest.RESULT_OK:
                    var story:StoryVO = StoryVO( request.resultData );
                    var firstOccurance:int = story.getText().indexOf( term );
                    sendNotification( AppConstants.SEARCH_RESULT, firstOccurance );
                    break;

                // RESULT_FAIL：通过通知发送要显示的失败消息
                case ServiceRequest.RESULT_FAIL:
                    var message:String = String( request.resultData );
                    sendNotification( AppConstants.REPORT_FAILURE, message );
                    break;
            }
        }
    }
}
```

故事阅读代理

类

StoryReadingProxy.as

代码

```
package com.futurescale.sa.model.proxy
{
    import com.futurescale.sa.model.request.ServiceRequest;
    import com.futurescale.sa.model.vo.StoryVO;

    import mx.rpc.AsyncToken;
    import mx.rpc.IResponder;
    import mx.rpc.events.FaultEvent;
    import mx.rpc.events.ResultEvent;
```

```
import mx.rpc.remoting.RemoteObject;

import org.puremvc.as3.patterns.observer.Notification;
import org.puremvc.as3.patterns.proxy.Proxy;

public class StoryReadingProxy extends Proxy implements IResponder
{
    public static const NAME:String = "StoryReadingProxy";
    private var service:RemoteObject;

    public function StoryReadingProxy()
    {
        super( NAME );
    }

    // 注册时创建服务
    override public function onRegister():void
    {
        service = new RemoteObject("StoryReadingService");
    }

    // 从远程服务加载故事
    public function loadStory( request:ServiceRequest ):void
    {
        var story:StoryVO = request.requestData as StoryVO;
        var token:AsyncToken =  service.loadStory( story );
        token.request = request;    // 保留请求
        token.addResponder( this ); // 由此代理响应
    }

    // 处理已加载的故事
    public function result( data:Object ):void
    {
        var event:ResultEvent      = ResultEvent(data);
        var token:AsyncToken       = event.token;
        var request:ServiceRequest = token.request;
        request.resultData         = event.result; // 已加载的故事
        if ( request.hasCallback ) {
            var note:Notification;
            note = new Notification( ServiceRequest.RESULT_OK, request );
            request.notifyObserver( note );
        }
    }

    // 处理故事加载失败
    public function fault( info:Object ):void
    {
        var event:FaultEvent       = FaultEvent(data);
        var token:AsyncToken       = event.token;
        var request:ServiceRequest = token.request;
        request.resultData         = event.message.toString();
        if ( request.hasCallback ) {
            var note:Notification;
            note = new Notification( ServiceRequest.RESULT_FAIL, request );
            request.notifyObserver( note );
```

```
            }
        }
    }
}
```

复用模型层

本书反复强调，Model 层应当始终不了解应用程序的其余部分。这意味着 Proxy 永远不应了解（也就是导入）视图组件、Mediator 或 Command。同时，它也不应发送定义在 ApplicationFacade、AppConstants 或其他应用程序专属位置的 Notification。

开发 StoryArchitect 这样的小型应用程序时，你可能会预想到，最终还需要平板电脑版和手机版等配套应用。这些潜在的应用可能拥有自己的视图组件、用例和业务逻辑。保持 Model 可复用将大有益处；不过，只要确信自己采取了适当的防范措施，就不必马上将它移到独立项目中。你可以等到开始开发第一个配套应用时，再把 Model 层代码移到单独的库项目中。如果已经确定会构建其他应用（也许它们正在并行开发），那么可以从一开始就将 model 包放在库项目中。

将 Model 层打包到独立库中的另一个充分理由，是可以针对其中的类编写单元测试。我们都知道，软件开发的现实条件并不总能给我们留下编写单元测试的时间，也并非所有开发者都具备编写测试的自律、经验或意愿。无论你有多少时间做单元测试、对它抱有什么态度，最值得争取覆盖的都是 Model 层。根据我的经验，用户界面行为和业务逻辑更适合交由称职且积极主动的 QA 人员测试。而数据的表示与持久化则非常适合单元测试；既然应用程序的行为和外观都依赖 Model 层的可靠性，至少花一点时间测试它就是合理的。在 Adobe FlashBuilder 中，你可以很快为某个类创建新的单元测试，工具会在项目中自动生成一个测试应用程序，与主应用程序放在一起。但是，把单元测试和应用程序挤在同一个项目中，可能意味着每次修改应用程序后，都得枯燥地等待测试代码编译，严重拖累工作效率。为 StoryArchitect 创建单元测试时，这个问题变得难以忍受，迫使我们进行拆分；但当 App、Model 和 Tests 分别处于独立项目后，工作秩序和效率就完全恢复了。

因此，无论是要让两个应用程序共享 Model 层，想在独立项目中测试它，还是为了将来复用而希望确认 Model 层与应用程序已经彻底解耦，把 Model 层移到一个新的库项目中，这项简单的工作都值得去做。它应该用不了 10

分钟，却能带来实实在在的好处。具体步骤取决于你使用的 IDE，以及是否使用代码仓库、使用什么代码仓库。

步骤 1：创建新的 Flex 库项目

本书希望保持对 IDE 的独立性。随着新的项目选项出现，对话框会发生变化，Flex 和 AIR 产品也仍在快速演进。但多年来适用于 Flex 及其库的基本步骤始终相同。由于我熟悉这些操作，这里会给出 FlashBuilder 的菜单路径。我相信其中大部分也适用于其他基于 Eclipse 的 Flex IDE，但实际情况可能有所不同。无论如何，对话框截图很快就会过时，对其他 IDE 的用户也没有用，所以这里不提供这些截图。

1. 创建新的 Flex 库项目。在 FlashBuilder 中：File → New → Flex Library Project → StoryArchitectModel。
2. 在项目中添加 libs 文件夹，并放入 PureMVC_AS3_2_0_4.swc 的副本；该文件可以在 PureMVC AS3 标准版的下载压缩包中找到。在 FlashBuilder 中：New → Folder → libs。
3. 将 libs 文件夹加入库项目的构建路径。在 FlashBuilder 中：Project → Properties → Flex Library Build Path → Add SWC Folder → libs。
4. 在库项目的 src 文件夹中创建基本包结构（例如 com.futurescale.sa），然后从应用程序中复制 model 包。如果你使用 SVN 等代码仓库，并且要将类从一个项目移出，不要直接从一个项目树复制并粘贴到另一个项目中，因为源项目的仓库信息也会被带过去，导致提交新项目时 IDE 无法正确处理。应当右键单击项目的 src 文件夹，选择 Team → Export，将文件夹结构导出到桌面，然后再从那里复制所需的文件夹和类。
5. 编译库，确认 bin 文件夹中生成了 StoryArchitectModel.swc。在 FlashBuilder 中：Project → Clean… → StoryArchitectModel。
6. 按照平时共享项目的流程，将项目添加到代码仓库中。

步骤 2：将库项目加入应用项目的 Flex 构建路径

现在，你已经创建了包含 Model 层副本的独立项目，并且能够成功生成 SWC 文件，可以开始在各个应用程序中使用这个库了，包括主应用、单元测试应用、移动版等。

1. 从应用程序中删除 model 包；现在改为引用库项目。
2. 将库项目加入应用项目的构建路径。在 FlashBuilder 中：Project → Properties → Flex Build Path → Add Project → StoryArchitect。
3. 编译应用程序，确认 bin-debug 文件夹中生成了 StoryArchitectModel.swf。在 FlashBuilder 中：Project → Clean… → StoryArchitect。
4. 启动应用程序并进行交互测试。
5. 按照平时的流程，将项目提交到代码仓库。

进阶练习：重构启动过程

现在，Model 层已经成为一个独立、可复用的库。App 和 Test 项目引用它，而它不引用这两个项目，从而确保应用程序中最重要的关注点分离得到了落实。此时你可能还想再做一件事，尤其是在计划或已经着手构建另一个同样使用该 Model 层的应用时：复用 StartupCommand 中的“模型准备”阶段。

· 在库项目中：
1. 创建新的 controller 包，与 model 包同级（例如 com.futurescale.sa.controller）。
2. 将 App 项目中的 StartupCommand 复制到 controller 包，并将其重命名为 PrepareModelCommand。
3. 在 PrepareModelCommand 中确认类名正确，删除所有“视图准备”代码及相关 import 语句。
4. 编译库，确认 bin 文件夹中生成了 StoryArchitectModel.swc。在 FlashBuilder 中：Project → Clean… → StoryArchitectModel。
· 在主应用项目中：
1. 将 StartupCommand 重命名为 PrepareViewCommand（务必取消选中更新引用的选项！）。
2. 在 PrepareViewCommand 中确认类名正确，删除所有“模型准备”代码及相关 import 语句。
3. 创建一个继承 MacroCommand 的新 StartupCommand 类。
4. 重写 initializeMacroCommand()，按顺序将 PrepareModelCommand 和 PrepareViewCommand 添加为子命令。
5. 编译应用程序，确认 bin-debug 文件夹中生成了 StoryArchitect.swf。

在 FlashBuilder 中：Project → Clean… → StoryArchitect。
6. 启动应用程序并进行交互测试。

· 现在你已经有了一个 MacroCommand，它执行独立的 SimpleCommand，分别处理“视图准备”和“模型准备”阶段。不妨顺势将 Command 的注册也从 ApplicationFacade 移到一个 SimpleCommand 中。在 App 项目中：

1. 复制 PrepareViewCommand，并将副本命名为 PrepareControllerCommand。
2. 在 PrepareControllerCommand 中确认类名正确，删除 execute() 方法中的所有代码及相关 import 语句。
3. 在 StartupCommand 重写的 initializeMacroCommand() 方法中，将 PrepareControllerCommand 添加为子命令，排在其他子命令之前。
4. 从 ApplicationFacade 中复制整段 Command 注册代码，并粘贴到 PrepareControllerCommand 的 execute() 方法中。在该 Command 中，需要为每个 registerCommand() 调用加上 facade. 前缀，例如 facade.registerCommand(AppConstants.MY_NOTE, MyCommand)。
5. 将 StartupCommand 的注册移到 startup() 方法中，并放在发送 STARTUP 通知之前。
6. 删除 ApplicationFacade 的 initializeController() 方法及所有相关 import 语句。
7. 看看 ApplicationFacade 现在变得多么精简。
8. 编译应用程序，确认 bin-debug 文件夹中生成了 StoryArchitectModel.swf。在 FlashBuilder 中：Project → Clean… → StoryArchitect。
9. 启动应用程序并进行交互测试。
10. 按照平时的流程，将项目提交到代码仓库。

应用程序外观

类

`ApplicationFacade.as`

代码

```
package com.futurescale.sa
{
```

```
        import com.futurescale.sa.controller.command.startup.StartupCommand;
        import com.futurescale.sa.controller.constant.AppConstants;

        import org.puremvc.as3.patterns.facade.Facade;

        public class ApplicationFacade extends Facade
        {
            /**
             * 单例实例的工厂方法。
             */
            public static function getInstance( ) : ApplicationFacade
            {
                if ( instance == null ) instance = new ApplicationFacade( );
                return instance as ApplicationFacade;
            }

            /**
             * 用于从应用程序启动 PureMVC
             * 运行机制的便捷方法。
             */
            public function startup( app:StoryArchitect ):void
            {
                registerCommand( AppConstants.STARTUP, StartupCommand );
                sendNotification( AppConstants.STARTUP, app );
            }
        }
    }
```

启动命令

类

StartupCommand.as

代码

```
    package com.futurescale.sa.controller.command.startup
    {
        import org.puremvc.as3.patterns.command.MacroCommand;

        /**
         * 为启动过程的各个阶段执行独立命令
         */
        public class StartupCommand extends MacroCommand
        {
            override protected function initializeMacroCommand():void
            {
                addSubCommand( PrepareControllerCommand );
                addSubCommand( PrepareModelCommand );
                addSubCommand( PrepareViewCommand );
            }
        }
    }
```

准备控制器命令

类

PrepareControllerCommand.as

代码

```
package com.futurescale.sa.controller.command.startup
{
    import com.futurescale.sa.controller.command.edit.AddItemCommand;
    import com.futurescale.sa.controller.command.edit.ApplyChangesCommand;
    import com.futurescale.sa.controller.command.edit.ApplySelectionCommand;
    import com.futurescale.sa.controller.command.edit.DeleteItemCommand;
    import com.futurescale.sa.controller.command.edit.DiscardChangesCommand;
    import com.futurescale.sa.controller.command.edit.RemoveSelectionCommand;
    import com.futurescale.sa.controller.command.series.ManageSeriesCommand;
    import com.futurescale.sa.controller.command.story.EditStoryCommand;
    import com.futurescale.sa.controller.command.story.ManageStoryCommand;
    import com.futurescale.sa.controller.constant.AppConstants;
    import com.futurescale.sa.model.proxy.StoryProxy;

    import org.puremvc.as3.interfaces.INotification;
    import org.puremvc.as3.patterns.command.SimpleCommand;

    /**
     * 准备 Controller。
     *
     * 注册 StartupCommand 之外的所有 Command
     */
    public class PrepareControllerCommand extends SimpleCommand
    {
        override public function execute( note:INotification ):void
        {
            // 由 Chooser 发起
            facade.registerCommand( AppConstants.MANAGE_STORY,
                                    ManageStoryCommand );
            facade.registerCommand( AppConstants.MANAGE_SERIES,
                                    ManageSeriesCommand );
            facade.registerCommand( AppConstants.EDIT_STORY,
                                    EditStoryCommand );
            facade.registerCommand( StoryProxy.STORY_ADDED,
                                    EditStoryCommand );

            // 由 Editor 发起
            facade.registerCommand( AppConstants.ADD_ITEM,
                                    AddItemCommand );
            facade.registerCommand( AppConstants.DELETE_ITEM,
                                    DeleteItemCommand );
            facade.registerCommand( AppConstants.APPLY_SELECTION,
                                    ApplySelectionCommand );
            facade.registerCommand( AppConstants.REMOVE_SELECTION,
                                    RemoveSelectionCommand );
            facade.registerCommand( AppConstants.DISCARD_CHANGES,
```

```
                                    DiscardChangesCommand );
            facade.registerCommand( AppConstants.APPLY_CHANGES,
                                    ApplyChangesCommand );
        }
    }
}
```

准备模型命令

类

PrepareModelCommand.as

代码

```
package com.futurescale.sa.controller.command.startup
{
    import com.futurescale.sa.model.proxy.CastProxy;
    import com.futurescale.sa.model.proxy.MilieuProxy;
    import com.futurescale.sa.model.proxy.SeriesProxy;
    import com.futurescale.sa.model.proxy.StoryProxy;

    import org.puremvc.as3.interfaces.INotification;
    import org.puremvc.as3.patterns.command.SimpleCommand;

    /**
     * 准备 Model。
     */
    public class PrepareModelCommand extends SimpleCommand
    {
        override public function execute( note:INotification ):void
        {
            // 创建 Proxy
            var castProxy:CastProxy     = new CastProxy();
            var milieuProxy:MilieuProxy = new MilieuProxy();
            var storyProxy:StoryProxy   =
                new StoryProxy( milieuProxy, castProxy );
            var seriesProxy:SeriesProxy =
                new SeriesProxy( storyProxy, milieuProxy, castProxy);

            // 注册 Proxy
            facade.registerProxy( castProxy );
            facade.registerProxy( milieuProxy );
            facade.registerProxy( storyProxy );
            facade.registerProxy( seriesProxy );
        }
    }
}
```

准备视图命令

类

PrepareViewCommand.as

代码

```
package com.futurescale.sa.controller.command.startup
{
    import com.futurescale.sa.view.context.SelectionContext;
    import com.futurescale.sa.view.mediator.ApplicationMediator;
    import com.futurescale.sa.view.mediator.ChooserMediator;
    import com.futurescale.sa.view.mediator.EditorMediator;
    import com.futurescale.sa.view.popup.mediator.AlertPopupMediator;
    import com.futurescale.sa.view.popup.mediator.ConfirmationPopupMediator;
    import com.futurescale.sa.view.popup.mediator.SeriesPopupMediator;
    import com.futurescale.sa.view.popup.mediator.StoryPopupMediator;

    import org.puremvc.as3.interfaces.INotification;
    import org.puremvc.as3.patterns.command.SimpleCommand;
    import org.puremvc.as3.patterns.proxy.Proxy;

    /**
     * 准备 View。
     *
     * 创建 SelectionContext，缓存它以供 Command 访问，
     * 并将它注入应用程序，与显示列表共享。
     * 同时，为初始视图组件建立中介关系，
     * 并注册弹出窗口的中介器。
     */
    public class PrepareViewCommand extends SimpleCommand
    {
        override public function execute( note:INotification ):void
        {
            // 从通知的 body 中获取应用程序
            var app:StoryArchitect = StoryArchitect( note.getBody() );

            // 注册一个便利的 Proxy 来保存 SelectionContext。
            //
            // SelectionContext 跟踪 View 中选中的项目，
            // 不属于领域模型，但可以利用框架的 Proxy 类
            // 快速缓存这个数据实体，
            // 它仅由 View 层和 Controller 层使用。
            var selectionContext:SelectionContext = new SelectionContext();
            var scProxy:Proxy = new Proxy( SelectionContext.NAME,
                                           selectionContext );
            facade.registerProxy( scProxy );

            // 为初始视图组件建立中介关系
            facade.registerMediator( new ApplicationMediator( app ) );
            facade.registerMediator( new EditorMediator( app.editor ) );
            facade.registerMediator( new ChooserMediator( app.chooser ) );
```

```
            // 注册弹出窗口中介器（见第 9 章）
            facade.registerMediator( new AlertPopupMediator() );
            facade.registerMediator( new ConfirmationPopupMediator() );
            facade.registerMediator( new StoryPopupMediator() );
            facade.registerMediator( new SeriesPopupMediator() );
        }
    }
}
```

重构后的项目

下面展示了第一轮开发迭代结束时，StoryArchitect 的 Model 项目（图 8-1）、App 项目（图 8-2）和 Test 项目（图 8-3）的源代码结构。

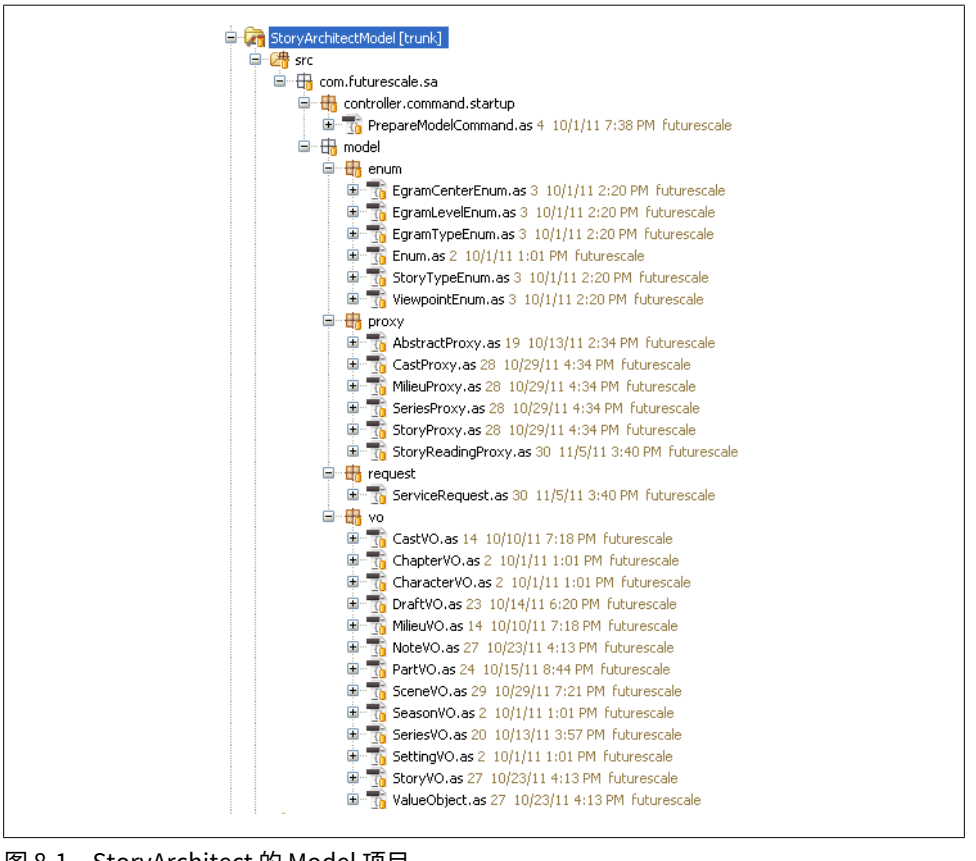


图 8-1　StoryArchitect 的 Model 项目

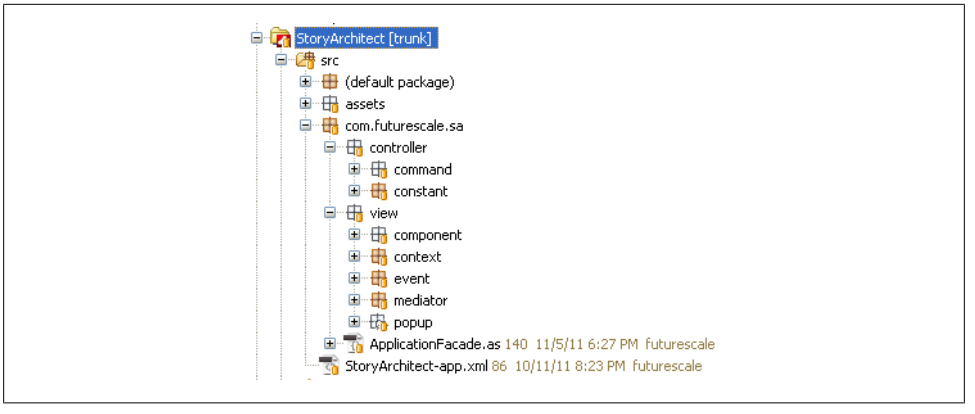


图 8-2　StoryArchitect 应用程序

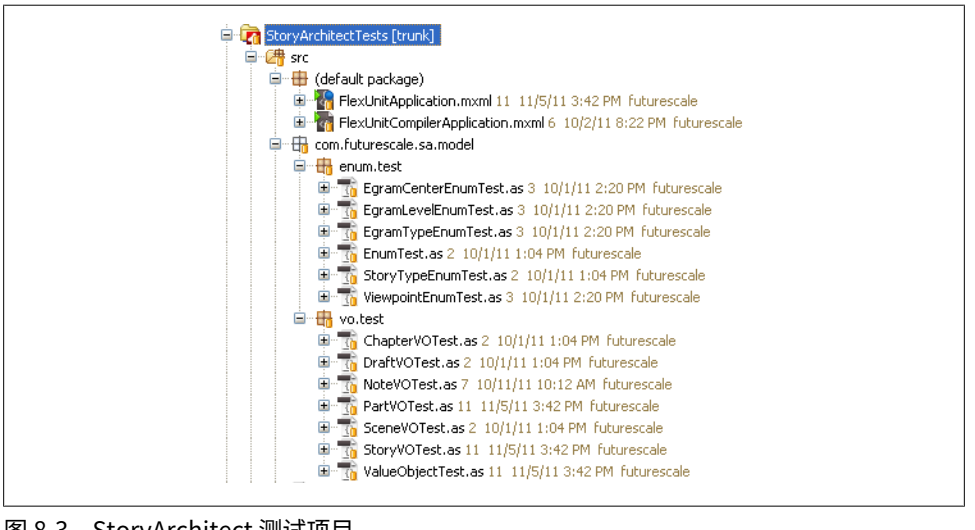


图 8-3　StoryArchitect 测试项目

第 9 章
视图进阶主题

PureMVC 运行机制初始化时，应用程序的大部分用户界面可能已经创建好，因此启动时就可以为其建立中介关系。但许多情况并不适合这种简单做法。本章将讨论视图组件生命周期中几类主要的棘手情况。

管理弹出窗口

到目前为止，本书只介绍了显示列表中视图组件的中介关系和交互。这些组件由主应用程序声明若干顶层组件，再由顶层组件声明其他组件，如此逐层构成。在 StoryArchitect 中，我们可以在 PrepareViewCommand 中很容易地为应用程序及其两个顶层组件建立中介关系，并接收其内部所有子组件发出的 Event。

然而，应用程序还使用了一些弹出窗口，它们从根本上需要另一种中介策略。这些对象是临时的，会在运行期间随时显示和移除。在 Flex 中，PopUpManager 单例负责弹出窗口的显示、居中和移除。若要让弹出窗口在应用程序上方居中，就必须传入应用程序本身的引用。过去，这个引用通过另一个 Flex 单例 Application.application 获取，但该方式已经弃用，取而代之的是 FlexGlobals.topLevelApplication。因此，如果你在开发一个大型应用程序，并在各处频繁创建弹出窗口，就可能在 IDE 的 Problems 视图中看到大量弃用警告，淹没其他可能更重要的警告，最终让你对所有警告都视而不见，也温和地提醒你：Flex 始终在变化。我曾在一个客户的应用程序中看到一百多条这样的警告。这意味着仅仅因为一个框架方法被弃用，就得费力地重新检查所有相关代码。

以往的做法

PureMVC 社区已经为如何最好地管理弹出窗口探索了相当长时间。基本做法包括：

- 在任何需要弹出窗口的 Command 中创建窗口，为它设置 Event 监听器，使用 PopUpManager 打开并居中显示，再从弹出窗口内部或 Command 的 Event 处理器中将其关闭。单例向来有“邪恶”的名声，前面提到的大量弃用警告也说明，它们确实可能带来麻烦。因为我们能够在任何地方访问单例，所以往往就这样做了，最终让代码充满不合理的依赖关系。更不用说还会重复大量与 PopUpManager 交互的代码，因为不管是哪种弹出窗口，所用的那五六行代码几乎总是相同的。
- 使用一个 PopupMediator 处理所有弹出窗口，在收到通知时创建所需的窗口。结果再由 PopupMediator 通过通知发出，触发另一个 Command 或 Mediator。这样将所有 PopUpManager 交互集中到一个角色中，确实不错；但也回到了第 8 章异步 Proxy 交互中见过的“分离调用方与响应方”模式。为了应对显示弹出窗口并等待用户提交数据所造成的异步中断，必须把用例拆成多个步骤。
- 让 PopupMediator 处理与 PopUpManager 的交互，但由 Command 创建弹出窗口，为窗口派发的 Event 设置监听器，并通过通知将组件发送给 PopupMediator。这样仍然可以把一个用例的全部或大部分逻辑放在单个 Command 中，但同时也让 Command 与视图组件产生了耦合。回看第 1 章的经典 MVC 图，确实写着 Controller 层可以直接更新 View 层，所以在 Command 中创建或操作视图组件似乎并非不合理。虽然 Mediator 的职责是充当应用程序中某个视图组件的唯一联系点，但在这里，让 Command 负责创建和监听看起来似乎是个好主意；否则，人们很容易把大量业务逻辑塞进 Mediator，而这是应当避免的。事实证明，即使是处理弹出窗口这种看似棘手的场景，让 Command 依赖具体视图组件也不是好主意。我最近参与过一个大型项目，其中大量业务逻辑使用 Halo（Flex 3）组件作为弹出窗口，也经常调用 Halo 的 Alert.show()。所有这些代码由 Flex 网页应用、AIR 桌面应用，以及后来新增的 Flex 移动应用共享。然而，Flex 移动项目只能使用 Spark（Flex 4）组件，根本不能使用 Halo。我们知道主界面会完全不同，却以为弹出窗口和警告框仍能工作，结果并非如此。于是，共享的业务逻辑现在都必须重构，移除对视图组件的依赖。这再次提醒我们，将业务逻辑与边界对象隔离是多么重要。

另一种做法

上面的解决方案过去都能工作，但显然各有不足。我们需要一种不同的做法，可以作为最佳实践广泛推荐。

还记得第 8 章中，我们如何让 Command 向 Proxy 发送一个规范化请求，执行异步操作，并在稍后收到包含结果的通知吗？仔细想想，这与弹出窗口面临的问题完全相同。在两种情况下，Command 都在执行业务逻辑，并在某个时刻需要一份必须异步获取的数据。我们希望 Command 能直接收到结果通知，不希望仅仅因为异步中断就不得不把业务逻辑拆成多个步骤。

Proxy 从服务获取数据，而弹出窗口从用户获取数据；这是两者唯一的区别。它们都是“异步边界交互”的例子。我们可以再次运用第 8 章中的模式：基于 PureMVC 的 Observer 类构建请求，将请求数据以及通知调用方的能力封装起来，通过这一规范化请求机制实现与弹出窗口的解耦交互。

添加、更改和删除故事的用例

StoryArchitect 中有若干弹出窗口，用于创建、修改或删除 StoryVO 和 SeriesVO，显示警告，以及在执行破坏性操作前请求确认。

以创建新故事为例。在这个用例中，你单击 Chooser 组件上的“添加故事”（+）按钮，如图 9-1 所示，随后显示 StoryPopup（见图 9-2），收集创建新 StoryVO 所需的基本信息。只要没有取消或关闭对话框，提交信息后，弹出窗口便会关闭，新故事被创建并保存到磁盘，索引也会加入这个新故事。接着，应用程序切换到 Editor，你就可以立即开始写作。

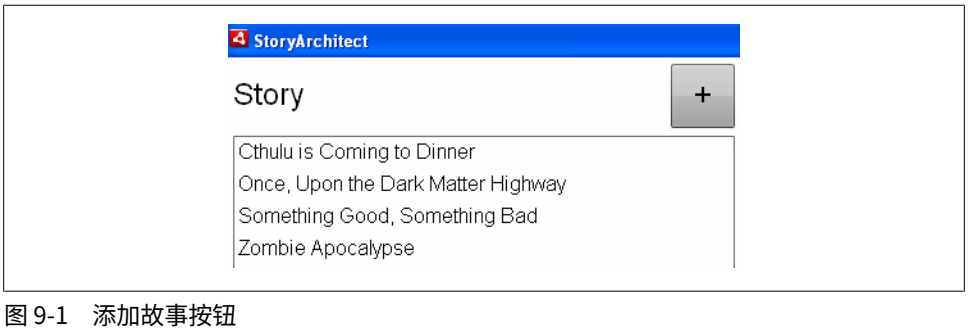


图 9-1　添加故事按钮

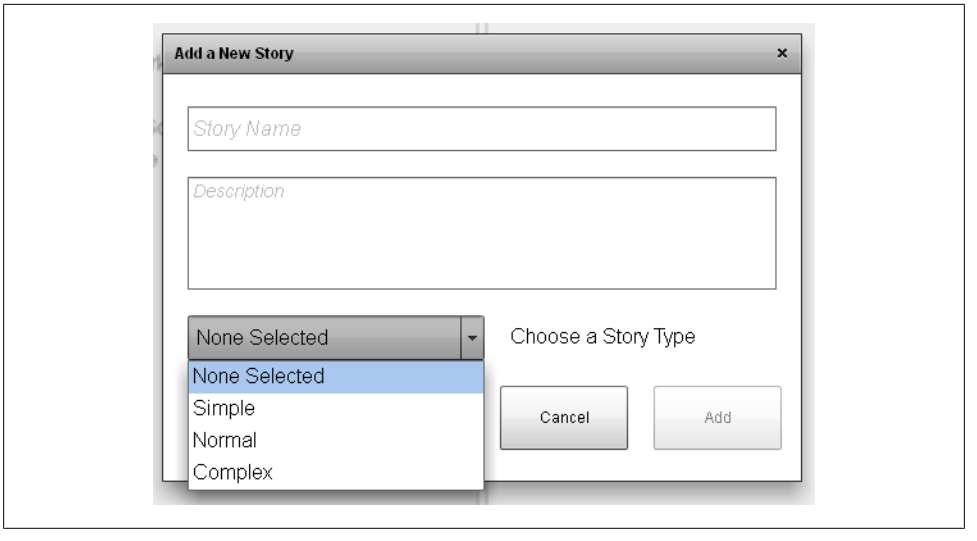


图 9-2　添加故事弹出窗口

你可以查看本书前面 Chooser 和 ChooserMediator 的代码，了解单击“添加故事”（+）按钮后，ManageStoryCommand 最终是如何被执行的。稍后我们会详细介绍弹出窗口本身的管理方式；但在阅读代码之前，先考虑另外几个相关用例，它们同样由 ManageStoryCommand 和这个弹出窗口类来处理。

我们还需要能够修改现有故事的名称和描述（但不能修改故事类型），或者将其删除。选择删除故事时，应先使用一个可复用的确认弹出窗口征求确认。

在 Chooser 中选中故事，再单击图 9-3 所示的“管理”按钮，就会打开同一个弹出窗口（见图 9-4），但此时不能编辑故事类型，只会显示创建故事时所选类型的说明。窗口标题也不同，还会出现一个在添加故事用例中不可见的“删除”按钮。

图 9-5 所示的删除故事确认操作，代表同一个用例中的又一次异步中断。如果把流程拆成一系列 Command 和 Notification，这里可能还得增加另一个 Command。而采用这种新方法，我们可以用单个命令处理添加、更改以及删除／确认等用例，并且命令本身不依赖任何具体的弹出窗口。

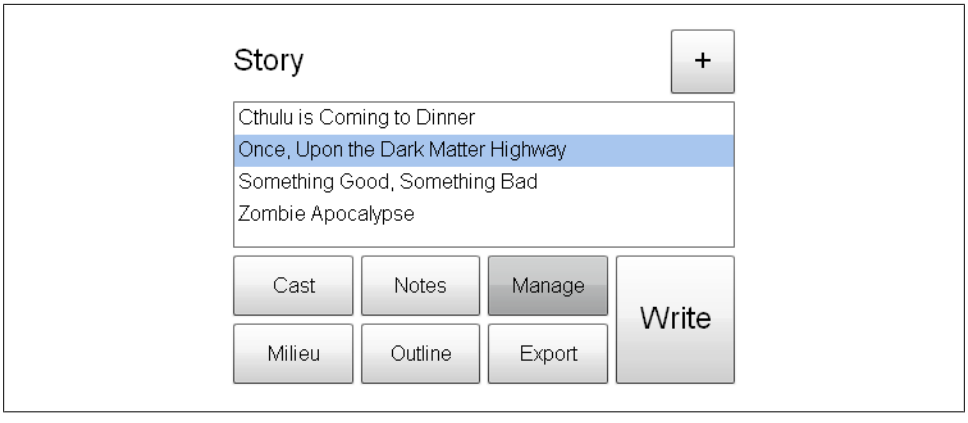


图 9-3　管理故事按钮



图 9-4　管理故事弹出窗口



图 9-5　删除故事确认窗口

采用这种方式与 Model 层通信时，结果通常只有成功／故障两种，而弹出窗口的交互可能更加复杂，所以我们为弹出窗口构建了一个略有不同的实现。除了规范化请求类，还会有一个自定义的 PopupActionEvent 类：异步 Notification 将这个事件传回调用方，而不是由原始请求携带结果返回。这样，弹出窗口就可以派发多种 PopupActionEvent 类型，并标明是否应当关闭窗口。

所有繁琐的弹出窗口处理都将放入 AbstractPopupMediator。窗口创建后，中介器会向它查询可能派发的 PopupActionEvent 类型列表。这样，AbstractPopupMediator 就知道应该设置哪些事件监听器，不需要由子类或发起调用的 Command 来做这件事。

每种弹出窗口都会有一个对应的 AbstractPopupMediator 子类。这个子类只需要关注用于调用该窗口的一个 Notification，并重写一个工厂方法，返回窗口的新实例即可。这样就能在需要时为弹出窗口注册不同版本的 Mediator，例如桌面和网页应用使用 Halo 版本，而移动应用使用 Spark 版本。

接下来逐个类查看代码。首先，简要了解各个组成部分：

弹出窗口请求
: 基于 Observer，将请求和定制弹出窗口的能力，与窗口派发 PopupActionEvent 时的异步通知能力结合起来。

弹出窗口操作事件
: 供弹出窗口向调用方回传信息的专用事件。

弹出窗口接口
: 弹出窗口使用的接口，要求提供 setData() 和 getEvents() 方法。

确认弹出窗口
: 实现 IPopup；大小、提示文字和标题可调整；发送 PopupActionEvent.OK 和 CANCEL。

故事弹出窗口
: 实现 IPopup；标题可调整；发送 PopupActionEvent.ADD、SAVE、DELETE 和 CANCEL。

抽象弹出窗口中介器
: 封装所有直接处理弹出窗口的操作的基类，包括查询窗口的 PopupActionEvent 类型、监听这些事件、打开、关闭、居中，以及通知请求方。

故事弹出窗口中介器
: AbstractPopupMediator 的子类，创建 StoryPopup。

确认弹出窗口中介器
: AbstractPopupMediator 的子类，创建 ConfirmationPopup。

管理故事命令
一个 SimpleCommand，负责 StoryVO 的添加、更改、删除逻辑以及与 Proxy 的交互，包括调用 StoryPopup 和 ConfirmationPopup，并处理它们传回的 PopupActionEvent。

弹出窗口请求

类

PopupRequest.as

代码

```
package com.futurescale.sa.view.popup.request
{
    import flash.display.DisplayObject;

    import mx.core.FlexGlobals;
    import mx.managers.PopUpManagerChildList;

    import org.puremvc.as3.patterns.observer.Observer;

    /**
     * PopupRequest 携带请求中介器创建弹出窗口
     * 所需的全部信息。
     */
    public class PopupRequest extends Observer
    {
        // 所有请求通知名称的前缀
        private static const NAME:String = "PopupRequest/";

        // 在此添加新的请求通知名称……
        public static const ALERT_POPUP:String        = NAME + "AlertPopup";
        public static const CONFIRMATION_POPUP:String = NAME + "ConfirmationPopup";
        public static const SERIES_POPUP:String       = NAME + "SeriesPopup";
        public static const STORY_POPUP:String        = NAME + "StoryPopup";
        public static const CHARACTER_POPUP:String    = NAME + "CharacterPopup";
        public static const SETTING_POPUP:String      = NAME + "SettingPopup";

        /**
         * 构造函数。
         * 示例：new PopupRequest( handlePopupNotification, this );
         */
        public function PopupRequest( callback:Function=null,
                                      caller:Object=null )
        {
            super( callback, caller );
            hasCallback = ( callback != null && caller != null );
        }

        // 请求包含回调（由构造函数设置）。
        public var hasCallback:Boolean = false;
```

```
        // 显示列表中的父对象，弹出窗口在其上方打开。
        public var parent:DisplayObject =
            FlexGlobals.topLevelApplication as DisplayObject;

        // 用于放置弹出窗口的子对象列表
        public var childList:String = PopUpManagerChildList.PARENT;

        // 弹出窗口是否应居中?
        public var center:Boolean = true;

        // 弹出窗口是否应为模态窗口?
        public var modal:Boolean = true;

        // 弹出窗口可使用的可选数据，
        // 例如标题或宽度、高度
        public var data:Object = {};

        // 创建新的警告窗口请求的便捷方法
        public static function getAlertRequest( message:String,
                                                windowTitle:String="Alert",
                                                callback:Function=null,
                                                caller:Object = null):PopupRequest {
            var request:PopupRequest = new PopupRequest( callback, caller );
            request.childList = PopUpManagerChildList.POPUP;
            request.data.windowTitle = windowTitle;
            request.data.message = message;
            return request;
        }
    }
}
```

弹出窗口操作事件

类

PopupActionEvent.as

代码

```
package com.futurescale.sa.view.popup.event
{
    import flash.events.Event;

    public class PopupActionEvent extends Event
    {
        // 所有弹出窗口操作事件类型的前缀
        private static const NAME:String = "PopupEvent/";

        // 在此添加新的事件名称……
        public static const CANCEL:String  = NAME + "cancel";
        public static const OK:String      = NAME + "ok";
        public static const ADD:String     = NAME + "add";
        public static const SAVE:String    = NAME + "save";
```

```
        public static const DELETE:String  = NAME + "delete";

        /**
         * 构造函数。
         *
         * 由弹出窗口派发，由 PopupMediatorBase 捕获，
         * 再传回原始调用方进行解释。
         */
        public function PopupActionEvent( type:String,
                                          data:Object = null,
                                          closePopup:Boolean=true ) {
            super( type );
            this.data         = data;
            this.closePopup = closePopup;
        }

        public var data:Object;           // 可选数据
        public var closePopup:Boolean;    // 关闭弹出窗口?

    }
}
```

弹出窗口接口

类

IPopup.as

代码

```
package com.futurescale.sa.view.popup.component
{
    import mx.core.IFlexDisplayObject;

    /**
     * 弹出窗口接口
     */
    public interface IPopup extends IFlexDisplayObject
    {
        function setData( data:Object ):void;
        function getEvents():Array;
    }
}
```

确认弹出窗口

类

ConfirmationPopup.mxml

代码

```
<?xml version="1.0" encoding="utf-8"?>
<!-- 确认弹出窗口 -->
<s:TitleWindow xmlns:fx="http://ns.adobe.com/mxml/2009"
               xmlns:s="library://ns.adobe.com/flex/spark"
               title="{windowTitle}" width="200" height="150" close="onCancel()"
               implements="com.futurescale.sa.view.popup.component.IPopup">

    <fx:Script>
        <![CDATA[
            import com.futurescale.sa.view.popup.event.PopupActionEvent;

            // IPopup 接口要求实现
            public function setData( data:Object ):void
            {
                if ( data.windowTitle ) windowTitle = data.windowTitle;
                if ( data.promptText )  promptText  = data.promptText;
                if ( data.width )       width       = data.width;
                if ( data.height )      height      = data.height;
            }

            // IPopup 接口要求实现
            public function getEvents( ):Array
            {
                return [ PopupActionEvent.OK, PopupActionEvent.CANCEL ]
            }

            private function onOk():void
            {
                dispatchEvent( new PopupActionEvent( PopupActionEvent.OK ) )
            }

            private function onCancel():void
            {
                dispatchEvent( new PopupActionEvent( PopupActionEvent.CANCEL ) )
            }

            [Bindable] private var windowTitle:String = "Confirmation";
            [Bindable] private var promptText:String  = "Are you sure?";


    </fx:Script>

    <!-- 布局 -->
    <s:VGroup width="100%" height="100%" gap="20"
              horizontalAlign="center" verticalAlign="middle">

        <!-- 消息 -->
        <s:Label text="{promptText}"/>

        <!-- 按钮 -->
        <s:HGroup gap="20">
            <s:Button label="Cancel" click="onCancel()"/>
            <s:Button label="OK"     click="onOk()"/>
        </s:HGroup>
```

```
        </s:VGroup>

    </s:TitleWindow>
```

故事弹出窗口

类

StoryPopup.mxml

代码

```
    <?xml version="1.0" encoding="utf-8"?>
    <!-- 故事弹出窗口 -->
    <s:TitleWindow implements="com.futurescale.sa.view.popup.component.IPopup"
                   xmlns:fx="http://ns.adobe.com/mxml/2009"
                   xmlns:s="library://ns.adobe.com/flex/spark"
                   xmlns:mx="library://ns.adobe.com/flex/mx"
                   close="onCancel()" title="{windowTitle}"
                   width="500" height="350">

        <fx:Script>
            <![CDATA[
                import com.futurescale.sa.model.enum.StoryTypeEnum;
                import com.futurescale.sa.model.vo.StoryVO;
                import com.futurescale.sa.view.popup.event.PopupActionEvent;

                import mx.controls.Text;

                // IPopup 接口要求实现
                public function setData( data:Object ):void
                {
                    if ( data.story ) {
                        editing = true;
                        windowTitle = "Manage Story";
                        story = data.story as StoryVO;
                    }
                }

                // IPopup 接口要求实现
                public function getEvents( ):Array
                {
                    return [ PopupActionEvent.ADD,
                             PopupActionEvent.SAVE,
                             PopupActionEvent.DELETE,
                             PopupActionEvent.CANCEL ]
                }

                private function reapFields( vo:StoryVO ):void
                {
                    vo.name = txtStoryName.text;
                    vo.description = txtStoryDesc.text;
                    vo.type = cmbStoryType.selectedItem;
```

```
        }

        private function onAdd():void
        {
            var newStory:StoryVO = new StoryVO();
            reapFields( newStory );
            dispatchEvent(
              new PopupActionEvent( PopupActionEvent.ADD, newStory ) );
        }

        private function onSave():void
        {
            reapFields( story );
            dispatchEvent(
              new PopupActionEvent( PopupActionEvent.SAVE, story ) );
        }

        private function onDelete():void
        {
            dispatchEvent(
              new PopupActionEvent( PopupActionEvent.DELETE, story ) );
        }

        private function onCancel():void
        {
            dispatchEvent(
              new PopupActionEvent( PopupActionEvent.CANCEL ) );
        }

        [Bindable] private var windowTitle:String = "Add a New Story";
        [Bindable] private var editing:Boolean=false;
        [Bindable] private var story:StoryVO;


    </fx:Script>

    <!-- 布局 -->
    <s:layout>
        <s:VerticalLayout gap="20" horizontalAlign="center" verticalAlign="middle"/>
    </s:layout>

    <!-- 故事名称 -->
    <s:TextInput id="txtStoryName" width="460" height="35" fontSize="16"
                 prompt="Story Name" text="{story.name}"/>

    <!-- 故事描述 -->
    <s:TextArea id="txtStoryDesc" width="460" height="88" fontSize="14"
                prompt="Description" text="{story.description}"/>

    <!-- 故事类型 -->
    <s:HGroup gap="20" horizontalAlign="center"
              verticalAlign="middle">

        <!-- 组合框 -->
        <s:DropDownList id="cmbStoryType" width="232" height="35"
```

```
                        selectedItem="{(editing)?story.type:StoryTypeEnum.NONE}"
                        dataProvider="{StoryTypeEnum.comboList}" fontSize="16"
                        labelField="name" enabled="{!(editing)}"/>

            <!-- 标签 -->
            <s:Label id="lblStoryType" width="208" height="35" fontSize="16"
                     text="{StoryTypeEnum(cmbStoryType.selectedItem).description}"
                     verticalAlign="middle"/>
        </s:HGroup>

        <!-- 按钮 -->
        <s:HGroup gap="20" width="100%" horizontalAlign="right" paddingRight="15">

            <!-- 取消 -->
            <s:Button width="100" height="50" label="Cancel" click="onCancel()"/>

            <!-- 添加 -->
            <s:Button width="100" height="50" label="Add" click="onAdd()"
                      visible="{!(editing)}" includeInLayout="{!(editing)}"
                      enabled="{cmbStoryType.selectedItem != StoryTypeEnum.NONE
                      &amp;&amp; txtStoryName.text != ''}"/>

            <!-- 删除 -->
            <s:Button width="100" height="50" label="Delete" click="onDelete()"
                      visible="{editing &amp;&amp; !story.isEpisode}"
                      includeInLayout="{editing}"/>

            <!-- 保存 -->
            <s:Button width="100" height="50" label="Save" click="onSave()"
                      visible="{editing}" includeInLayout="{editing}"
                      enabled="{cmbStoryType.selectedItem != StoryTypeEnum.NONE
                                  &amp;&amp; txtStoryName.text != ''}"/>
        </s:HGroup>

    </s:TitleWindow>
```

抽象弹出窗口中介器

类

AbstractPopupMediator.as

代码

```
    package com.futurescale.sa.view.popup.mediator
    {
        import com.futurescale.sa.view.popup.component.IPopup;
        import com.futurescale.sa.view.popup.event.PopupActionEvent;
        import com.futurescale.sa.view.popup.request.PopupRequest;

        import mx.managers.PopUpManager;

        import org.puremvc.as3.interfaces.INotification;
        import org.puremvc.as3.patterns.mediator.Mediator;
```

```
import org.puremvc.as3.patterns.observer.Notification;

public class AbstractPopupMediator extends Mediator
{
    public function AbstractPopupMediator( name:String )
    {
        super( name );
    }

    /**
     * 在子类中重写。
     * 只需创建具体的弹出窗口。
     */
    protected function popupFactory():IPopup
    {
        return null;
    }

    /**
     * 收到请求通知时，由 handleNotification 方法调用。
     * 使用 popupFactory() 创建弹出窗口，将请求中的数据交给它，
     * 调用 setEventInterests 添加监听器，然后
     * 显示弹出窗口，并按需将其居中。
     */
    protected function openPopup( ) : void
    {
        var popup:IPopup = popupFactory();
        if (popup) {
            popup.setData( request.data );
            setEventInterests( popup );
            PopUpManager.addPopUp( popup,
                                   request.parent,
                                   request.modal,
                                   request.childList );
            if ( request.center ) PopUpManager.centerPopUp( popup );
        }
    }

    /**
     * 请求设置完成后、弹出窗口显示之前，由 openPopup 调用。
     * 查询弹出窗口将要派发的事件，
     * 并为每种事件设置监听器。
     */
    protected function setEventInterests( popup:IPopup ):void
    {
        for each ( var interest:String in popup.getEvents() )
        {
            popup.addEventListener( interest,
                                    handlePopupAction,
                                    false, 0, true );
        }
    }

    /**
     * 子类只注册一种感兴趣的通知，
```

```
         * 所有子类的这种通知都在这里以相同方式处理。
         */
        override public function handleNotification( note:INotification ):void
        {
            request = note.getBody() as PopupRequest;
            openPopup( );
        }

        /**
         * 子类只设置一种感兴趣的通知，
         * 所有子类的这种通知都在这里以相同方式处理。
         * 如果事件指定关闭窗口，就先将其关闭，然后
         * 将 PopupEvent 以及……通知调用方〔原注释未完〕。
         */
        protected function handlePopupAction( event:PopupActionEvent ):void
        {
            var popup:IPopup = event.target as IPopup;
            if ( event.closePopup ) removePopup( popup );
            var note:Notification = new Notification( event.type, event );
            if (request.hasCallback) request.notifyObserver( note );
            request = null;
        }

        /**
         * 当 PopupActionEvent 的 closePopup 属性为 true 时调用
         */
        protected function removePopup( popup:IPopup ):void
        {
            PopUpManager.removePopUp( popup );
        }

        // 弹出窗口存续期间临时保存请求，
        // 以便中介器通知调用方。
        protected var request:PopupRequest;

    }
}
```

故事弹出窗口中介器

类

StoryPopupMediator.as

代码

```
package com.futurescale.sa.view.popup.mediator
{
    import com.futurescale.sa.view.popup.component.IPopup;
    import com.futurescale.sa.view.popup.component.StoryPopup;
    import com.futurescale.sa.view.popup.request.PopupRequest;

    public class StoryPopupMediator extends AbstractPopupMediator
    {
```

```
        public static const NAME:String = "StoryPopupMediator";

        public function StoryPopupMediator()
        {
            super( NAME );
        }

        override public function listNotificationInterests():Array
        {
            return [ PopupRequest.STORY_POPUP ];
        }

        override protected function popupFactory():IPopup
        {
            return new StoryPopup();
        }
    }
}
```

确认弹出窗口中介器

类

ConfirmationPopupMediator.as

代码

```
package com.futurescale.sa.view.popup.mediator
{
    import com.futurescale.sa.view.popup.component.ConfirmationPopup;
    import com.futurescale.sa.view.popup.component.IPopup;
    import com.futurescale.sa.view.popup.request.PopupRequest;

    public class ConfirmationPopupMediator extends AbstractPopupMediator
    {
        public static const NAME:String = "ConfirmationPopupMediator";

        public function ConfirmationPopupMediator()
        {
            super( NAME );
        }

        override public function listNotificationInterests():Array
        {
            return [ PopupRequest.CONFIRMATION_POPUP ];
        }

        override protected function popupFactory():IPopup
        {
            return new ConfirmationPopup();
        }

    }
}
```

管理故事命令

类

ManageStoryCommand.as

代码

```
package com.futurescale.sa.controller.command.story
{
    import com.futurescale.sa.model.proxy.StoryProxy;
    import com.futurescale.sa.model.vo.StoryVO;
    import com.futurescale.sa.view.popup.event.PopupActionEvent;
    import com.futurescale.sa.view.popup.request.PopupRequest;

    import org.puremvc.as3.interfaces.INotification;
    import org.puremvc.as3.patterns.command.SimpleCommand;

    /**
     * 添加、更改或删除故事。
     *
     * 显示 StoryPopup；如果添加或保存了故事，
     * 就将其持久化。如果要删除，则先确认，
     * 然后再删除。
     */
    public class ManageStoryCommand extends SimpleCommand
    {
        private var story:StoryVO;
        private var storyProxy:StoryProxy;

        override public function execute( note:INotification ):void
        {
            var storyStub:StoryVO = StoryVO( note.getBody() ); // 添加时为 null
            storyProxy = StoryProxy( facade.retrieveProxy( StoryProxy.NAME ) );
            if ( storyStub    ) story = storyProxy.loadStory( storyStub );

            // 请求弹出窗口
            var request:PopupRequest =
                new PopupRequest( handleEditPopupNote, this );
            request.data.story = story;
            sendNotification( PopupRequest.STORY_POPUP, request );
        }

        /**
         * 处理弹出窗口操作。
         */
        private function handleEditPopupNote( actionNote:INotification ):void
        {
            var event:PopupActionEvent = PopupActionEvent( actionNote.getBody() );
            story = StoryVO( event.data );
            switch ( actionNote.getName() )
            {
                case PopupActionEvent.ADD:
                    storyProxy.addStory( story );
```

```
                    break;

                case PopupActionEvent.SAVE:
                    storyProxy.saveStory( story );
                    break;

                case PopupActionEvent.DELETE:
                    var confirm:PopupRequest =
                        new PopupRequest( handleConfirmPopupNote, this );
                    confirm.data.promptText = "Delete Story?";
                    sendNotification( PopupRequest.CONFIRMATION_POPUP, confirm );
                    break;

                case PopupActionEvent.CANCEL:
                    break;
            }
        }

        /**
         * 处理 ConfirmationPopup 操作。
         */
        private function handleConfirmPopupNote( actionNote:INotification ):void
        {
            var event:PopupActionEvent = PopupActionEvent( actionNote.getBody() );
            switch ( actionNote.getName() )
            {
                case PopupActionEvent.OK:
                    storyProxy.deleteStory(story);
                    break;

                case PopupActionEvent.CANCEL:
                    break;
            }
        }
    }
}
```

延迟实例化

还有一种与弹出窗口类似、但没那么复杂的问题：Flex 有时会推迟已声明组件的实例化。

例如，设想你有一个划分为多个功能区的会计应用程序，分别用复杂的视图组件处理应收账款、应付账款、总账等功能。在主应用程序的 MXML 中，你声明了一个名为 AppSections 的组件，它基于 Flex Halo 的 TabNavigator 类。AppSections 将各个自定义会计功能区视图组件各声明一个，作为自己的子组件。你希望为 AppSections 建立中介关系，使其他角色能在需要时发送 Notification，通过程序切换当前显示的功能区。同时，你还希望为它的每个子组件建立中介关系，因为它们代表复杂的子系统，很可能需要各自的 Mediator。

问题在于，默认情况下，TabNavigator（以及 Accordion 和 ViewStack 组件）只会创建启动时需要显示的第一个子组件，其他子组件则会延迟到导航至它们时才实例化。这样做是为了优化启动时间。对于我们假设的会计应用这类复杂系统，创建那些隐藏视图可能确实需要一些时间，让应用程序显得启动迟缓。我们都知道第一印象有多重要。因此，应用程序先快速启动；首次访问某个复杂子组件时，因为要创建它，会有短暂延迟。此后它便一直保留在显示列表中，再切换过去就很快了。可以设置 creationPolicy 属性，要求构建所有子组件，而不仅仅是第一个。如果子组件都很简单，你可能只需将这个属性设为 "all"，然后像 PrepareViewCommand 中那样，用简单的方式为子组件建立中介关系。但如果需要依靠延迟实例化加快首次呈现，就需要另一种中介策略。

在这种情况下，启动时不能为 TabNavigator 的所有子组件建立中介关系，因为它们还不存在。可以先为第一个子组件建立中介关系，再监听其他组件的创建，在它们创建后进行处理。

应用功能区 TabNavigator

类

AppSections.mxml

代码

```
<?xml version="1.0" encoding="utf-8"?>
<!-- 应用程序分区 -->
<mx:TabNavigator  xmlns:mx="http://www.adobe.com/2006/mxml"
                  xmlns:view="com.mycompany.myapp.view.components.*"
                  selectedIndex="{currentSection}">
    <mx:Script>
        <![CDATA[

            import com.mycompany.myapp.view.event.AppEvent;

            public static const AR_SECTION:Number = 0;
            public static const AP_SECTION:Number = 1;
            public static const GL_SECTION:Number = 2;

            [Bindable] public var currentSection:Number = AR_SECTION;

            private function sendEvent( type:String ):void
            {
                dispatchEvent( new AppEvent( type ) );
            }

    </mx:Script>
```

```
        <!-- 应收账款分区 -->
        <view:AcctRcv   id="acctRcv"/><!-- 将自动创建 -->

        <!-- 应付账款分区 -->
        <view:AcctPay   id="acctPay"
                        creationComplete="sendEvent( AppEvent.ACCT_PAY_CREATED )"/>

         <!-- 总账分区 -->
        <view:GenLedger id="genLedger"
                        creationComplete="sendEvent( AppEvent.GEN_LEDGER_CREATED )"/>

    </mx:TabNavigator>
```

应用功能区中介器

类

AppSectionsMediator.as

代码

```
package com.mycompany.myapp.view.mediator
{
    import com.mycompany.myapp.view.component.AppSections;
    import com.mycompany.myapp.view.event.AppEvent;

    import org.puremvc.as3.interfaces.INotification;
    import org.puremvc.as3.patterns.mediator.Mediator;

    public class AppSectionsMediator extends Mediator
    {
        public static const NAME:String = 'AppSectionsMediator';

        public function AppSectionsMediator( viewComponent:AppSections )
        {
            super( NAME, viewComponent );
        }

        protected function get appSections():AppSections
        {
            return viewComponent as AppSections;
        }

        override public function onRegister():void
        {
            // 为默认已存在的第一个子组件建立中介关系
            facade.registerMediator( new AcctRcvMediator( appSections.acctRcv ) );

            // 监听其他子组件的创建
            appSections.addEventListener( AppEvent.ACCT_PAY_CREATED,
                                          onAPCreated  );
            appSections.addEventListener( AppEvent.GEN_LEDGER_CREATED,
                                          onGLCreated );
```

```
        }

        override public function listNotificationInterests():Array
        {
            return [ AppConstants.SHOW_ACCT_RCV,
                     AppConstants.SHOW_ACCT_PAY,
                     AppConstants.SHOW_GEN_LEDGER ];
        }

        override public function handleNotification( note:INotification ):void
        {
            // 处理传入的请求，以显示不同组件
            switch ( note.getName() )
            {
                case AppConstants.SHOW_ACCT_RCV:
                    appSections.currentSection = AppSections.AR_SECTION;
                    break;

                case AppConstants.SHOW_ACCT_PAY:
                    appSections.currentSection = AppSections.AP_SECTION;
                    break;

                case AppConstants.SHOW_GEN_LEDGER:
                    appSections.currentSection = AppSections.GL_SECTION;
                    break;
            }
        }

        // AcctPay 组件创建后，为其建立中介关系
        protected function onAPCreated( event:AppEvent ):void
        {
            var apm:AcctRcvMediator =
                new AcctPayMediator( appSections.acctPay );
            facade.registerMediator( apm );
        }

        // GenLedger 组件创建后，为其建立中介关系
        protected function onGLCreated( event:Event ):void
        {
            var glm:GenLedgerMediator =
                new GenLedgerMediator( appSections.genLedger );
            facade.registerMediator( glm );
        }
    }
}
```

动态添加视图组件

这与延迟实例化类似，只是在这里，负责实例化组件并将它添加到显示列表中的不是 Flex，而是我们自己。我们可能需要、也可能不需要为该组件建立中介关系，但都希望按照业务逻辑的要求，在需要时将它添加进去。

考虑一个简单的游戏：我们基于 Flex Spark 的 Group 类定义了 GameBoard 组件。Group 是一个绝对定位容器，即按 x 和 y 属性布置子组件。现在希望向 GameBoard 添加两种视图组件：Player 组件和 Fixture 组件。Player 可能具有生命值、工具等多个属性；场景物件则可能是钱袋、树木或炸弹，碰到它们时可能得分、被阻挡，或者死亡。这些配置可以持久化，从而保存游戏进度。

因此，我们创建了名为 PlayerConfigVO 和 FixtureConfigVO 的值对象，用于在新的 Player 或 Fixture 组件放入 GameBoard 之前对其初始化。这些 VO 可以从存储中读取，也可以由程序创建，然后通过 Notification 发送给 GameBoardMediator。在本例中，假定只有 Player 尚不存在时才添加它；添加时要为其建立中介关系，并放在 GameBoard 的中央。Fixture 则只需添加到 GameBoard 的随机位置，不建立中介关系；它们的事件以后也许会由 GameBoard 本身或 GameBoardMediator 处理。

在本例中，视图组件和配置 VO 的具体实现并不重要。我们关注的是 GameBoard 与 GameBoardMediator 如何协作，使组件能够动态加入显示列表，无论这些组件是否需要建立中介关系。

游戏面板

类

GameBoard.mxml

代码

```
<?xml version="1.0" encoding="utf-8"?>
<s:Group xmlns:fx="http://ns.adobe.com/mxml/2009"
         xmlns:s="library://ns.adobe.com/flex/spark"
         xmlns:mx="library://ns.adobe.com/flex/mx">

    <fx:Script>
        <![CDATA[

            // 将 Player 组件添加到游戏面板中央
            public function addPlayerToBoard( player:Player ):void
            {
                player.x = this.width/2;
                player.y = this.height/2;
                addElement( player );
            }

            // 将 Fixture 组件添加到游戏面板的随机位置
            public function addFixtureToBoard( fixture:Fixture ):void
            {
```

```
                fixture.x = Math.random() * this.width;
                fixture.y = Math.random() * this.height;
                addElement( fixture );
            }

        </fx:Script>

    </s:Group>
```

游戏面板中介器

类

GameBoardMediator.as

代码

```
    package com.me.myapp.view.mediator
    {
        import com.mycompany.myapp.view.component.GameBoard;
        import com.mycompany.myapp.view.component.Player;
        import com.mycompany.myapp.view.component.Fixture;
        import com.mycompany.myapp.model.vo.PlayerConfigVO;
        import com.mycompany.myapp.model.vo.FixtureConfigVO;
        import com.mycompany.myapp.view.event.AppEvent;

        import org.puremvc.as3.interfaces.INotification;
        import org.puremvc.as3.patterns.mediator.Mediator;

        public class GameBoardMediator extends Mediator
        {
            public static const NAME:String = 'GameBoardMediator';

            public function GameBoardMediator( viewComponent:GameBoard )
            {
                super( NAME, viewComponent );
            }

            protected function get gameBoard():GameBoard
            {
                return viewComponent as GameBoard;
            }

            override public function listNotificationInterests():Array
            {
                return [ AppConstants.ADD_PLAYER,
                         AppConstants.ADD_FIXTURE ];
            }

            override public function handleNotification( note:INotification ):void
            {
                // 处理传入的请求，以添加不同组件
                switch (note.getName())
                {
```

```
                case AppConstants.ADD_PLAYER:
                    var pConfig:PlayerConfigVO = PlayerConfigVO( note.getBody() );
                    addPlayer( pConfig );
                    break;

                case AppConstants.ADD_FIXTURE:
                    var fConfig:FixtureConfigVO = FixtureConfigVO( note.getBody() );
                    addFixture( fConfig );
                    break;
            }
        }

        // 向 GameBoard 添加已建立中介关系的 Player 组件
        protected function addPlayer( playerConfig:PlayerConfigVO ):void
        {
            // 仅在 Player 尚不存在时添加
            if ( ! facade.hasMediator( PlayerMediator.NAME ) ) {
                var player:Player = new Player( playerConfig );
                var pm:PlayerMediator = new PlayerMediator( player );
                facade.registerMediator( pm );
                gameBoard.addPlayerToBoard( player );
            }
        }

        // 向 GameBoard 添加未建立中介关系的 Fixture 组件
        protected function addFixture( fixtureConfig:FixtureConfigVO ):void
        {
            var fixture:Fixture = new Fixture( fixtureConfig );
            gameBoard.addFixtureToBoard( fixture );
        }
    }
}
```

Flex Mobile 的中介策略

Flex Mobile 在构建和销毁视图层次结构方面采用了截然不同的模式，因此我们在桌面和网页应用中常用的中介方式不再适用。不过不用担心，PureMVC 具有极强的适应性；我们只需采用一种考虑这种陌生视图组件生命周期的新策略。本节将对比普通 Flex 与 Flex Mobile 的组件生命周期和中介策略，然后介绍一个包含四个 View 的常见应用场景，最后通过代码展示这个假想应用程序的所有重要部分。

典型的 Flex 或 AIR 中介方式

通常，在 Flex（浏览器）或 AIR（桌面）应用程序中，由 Application 或 WindowedApplication 声明初始组件层次结构，创建完成后再为其建立中介关系。

每当要向 Application 或 WindowedApplication 添加视图组件时，就调用应用程序的 addChild() 方法，传入要添加的视图组件

实例。最好由应用程序自身在一个公共方法中执行这个调用，再让 Mediator 调用该公共方法。

处理应用程序的 applicationComplete 事件时，我们会把应用程序引用传给 ApplicationFacade.startup()。在那里，该引用被放入 Notification，发送给 StartupCommand，由后者为应用程序以及显示列表中其他需要中介关系的顶层组件建立中介关系。Mediator 还可以进一步为其视图组件的顶层子组件建立中介关系。

当视图组件被延迟创建时，例如 ViewStack 的子组件直到导航至它们时才创建，对应的 Mediator 应监听 Event.ADDED，并在事件处理器中为 Event 的 target 建立中介关系。例如，ViewStack 组件的 Mediator 会在各个子组件创建时，动态为它们建立中介关系。

你可能随时都希望动态创建一个视图组件，为其建立中介关系，并加入显示列表。只需创建该组件，为它注册 Mediator，再通过通知将它发送出去。ApplicationMediator 响应该通知，取出组件并将其交给应用程序，由应用程序把它添加为子组件。如果新视图组件要添加到显示列表中更深的层级，那么可以由另一个 Mediator 处理通知，而不是 ApplicationMediator，但基本思路相同。

网页或桌面应用程序中的 Mediator 通常长期存在，在整个运行期间都保持有效，因为视图组件很少被销毁。使用 Flex View States 时是一个例外：状态变化会导致视图组件被创建和销毁。从生命周期角度看，Flex View States 与 Flex Mobile 的 View 非常相似。

Flex Mobile 的中介方式

Spark 的 ViewNavigator 组件本质上是一个栈，可以将 View 压入或弹出。ViewNavigatorApplication 是针对移动设备优化的顶层应用程序，使用 ViewNavigator 作为主容器。你把应用功能放入一系列 View 中，用户在它们之间导航。

它与 Application 或 WindowedApplication 的一个主要区别是：你不会先创建 View 再将其压入 ViewNavigator，而是直接传入类名，由 ViewNavigator 替你创建。这意味着，不能再采用先创建组件、为其建立中介关系，再通过 Notification 发送给某个 Mediator 加入显示列表的方式。

ViewNavigatorApplication 可以指定 firstView，并自动创建它；你可以在启动时为其建立中介关系。其他 View 只有在压入栈中，或者通过弹栈返回该视图时才会创建。View 从 ViewNavigator 中弹出时，或者另一个 View 压入 ViewNavigator 并覆盖在它上面时，都会被销毁，除非……

你可以在 View 上声明 destructionPolicy="never" 属性。不过，这只意味着当另一个视图压入 ViewNavigator、位于它上方时，这个 View 会保留在内存中。如果把设置了 destructionPolicy="never" 的 View 弹出，它实际上仍会被销毁。所以，这里的“never”真正表示的是“有其他视图压在这个 View 上面时，永远不要销毁它”。

将 View 压入 ViewNavigator 时，还可以提供一个任意 Object 作为数据，在 View 创建时传给它。这个对象会出现在 View 的 data 属性中。此外，ViewNavigator 有一个名为 context 的属性，同样可以是任意 Object。它是只读属性，其值在压入新 View 时，作为可选参数传给 ViewNavigator.pushView()。本例将利用这个值判断当前处于哪个 View，而无需取得活动 View 再检查其类型。

在 Flex Mobile 中，View 会不断创建和销毁，所以必须动态为其建立中介关系。Mediator 的存续时间应与它所中介的组件相同。因此，ApplicationMediator 将是应用程序中唯一长期存在的 Mediator，其余都是临时的。

你可能还需要管理什么时候向 ViewNavigator 压入一个 View，什么时候通过弹栈返回某个 View。每次想回到特定页面时，当然都可以压入一个新 View；但有时你希望某个 View 一直保留在内存中，直到再次回到它。这也意味着，需要考虑哪些 View 有必要设置 destructionPolicy="never"。

Flex Mobile 示例

设想一个应用程序：用户可以登录、查看自己的事项列表、维护这些事项，或者打开信息页面，查看当前应用版本、移动设备剩余空间，以及这些事项目前占用的空间。

应用程序以 LoginView 作为 firstView。登录后，进入 ListView，查看自己的事项列表。从那里，可以退出登录，返回 LoginView；也可以在 InfoView 中查看应用信息，或者在 FormView 中编辑某个事项。

ListView 可能需要一些时间才能呈现，尤其是在事项很多时。你会希望给它设置 destructionPolicy="never"，这样在上方压入 InfoView 或 FormView 时，它就不会被销毁。从 InfoView 或 FormView 弹栈返回 ListView 时，显示的仍是同一个已建立中介关系的 ListView。但退出登录时，ListView 会从 ViewNavigator 中弹出并被销毁。这很合理，因为如果随后以另一个身份登录，本来也不希望前一个用户的列表仍留在内存中。另外，为了演示压入 View 时传递数据的能力，InfoView 将接收一个 InfoVO。我们假定其他 View 都按通常的方式从各自的 Mediator 获取数据。

这一切从应用程序开始。它需要公开若干方法，用于显示不同的 View，并封装所有压栈和弹栈逻辑；这样 ApplicationMediator 只需知道显示指定 View 应调用哪个方法。此外，我们还要为应用程序创建的初始 LoginView 组件建立中介关系，因此需要一个方法，从应用程序获取当前活动 View 的引用。

以下按启动过程中的出现顺序，列出重要角色及其职责：

IMyMobileApp 接口
列出应用程序应当实现的方法。虽然不是必需的，但它很好地演示了如何在 Mediator 与视图组件之间建立契约：帮助组件封装自己的实现，并允许用实现相同接口的其他组件替换它。

MyMobileApp 应用程序
基于 ViewNavigatorApplication；创建初始 LoginView 实例，启动 PureMVC，并实现 IMyMobileApp 接口。

应用程序外观
基于 Facade；实现 getInstance() 和 startup() 方法。

启动命令
依次准备 Controller、Model 和 View。

应用程序中介器
监听应用程序的 Event.ADDED，将其中的 View 通过 MEDIATE_VIEW 通知发送出去；关注切换 View 的通知，响应时调用应用程序中相应的方法；并为应用程序的 firstView 建立中介关系。

视图中介命令
为 View 创建并注册适当的 Mediator，在此之前先移除该 Mediator 已注册的实例。

列表视图
一个普通的 View，只是需要设置 destructionPolicy="never"。

现在来看代码：

IMyMobileApp 接口

```
package com.mycompany.myapp.view
{
    import com.mycompany.myapp.model.vo.InfoVO;

    import flash.events.IEventDispatcher;

    import spark.components.View;

    public interface IMyMobileApp extends IEventDispatcher
```

```
    {
        function getActiveView():View;
        function showLoginView():void;
        function showListView():void;
        function showInfoView( infoVO:InfoVO ):void;
        function showFormView():void;
    }
}
```

MyMobileApp 应用程序

```
<?xml version="1.0" encoding="utf-8"?>
<!-- Flex Mobile 应用程序 -->
<s:ViewNavigatorApplication
    firstView="com.mycompany.myapp.view.component.LoginView"
    implements="com.mycompany.myapp.view.IMyMobileApp"
    applicationComplete="facade.startup(this)"
    xmlns:s="library://ns.adobe.com/flex/spark"
    xmlns:fx="http://ns.adobe.com/mxml/2009">

    <fx:Script>
    <![CDATA[
    import com.mycompany.myapp.ApplicationFacade;
    import com.mycompany.myapp.model.vo.InfoVO;
    import com.mycompany.myapp.view.component.InfoView;
    import com.mycompany.myapp.view.component.ListView;
    import com.mycompany.myapp.view.component.LoginView;
    import com.mycompany.myapp.view.component.FormView;

    import spark.components.View;

    // 获取 Facade
    private var facade:ApplicationFacade = ApplicationFacade.getInstance();

    // 只读上下文
    private function get context():String
    {
        // 如果没有 context，就说明当前位于由 firstView 设置的 LoginView；
        // 我们无法为 firstView 设置 context
        return ( navigator.context ) ? String( navigator.context ) : LOGIN;
    }

    // 上下文取值
    private static const LOGIN:String = "LoginView";
    private static const LIST:String  = "ListView";
    private static const FORM:String  = "FormView";
    private static const INFO:String  = "InfoView";

    //--------------------------------------------------------
    // 以下方法满足 IMyMobileApp 接口要求
    //--------------------------------------------------------

    // 获取活动视图
    public function getActiveView():View
    {
```

```
            // LoginView 被指定为 firstView，
            // 因此需要按传统方式为它建立中介关系，
            // 这意味着必须公开当前活动视图
            return navigator.activeView;
        }

        // 显示登录视图
        public function showLoginView( ):void
        {
            // 仅在当前不处于 LoginView 时返回它
            if ( context != LOGIN ) navigator.popToFirstView();
        }

        // 显示列表视图
        public function showListView( ):void
        {
            // 仅在当前不处于 ListView 时执行操作
            if ( context != LIST )
            {
                // 如果从 InfoView 或 FormView 返回，只需弹栈；
                // 否则压入一个新的 ListView
                var returning:Boolean = ( context == INFO || context == FORM );
                if ( returning ) {
                    navigator.popView();
                } else {
                    navigator.pushView( ListView,  null, LIST );
                }
            }
        }

        // 显示表单视图
        public function showFormView(  ):void
        {
            // 仅在当前不处于 FormView 时压入它
            if ( context != FORM )
                navigator.pushView( FormView, null, FORM );
        }

        // 显示信息视图（传入 InfoVO）
        public function showInfoView( infoVO:InfoVO ):void
        {
            // 仅在当前不处于 InfoView 时压入它
            if ( context != INFO )
                navigator.pushView( InfoView, infoVO, INFO );
        }
    </fx:Script>

    </s:ViewNavigatorApplication>
```

应用程序外观

```
    package com.mycompany.myapp
    {
        import com.mycompany.myapp.controller.command.StartupCommand;
        import com.mycompany.myapp.controller.constants.AppConstants;
```

```
    import com.mycompany.myapp.view.IMyMobileApp;

    public class ApplicationFacade
    {

        public static function getInstance() : ApplicationFacade
        {
            if ( instance == null ) {
                instance = new ApplicationFacade();
            }
            return ApplicationFacade( instance );
        }

        public function startup( app:IMyMobileApp ):void
        {
            registerCommand( AppConstants.STARTUP, StartupCommand );
            sendNotification( ViewerConstants.STARTUP, app );
        }
    }
}
```

启动命令

```
package com.mycompany.myapp.controller.command
{
    import com.mycompany.myapp.model.proxy.InfoProxy;
    import com.mycompany.myapp.model.proxy.ThingProxy;
    import com.mycompany.myapp.view.IMyMobileApp;
    import com.mycompany.myapp.view.mediator.ApplicationMediator;

    import org.puremvc.as3.interfaces.INotification;
    import org.puremvc.as3.patterns.command.SimpleCommand;

    public class StartupCommand extends SimpleCommand
    {
        override public function execute( note : INotification ) : void
        {
            // 准备 Controller
            facade.registerCommand( AppConstants.MEDIATE_VIEW, MediateViewCommand );

            // 准备 Model
            facade.registerProxy( new ThingProxy( ) );
            facade.registerProxy( new InfoProxy( ) );

            // 准备 View
            var app:IMyMobileApp = IMobileApp( note.getBody() );
            facade.registerMediator( new ApplicationMediator( app ) );
        }
    }
}
```

应用程序中介器

```
package com.mycompany.myapp.view.mediator
{
```

```
import com.mycompany.myapp.controller.constants.AppConstants;
import com.mycompany.myapp.view.IMyMobileApp;
import com.mycompany.myapp.model.vo.InfoVO;

import flash.events.Event;

import org.puremvc.as3.interfaces.INotification;
import org.puremvc.as3.patterns.mediator.Mediator;

import spark.components.View;

public class ApplicationMediator extends Mediator
{
    public static const NAME:String = "ApplicationMediator";

    // 构造函数：通过接口处理组件
    public function ApplicationMediator( app:IMyMobileApp )
    {
        super( NAME, app );
    }

    // 注册时调用：列出感兴趣的通知
    override public function listNotificationInterests():Array
    {
        return [ AppConstants.SHOW_LOGIN,
                 AppConstants.SHOW_LIST,
                 AppConstants.SHOW_FORM,
                 AppConstants.SHOW_INFO,
               ];
    }

    // 注册时调用：监听事件，为初始视图建立中介关系
    override public function onRegister():void
    {
        // 监听 View 的创建
        app.addEventListener( Event.ADDED, handleAdded );


        // 为 firstView 建立中介关系
        sendNotification( AppConstants.MEDIATE_VIEW, app.getActiveView() );
    }

    // 处理切换视图的通知
    override public function handleNotification( note:INotification ):void
    {
        switch ( note.getName() )
        {
            case AppConstants.SHOW_LOGIN:
                app.showLoginView();
                break;

            case AppConstants.SHOW_LIST:
                app.showListView();
                break;
```

```
                case AppConstants.SHOW_FORM:
                    app.showFormView();
                    break;

                case AppConstants.SHOW_INFO:
                    app.showInfoView( InfoVO( note.getBody() ) );
                    break;
            }
        }

        // 处理 ADDED 事件
        private function handleAdded( event:Event ):void
        {
            // 只处理 View
            if ( event.target is View ) {
                sendNotification( AppConstants.MEDIATE_VIEW, event.target );
            }
        }

        // 将视图组件转换为正确的类型
        private function get app():IMyMobileApp
        {
            return viewComponent as IMyMobileApp;
        }
    }
}
```

视图中介命令

```
package com.mycompany.myapp.controller.command
{
    import com.mycompany.myapp.view.component.InfoView;
    import com.mycompany.myapp.view.component.ListView;
    import com.mycompany.myapp.view.component.LoginView;
    import com.mycompany.myapp.view.component.FormView;
    import com.mycompany.myapp.view.mediator.InfoViewMediator;
    import com.mycompany.myapp.view.mediator.ListViewMediator;
    import com.mycompany.myapp.view.mediator.LoginViewMediator;
    import com.mycompany.myapp.view.mediator.FormViewMediator;

    import org.puremvc.as3.interfaces.INotification;
    import org.puremvc.as3.interfaces.IMediator;
    import org.puremvc.as3.patterns.command.SimpleCommand;

    import spark.components.View;

    public class MediateViewCommand extends SimpleCommand
    {
        /**
         * 每个 View 在离开时都会被销毁，
         * 因此 Mediator 也必须是临时的。应移除已有的
         * Mediator，并随新的 View 一起注册
         * 新的 Mediator。与这些 View 的子组件
         * 关联的其他 Mediator，应在该 View 的 Mediator 的
         * onRegister() 和 onRemove()
```

```
         * 方法中进行注册和移除。
         */
        override public function execute( note:INotification ):void
        {
            var view:View = View( note.getBody() );
            var mediator:IMediator;
            switch ( view.className )
            {
                case "LoginView":
                    facade.removeMediator( LoginViewMediator.NAME );
                    mediator = new LoginViewMediator( view as LoginView );
                    break;

                case "ListView":
                    facade.removeMediator( ListViewMediator.NAME );
                    mediator = new ListViewMediator( view as ListView );
                    break;

                case "InfoView":
                    facade.removeMediator( InfoViewMediator.NAME );
                    mediator = new InfoViewMediator( view as InfoView );
                    break;

                case "FormView":
                    facade.removeMediator( FormViewMediator.NAME );
                    mediator = new FormViewMediator( view as FormView );
                    break;
            }
            if (mediator) facade.registerMediator( mediator );
        }
    }
}
```

列表视图

```
<?xml version="1.0" encoding="utf-8"?>
<!-- 列表视图 -->
<s:View xmlns:s="library://ns.adobe.com/flex/spark"
    xmlns:fx="http://ns.adobe.com/mxml/2009"
    destructionPolicy="never">

    <!-- 列表相关操作在此进行 -->

</s:View>
```

第 10 章
继续前行

如果你已经是经验丰富的 PureMVC 开发者，现在就不必再向尚未掌握它的同事解释那么多了，只需把本书借给他们，或告诉他们到哪里购买。如果你刚刚起步，希望本书已经帮助你充分了解 PureMVC 的开发过程，以及掌握它所需思维方式中的要点。当你继续提升 PureMVC 技能时，下面还有一些知识可以助你前行。

在 PureMVC 中使用调试器

这里从调试的角度，简要介绍 PureMVC 的优势。如果你从未用过 IDE 的调试器，其实它很简单，我会在这里展示基本步骤。如果你已经熟悉调试，相信只要设置第一个断点并查看变量，就能体会 PureMVC 对应用程序内部结构的组织带来的好处。你可能知道，在应用程序中设置断点时，如果当前类没有持有某个对象的引用，通常就无法查看那个对象。所以，虽然能了解断点所在类的状态，却很难知道其他地方正在发生什么，除非像圣诞夜布设路障一样，在程序各处设置断点。

在 PureMVC 中，只要在任何 Mediator、Command 或 Proxy 的任意可执行代码行设置断点，就能获得 Facade 的引用，进而获得 Model、View 和 Controller 的引用。通过它们，又可以访问全部 Proxy、Mediator 实例和 Command 注册信息。当然，Proxy 和 Mediator 还让你能够访问服务、VO 和视图组件。你可以沿着这些引用深入应用程序的任何其他部分，查看其中的属性。图 10-1 至图 10-7 的一系列截图，简要展示了如何查看 StoryArchitect 应用程序。

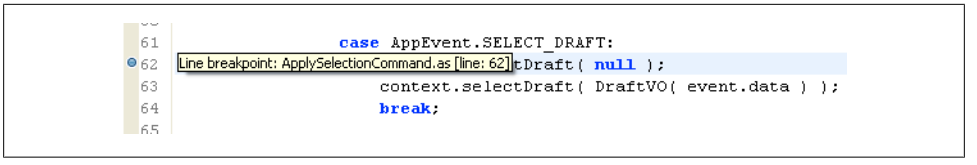


图 10-1　设置断点



图 10-2　启动调试器

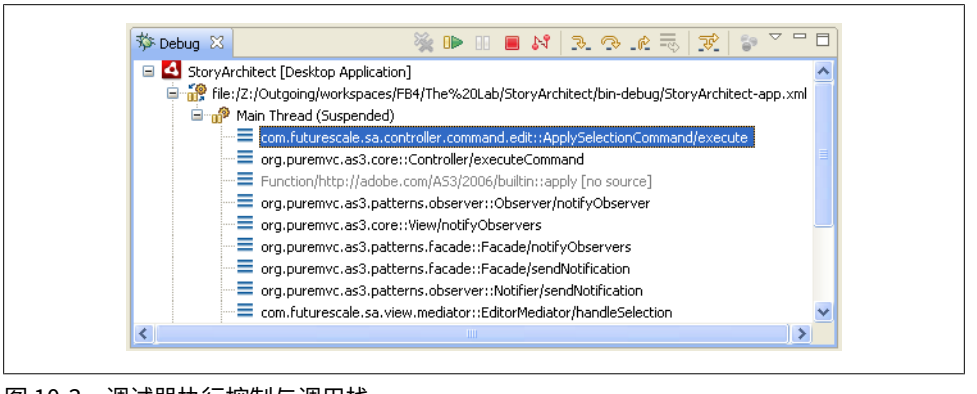


图 10-3　调试器执行控制与调用栈

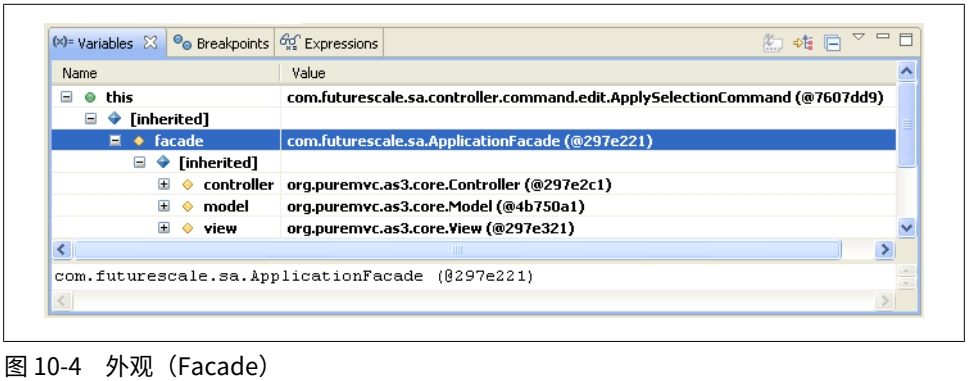


图 10-4　外观（Facade）

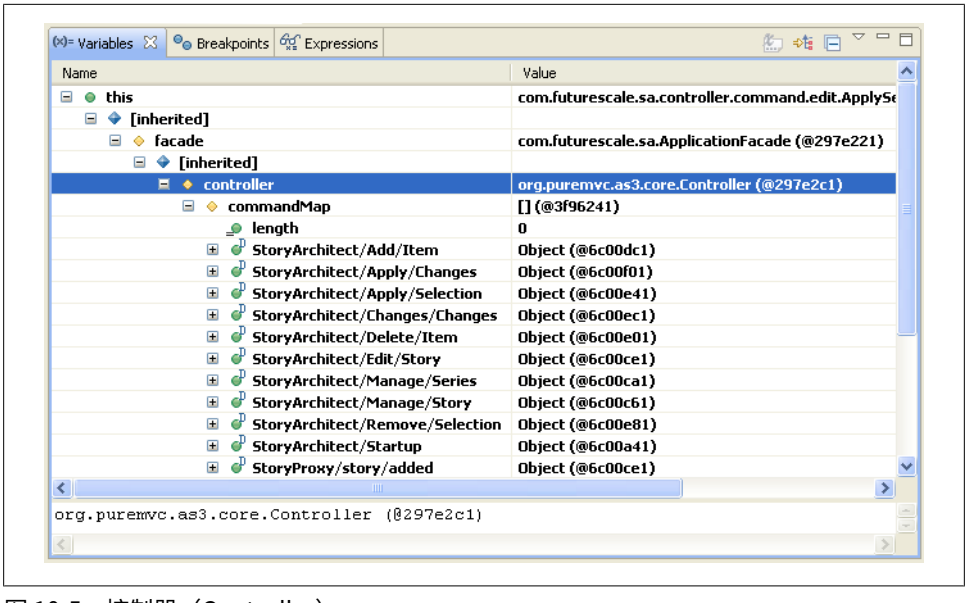


图 10-5　控制器（Controller）

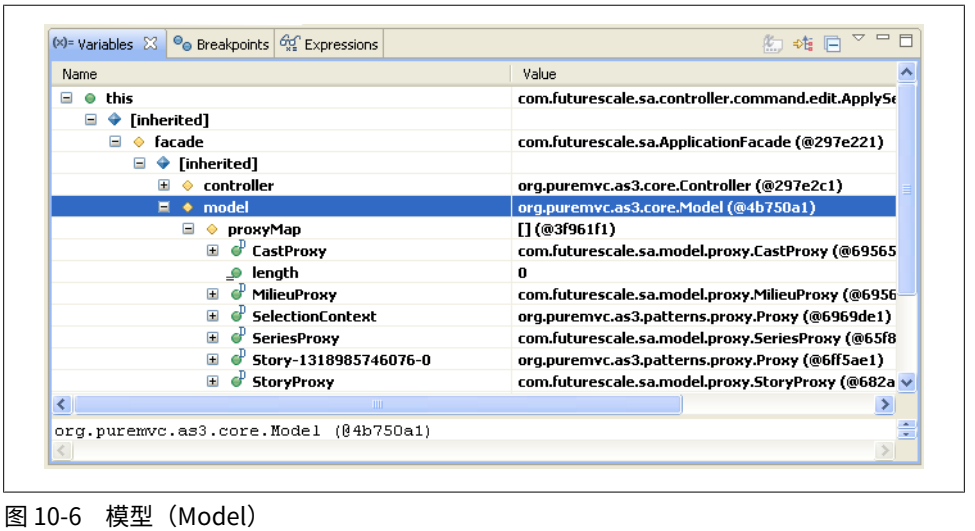


图 10-6　模型（Model）

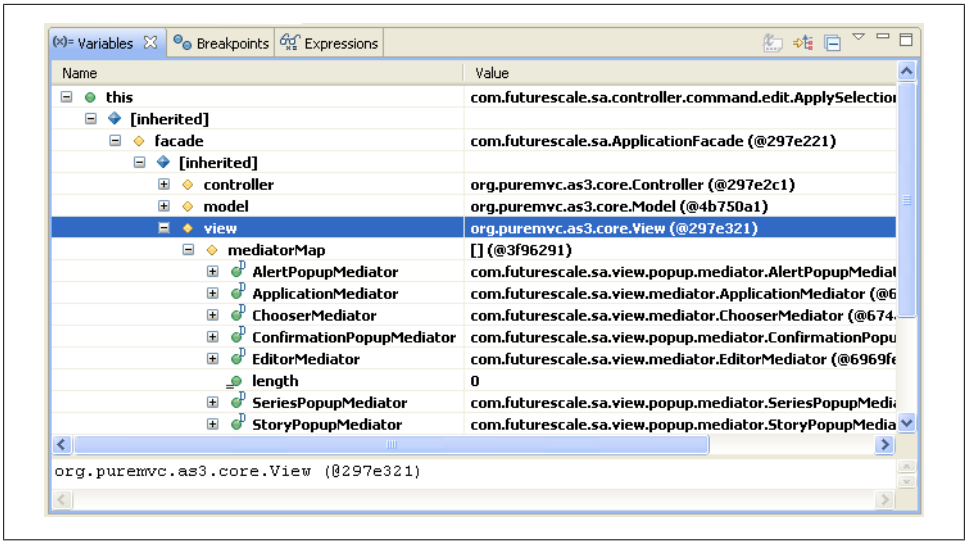


图 10-7　视图（View）

PureMVC 实用工具

开源框架有一个特点：如果没有清楚界定并固定其范围，“可怕的功能膨胀怪兽”就会随意摆布代码库。功能不断增长，带来缺陷；修复缺陷的版本又塞入新功能，如此循环，没有尽头。开发者不得不持续升级，以确保代码仍能工作，并且拥有最新、最强的能力。PureMVC 的设计范围很简单，就是实现 MVC 的分离：前面的调试器中可以直观看到这种分离，而你也已经在本书中看到了它的价值。为解决常见问题而增加的功能，应当作为独立的实用工具提供，让你按需选用。

在 PureMVC 网站 AS3 的 Code 栏目下，可以找到适用于 Flex、Flash 和 AIR 的实用工具。以下几个非常实用，值得先了解：

· AsyncCommand（Flash／Flex／AIR）
  http://trac.puremvc.org/Utility_AS3_AsyncCommand
  作者：Duncan Hall（duncan.hall@puremvc.org）

AsyncCommand 和 AsyncMacroCommand 类似于 SimpleCommand 和 MacroCommand，但它们解决了这样一个问题：顺序执行一系列 Command，而每个 Command 可能必须完成一个或多个异步操作，才能执行下一个 Command。包含标准版和 MultiCore 版。

· Desktop Citizen（AIR）

http://trac.puremvc.org/Utility_AS3_AIR_DesktopCitizen
作者：Cliff Hall（cliff@puremvc.org）
这个工具让基于 PureMVC 的 AIR 应用程序在每次启动时，都能恢复此前的窗口大小、位置和窗口状态（最大化、最小化或普通状态）。它还提供了在应用窗口关闭前请求确认，或执行某些关闭流程的机制。

- Loadup（Flash／Flex／AIR）

http://trac.puremvc.org/Utility_AS3_Loadup
作者：Philip Sexton（philip.sexton@puremvc.org）
这个工具解决了如何在应用运行期间任意时刻管理资源异步加载的问题。它支持加载顺序依赖、不同媒体类型，并能够跟踪进度。包含标准版和 MultiCore 版。

- State Machine（状态机，Flash／Flex／AIR）

http://trac.puremvc.org/Utility_AS3_StateMachine
作者：Neil Manuell 和 Cliff Hall（neil.manuell@puremvc.org、cliff@puremvc.org）
这个工具提供了简单而有效的有限状态机（FSM）实现，允许定义离散状态、从各个状态可以进行的有效转换，以及触发这些转换的动作。它还提供一种机制，可用 XML 定义整个状态机，并将一个配置完整的 StateMachine 注入 PureMVC 应用程序。包含标准版和 MultiCore 版。

- Undo（撤销，Flash／Flex／AIR）

http://trac.puremvc.org/Utility_AS3_Undo
作者：Dragos Dascalita（dragos.dascalita@puremvc.org）

UndoableCommandBase 和 UndoableMacroCommandBase 类似于 SimpleCommand 和 MacroCommand，只是它们通过维护已执行 IUndoableCommand 的历史记录，让你能够在应用程序中实现撤销功能。目前仅提供标准版。

其他资源

世界各地都有很多人在使用 PureMVC，其中不少人抽出时间，以各种方式回馈社区。他们的贡献包括论坛中有益的讨论、将框架移植到其他编程语言、把《最佳实践》文档翻译成其他语言，以及示例程序、实用工具、调试工具、项目模板、培训课程、富有见地的博客文章等。

无法在这里全部列出，下面介绍一些值得关注的资源：

- PureMVC 项目网站

http://puremvc.org
提供方：Futurescale, Inc.
先从这里开始，探索文档、社区和代码。

· PureMVC 论坛

注册地址：http://contact.futurescale.com
阅读和发帖地址：http://forums.puremvc.org
如果你有兴趣积极使用 PureMVC，就应注册论坛账号，并熟悉已有资料。多年来，社区已经反复讨论了日常 PureMVC 开发中最重要的问题。你遇到的问题很可能早就有人回答过；如果还没有，欢迎加入我们并分享你的想法。遗憾的是，持续不断的垃圾信息机器人迫使我们关闭了普通的论坛注册流程，改用防机器人的注册机制。通过上面的地址使用向导加入时，还可以体验另一个有趣的 Flex／PureMVC 应用。请从“Request Access to Futurescale-operated Forums”（申请访问 Futurescale 运营的论坛）开始操作。

· Darkstar PureMVC 漫游器

http://darkstar.puremvc.org
提供方：Futurescale, Inc.
PureMVC 的世界正在不断扩展；用 Darkstar 来探索它。随着 PureMVC 越来越受欢迎，其网站汇集了大量内容，但谁愿意一页接一页地浏览网页，才能找到所需资料呢？这个交互式网络节点浏览器可以帮助你探索 PureMVC 项目的代码、社区和文档资源。它使用 Flex 和 PureMVC 标准版构建。

· KapLab PureMVC 控制台

http://lab.kapit.fr/display/puremvcconsole
提供方：KapIT
这款专业工具面向使用 PureMVC AS3 框架（标准版或 MultiCore 版）的 Flex 和 AS3 应用开发者，可以深入展示重要框架角色的所有实例正在发生什么，包括 Notification、Command、Mediator 和 Proxy。

· PureMVC-Gen

https://github.com/gjastrab/puremvc-gen
作者：Greg Jastrab
PureMVCGen 是一个 Ruby gem，封装了基于 ANT 的代码生成工具，用于生成 PureMVC ActionScript 代码。目前仅适用于 PureMVC 标准版。

· 用管道连接状态机：结合 Pipes 与有限状态机的 PureMVC MultiCore
http://bit.ly/xpeco
作者：Joel Hooks
这是一篇介绍使用 PureMVC 进行模块化编程的优秀入门文章。它在一个简单应用中结合了这些关键工具，充分展示 MultiCore 的强大与简洁，并提供完整应用及源代码。
· MultiCore 模块化 Pipes 应用中的垃圾回收

http://bit.ly/tuG4QQ
作者：Simon Bailey
这是对模块化、使用管道通信的 PureMVC 应用中如何正确处理 GC 的深入研究，涵盖销毁模块时确保内存被释放所必需的全部步骤。附带完整应用和源代码；添加或移除模块时，应用会显示内存统计信息。
· PureMVC TV

http://puremvc.tv
提供方：Futurescale, Inc.
PureMVC 项目架构师制作的一系列演示，以自定义幻灯片形式呈现，并配有语音讲解。内容包括《标准版概览》《MultiCore 版概览》《StateMachine 工具概览》，以及《进入节奏》：使用 MultiCore、StateMachine 和 Pipes 工具构建音乐播放器应用。

关于作者

Cliff Hall 是一位拥有超过 25 年行业经验的软件架构师。

他的职业经历跨度很大，从使用机器语言为 Commodore 64 和 Apple II 编写游戏，到实现大型、面向对象的企业级应用程序。

1993 年，他创办了《Synapse：兼收并蓄的多媒体杂志》，这是第一份基于 CD-ROM 的数字杂志（也称“digizine”）。在第一批网页浏览器出现将近一年前，它就已经通过自然协调的用户界面，提供多媒体文本、音频、视频、动画和交互式虚拟现实。

他也是音乐人，曾为天文馆节目创作配乐，演出场地从曼哈顿的俱乐部到波多黎各雨林中的艺术节，各不相同。他目前的音乐录制项目 Sea of Arrows，可在 http://seaofarrows.com 找到。

他是 Futurescale, Inc. 的所有者，为世界各地的大型企业和前沿创业公司开发富互联网应用程序，也是开源 PureMVC 框架的架构师。

如需邀请他开发你的下一个应用程序，请访问 http://contact.futurescale.com。